

Modellieren mit Diskreten Strukturen

Lehr- und Arbeitsbuch zur Lehrveranstaltung

Diskrete Strukturen

Peter Riegler

unter Mitarbeit von Jessica Kirsch

19. September 2022

Inhaltsverzeichnis

1	Grundlegende Strukturen	3
1.1	Hinweise zu diesem Text	3
1.2	Aufwärmübungen	4
1.3	Abstraktion und Modellbildung	5
1.4	Kollektionen	7
1.5	Funktionen	9
1.6	Einführung in <i>setlX</i>	11
1.7	Lernziele	11
1.8	Übungsaufgaben	12
2	Aussagenlogik	13
2.1	Aufwärmübungen	13
2.2	Gegenstand der Logik	14
2.3	Aussagen und Aussageformen	15
2.4	Logische Operatoren	19
2.4.1	Negation, Konjunktion und Disjunktion	19
2.4.2	Modellieren von Alltagssprache - Teil 1	23
2.4.3	Subjunktion, Bijunktion und Alternation	24
2.4.4	Bedeutung der Subjunktion	27
2.4.5	Modellieren von Alltagssprache - Teil 2	28
2.4.6	Die 16 binären logischen Operatoren	30
2.5	Präzedenz	32
2.6	Negation sprachlicher Aussagen	34
2.7	Empfohlene Vertiefungen	35
2.7.1	Beziehung von Aussagenlogik und Sprache	35
2.7.2	Gegenteil und Gegensatz	36
2.7.3	Bedeutungen des Gleichheitszeichens	36
2.8	Lernziele	38
2.9	Übungsaufgaben	38
3	Tautologien	41
3.1	Aufwärmübungen	41
3.2	Tautologie und Kontradiktion	42
3.3	Notwendige und hinreichende Bedingungen	44
3.4	Modellieren mit Subjunktion, Bijunktion, Implikation bzw. Äquivalenz	46
3.5	Definitionen und Theoreme	47
3.6	Äquivalenzumformungen	48
3.7	Lernziele	49
3.8	Übungsaufgaben	49

4	Aussagenlogisches Rechnen	51
4.1	Aufwärmübungen	51
4.2	Rechnen mit Wahrheitstabellen	52
4.3	Algebraische Eigenschaften logischer Operatoren	52
4.4	Symbolisches Rechnen mit logischen Ausdrücken	53
4.5	Faule Auswertung	55
4.6	Allgemeine Strukturen	55
4.7	Minimalset binärer logischer Operatoren	57
4.8	Lernziele	58
4.9	Übungsaufgaben	58
5	Mengen	59
5.1	Aufwärmübungen	59
5.2	Mengen und andere Kollektionen	60
5.3	Element und Teilmenge	61
5.4	Mengen-Konstruktoren	63
5.5	Operationen auf Mengen	65
5.6	Leere Menge	68
5.7	Algebraische Eigenschaften	69
5.8	Potenzmenge	70
5.9	Kartesisches Produkt	70
5.10	Empfohlene Vertiefungen	71
	5.10.1 Boolesche Algebra	71
	5.10.2 Beweis mengentheoretischer Beziehungen	72
5.11	Lernziele	75
5.12	Übungsaufgaben	75
6	Prädikatenlogik	77
6.1	Aufwärmübungen	77
6.2	Prädikate	78
6.3	Quantifizierte Aussagen	78
6.4	Modellieren mit Quantoren	79
6.5	Negation quantifizierter Aussagen	81
6.6	Gebundene und freie Variablen	81
6.7	Geschachtelte quantifizierte Aussagen	83
6.8	Syntax	84
6.9	Quantoren als große Operatoren	85
6.10	Tautologien und quantifizierte Aussagen	87
6.11	Empfohlene Vertiefungen	89
	6.11.1 Sichtbarkeitsbereich von gebundenen Variablen	89
	6.11.2 Rekursive Definition	90
	6.11.3 Große Operatoren der Mengenlehre	90
	6.11.4 Große Operatoren in <i>setlX</i>	91
6.12	Lernziele	91
6.13	Übungsaufgaben	91
7	Relationen	93
7.1	Aufwärmübungen	93
7.2	Modellieren von Beziehungsmustern	94
7.3	Eigenschaften von Relationen	95
7.4	Graphen	98
7.5	Äquivalenzrelationen	100
7.6	Ordnungsrelationen	102
7.7	Operationen auf Relationen	103

7.8	Empfohlene Vertiefungen	106
7.8.1	Verallgemeinerung	106
7.8.2	Graphentheorie	106
7.8.3	Relationales Datenmodell	107
7.9	Lernziele	107
7.10	Übungsaufgaben	107
8	Funktionen	109
8.1	Aufwärmübungen	109
8.2	Perspektiven auf Funktionen	110
8.2.1	Funktion als Transformationsprozess	110
8.2.2	Funktion als spezielle Relation	112
8.2.3	Funktion als Kovariation	113
8.3	Formale Notation	114
8.4	Bild- und Wertemenge	115
8.5	Formale Definition	116
8.6	Repräsentationen von Funktionen	119
8.7	Modellieren mit Funktionen	119
8.8	Lernziele	124
8.9	Übungsaufgaben	124
9	Funktionen als Objekte	125
9.1	Aufwärmübungen	125
9.2	Umkehrfunktion	126
9.3	Besondere Eigenschaften	128
9.3.1	Surjektivität	128
9.3.2	Injektivität	129
9.3.3	Bijektivität	130
9.3.4	Bilden der Umkehrfunktion	130
9.4	Operationen auf Funktionen	132
9.4.1	Addition von Funktionen	132
9.4.2	Gleichheit von Funktionen	135
9.4.3	Umkehrfunktion	137
9.5	Komposition von Funktionen	137
9.6	Totale und partielle Funktionen	142
9.7	Empfohlene Vertiefungen	143
9.7.1	Currying	143
9.7.2	Operatorpositionierung	144
9.7.3	Besondere Funktionen	144
9.8	Lernziele	144
9.9	Übungsaufgaben	144
10	Kombinatorik	147
10.1	Aufwärmübungen	147
10.2	Additives und multiplikatives Zählen	149
10.2.1	Additives Zählen	149
10.2.2	Multiplikatives Zählen	150
10.3	Elementares Zählen	150
10.3.1	Umkehraufgaben	150
10.3.2	Kleiner werdende Mengen	151
10.3.3	Permutationen	152
10.4	Kombinatorisches Modellieren	153
10.4.1	Reihenfolge und Mehrfachvorkommen relevant	154
10.4.2	Reihenfolge relevant, Mehrfachvorkommen nicht relevant	154

10.4.3	Reihenfolge nicht relevant, Mehrfachvorkommen nicht relevant	155
10.4.4	Reihenfolge nicht relevant, Mehrfachvorkommen relevant	156
10.5	Mit dem Rechner zählen	157
10.6	Empfohlene Vertiefungen	158
10.6.1	Verteilen statt Auswählen	158
10.6.2	Binomialkoeffizient	160
10.7	Lernziele	162
10.8	Übungsaufgaben	162
11	Zahlentheorie	165
11.1	Aufwärmübungen	165
11.2	Ganzzahlige Division	165
11.3	Restklassen	167
11.4	Kongruenz	169
11.5	Modulare Arithmetik	170
11.6	Modulare arithmetische Operatoren	175
11.7	Empfohlene Vertiefungen	176
11.8	Lernziele	177
11.9	Übungsaufgaben	177
12	Algebraische Strukturen	179
12.1	Aufwärmübungen	179
12.2	Übergreifende Strukturen	180
12.3	Gruppen	180
12.4	Körper	187
12.5	Lernziele	189
12.6	Übungsaufgaben	189

Kapitel 1

Grundlegende Strukturen

*You can't do much carpentry with your bare hands and
you can't do much thinking with your bare brain.*
(Bo Dahlbom)

In diesem Kurs geht es um Denkzeuge – Werkzeuge des Denkens. Was der Hobel für Zimmerleute ist, sind Begriffe für denkende Menschen. Wer sich keinen Begriff von Viren gemacht hat, wird Viren nicht erkennen. Wer sich keinen Begriff von Relationen gemacht hat, wird sie nicht erkennen und auch nicht als Denkzeug verwenden können.

Wissenschaftliche Begriffsbildung ist mit einigen Herausforderungen verbunden. Da ist zum einen die Tatsache, dass viele Begriffe der Alltagssprache entlehnt sind, aber im wissenschaftlichen Kontext eine andere oder auch nur etwas andere Bedeutung haben. Eine Maus in der Informatik eignet sich nicht als Katzenfutter.

Eine weitere Herausforderung besteht darin, nahe beieinander liegende Begriffe, die unterscheidbar und unterscheidungswürdig sind, auseinanderzuhalten. Das ist in der Regel mit anfänglichen Schwierigkeiten verbunden. „Gegenteil und Gegensatz“ könnte eines der ersten Begriffspaare in dieser Kategorie sein, wo Ihnen diese Schwierigkeit begegnen wird.

1.1 Hinweise zu diesem Text

Lesen Sie diesen Text - wie jeden Fachtext - immer mit Stift und Papier. Mit Papier ist nicht das Papier gemeint, auf das dieser Text gedruckt ist, sondern zusätzliches Papier. Der Stift ist nicht ausschließlich zum Unterstreichen da, sondern damit Sie sich Notizen auf dem zusätzlichen Papier machen. Diese Notizen können darin bestehen, dass sie Beispiele nachvollziehen, selbst Beispiele kreieren, Dinge ausprobieren, Skizzen machen, Textstellen exzerpieren, notieren, was Ihnen unklar ist, und vieles mehr.

Experten lesen immer mit Stift und Papier. Und sie lesen nicht unbedingt sequentiell, sondern blättern zurück, überfliegen gelegentlich Textpassagen, um sie erst später im Detail zu studieren.

Lesen Sie diesen Text - wie jeden Fachtext - mit ausreichenden „Unterbrechungen“. Solche Unterbrechungen können Pausen sein, in denen Sie etwas anderes tun als sich mit den Gegenständen des Texts zu befassen. Es sollten aber auch Momente des Innehaltens sein, in denen Sie auf das Gelesene zurückblicken, Dinge ausprobieren, Notizen machen *etc.*

Vermeiden Sie es die Kapitel dieses Textes an einem Stück zu lesen, auch wenn die Leseaufträge sich immer auf ein Kapitel beziehen. Ein geeigneter Zeitpunkt für die genannten „Unterbrechungen“ ist z. B., wenn Sie einen Abschnitt gelesen haben. Gelegentlich wird der Text Ihnen vorschlagen, erst einmal nicht weiterzulesen. Aber nur gelegentlich, denn Sie sollen und müssen lernen selbst zu erkennen, wann die Zeit für eine „Unterbrechung“ gekommen ist.

Machen Sie sich immer klar, dass Fachtexte, wie alle wichtigen Texte mehrfach gelesen werden müssen. Machen Sie sich auch immer klar, dass niemand von Ihnen erwartet, dass Sie alles nach dem ersten Durchlesen verstanden haben. Das gelingt auch Experten selten. Machen Sie sich auch immer

klar, dass es in Ordnung ist, wenn Sie zunächst nicht alles verstehen. Um diese Punkte zu klären ist ja die Kontaktzeit der Lehrveranstaltung da - zumindest während des Studiums. Später werden Sie jedoch nicht für jeden Fachtext auf die Hilfe einer Lehrveranstaltung zurückgreifen können.

Die Aufwärmübungen zu Beginn eines jeden Kapitels sind wie beim Sport dazu da, vor der eigentlichen Aktivität – hier dem Studium der Inhalte – gemacht zu werden: zum Aufwärmen des Gehirns, zum Strecken, zum Dehnen. Sie dienen der Aktivität, also dem Lernen. Sie sind nicht dazu da zu sehen, zu welchem Grad Sie die Kursinhalte beherrschen. Diesen Zweck erfüllen andere Aufgaben wie z. B. die Übungsaufgaben. Sie sollten die Aufwärmübungen ernsthaft und schriftlich beantworten.

Die Leitfragen am Ende der Aufwärmübungen haben einen doppelten Zweck. Sie dienen dem Aufwärmen, aber auch einer *ersten* Lernzielkontrolle, d. h. Sie sollten sie *nach* dem Studium eines Kapitels beantworten können. Sollte das nicht der Fall sein, ist dies ein Signal, dass Sie den Text (noch) nicht aufmerksam genug gelesen haben.

In diesem Text treten immer wieder Textpassagen auf, die klein gedruckt sind. Solche Textpassagen können Sie beim ersten Lesen ausblenden. Sie müssen sie allerdings beim erneuten Lesen mitlesen, denn sie enthalten wichtige Verknüpfungen zu anderen Themen innerhalb oder außerhalb dieses Kurses, die für diesen Kurs wichtig sind. Fußnoten dagegen enthalten weitergehende oder Hintergrundinformation, die für diesen Kurs nicht elementar sind.

Formale Ausdrücke, Gleichungen etc., auf die später verwiesen werden soll, werden fortlaufend durchnummeriert und in der Form ((Kapitelnummer).(fortlaufende Nummer)) referenziert, z. B. verweist (2.1) auf die so bezeichnete Gleichung auf Seite 21.

Quadrate der Form ■ signalisieren, dass ein Kontext beendet ist, in der Regel der Kontext eines Beispiels.

Hervorhebungen im Text sind *kursiv* gesetzt, ebenso Begriffe, die nicht unbedingt Bestandteil der deutschen Sprache sind, wie z. B. *black box*.

Das Symbol  fordert Sie auf, an der damit markierten Stelle etwas einzutragen und zwar *bevor Sie weiterlesen*.

Empfohlene Vertiefungen sind optional, also nicht Bestandteil der Lernziele, aber durchaus relevant. Gerade wenn Sie sich nicht ausgelastet fühlen, sollten Sie diese nicht verpassen.

Lernziele beschreiben, welche Fertigkeiten und Kenntnisse Sie sich spätestens am Kursende angeeignet haben müssen. Der Grad des Erreichens der Lernziele bestimmt Ihre Abschlussnote.

Jedes Kapitel schließt mit Übungsaufgaben. Diese sind dazu da, dass Sie sie bearbeiten, *nachdem* das Kapitel in der Kontaktzeit der Lehrveranstaltung abgeschlossen wurde.

1.2 Aufwärmübungen

1. Was bedeutet abstrakt für Sie?



2. Bei welchen der folgenden Fragestellungen kommt es auf die Reihenfolge der Geldscheine an?
Bei welchen Fragestellungen kommt es darauf an, ob ein Geldschein mehrfach vorhanden ist?

- (a) Die Geldbeträge, die es im Euroraum als Geldscheine gibt
- (b) Die Arten von Geldscheinen, die ich in meinem Portmonee habe
- (c) Der Betrag, den ich an Papiergeld in meinem Portmonee habe



3. Leitfragen:

- (a) Wie unterscheidet sich die Bedeutung von „abstrakt“ in der Informatik von der alltags-sprachlichen Bedeutung?
- (b) Warum sind Modelle in wissenschaftlichen Disziplinen wichtig?
- (c) Warum ist es sinnvoll, verschiedene Arten von Kollektionen zu unterscheiden?

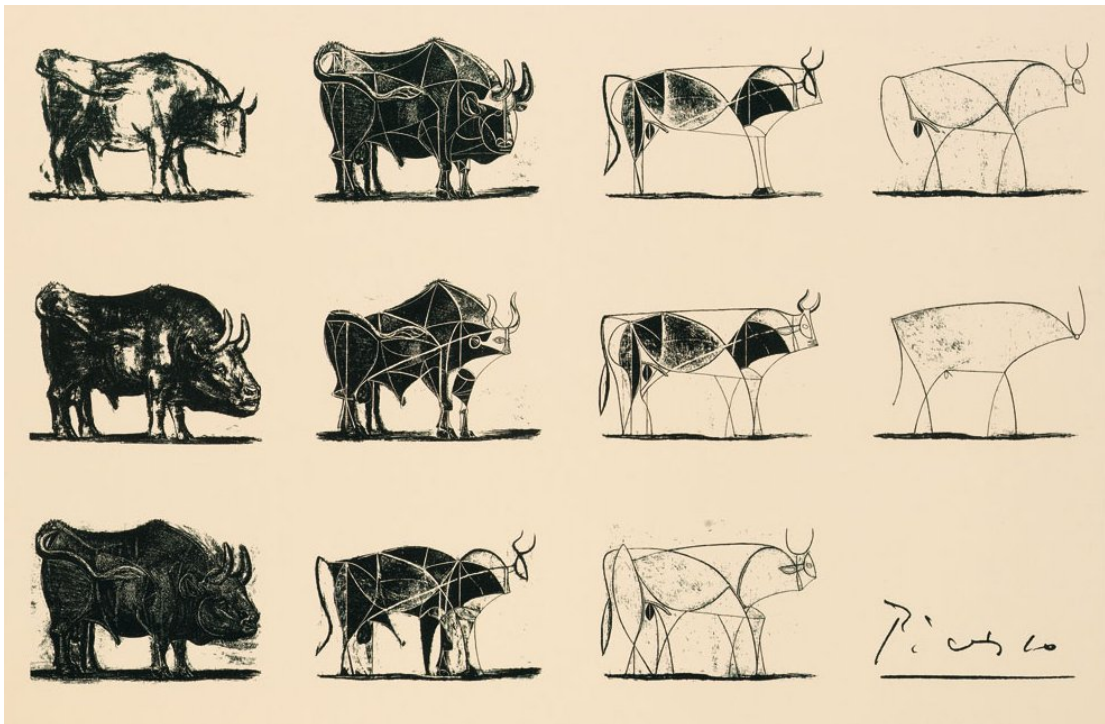


Abbildung 1.1: Die Lithografien „Der Stier“ von Pablo Picasso verbildlichen einen Abstraktionsprozess.² An dessen Ende steht eine Strichzeichnung, die immer noch als Stier erkennbar ist, weil sie alles Wesentliche enthält, während alles Unwesentliche wegabstrahiert wurde.



1.3 Abstraktion und Modellbildung

Zwei Themen, die uns im Laufe dieses Kurses immer wieder begegnen werden, sind Abstraktion und Modellbildung. „Abstrakt“ ist für viele Menschen gleichbedeutend mit „unverständlich“. Abstraktion bedeutet aber „Unwesentliches weglassen“ mit dem Ziel, das Wesentliche besser verstehen zu können.¹

Abstrakt im Sinne von unverständlich ist also eher das, wo wir den Abstraktionsprozess nicht nachvollziehen können. Eines der abstraktesten Konzepte der Mathematik (und der Menschheit) überhaupt ist dagegen für viele Menschen problemlos greifbar: die (natürliche) Zahl.

Was ist beispielsweise 42? 42 Autos, 42 Kilometer usw. sind ziemlich konkret. Aber 42? Für die Zahl 42 ist es unerheblich, was es 42-mal gibt (und ob es das überhaupt gibt). Dieses Was ist beim Zahlenkonzept wegabstrahiert.

Die Bedeutung „Unwesentliches weglassen“ steht auch hinter „abstrakt“ in abstrakte Kunst. Abb. 1.1 verdeutlicht das an Hand eines Bilderzyklus von Picasso.

¹Das Sprichwort „Den Wald vor lauter Bäumen nicht sehen“ beschreibt einen Zustand, in dem jemand nicht abstrahieren kann, weil es nicht gelingt Unwesentliches wegzulassen.

Abstraktion ist ein fortwährender Prozess. Die schon abstrakten Konstrukte Kugel, Quader, Kegel werden in einer weiteren Abstraktionsstufe zu dreidimensionalen Körpern. Die Form wurde wegabstrahiert, geblieben ist als Wesentliches die Dreidimensionalität.

Beispiel 1.1: In der Programmierung erlauben abstrakte Klassen einen hohen Grad an Abstraktion. Der folgende Java-Code geht den Weg vom abstrakten Konzept Körper zu weniger abstrakten Konzepten wie Quader oder Kugel (es wird hier nicht von Ihnen erwartet, dass Sie den Code lesen und verstehen können; versuchen Sie ihn allerdings so gut wie möglich zu verstehen):

```
public abstract class Koerper {
    public abstract double Volumen();
    public abstract double Oberflaeche();
}

public class Quader extends Koerper {
    protected double h,l,b;
    public Quader() {h=0.0; l=0.0; b=0.0;}
    public Quader(double h, double l, double b) {this.h=h; this.l=l; this.b=b;}
    public double Volumen() {return h*l*b;}
    public double Oberflaeche() {return 2*(h*l+l*b+b*h);}
    public double getHoehe() {return h;}
    public double getLaenge() {return l;}
    public double getBreite() {return b;}
}

public class Kugel extends Koerper {
    ...
}
```

Der Weg dieses Code-Ausschnitts ist deduktiv, also vom abstrakten Allgemeinen (Körper) zum Speziellen (Kugel, Quader). Hier zeigt sich eine wichtige Parallele der Informatik zur Mathematik: Das Programm als Produkt der Arbeit von Informatikern ist abstrakt und das Vorgehen innerhalb des Programms ist deduktiv, ebenso wie der mathematische Text als Produkt der Arbeit von Mathematikern und das Vorgehen in Mathematik-Büchern. ■

Abstraktion ist ein wesentlicher Aspekt des Modellierens. Modellbildung abstrahiert mittels Erstellen des Modells von der Realität. Meist ist die Realität zu komplex, um sie vollständig zu beschreiben. Beim Modellieren geht es nicht um Vollständigkeit. Vielmehr sollen die wesentlichen Eigenschaften identifiziert und beschrieben werden.

Beispiel 1.2: Wenn Sie Kugeln modellieren, wird der Kugelradius sicher ein wesentlicher Aspekt ihres Modells sein. Aspekte wie Material, Rauigkeit der Oberfläche oder den Ort, an dem sich die Kugel befindet, werden Sie vermutlich wegabstrahieren. ■

Modelle sind Beschreibungen, die sich auf das Wesentliche beschränken. Was wesentlich ist, hängt dabei mit Ziel und Zweck des Modells zusammen.

Beispiel 1.3: Das Rutherfordsche Atommodell beschreibt zwei wesentliche Aspekte von Atomen: Den Aufbau aus Kern und Elektronen und die Bewegung der Elektronen auf Bahnen um den Kern. Es beschreibt allerdings nicht die Quantisierung der Elektronenenergien und auch nicht die Wechselwirkung von Licht mit Atomen.

Das Bohrsche Atommodell unterscheidet sich vom Rutherfordschen dadurch, dass es gerade diese Wechselwirkung als wesentlich ansieht und das Konzept der Elektronenbahn durch das des Orbitals ersetzt. ■

Abstraktion und Modellieren gehen beide mit der Verwendung von Symbolen und Begriffen einher. Symbole und Begriffe repräsentieren nicht nur Dinge, sie verallgemeinern und abstrahieren

²Bildquelle: <https://pbs.twimg.com/media/C7DpoqQV0AAaEXq.jpg> zuletzt besucht am 18.9.2018

auch immer. Das Wort Baum repräsentiert das allgemeine *Konzept* Baum. Es bezeichnet keinen spezifischen Baum bzw. nur dann, wenn zusätzliche Information durch Wahrnehmung oder in Form von Worten vorhanden ist („Der Baum vor meinem Fenster“). Wir brauchen Symbole bzw. Begriffe und damit auch Abstraktion, um unsere Welt beschreiben und verstehen zu können, nicht nur um kommunizieren zu können.

Ein Aspekt von Modellieren, der erfahrungsgemäß zunächst Schwierigkeiten bereitet, besteht darin, dass Modelle scheinbar willkürlich sind: Es gibt kein *richtiges* Modell. Die Kategorien *richtig* und *falsch* sind auf Modelle nicht anwendbar. Es geht vielmehr darum, ob diese *zweckmäßig* sind (oder eben nicht). Es kann also mehrere Modelle desselben Betrachtungsgegenstands geben, wenn diese z. B. unterschiedliche Zwecke verfolgen. Bei den Atommodellen ist das bspw. der Fall.

Ein Modell ist nie eine richtige Beschreibung der Wirklichkeit. Es *ist nicht* die Wirklichkeit. Ein Modell verhält sich zur Wirklichkeit wie eine Karte zum dargestellten Terrain.

Eine Karte *ist nicht* das Terrain. Eine Karte ist *ein Modell* des Terrains. Unterschiedliche Karten erfüllen unterschiedliche Zwecke. Beispielsweise sind eine Straßenkarte und ein Liniennetzplan von Wolfenbüttel beides Modelle derselben Wirklichkeit. Beide sind „falsch“ in dem Sinne, dass sie die Wirklichkeit nicht völlig korrekt wiedergeben. Das ist auch nicht ihr Zweck.

Ein Liniennetzplan (vgl. Abb. 1.2) gibt nicht genau wieder, *wo* ein Bus fährt, sondern nur die Haltestationen und deren Beziehung untereinander. Er gibt keine Informationen über Straßenkreuzungen, Fußgängerzonen usw. Eine Straßenkarte tut dies, aber sie gibt keine Information über Busse und Umstiegsmöglichkeiten. Keine der Karten gibt eine Information über das Geländeprofil. Eine topographische Karte tut dies.

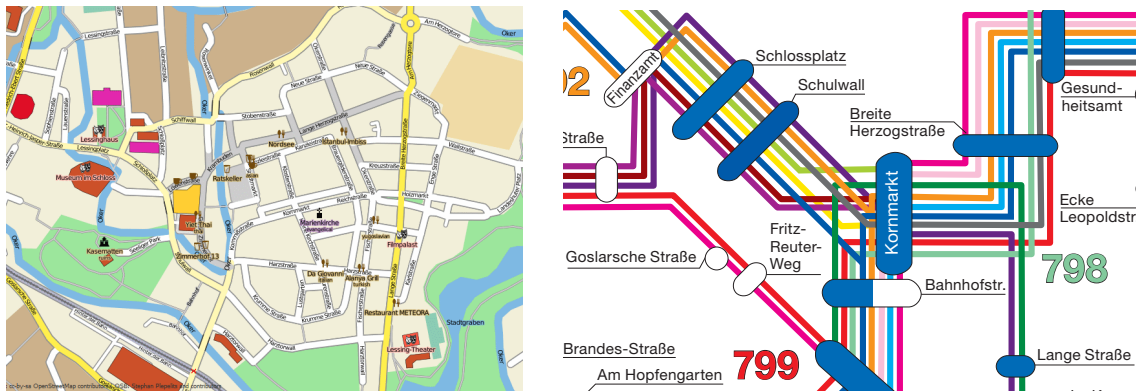


Abbildung 1.2: Straßenkarte ³ (links) und Liniennetzplan ⁴ (rechts) beschreiben unterschiedliche Aspekte desselben Terrains. Insbesondere entsprechen Abstände auf dem Liniennetzplan nicht den Abständen auf der Straßenkarte.

Es kann auch unterschiedliche Modelle mit demselben Zweck geben, die sich aber z. B. darin unterscheiden, zu welchem Grad Details der Wirklichkeit wiedergegeben werden. Letztendlich ist dies auch wieder eine Frage des Zwecks. Ein Autoatlas von Deutschland zeigt an, dass die B79 durch Wolfenbüttel verläuft, ebenso wie ein Stadtplan von Wolfenbüttel. Der Stadtplan enthält jedoch mehr Details. Beide Karten sind auf ihre Weise „korrekt“, aber sie erfüllen unterschiedliche Zwecke.

1.4 Kollektionen

Menschen lieben es Dinge zu sammeln und in der Realität Ansammlungen zu sehen. Wir kategorisieren unsere Umwelt in Tiere und Pflanzen, Zahlen usw. Diese Ansammlungen bzw. Kollektionen

³Bildquelle: Abbildung erzeugt mit openstreetmap.org und openstreetbrowser.org [CC BY-SA 4.0 (<https://creativecommons.org/licenses/by-sa/4.0/>)], via Wikimedia Commons https://commons.wikimedia.org/wiki/File:Wolfenbuettel_Druckversion.png, zuletzt besucht am 12.08.2019

⁴Bildquelle: mit freundlicher Genehmigung der Kraftverkehrsgesellschaft mbH Braunschweig

gibt es nicht. Sie sind Teil des Modells, das wir uns von unserer Umwelt bilden.

Untersucht man die Kollektionen, die bei Modellbildungen entstehen, dann kann man vier Arten erkennen, die durch zwei Aspekte gekennzeichnet sind:

- Die Reihenfolge der Objekte in der Kollektion
- Das mögliche Mehrfachvorkommen der Objekte in der Kollektion

Das führt auf das folgende Schema der vier möglichen Arten von Kollektionen:

	Reihenfolge ist relevant	Reihenfolge ist nicht relevant
Mehrfachvorkommen ist möglich oder relevant	Tupel	
Mehrfachvorkommen ist nicht möglich oder relevant		Menge

Im Rahmen dieses Kurses werden wir zur Modellbildung v. a. die beiden Arten von Kollektionen benötigen, die in diesem Schema mit *Tupel* bzw. *Menge* bezeichnet sind.

Mengen sind Kollektionen, bei denen die Reihenfolge der Objekte in der Kollektion nicht relevant ist und ebenfalls nicht, ob diese mehrfach vorkommen. Mengen werden üblicherweise durch geschweifte Klammern gekennzeichnet. D. h. die Verwendung von geschweiften Klammern dient als visuelles Signal, dass es weder auf Reihenfolge noch auf Vielfachheit ankommt. Die Objekte in einer mengenartigen Kollektion bezeichnen wir als ihre *Elemente*.

Beispiel 1.4: Wenn wir nach den Arten von Geldmünzen fragen, die im Euroraum verwendet werden, ist das beste Modell für die Kollektion der Münzbeträge eine Menge, welche die acht Münzarten umfasst:

$$\{1\text{¢}, 2\text{¢}, 5\text{¢}, 10\text{¢}, 20\text{¢}, 50\text{¢}, 1\text{€}, 2\text{€}\}$$

Dass die Geldbeträge dabei geordnet aufgeführt ist, hilft, die Menge leichter zu erfassen. Tatsächlich hat die Reihenfolge, in der die Beträge aufgeführt sind, keine Bedeutung bei der Beantwortung der Frage nach den Arten der Geldmünzen.

$$\{1\text{€}, 2\text{¢}, 10\text{¢}, 1\text{¢}, 5\text{¢}, 20\text{¢}, 50\text{¢}, 2\text{€}\}$$

ist also dieselbe Menge. ■

Beispiel 1.5: Fragen wir nach den Buchstaben, die im Titel dieses Kapitels vorkommen, ist das beste Modell für die Kollektion der Buchstaben eine Menge: $\{G, r, u, n, d, l, e, g, S, t, k\}$. Natürlich können wir die vorkommenden Buchstaben auch geordnet aufschreiben: $\{G, S, d, e, g, k, l, n, r, t, u\}$. Dennoch hat die Reihenfolge keine Bedeutung. ■

Tupel sind Kollektionen, bei denen die Reihenfolge der Objekte in der Kollektion relevant ist und ebenfalls, ob diese mehrfach vorkommen. Im Rahmen dieses Kurses werden wir Tupel durch eckige Klammern kennzeichnen.⁵ Die Verwendung von eckigen Klammern dient also als visuelles Signal, dass Reihenfolge und Vielfachheit relevant sind. Die Objekte in einer tupelartigen Kollektion werden *Komponenten* genannt.

Beispiel 1.6: Das Tupel $[G, r, u, n, d, l, e, g, e, n, d, e, S, t, r, u, k, t, u, r, e, n]$ ist das beste Modell für die Kollektion der Buchstaben im Titel dieses Kapitels. Dagegen ist das Tupel $[S, t, r, u, k, t, u, r, e, n, l, e, g, e, n, d, e, G, r, e, n, d, u, n]$ nicht geeignet, da es ja auf die Reihenfolge ankommt. ■

Beispiel 1.7: Wir könnten auch danach fragen, welche und wie viele Buchstaben man in einem Setzkasten braucht, um den Titel dieses Kapitels drucken zu können. Das geeignete Modell der Antwortkollektion berücksichtigt die Vielfachheit, aber nicht die Reihenfolge:

$$\langle G, S, d, d, e, e, e, g, k, l, n, n, n, r, r, t, t, u, u, u \rangle.$$

⁵Auch runde Klammern sind üblich, um Tupel darzustellen.

Dabei wurden hier keine eckigen bzw. geschweiften Klammern verwendet, um zu betonen, dass es sich nicht um ein Tupel oder um eine Menge handelt. ■

Tupel werden nach der Anzahl ihrer Komponenten kategorisiert: Tupel mit zwei Komponenten heißen Paare oder auch 2-Tupel, Tupel mit drei Komponenten Tripel bzw. 3-Tupel usw.

Damit zwei Mengen gleich sind, müssen sie die gleichen Elemente haben. Reihenfolge und Vielfachheit spielen dabei natürlich keine Rolle. Daher ist z. B.

$$\begin{aligned}\{1, 2, 3\} &= \{3, 2, 1, 3\} \\ \{1, 2, 3\} &\neq \{1, 2, 4\}.\end{aligned}$$

Damit zwei Tupel gleich sind, müssen sie komponentenweise gleich sein. An den entsprechenden Positionen der beiden Tupel müssen also gleichwertige Objekte stehen. Daher ist z. B.

$$\begin{aligned}[1, 2, 3] &= [1, 1 + 1, 1 + 2] \\ [1, 2, 3] &\neq [3, 2, 1] \\ [1, 2, 3] &\neq [1, 2, 3, 4] \\ [1, 2, 3] &\neq [1, 2, 4].\end{aligned}$$

Tupel und Mengen sind die Atome, die wir häufig verwenden werden, um mit ihnen im Rahmen der Modellbildung „molekülartige“ Strukturen zu beschreiben. Das können z. B. Mengen von Mengen sein, z. B. Potenzmengen in Abschnitt 5.8. Das können Mengen von Tupeln sein, wie bei den Relationen in Kapitel 7. Es könnten auch Tupel von Tupeln sein, wie bei Matrizen, die wir im Rahmen dieses Kurses jedoch nicht benötigen werden.

Diese „Moleküle“ können dann verwendet werden, um komplexere Strukturen zu modellieren. Tupel von Mengen und Relationen sind bspw. geeignet um Automaten zu modellieren.

Beispiel 1.8: Das geeignete Modell für einen Funktionsgraphen ist die Menge von Tupeln. Jedes Tupel hat zwei Komponenten und repräsentiert einen Punkt des Graphen. Hier kommt es auf die Reihenfolge an, denn $[1, 2]$ und $[2, 1]$ repräsentieren unterscheidbare Punkte. Es kommt auch auf die Vielfachheit an, denn $[1, 1]$ repräsentiert einen Punkt, $[1]$ tut das nicht.

Ein Funktionsgraph ist also eine Menge von Tupeln. Hier kommt es nicht auf die Reihenfolge der Tupel in der Menge an. Es ist auch nicht von Belang, wenn ein Punkt als Tupel mehrfach in der Menge aufgeführt ist. ■

Möglicherweise wäre nun ein passender Zeitpunkt, um auf das zurückzublicken, was Sie bisher gelesen haben, oder eine Pause zu machen.

1.5 Funktionen

Funktionen stellen eine weitere wichtige Klasse von Modellen dar. Wir werden uns ihnen im Rahmen dieses Kurses schrittweise nähern. Zunächst werden wir Funktionen einfach als *black boxes* bzw. Eingabe-Ausgabe-Beziehungen betrachten, die eine Eingabe eindeutig in eine Ausgabe verwandeln. Das bedeutet, wann immer wir dieselbe Eingabe in die *black box* stecken, werden wir auch garantiert dieselbe Ausgabe bekommen. Das folgende Blockdiagramm visualisiert diese Vorstellung von Funktionen:

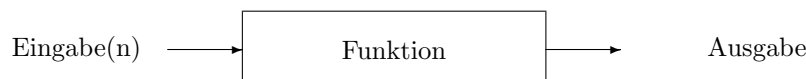


Abbildung 1.3: Funktion als *black box*, die zu jeder Eingabe immer eindeutig eine Ausgabe produziert.

Wir nennen diese Sichtweise auf Funktionen *Funktionenmaschine*. Dabei interessiert uns nicht, *wie* die Funktion das leistet, *was* sie leistet. Was in der *black box* genau vorgeht, interessiert also nicht.

Beispiel 1.9: Das Berechnen der vierten Potenz reeller Zahlen kann als Funktion modelliert werden. Jede eingegebene reelle Zahl wird eindeutig zu einer Ausgabe verrechnet. Ob die Funktion dies dadurch tut, dass sie die Eingabe dreimal mit sich selbst multipliziert oder indem sie in einem ersten Schritt die Zahl mit sich selbst multipliziert und das Resultat wieder mit sich selbst multipliziert, ist dabei sekundär.⁶ ■

Diesen wesentliche Aspekt verdeutlicht die Kuchenmaschine aus der Sendung mit der Maus in Abb. 1.4 besonders gut.

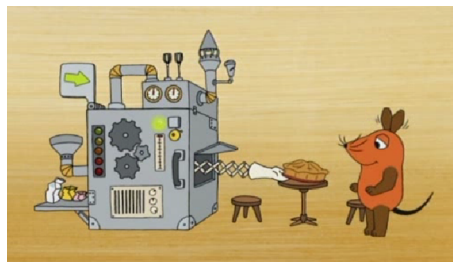


Abbildung 1.4: Kuchenmaschine als Kapsel bzw. *black box*.⁷

Abhängig von der Eingabe (links von der Maschine) produziert sie eindeutig einen bestimmten Kuchen als ihre Ausgabe. Das beschreibt, *was* die Funktion Kuchenmaschine tut. *Wie* das Ganze passiert, ist dabei nicht von Interesse, auch wenn diese Wie-Frage beantwortet werden kann, siehe Abb. 1.5.

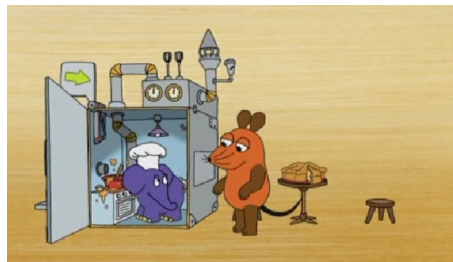


Abbildung 1.5: Die geöffnete Kuchenmaschine verrät, *wie* die Funktion arbeitet.⁸

Funktionen können auch mehr als eine Eingabe haben. Das ist bei der Kuchenmaschine der Fall (300g Mehl, 2 Eier *etc.*). Das folgende Blockdiagramm zeigt dies für die Modulo-Funktion, die den Rest der ganzzahligen Eingabe a nach Division mit der natürlichen Zahl m berechnet. Das Ergebnis wird üblicherweise mit $a \bmod m$ symbolisiert.

⁶In der Welt des Rechnens auf Computern macht es sehr wohl einen Unterschied, *wie* die vierte Potenz berechnet wird: $x \cdot x \cdot x \cdot x$ und $(x \cdot x) \cdot (x \cdot x)$ können zu unterschiedlichen Ergebnissen führen.

⁷Bildquelle: <https://www.wdrmaus.de/filme/mausspots/kuchenmaschine.php5> 00:39; zuletzt besucht am 18.9.2018

⁸Bildquelle: <https://www.wdrmaus.de/filme/mausspots/kuchenmaschine.php5> 00:52; zuletzt besucht am 18.9.2018



Abbildung 1.6: Modulo als Funktionenmaschine mit zwei Eingaben und einer Ausgabe.

Diese Funktionenmaschine berechnet dann bspw. für $a = 42$ und $m = 17$ das Resultat $42 \bmod 17 = 8$.

1.6 Einführung in *setlX*

Ein wesentliches Denkzeug in diesem Kurs ist *setlX*. Das ist eine Programmiersprache, die nahe an den Inhalten dieser Veranstaltung ist und leicht erlaubt, Dinge auf dem Rechner auszuprobieren und Rechenarbeit an den Rechner zu delegieren. Dabei ist es nicht nötig, viel Code zu schreiben. Meistens wird es nur eine Zeile sein.

In diesen Text sind regelmäßig Berechnungen mit *setlX* integriert. Das soll Sie u. a. an die Verwendung als Denkzeug herantführen. Damit das funktioniert, müssen Sie Berechnungen auch tatsächlich in *setlX* durchführen - auch dann, wenn der Text das Berechnungsergebnis bereits nennt. Das wird umso wirksamer sein, je mehr Fragen Sie sich dabei stellen. Sinnvolle Fragen sind u. a. „Was erwarte ich als Ergebnis?“ und „Warum ist das Ergebnis anders als ich erwartet habe? Wo habe ich falsch gedacht?“

Beispiel 1.10: Als einfache Einstiegsübung dividieren wir mittels *setlX* 2 durch 10. Was erwarten Sie als Ergebnis?

```
=> 2/10;
~< Result: ■■■ >~
```

Das Ergebnis ist hier geschwärzt. ■

Als Programmiersprache ist *setlX* eher deklarativ, etwa im Gegensatz zu Java, was eher eine imperative Programmiersprache ist. Bei der imperativen Programmierung beschreibt man im Code, *wie* die Dinge berechnet werden. Bei der deklarativen Programmierung beschreibt man, *was* berechnet wird.

Eine Installationsleitung zu *setlX* zusammen mit einer Einführung finden Sie im elektronischen Teil des Kurses.

1.7 Lernziele

Am Ende eines jeden Kapitels werden Sie eine Aufstellung der wichtigsten Lernziele finden. Die Lernziel-Dimensionen „sozial“ und „persönlich“ sind themenübergreifend und werden nicht in jedem Kapitel erneut wiedergegeben.

Studierende	Kennen	Können	Verstehen
fachlich	☐ benennen die Unterschiede zwischen Mengen und Tupel	☐ werten modulo-Funktion per Hand aus	☐ erkennen bzw. wägen ab, ob ein Objekt geeignet als Menge oder Tupel modelliert werden kann
methodisch	☐ kennen die vorgestellten Sprachkonstrukte von setlx ☐ erläutern die wissenschaftliche Bedeutung von „abstrakt“	☐ schreiben Funktionen in setlx ☐ führen vorgegebene Berechnungen in setlx durch	☐ erkennen, ob etwas als Funktionenmaschine modelliert werden kann
sozial	☐ beschreiben Lernen als einen Prozess mit sozialen Elementen	☐ vertreten im Fachgespräch eigene Position und erwägen Argumente anderer ☐ erläutern Argumente, Rechenschritte usw. nachvollziehbar	☐ erklären zentrale Konzepte
persönlich	☐ sehen (Lern-)Prozess als Ziel einer Übungsaufgabe (statt richtiger Antwort) ☐ benennen Relevanz der Inhalte	☐ erarbeiten neue Inhalte aus Texten ☐ trauen sich Arbeit an neuen Aufgaben zu	☐ übernehmen Verantwortung für eigenes Lernen ☐ hinterfragen eigene Meinung

Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben. Überlegen Sie sich, was sie tun müssen, um die verbleibenden Lernziele zu erreichen.



Wie können Sie nachweisen, dass Sie diese Lernziele tatsächlich erreicht haben?



Wie beantworten Sie nun die Leitfragen?



1.8 Übungsaufgaben

1. Welche Begriffe kennen Sie, die synonym zu Tupel sind, z. B. im Kontext der Programmierung?
2. Welche Art von Kollektion ist jeweils das geeignete Modell?
 - (a) Kollektion von Pokémon-Karten
 - (b) natürliche Zahlen
 - (c) Lottozahlen
 - (d) Telefonnummer

Kapitel 2

Aussagenlogik

Thus computer scientists have devoted much of their talent and energy to developing powerful descriptive formalisms. One might even say that computer science is wrongly so called: Most of it is not the science of computers, but the science of descriptions and descriptive languages.
(Seymour Papert)

2.1 Aufwärmübungen

1. Gehen Sie davon aus, dass die folgenden Sätze falsch sind, und formulieren Sie jeweils den dazu passenden wahren Satz.
 - (a) Das Glas ist voll.
 - (b) Das Buch ist schwarz.
 - (c) Hans mag Wein, aber kein Bier.
 - (d) Denise boxt und engagiert sich im Tierschutz.



2. Wenn-dann-Sätze haben die Struktur „Wenn A , dann B “, z. B.
 - (a) „Wenn es regnet, dann ist die Straße nass.“
 - (b) „Wenn Du zu meiner Party kommst, lernst Du nette Leute kennen.“
 - (c) „Wenn eine Zahl durch 4 teilbar ist, dann ist sie auch durch 2 teilbar.“

Wenn-dann-Sätze verknüpfen zwei Aussagen A und B miteinander (A = „Es regnet“ und B = „Die Straße ist nass“ im ersten Beispiel). Aussagen können wahr oder falsch sein. Können auch Wenn-dann-Sätze als Ganzes wahr oder falsch sein? 

Falls Wenn-dann-Sätze wahr sein können, müssen die miteinander verknüpften Aussagen dazu dann wahr oder falsch sein? 

Falls Wenn-dann-Sätze falsch sein können, müssen die verknüpften Aussagen dazu dann wahr oder falsch sein? 

3. Leitfragen:

- (a) Warum kann man alleine mit Hilfe von Logik nicht feststellen, was wahr bzw. falsch ist?
- (b) Warum ist der folgende Satz wahr? „Wenn der Tag nach Samstag Montag ist, dann ist immer Mittwoch.“
- (c) Warum ist das Und im folgenden Satz kein Und im Sinne der Logik? „Die Trennung von EU und Großbritannien war 2019 ein wichtiges politisches Thema.“

2.2 Gegenstand der Logik

In der Logik geht es um logische Zusammenhänge. Vermutlich halten Sie das für logisch. Allerdings bedeutet „logisch“ in den beiden vorangegangenen Sätzen zwei völlig unterschiedliche Dinge. Wenn wir sagen, dass eine Aussage oder ein Sachverhalt für uns logisch ist, dann meinen wir damit meist, dass die Sache für uns *intuitiv* klar ist. Die Korrektheit der fraglichen Aussage ergibt sich für uns ohne Nachdenken und von selbst. Sie braucht keine Begründung.

Das Problem mit der Intuition ist oft, dass sie zwar in vielen Fällen hilfreich ist, aber im Allgemeinen auf falschen Vorstellungen beruht. Für die meisten Menschen ist intuitiv klar, dass ein Gegenstand sich nur bewegen kann, wenn dauerhaft eine Kraft auf ihn wirkt. Das ist eine in vielen Fällen hilfreiche Vorstellung. Denken Sie z. B. an Fahrradfahren. Diese Vorstellung ist aber falsch.¹

Es ist ein Charakteristikum von Wissenschaft, dass sie uns die Grenzen unserer Intuition aufzeigt. Auch die Wissenschaft der Logik wird dies mehrfach im Rahmen dieses Kurses tun.

Die Wissenschaft der Logik befasst sich mit wahren und falschen Aussagen und deren Verknüpfung. Sie befasst sich insbesondere mit folgerichtigen, also logischen (im Sinne der Logik) Verknüpfungen. Man kann Logik als beschreibenden Formalismus sehen, ganz im Sinne des Zitats von Papert am Anfang dieses Kapitels. Dieser Formalismus erlaubt, Aussagen und deren Verknüpfungen so zu formulieren, dass man damit rechnen kann und dadurch auch ein Computer „logisch handeln“ kann.

In der Informatik kommen wir nicht ohne den beschreibenden Formalismus der Logik aus. Logik wird u. a. benötigt, um Anforderungen eindeutig und widerspruchsfrei zu formulieren. Unsere Alltagssprache ist dazu nicht geeignet, wie wir immer wieder sehen werden. Dies trifft auch auf unsere Intuition zu. Intuitionen sind allerdings nichts Statisches. Sie können sich verändern und weiterentwickeln. Setzen wir uns also zum Ziel, dass Logik für Sie intuitiv wird.

Logik ist gewissermaßen der einfachst denkbare Formalismus. Logik teilt alles in zwei Kategorien auf: Etwas ist so, oder es ist nicht so. Ja oder nein. Wahr oder falsch. Wenn Logik als Werkzeug zur Modellierung verwendet wird, wird soweit abstrahiert, dass nur noch in zwei, sich gegenseitig ausschließenden Kategorien gedacht wird. Beispielsweise wird nicht mehr nach dem Wert einer Zahl gefragt, sondern nur noch, ob diese größer als 42 ist oder nicht.

Wahrheits-
werte und
Symbole für
diese

Die beiden, sich gegenseitig ausschließenden Kategorien der Logik sind *wahr* und *falsch*. In vielen Programmiersprachen werden diese beiden Kategorien mit `true` bzw. `false` bezeichnet, manchmal auch mit 1 oder 0. Im letzten Fall wird eine strukturelle Verwandtschaft zu binären Zahlen betont, auf die wir im Laufe des Kurses mehrfach stoßen werden. Diese Verwandtschaft ermöglicht letztendlich digitales Rechnen und damit die ganze Digitaltechnik.

Wir werden in diesem Kurs „wahr“ mit dem Symbol $\mathbb{1}$ abkürzen und „falsch“ mit $\mathbb{0}$. Dadurch wird einerseits die erwähnte Verwandtschaft mit binären Zahlen leicht sichtbar, andererseits besteht aber auch keine Verwechslungsgefahr. Es ist immer klar, dass $\mathbb{0}$ für „falsch“ steht und 0 für die Ziffer Null.

$\mathbb{0}$ und $\mathbb{1}$ werden auch als *Wahrheitswerte* oder *Boolesche Werte* bezeichnet. Die Menge $\mathbb{B} = \{\mathbb{0}, \mathbb{1}\}$ wird entsprechend die Menge der Wahrheitswerte oder *Boolesche Menge* genannt.

¹Tatsächlich ist es so, dass die Kraft die Ursache der Bewegungsänderung ist, aber nicht die Ursache der Bewegung. Dieses Erkenntnis, das Trägheitsgesetz, ist eine der ersten großen geistigen Leistungen der modernen Wissenschaft.

2.3 Aussagen und Aussageformen

Das, was wahr oder falsch sein kann, wird in der Logik als Aussage bezeichnet.

Definition: Aussage

Eine Aussage ist ein Satz, der entweder wahr oder falsch ist.

Aussage

Beachten Sie dabei einen Punkt, der ganz wichtig ist: Damit ein Satz eine Aussage ist, müssen wir nicht *wissen*, ob er wahr ist oder nicht. So weiß ich nicht, ob Sie, die Sie diese Zeilen lesen, eine Frau sind, aber der Satz „Sie sind eine Frau“ ist entweder wahr oder falsch und somit eine Aussage.

Damit ein Satz eine Aussage ist, muss es noch nicht einmal prinzipiell entscheidbar sein, ob er wahr ist oder nicht. Selbst Sätze, von denen wir noch nicht einmal wissen können oder entscheiden können, ob sie wahr oder falsch sind, können Aussagen sein. So können wir beispielsweise gar nicht wissen, ob der Satz „Genau jetzt in hundert Jahren regnet es in Wolfenbüttel“ wahr ist. Dennoch wird es zum angegebenen Zeitpunkt entweder regnen oder eben nicht regnen. Also ist der Satz entweder wahr oder falsch und damit eine Aussage, auch wenn wir heute nicht wissen können, was der Fall sein wird.

Die mit den großen offenen Fragen der Informatik verbundenen Aussagen, etwa „Eine Aufgabe ist algorithmisch lösbar, wenn es eine Turing-Maschine gibt, die sie löst“ oder „Die Komplexitätsklassen P und NP sind identisch“, fallen ebenfalls in die Kategorie von Aussagen, deren Wahrheitswert wir (noch?) nicht kennen, die aber entweder wahr oder falsch sind.

Festzustellen, ob eine Aussage wahr oder falsch ist, ist übrigens kein Anliegen der Logik und auch nicht ihre Aufgabe. Das ist Aufgabe der einzelnen Wissenschaften. Der Wahrheitswert der Aussage „Auf dem Mars gibt es Wasser“ lässt sich mit Mitteln der Logik nicht bestimmen. Das ist in diesem Fall Aufgabe der Astronomie.

Logik als Teil der Mathematik bzw. Informatik ist wie diese auch Strukturwissenschaft. Es geht also um die Strukturen, die beim Hantieren mit Wahrheitswerten auftreten.

Betrachten wir einige Beispiele. Nutzen Sie die Gelegenheit selbst zu überlegen, ob die jeweiligen Objekte Aussagen sind oder nicht.

Beispiel 2.1: Aussage oder keine Aussage?

- (a) Wolfenbüttel ist eine niedersächsische Stadt.
- (b) Deutschland wird 2022 Fußballweltmeister werden.
- (c) Wie geht es Ihnen?
- (d) Ist heute Sonntag?
- (e) $5 > \sqrt{3}$
- (f) $1+1=3$
- (g) $\sqrt{2}$

 Aussagen sind:

 Keine Aussagen sind:

Satz (a) ist eine Aussage. Wir können sogar den Wahrheitswert benennen (wahr). Satz (b) ist ebenfalls eine Aussage, auch wenn wir den Wahrheitswert nicht kennen. Aber entweder wird Deutschland 2022 Fußballweltmeister oder eben nicht. Der Satz ist also entweder wahr oder falsch und damit eine Aussage. Satz (c) ist keine Aussage, sondern eine Frage. Die Frage, ob „Wie geht es Ihnen?“ wahr oder falsch ist, ist sinnlos. Satz (d) ist ebenfalls eine Frage und daher keine Aussage. Zwar gibt es auf die Frage nur zwei Antworten, die sich wie die beiden Wahrheitswerte gegenseitig ausschließen, diese sind aber eben mögliche Antworten und nicht mögliche Wahrheitswerte. Ausdruck (e) ist eine wahre Aussage, Ausdruck (f) eine falsche. Ausdruck (g) ist keine Aussage, $\sqrt{2}$ ist weder wahr noch falsch.

■

Beispiel 2.2: Aussage oder keine Aussage?

- (a) $x + 1 = 4$
- (b) $5x > 3x$
- (c) Dieser Student ist in Wolfenbüttel geboren.

Überlegen Sie sich auch hier zunächst wieder selbst, ob Aussagen vorliegen oder nicht:

 Aussagen sind:

 Keine Aussagen sind:

Ist Ihnen die Klassifikation leicht gefallen? Alle drei Objekte können wahr oder falsch sein. Aber es sind keine Aussagen! Aussagen sind Sätze, die *entweder* wahr *oder* falsch sind. In den drei obigen Sätzen ist aber beides möglich! Beispielsweise ist (a) für $x = 3$ wahr, aber für $x = 0$ falsch. Der Wahrheitswert von (a) hängt also vom Wert von x ab. Satz (a) ist also nicht *entweder* wahr *oder* falsch und damit keine Aussage. Ebenso verhält es sich bei (b). Auch bei (c) hängt der Wahrheitswert des Satzes davon ab, wer „dieser Student“ ist. ■

Dieses Beispiel zeigt, dass es Sätze gibt, die zwar „aussagenartig“ sind, aber keine Aussagen. Ihnen gemeinsam ist, dass sie Variablen enthalten. Erst wenn die Variablen mit einem Wert belegt werden, werden die Sätze zu Aussagen. In den ersten beiden Fällen ist x die Variable, im dritten Satz ist es „dieser Student“.

Sätze, die erst dadurch zu Aussagen werden, indem Variablen konkret mit Werten belegt werden, werden als Aussageformen bezeichnet. Wir werden uns im Kapitel 6 eingehend mit ihnen und ihrer Logik, der Prädikatenlogik, befassen. Bis dahin beschäftigen wir uns fast ausschließlich mit der Logik von Aussagen, der *Aussagenlogik*. Dennoch ist es schon jetzt wichtig, dass Sie sicher zwischen Aussage und „ihrer nächsten Verwandten“, der Aussageform, unterscheiden können. Wir sind also an einer weiteren Stelle, an der Sie lernen müssen, zwei nahe beieinander liegende Begriffe auseinanderzuhalten.

Definieren wir zu diesem Zweck also erst mal den Begriff:

Definition 2.3: Aussageform

Aussageformen sind Sätze oder Ausdrücke, die variable Elemente enthalten und durch Belegung dieser variablen Elemente mit konkreten Werten zu einer Aussage werden.

Die folgenden Beispiele sollen Ihnen helfen, sich ein klareres Bild vom Unterschied zwischen Aussage und Aussageform zu entwickeln. Dazu ist es wichtig, dass Sie zunächst selbst überlegen, ob eine Aussage, eine Aussageform oder keines von beiden vorliegt, und erst dann weiter lesen. Notieren Sie jeweils Ihre Antwort und unterstreichen Sie bei Aussageformen die variablen Elemente.

- $x^2 - 1 = 0$ 
- „ $x^2 - 1 = 0$ ist gleichbedeutend damit, dass $x = -1$ oder $x = 1$ gilt.“ 
- „ $x^2 - 1 = 0$ hat die Lösungen $x = -1$ und $x = 1$.“ 
- $x^2 - 1$ 
- „Wenn eine Zahl durch 4 teilbar ist, ist sie auch durch 2 teilbar.“ 
- „Wenn eine Zahl durch p^2 teilbar ist, ist sie auch durch p teilbar.“ 

Beispiel 2.4: $x^2 - 1 = 0$ ist eine Aussageform mit dem variablen Element x . Belegen wir x mit dem Wert -1 erhalten wir die Aussage $(-1)^2 - 1 = 0$ (mit dem Wahrheitswert $\mathbb{1}$). Belegen wir x mit 0 , erhalten wir die Aussage $0^2 - 1 = 0$ (mit dem Wahrheitswert $\mathbb{0}$). ■

Beispiel 2.5: „ $x^2 - 1 = 0$ ist gleichbedeutend damit, dass $x = -1$ oder $x = 1$ gilt.“ Dies ist ebenfalls eine Aussageform mit der Variablen x . Belegen wir x mit dem Wert -1 erhalten wir die Aussage „ $(-1)^2 - 1 = 0$ ist gleichbedeutend damit, dass $-1 = -1$ oder $-1 = 1$ gilt.“ Ausgewertet führt dies

zu „1 ist gleichbedeutend damit, dass 1 oder 0 gilt.“ Das hat den Wahrheitswert 1. (Sollte Ihnen das nicht klar sein: Wir werden dies im Abschnitt 2.4.1 klären.)

Belegen wir x mit dem Wert 0 erhalten wir die Aussage „ $0^2 - 1 = 0$ ist gleichbedeutend damit, dass $0 = -1$ oder $0 = 1$ gilt.“ Ausgewertet führt dies zu „0 ist gleichbedeutend damit, dass 0 oder 0 gilt.“ Der Wahrheitswert ist ebenfalls 1. ■

Beispiel 2.6: „ $x^2 - 1 = 0$ hat die Lösungen $x = -1$ und $x = 1$.“ Dies ist keine Aussageform.

Wenn Sie anderer Meinung sind, dann vermutlich deshalb, weil Sie x als variables Element identifiziert haben. Dem spricht nichts entgegen. Nur besagt die Definition von Aussageformen nicht, dass diese Variablen enthalten, sondern dass sie durch die Belegung dieser Variablen mit konkreten Werten zu Aussagen werden. Belegen wir bspw. x mit dem Wert 0 erhalten wir effektiv den Satz „ $-1 = 0$ hat die Lösungen $0 = -1$ und $0 = 1$.“ Ist dieser Satz wahr oder falsch? Weder noch! Er ist bedeutungslos („semantisch falsch“ in der Sprache der Informatik, aber eben nicht „logisch falsch“), denn im Kontext von $-1 = 0$ kann man nicht von einer Lösung sprechen. Es gibt hier nichts zu lösen.

Es handelt sich hier vielmehr um eine Aussage (mit dem Wahrheitswert 1). ■

Beispiel 2.7: $x^2 - 1$ ist ebenfalls keine Aussageform, wie man wieder durch Belegen sieht. Für $x = 0$ erhalten wir -1 , was keinen Wahrheitswert hat. Es ist einfach nur eine Zahl.

$x^2 - 1$ ist auch keine Aussage. Es ist weder wahr noch falsch. ■

Beispiel 2.8: „Wenn eine Zahl durch 4 teilbar ist, ist sie auch durch 2 teilbar.“ Hier liegt eine Aussageform vor. Das variable Element ist „eine Zahl“. Belegen wir diese Zahl bspw. mit dem Wert 8 erhalten wir die wahre Aussage „Wenn 8 durch 4 teilbar ist, ist 8 auch durch 2 teilbar.“ Belegen wir die Zahl mit dem Wert 6 erhalten wir die Aussage „Wenn 6 durch 4 teilbar ist, ist 6 durch 2 teilbar.“ Sie hat den Wahrheitswert 1, wie wir in Abschnitt 2.4.3 sehen werden.

Noch deutlicher sichtbar wird der Charakter der Aussageform, wenn wir das variable Element durch ein Symbol ersetzen, z. B. z : „Wenn z durch 4 teilbar ist, ist z auch durch 2 teilbar.“ ■

Beispiel 2.9: „Wenn eine Zahl durch p^2 teilbar ist, ist sie auch durch p teilbar.“ Hier liegt wieder eine Aussageform vor, allerdings mit zwei variablen Elementen: „eine Zahl“ und p . Erst wenn beide Variablen mit konkreten Werten belegt werden, wird daraus eine Aussage, z. B. durch Belegung von „eine Zahl“ mit 27 und mit $p = 3$: „Wenn 27 durch 3^2 teilbar ist, ist 27 auch durch 3 teilbar.“ ■

In diesen Beispielen ging es immer um die Frage, ob ein Satz oder ein Ausdruck eine Aussageform ist. Eine bessere Formulierung der Fragestellung lautet: Lässt sich der Satz oder Ausdruck mit Hilfe einer Aussageform *modellieren*? Die Verwendung des Verbs „ist“ bzw. „sein“ ist hier eigentlich nicht angebracht, denn Aussageformen gibt es nicht. Sie *sind* kein Element der Realität. Sie sind reine Konstrukte des menschlichen Geistes.

Das mag Ihnen philosophisch oder haarspalterisch vorkommen. Die vielfältigen Bedeutungen von „sein“ und dabei insbesondere die Bedeutungen „hat die Eigenschaft“ und „kann beschrieben werden als“ sind unterscheidbar und unterscheidungswürdig. Geschieht dies nicht, ist dies häufig Ursache von Fehlverständnissen oder Verständnisschwierigkeiten. Viele bedeutende Denkerinnen und Denker haben auf diesen Punkt hingewiesen.

Man kann Aussageformen auch so betrachten: Sie stellen viele Aussagen gleichzeitig dar, nämlich genau so viele Aussagen wie es Möglichkeiten gibt, die Variablen der Aussageformen mit Werten zu belegen.

Beispiel 2.10: Der Satz „Die Dezimalziffer ist größer als fünf“ kann als Aussageform mit der Variablen „Dezimalziffer“ beschrieben werden. Verwenden wir das Symbol z für diese Variable, können wir auch $z > 5$ schreiben.

Die eine Aussageform $z > 5$ steht zugleich für zehn Aussagen, nämlich je eine für jeden möglichen Wert von z :

$$0 > 5, \quad 1 > 5, \quad 2 > 5, \quad 3 > 5, \quad 4 > 5, \quad 5 > 5, \quad 6 > 5, \quad 7 > 5, \quad 8 > 5, \quad 9 > 5$$

Hier sind genau vier der zehn möglichen Aussagen, für die $z > 5$ steht, wahr. ■

Beispiel 2.11: Auch der Satz „Die Dezimalziffer ist größer als neun“ kann als Aussageform mit der Variablen „Dezimalziffer“ beschrieben werden. Verwenden wir wieder das Symbol z für diese Variable, können wir auch $z > 9$ schreiben.

Auch die eine Aussageform $z > 9$ steht zugleich für zehn Aussagen, je eine für jeden möglichen Wert von z . Alle diese Aussagen haben den Aussagewert falsch. ■

Bis auf weiteres werden wir uns zunächst vor allem mit Aussagen beschäftigen. Vieles von dem, was wir erarbeiten, lässt sich sofort auf Aussageformen übertragen. Außerdem werden Sie in Kapitel 6 sehen, dass Aussagen als spezielle Aussageformen betrachtet werden können. Bis dahin werden wir streng zwischen Aussagen und Aussageformen unterscheiden.

Ein Standard-„Denkzeug“ der Informatik ist die Funktion. Lassen Sie uns Aussagen mit diesem Denkzeug betrachten: Aussagen sind Sätze, die entweder wahr oder falsch sind. Das können wir uns so vorstellen, dass gewisse Sätze durch eine Funktion `hat_den_Wahrheitswert` als wahr oder falsch gekennzeichnet werden, vgl. Abb. 2.1. Gültige Eingaben für diese Funktion sind Aussagen. Alles, was keine Aussage ist, kann von dieser Funktion nicht als wahr oder falsch klassifiziert werden.

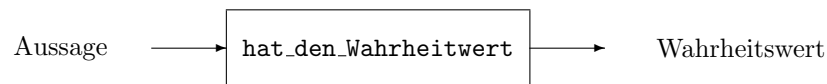


Abbildung 2.1: Aussagen sind die Argumente einer Funktion, die den Aussagen ihren Wahrheitswert zuweist.

Programmiersprachen stellen für sehr einfache Aussagen die Funktion `hat_den_Wahrheitswert` bereit (auch wenn sie nicht so genannt wird). Meistens sind dies Aussagen, die Vergleiche (z. B. von Zahlen) beinhalten. Schauen wir uns das in *setlX* an:

```

=> 2>1;
~< Result: true >~
=> 2>4;
~< Result: false >~
=> 'a'=='b';
~< Result: false >~
  
```

Die Funktion `hat_den_Wahrheitswert` benötigt als Eingabe unbedingt eine Aussage. Das bedeutet, dass sie z. B. Aussageformen nicht akzeptieren darf, weil sie ja keine Aussagen sind. Entsprechend kann *setlX* mit Aussageformen wie z. B. $x > 1$ (zunächst) auch nichts anfangen:

```

=> x>1;
Error in "x > 1":
Error in substitute comparison "1 < x":
Right-hand-side of '1 < om' is not a number.
  
```

Zunächst muss die Variable in der Aussageform mit einem Wert belegt werden und so zu einer Aussage werden, damit überhaupt die Chance besteht, dass ein Computer den Wahrheitswert bestimmen kann.

Auch Aussageformen können als Funktionen gedacht werden. Sie bilden aber nicht Aussagen auf Wahrheitswerte ab, sondern Variablen auf Aussagen, siehe Abb. 2.2. Solche Funktionen können natürlich programmiert werden. Die folgende Funktion für die Aussageform $x > 1$ bildet die Variable x auf eine Aussage ab, deren Wahrheitswert dann sofort von *setlX* bestimmt werden kann:

```

=> ist_groesser_als_1:=procedure(x){return x>1;};
~< Result: procedure(x) { return x > 1; } >~
=> ist_groesser_als_1(0);
~< Result: false >~
  
```



Abbildung 2.2: Aussageformen können als Funktionen gedacht werden, die Variablen in Aussagen umwandeln.

Möglicherweise wäre nun ein passender Zeitpunkt, um auf das zurückzublicken, was Sie bisher gelesen haben, oder eine Pause zu machen.

2.4 Logische Operatoren

Aussagen sind häufig zusammengesetzt, d. h. sie lassen sich in Teilaussagen zerlegen.

Beispiel 2.12: Die folgenden Aussagen sind zusammengesetzt:

- „Es regnet, aber die Sonne scheint.“
- „Entweder regnet es oder es scheint die Sonne.“
- „Wenn es regnet, scheint die Sonne nicht.“
- „Es regnet, weil die Sonne scheint.“

Alle lassen sich in zwei Teilaussagen zerlegen, nämlich „es regnet“ und „die Sonne scheint“. ■

2.4.1 Negation, Konjunktion und Disjunktion

Der Gegenstand der Logik ist nicht die Wahrheit von Aussagen, sondern was passiert, wenn Aussagen miteinander verknüpft werden. Die bedeutendsten logischen Verknüpfungen sind die Negation (auch Verneinung genannt, Symbol $\neg$, gesprochen „nicht“), die Konjunktion (Symbol $\wedge$, gesprochen „und“) und die Disjunktion (Symbol $\vee$, gesprochen „oder auch“).

Definition 2.13: Die *Konjunktion* $A \wedge B$ zweier Aussagen A und B ist nur dann wahr, wenn beide Aussagen wahr sind. Die *Disjunktion* $A \vee B$ zweier Aussagen ist nur dann falsch, wenn beide Aussagen falsch sind. Die *Negation* $\neg A$ einer Aussage A ist wahr, wenn diese falsch ist, und umgekehrt.

Die vermutlich einprägsamste Art diese Definition darzustellen ist mittels *Wahrheitstabellen*. Sie zeigen die Wahrheitswerte für jeden möglichen Wahrheitswert der beiden verknüpften Teilaussagen:

A	$\neg A$	A	B	$A \wedge B$	A	B	$A \vee B$
0	1	0	0	0	0	0	0
0	1	0	1	0	0	1	1
1	0	1	0	0	1	0	1
		1	1	1	1	1	1

Überzeugen Sie sich zunächst davon, dass der Wortlaut der obigen Definitionen jeweils mit der Tabelle konsistent ist.

Die drei Wahrheitstabellen zeigen einen wesentlichen, strukturellen Unterschied zwischen der Negation einerseits und der Konjunktion bzw. Disjunktion andererseits auf: Die Negation ist ein unärer Operator, sie wirkt auf einen einzigen Operanden. Konjunktion und Disjunktion sind binäre Operatoren, sie verknüpfen zwei Operanden.

unäre und
binäre Ope-
rationen

Beispiel 2.14: Schauen wir uns zur Sicherheit ein einfaches Beispiel an, ausgehend von den beiden Aussagen

$$\begin{aligned} A &= (\pi > 3) \\ B &= (-1 \geq 0). \end{aligned}$$

Wir sind hier in der komfortablen Situation, dass wir die Wahrheitswerte kennen, nämlich $A = \mathbb{1}$ und $B = \mathbb{0}$. Daher ist

$$\begin{aligned} \neg A &= \mathbb{0} \\ \neg B &= \mathbb{1}. \end{aligned}$$

Wenn wir nicht die abkürzenden Symbole A bzw. B für die beiden Aussagen verwenden, können wir statt $\neg A$ auch $\neg(\pi > 3)$ bzw. $\pi \leq 3$ schreiben.

$$\neg(\pi > 3) = \mathbb{0} \text{ bzw. } (\pi \leq 3) = \mathbb{0}$$

Entsprechend für B , also

$$\neg(-1 \geq 0) = \mathbb{1} \text{ bzw. } (-1 < 0) = \mathbb{1}.$$

Sie sollen daran auch sehen, dass es möglich ist, die Negation einer Aussage zu bilden, ohne den Wahrheitswert der Aussage berücksichtigen oder kennen zu müssen. Es gilt $\neg(x > 3) = (x \leq 3)$, egal ob wir den Wahrheitswert von $x > 3$ kennen.

Nun zu Konjunktion und Disjunktion:

$$\begin{aligned} (A \wedge B) &= (\pi > 3 \wedge -1 \geq 0) = \mathbb{0}, & \text{denn es sind nicht beide Teilaussagen wahr} \\ (A \vee B) &= (\pi > 3 \vee -1 \geq 0) = \mathbb{1}, & \text{denn es sind nicht beide Teilaussagen falsch} \\ (A \vee \neg B) &= (\pi > 3 \vee -1 < 0) = \mathbb{1}, & \text{denn es sind nicht beide Teilaussagen falsch} \\ (A \wedge A) &= (\pi > 3 \wedge \pi > 3) = \mathbb{1}, & \text{denn es sind beide Teilaussagen wahr} \\ (A \vee A) &= (\pi > 3 \vee \pi > 3) = \mathbb{1}, & \text{denn es sind nicht beide Teilaussagen falsch} \end{aligned}$$

und so weiter. ■

Vielleicht sind Sie wegen der vielen Klammern, wie z. B. in

$$(A \vee \neg B) = (\pi > 3 \vee -1 < 0) = \mathbb{1}$$

verwirrt und fragen sich, warum diese nicht einfach weggelassen werden können, also Folgendes geschrieben werden kann:

$$A \vee \neg B = \pi > 3 \vee -1 < 0 = \mathbb{1}.$$

Das ist leider nicht möglich, denn dies würde

$$A \vee (\neg B = \pi > 3) \vee (-1 < 0 = \mathbb{1})$$

bedeuten. Wir werden dies in Abschnitt 2.5 eingehender thematisieren.

komplexe
Ausdrücke

Konjunktion und Disjunktion verbinden als binäre Operatoren zwei Aussagen miteinander. Diese Aussagen können selbst wieder Negationen, Konjunktionen oder Disjunktionen von Aussagen sein. Mit anderen Worten: logische Ausdrücke können beliebig komplex sein, wie z. B. $A \vee \neg B \vee (A \wedge \neg C)$. Egal wie komplex die Ausdrücke sind, die Definition der Operationen erlaubt uns immer den Wahrheitswert der Gesamtaussage anzugeben, wenn wir die Wahrheitswerte der Teilaussagen kennen.

Beispiel 2.15: Schauen wir uns die drei etwas komplexeren Ausdrücke

- (a) $\neg A \vee B$
- (b) $(A \vee \neg B) \wedge (\neg A \vee B)$
- (c) $A \vee \neg B \vee (A \wedge \neg C)$

an. Um die Wahrheitswerte von $\neg A \vee B$ zu bestimmen, muss zunächst als Zwischenschritt $\neg A$ gebildet werden und dann davon die Disjunktion mit B . In der nachfolgenden Wahrheitstabelle (links) ist das gemacht. Dabei wurde für den Zwischenschritt eine eigene Spalte eingefügt.

Entsprechend braucht es bei $(A \vee \neg B) \wedge (\neg A \vee B)$ mehrere Zwischenschritte: $\neg B$ muss gebildet werden und daraus $A \vee \neg B$. Das nächste benötigte Zwischenergebnis, $\neg A \vee B$, brauchen wir hier nicht mehr schrittweise zu berechnen, weil wir es aus Aufgabenteil (a) übernehmen können. Im letzten Schritt muss die Konjunktion der beiden Teilaussagen $A \vee \neg B$ und $\neg A \vee B$ gebildet werden. Die nachfolgende Wahrheitstabelle weist wieder jedes Zwischenschrittergebnis als eigene Spalte aus:

A	B	$\neg A$	$\neg A \vee B$	A	B	$\neg B$	$A \vee \neg B$	$\neg A \vee B$	$(A \vee \neg B) \wedge (\neg A \vee B)$
0	0	1	1	0	0	1	1	1	1
0	1	1	1	0	1	0	0	1	0
1	0	0	0	1	0	1	1	0	0
1	1	0	1	1	1	0	1	1	1

Aufgabenteil (c) ist komplexer, weil drei Wahrheitswerte A, B, C verknüpft werden. Das hat zur Konsequenz, dass die Wahrheitstabelle mehr Zeilen braucht. Wieder bekommt jeder Zwischenschritt eine eigene Spalte, wobei das Ergebnis für den Zwischenschritt $A \vee \neg B$ aus Teil (b) übernommen wurde.

Zeile	A	B	C	$A \vee \neg B$	$\neg C$	$A \wedge \neg C$	$A \vee \neg B \vee (A \wedge \neg C)$
0	0	0	0	1	1	0	1
1	0	0	1	1	0	0	1
2	0	1	0	0	1	0	0
3	0	1	1	0	0	0	0
4	1	0	0	1	1	1	1
5	1	0	1	1	0	0	1
6	1	1	0	1	1	1	1
7	1	1	1	1	0	0	1

Was fällt Ihnen auf, wenn Sie die Ergebnisspalte mit der fünften Spalte für $A \vee \neg B$ vergleichen?



Aus dem vorangegangenen Beispiel können wir zwei Erkenntnisse ziehen: Komplexe Ausdrücke können gelegentlich einfacher geschrieben werden. Aus der letzten Wahrheitstabelle ergibt sich beispielsweise, dass

$$(A \vee \neg B \vee (A \wedge \neg C)) = (A \vee \neg B), \quad (2.1)$$

denn die beiden entsprechenden Spalten in der Wahrheitstabelle sind identisch. Das hat u. a. Konsequenzen für die Programmierung. Der Befehl „Mache etwas, wenn A alleine wahr ist oder B alleine falsch ist oder A wahr und gleichzeitig C falsch ist“ wird in den meisten Programmiersprachen in der Form

```
if( A || !B || (A && !C) ) then ...
```

geschrieben. Die Logik sagt uns, dass dies einfacher in der Form

```
if( A || !B ) then ...
```

geschrieben werden kann.

Die zweite Erkenntnis betrifft die Anzahl der Zeilen, die eine Wahrheitstabelle neben der Kopfzeile benötigt. Besteht ein logischer Ausdruck aus nur einer Variablen, braucht die Wahrheitstabelle zum Auswerten dieses Ausdrucks zwei Zeilen, denn die Variable kann die beiden Werte 0 oder 1 annehmen. Die Wahrheitstabelle für die Negation ist ein Beispiel für diesen Fall.

Umfasst ein logischer Ausdruck zwei Variablen wie z. B. in den Teilaufgaben (a) und (b) des obigen Beispiels, so gibt es zwei mögliche Werte für die erste Variable und für jeden Wert der ersten Variable wieder zwei mögliche Werte der zweiten Variable. (Wenn die erste Variable den Wert 0 hat,

Vereinfachen
von Aus-
drücken

Anzahl der
Zeilen in
Wahrheits-
tabelle

kann die zweite Variable die Werte 0 oder 1 haben.) Insgesamt gibt es also $2 \cdot 2 = 4$ Möglichkeiten. Die Wahrheitstabelle benötigt vier Zeilen.

Umfasst ein logischer Ausdruck drei Variablen wie z. B. in Teilaufgabe (c) des obigen Beispiels, so gibt für jeden der vier Fälle für die ersten beiden Variablen jeweils zwei Werte für die dritte Variable, insgesamt also $4 \cdot 2 = 8$ Möglichkeiten. Die Wahrheitstabellen benötigt acht Zeilen, wie dies auch in Teilaufgabe (c) der Fall war.

Treibt man diese kombinatorische Überlegung weiter (wir werden dies in Kapitel 10 tun), kommt man zur Erkenntnis, dass bei n Variablen 2^n Zeilen benötigt werden. Die Anzahl der benötigten Zeilen wächst also exponentiell und damit sehr schnell an.

systematisches
Belegen der
Variablen

Bei der Wahl der Wahrheitswerte für die einzelnen Variablen sollte man natürlich systematisch vorgehen. Eine Möglichkeit dies zu tun, ist die Kette der Wahrheitswerte als Binärzahl zu lesen und entsprechen hoch zu zählen. In der Tabelle aus Teilaufgabe (c) ist an der Zeilennummer zu sehen, dass die Belegung der drei Wahrheitswerte als Bitfolge gelesen immer genau der Zeilennummer entspricht. Beispielsweise haben in Zeile Nr. 6 = 110_2 die drei Variablen die Folge der Wahrheitswerte 110 .

Drei logische Ausdrücke, die häufig vorkommen, sind die Negation der Negation (also $\neg(\neg A)$), die Negation der Konjunktion (also $\neg(A \wedge B)$) und die Negation der Disjunktion (also $\neg(A \vee B)$). Hier sind die Wahrheitstabellen für die ersten beiden Fälle:

A	$\neg A$	$\neg(\neg A)$	A	B	$A \wedge B$	$\neg(A \wedge B)$	$\neg A$	$\neg B$	$\neg A \vee \neg B$
0	1	0	0	0	0	1	1	1	1
0	1	0	0	1	0	1	1	0	1
1	0	1	1	0	0	1	0	1	1
1	0	1	1	1	1	0	0	0	0

Aus der ersten Wahrheitstabelle ergibt sich $\neg(\neg A) = A$, denn die Ergebnisspalte ist mit der ersten Spalte identisch. Zwei aufeinander folgende Negationen neutralisieren sich also.

Die Ergebnisspalte der zweiten Wahrheitstabelle besagt, dass $\neg(A \wedge B)$ nur dann falsch ist, wenn beide Aussagen A und B wahr sind, d. h. wenn $\neg A$ und $\neg B$ jeweils falsch sind. Das ist der Wortlaut der Definition der Disjunktion („nur dann falsch, wenn“) angewandt auf die Aussagen $\neg A$ und $\neg B$, also $\neg A \vee \neg B$. Daher gilt

$$\neg(A \wedge B) = (\neg A \vee \neg B).$$

Die Negation der Konjunktion ist also die Disjunktion der negierten Aussagen. Sie können dies auch mit Hilfe der letzten drei Spalten der rechten Wahrheitstabelle sehen.

Bleibt noch die Negation der Disjunktion $\neg(A \vee B)$. Finden Sie bitte selbst heraus, in welcher Form diese noch geschrieben werden kann.



Wie gesagt, kommen diese Fälle häufig vor. Daher halten wir fest:

Negation von Negation, Konjunktion und Disjunktion

$$\neg(\neg A) = A \quad (2.2)$$

$$\neg(A \wedge B) = (\neg A \vee \neg B) \quad (2.3)$$

$$\neg(A \vee B) = (\neg A \wedge \neg B) \quad (2.4)$$

Weil die Symbole $\neg$, $\vee$ und $\wedge$ nicht auf der Tastatur verfügbar sind, verwenden Programmiersprachen in der Regel andere Symbole. Tabelle 2.1 auf S. 26 nennt Ihnen die entsprechenden Symbole, die *setlX* verwendet. Außerdem sind weitere, übliche Symbole aufgelistet, die für die Notation der logischen Operatoren auf Papier verwendet werden.

Beispiel 2.16: Wir nutzen *setlX*, um (2.4) zu verifizieren:

```
=> A:=false; B:=false; !(A||B)==(!A&&!B);
~< Result: true >~
=> A:=false; B:=true; !(A||B)==(!A&&!B);
~< Result: true >~
=> A:=true; B:=false; !(A||B)==(!A&&!B);
~< Result: true >~
=> A:=true; B:=true; !(A||B)==(!A&&!B);
~< Result: true >~
```

Beziehung (2.4) ist also tatsächlich für jede mögliche Belegung von A und B wahr. ■

2.4.2 Modellieren von Alltagssprache - Teil 1

Mit den bisherigen, wenigen Konzepten der Logik können wir Alltagssprache modellieren. Schauen Sie sich dazu die folgende Tabelle an:

Nr.	Gesamtaussage	Teilaussage A	Teilaussage B	Modell
1	Anna studiert Informatik und ihr Freund Cedric studiert Biologie.	Anna studiert Informatik.	Cedric studiert Biologie.	$A \wedge B$
2	Anna wohnt in Wolfenbüttel, aber Bertram wohnt in Braunschweig.	Anna wohnt in Wolfenbüttel.	Bertram wohnt in Braunschweig.	$A \wedge B$
3	Anna wohnt in Wolfenbüttel und studiert Informatik.	Anna wohnt in Wolfenbüttel.	Anna studiert Informatik.	$A \wedge B$
4	Bertram studiert Mathematik und Sport.	Bertram studiert Mathematik.	Bertram studiert Sport.	$A \wedge B$
5	Bertram studiert nicht Informatik.	Bertram studiert Informatik.		$\neg A$
6	Bertram studiert nicht Informatik und Cedric studiert nicht Romanistik.	Bertram studiert Informatik.	Cedric studiert Romanistik.	$\neg A \wedge \neg B$
7	Es regnet nicht oder es schneit nicht.	Es regnet.	Es schneit.	$\neg A \vee \neg B$

Wie Sie sehen, können Aussagen häufig in Teilaussagen zerlegt werden (hier die Nummern 1-4 und 6, 7). Aussagen können als Negationen anderer Aussagen aufgefasst werden (Nr. 5) oder aus negierten Teilaussagen zusammengesetzt sein (Nr. 6, 7).

Die Modellierung geht auch hier mit einem Abstraktionsprozess einher. Besonders deutlich sichtbar ist dies am Endprodukt, eben den Modellen. Es geht in den Modellen nicht mehr um Personen und was diese tun, sondern nur noch darum, ob und wie die Ausgangssätze als logische Verknüpfung von Teilaussagen aufgefasst werden können.

Auch bei der Formulierung der Teilaussagen hat teilweise schon eine Abstraktion stattgefunden, so bspw. im Satz Nr. 1. Dort wurde in der Formulierung der Teilaussage B die Beziehung zwischen Anna und Cedric wegabstrahiert, denn für die logische Struktur ist dies unbedeutend. Ohne diesen Abstraktionsschritt wäre die Aussage „Ihr Freund Cedric studiert Biologie“ entstanden. Dies Aussage kann aber nicht alleine stehen. Sie ist nur im Kontext mit der Teilaussage A in Gänze verständlich.

Die Negation von Satz Nr. 5 (Modell $\neg A$) ist $\neg(\neg A) = A$, in Alltagssprache also „Bertram studiert Informatik.“ Die Negation von Satz Nr. 7 (Modell $\neg A \vee \neg B$) ist formal und aufgrund von Beziehung (2.4)

$$\begin{aligned}\neg(\neg A \vee \neg B) &= (\neg(\neg A) \wedge \neg(\neg B)) \\ &= (A \wedge B).\end{aligned}$$

Der negierte Satz von „Es regnet nicht oder es schneit nicht“ lautet daher „Es regnet und es schneit.“ All das dürfte Ihrem Sprachempfinden entsprechen.

Die Modellierung funktioniert übrigens nur deshalb, weil „und“, „oder“ bzw. „nicht“ in den obigen Beispielsätzen der Konjunktion, Disjunktion bzw. Negation in der Logik entsprechen. Vermutlich ist Ihnen das intuitiv klar.

Nur ist es hier so, wie es meistens mit der Intuition ist. Man kommt ziemlich weit mit ihr, im Allgemeinen führt sie aber in die Irre. Dass die Angelegenheit diffiziler ist als sie auf den ersten Blick erscheint, können Sie an Satz Nr. 4 sehen. Dort verbindet „und“ keine zwei Aussagen, sondern zwei Objekte (im grammatikalischen Sinn). Es kann daher nicht als logischer Operator aufgefasst werden und somit schon gar nicht als Konjunktion. Nun ist es allerdings so, dass dieser Satz gleichbedeutend ist mit „Bertram studiert Mathematik und Bertram studiert Sport.“ In diesem neuen Satz verbindet „und“ tatsächlich zwei Teilaussagen und kann daher als Konjunktion modelliert werden.

Anders ist die Situation beim Satz „Mathematik und Informatik haben vieles gemeinsam.“ Dieser Satz lässt sich nicht einfach umschreiben in zwei Teilsätze, die mit „und“ verbunden sind.

Nicht jedes „und“ ist also eine Konjunktion und auch nicht jede Konjunktion muss durch ein „und“ ausgedrückt sein. Satz Nr. 2 ist ein Beispiel dafür. Dort drückt „aber“ die Konjunktion aus – gleichbedeutend mit „und im Gegensatz dazu“. Dieser Gegensatz geht hier allerdings durch den Abstraktionsprozess beim Modellieren verloren. Die Logik kümmert sich nicht um solche feineren Nuancen der Alltagssprache. Ihr geht es nur um wahr und falsch. Wir werden uns in den Kapiteln 3 und 6 noch eingehender mit der Beziehung von Logik und Alltagssprache befassen.

2.4.3 Subjunktion, Bijunktion und Alternation

Drei weitere binäre Operatoren werden in der Logik häufig benötigt. Ihre Wahrheitstabellen sind:

A	B	$A \rightarrow B$	A	B	$A \leftrightarrow B$	A	B	$A \nleftrightarrow B$
0	0	1	0	0	1	0	0	0
0	1	1	0	1	0	0	1	1
1	0	0	1	0	0	1	0	1
1	1	1	1	1	1	1	1	0

Lassen Sie uns zunächst die letzte dieser Operationen anschauen, da Sie Ihnen vermutlich am vertrautesten ist: $A \nleftrightarrow B$ ist genau dann wahr, wenn A und B verschiedene Wahrheitswerte haben. Sonst ist $A \nleftrightarrow B$ falsch. Die beste sprachliche Umschreibung von $\nleftrightarrow$ ist „entweder ... oder ...“. Im Gegensatz zum Operator der Disjunktion $\vee$ drückt der Operator $\nleftrightarrow$ eine echte Alternative aus: Entweder das eine oder das andere, aber nicht beides zugleich. Daher wird $A \nleftrightarrow B$ auch die *Alternation* von A und B genannt. Eine weitere gebräuchliche Bezeichnung ist *ausschließende Disjunktion*. Der Operator $\nleftrightarrow$ wird in der Informatik meistens als *xor* (kurz für *exclusive or*) bezeichnet.

Von der mittleren Operation $A \leftrightarrow B$ haben Sie hoffentlich gesehen, dass sie die Negation der Alternation ist:

$$(A \nleftrightarrow B) = \neg(A \leftrightarrow B) \quad (2.5)$$

$$(A \leftrightarrow B) = \neg(A \nleftrightarrow B) \quad (2.6)$$

Die Symbolik ist also wie bei $=$ und $\neq$: Durchstreichen des Operators steht für dessen Negation.

Die Verknüpfung $A \leftrightarrow B$ liefert genau dann wahr, wenn die beiden verknüpften Aussagen gleiche Wahrheitswerte haben. Die beste sprachliche Beschreibung ist „ A genau dann, wenn B “, wie

bspw. in „Es schneit genau dann, wenn ich keine Winterreifen habe.“ Wir bezeichnen diese Operation als *Bijunktion*. Weitere gebräuchliche Bezeichnungen sind *logische Äquivalenz* oder einfach nur *Äquivalenz*.

Damit sind wir im Begriffsdschungel der Logik angelangt. Die Vielfalt von Synonymen und der wenig einheitliche Gebrauch von Begriffen erschweren leider die Orientierung in der Logik. Die eigentliche Herausforderung ist jedoch eine ganz andere: Es gibt ein zur Bijunktion verwandtes Konzept. Diese beiden Konzepte zu trennen und zu verstehen, dass sie unterscheidbar und unterscheidungswürdig sind, ist eine geistige Leistung. Für dieses verwandte Konzept hat sich sogar ein einheitlicher Begriff etabliert: *Äquivalenz*. Wir werden ihr im Abschnitt 3.2 begegnen und sie dort von der Bijunktion abgrenzen. An dieser Stelle sollte Ihnen schon klar sein, dass wir aus den angedeuteten Gründen die Bijunktion keinesfalls *Äquivalenz* nennen sollten.

Damit sind wir bei der ersten Operation angelangt. $A \rightarrow B$ ist nur falsch, wenn A wahr und B falsch ist. Die beste sprachliche Formulierung ist „wenn A , dann sicher B (aber vielleicht auch sonst)“, oft lax als „wenn A , dann B “ formuliert. Wir bezeichnen $A \rightarrow B$ als *Subjunktion* von A und B .

Weitere gebräuchliche Bezeichnungen für die Subjunktion sind *logische Implikation* oder einfach nur *Implikation*. Die Situation ist vergleichbar mit der bei der Bijunktion. Es gibt einen Begriffsdschungel und es gibt ein verwandtes, aber unterscheidbares und unterscheidungswürdiges Konzept, von dem wir die Subjunktion abgrenzen müssen: Die Implikation. Mehr dazu in Abschnitt 3.2.

Definition 2.17: Die *Subjunktion* $A \rightarrow B$ zweier Aussagen A und B ist nur dann falsch, wenn A wahr und B falsch ist. Die *Bijunktion* zweier Aussagen ist genau dann wahr, wenn beide Aussagen den gleichen Wahrheitswerte haben. Die *Alternation* ist genau dann wahr, wenn beide Aussagen unterschiedliche Wahrheitswerte haben.

Gerade die Subjunktion und der Zusammenhang mit Wenn-dann-Sätzen bereiten erfahrungsgemäß beim Studium der Logik Schwierigkeiten. Wir verschieben diese Problematik auf den nachfolgenden Abschnitt 2.4.4 und untersuchen hier zunächst einmal die formalen Eigenschaften der Subjunktion. Diese werden sich im Abschnitt 2.4.4 als hilfreich erweisen.

Zur Untersuchung der formalen Eigenschaften der Subjunktion vergleichen Sie bitte die Wahrheitstabellen zu $A \rightarrow B$ auf S. 24 und zu $\neg A \vee B$ auf S. 21. Was ergibt dieser Vergleich?



Die Subjunktion kann also durch eine Kombination aus Negation und Disjunktion umschrieben werden.

$$(2.7) \quad (A \rightarrow B) = (\neg A \vee B)$$

Die Wahrheitstabellen der beiden Ausdrücke sind bis auf die Überschriften der Ergebnisspalten identisch. Das bedeutet, dass die logischen Ausdrücke in diesen Überschriften gleichwertig sein müssen. Es gilt also

Natürlich gibt es die Operatoren von Subjunktion, Bijunktion und Alternation auch in *setlX*. Tabelle 2.1 nennt die Operatorsymbole.

Sie haben nun also die Möglichkeit mit *setlX* alle wichtigen logischen Operatoren auszuprobieren und damit herumzuspielen. Nutzen Sie diese Gelegenheit!

Beispiel 2.18: Wir nutzen *setlX*, um (2.7) zu verifizieren:

```
=> A:=false; B:=false; (A=>B)==(!A||B);
~< Result: true >~
=> A:=false; B:=true; (A=>B)==(!A||B);
~< Result: true >~
=> A:=true; B:=false; (A=>B)==(!A||B);
~< Result: true >~
=> A:=true; B:=true; (A=>B)==(!A||B);
```

Tabelle 2.1: Verschiedene Notationssysteme für die bedeutendsten logischen Operatoren.

Operator	Symbol in diesem Text	<i>setlX</i>	weitere Symbole
Negation	$\neg$	!	$\neg$ (z. B. $\neg A$)
Konjunktion	$\wedge$	&&	$\cdot$
Disjunktion	$\vee$		+
Subjunktion	$\rightarrow$	=>	$\Rightarrow$
Bijunktion	$\leftrightarrow$	<==>	$\Leftrightarrow$
Alternation	$\nleftrightarrow$	<!=>	$\nleftrightarrow, \oplus$

~< Result: true >~

Beziehung (2.7) ist also tatsächlich für jede mögliche Belegung von A und B wahr. ■

Die sechs logischen Operatoren können natürlich beliebig kombiniert werden, solange die folgenden Syntaxregeln eingehalten werden:²

1. Der unäre Operator der Negation muss immer auf eine einzige Aussage oder Aussageform wirken. Der Ausdruck muss also von der Form $\neg A$ sein oder mit Klammern umgeben von der Form $(\neg A)$.
2. Binäre Operatoren müssen immer genau zwei Aussagen, zwei Aussageformen oder eine Aussage und eine Aussageform verbinden. Steht $\diamond$ für einen binären logischen Operator, muss der Ausdruck von der Form $A \diamond B$ oder mit Klammern umgeben von der Form $(A \diamond B)$ sein.

Beispiel 2.19: Der folgende Ausdruck ist eine eher unspektakuläre Kombination aus Subjunktion und Negation:

$$\neg B \rightarrow \neg A$$

Die Wahrheitstabelle ist:

A	B	$\neg B$	$\neg A$	$\neg B \rightarrow \neg A$
0	0	1	1	1
0	1	0	1	1
1	0	1	0	0
1	1	0	0	1

Die Ergebnisspalte ist mit der der Subjunktion auf S. 24 identisch. Daher gilt

$$(\neg B \rightarrow \neg A) = (A \rightarrow B) \quad (2.8)$$

Der Wahrheitswert einer Subjunktion ändert sich also nicht, wenn die Operanden vertauscht und gleichzeitig negiert werden. In Worten: „Wenn A , dann B “ ist gleichbedeutend mit „Wenn nicht B , dann nicht A “. ■

Beispiel 2.20: Betrachten wir als ein komplexeres Beispiel den Ausdruck

$$(A \rightarrow B) \wedge (B \rightarrow A)$$

und werten ihn mittels Wahrheitstabelle aus:

A	B	$A \rightarrow B$	$B \rightarrow A$	$(A \rightarrow B) \wedge (B \rightarrow A)$
0	0	1	1	1
0	1	1	0	0
1	0	0	1	0
1	1	1	1	1

²Die Formulierung der Syntaxregeln ist notwendigerweise etwas holprig, weil uns noch die geeignete Beschreibungsförm der Grammatik fehlt. Sie können diese im Laufe Ihres weiteren Studiums im Rahmen der Theorie der formalen Sprachen in der Theoretischen Informatik kennenlernen.

Ein Vergleich mit der Wahrheitstabelle für die Bijunktion auf S. 24 zeigt, dass diese beiden Operationen identisch sind. Es gilt also

$$(A \leftrightarrow B) = ((A \rightarrow B) \wedge (B \rightarrow A)) \quad (2.9)$$

In Worten: „ A genau dann, wenn B “ ist gleichbedeutend mit „Wenn A , dann B und, wenn B , dann A .“ ■

Wir fassen die Ergebnisse aus (2.7) und (2.9) zusammen:

Alternative Ausdrücke für Subjunktion und Bijunktion:

$$(A \rightarrow B) = (\neg A \vee B) \quad (2.10)$$

$$(A \rightarrow B) = (\neg B \rightarrow \neg A) \quad (2.11)$$

$$(A \leftrightarrow B) = ((A \rightarrow B) \wedge (B \rightarrow A)) \quad (2.12)$$

Bei Konjunktion und Disjunktion haben wir die Wirkung der Negation auf diese Operationen untersucht und sind zu den Ergebnissen (2.3) und (2.4) gekommen. Die Wirkung der Negation auf Alternation und Bijunktion haben wir schon in (2.5) und (2.6) festgehalten. Es bleibt noch die Wirkung der Negation auf die Subjunktion, also $\neg(A \rightarrow B)$, zu untersuchen. Dazu könnten wir wieder die Wahrheitstabelle aufstellen. Wir gehen hier jedoch einen anderen Weg, weil wir inzwischen genügend Formelmaterial angesammelt haben, dass wir auch symbolisch rechnen können:

$$\begin{aligned} \neg(A \rightarrow B) &= \neg(\neg A \vee B) \quad \text{wegen (2.10)} \\ &= (\neg(\neg A) \wedge \neg B) \quad \text{wegen (2.4)} \\ &= (A \wedge \neg B) \quad \text{wegen (2.2)} \end{aligned}$$

Wir werden in Kapitel 4 das symbolische Rechnen mit logischen Ausdrücken auf systematische Füße stellen. Hier fassen wir zunächst die Ergebnisse zur Negation zusammen:

Negation von Subjunktion, Bijunktion und Alternation

$$\neg(A \rightarrow B) = (A \wedge \neg B) \quad (2.13)$$

$$\neg(A \leftrightarrow B) = (A \nleftrightarrow B) \quad (2.14)$$

$$\neg(A \nleftrightarrow B) = (A \leftrightarrow B) \quad (2.15)$$

2.4.4 Bedeutung der Subjunktion

Haben Sie sich gefragt, wie es sein kann, dass eine Subjunktion und damit ein Wenn-dann-Satz falsch sein kann? Haben Sie sich gefragt, wie es sein kann, dass die Aussage im Wenn-Satz falsch ist?

Hier kommen die Antworten! Sie werden sehen, dass das, was Sie in Frage stellen, tatsächlich Ihrem Sprachgefühl entspricht. Der Satz

„Wenn Du zu am Freitag zu meiner Party kommst, lernst Du nette Leute kennen.“

kann als Subjunktion $P \rightarrow L$ modelliert werden. Stellen Sie sich vor, ein Freund sagt diesen Satz. Es gibt nun zwei Möglichkeiten:

1. Ihr Freund sagt die Wahrheit. In diesem Fall hat $P \rightarrow L$ den Wahrheitswert $\mathbb{1}$.
2. Ihr Freund lügt. In diesem Fall hat $P \rightarrow L$ den Wahrheitswert $\mathbb{0}$.

Formal sagt uns die Wahrheitstabelle, in welchen Fällen Ihr Freund die Wahrheit sagt bzw. lügt.

P	L	$P \rightarrow L$
0	0	1
0	1	1
1	0	0
1	1	1

Hier geht es darum, zu sehen, was unser Sprachempfinden darüber sagt, und zu analysieren, inwiefern Sprachempfinden und formales Modell der Logik übereinstimmen. Dazu gehen wir systematisch alle vier Fälle für das Aussagenpaar $[P, L]$ durch:

1. Sie gehen zur Party und lernen tatsächlich nette Leute kennen ($P = 1, L = 1$). Ihr Freund hat also nicht gelogen. $P \rightarrow L$ hat den Wahrheitswert 1.
2. Sie gehen nicht zur Party (weil Sie z. B. alleine zu Hause Diskrete Strukturen studieren) und lernen so am Freitag auch keine netten Leute kennen ($P = 0, L = 0$). Ihr Freund hat ebenfalls nicht gelogen. Er hat ja keine Aussage darüber gemacht, was passiert, wenn Sie nicht zu seiner Party kommen. $P \rightarrow L$ hat den Wahrheitswert 1.
3. Sie gehen nicht zur Party (Sie haben sich ja entschieden Diskrete Strukturen zu studieren) und lernen am Freitag (anderweitig) nette Leute kennen (nämlich die netten Kommilitonen, die Ihnen vorgeschlagen haben, eine Arbeitsgruppe zu bilden), also $P = 0, L = 1$. Aus demselben Grund wie im vorausgegangenen Fall hat Ihr Freund nicht gelogen. $P \rightarrow L$ hat den Wahrheitswert 1.
4. Sie gehen zur Party, aber lernen keine nette Leute kennen ($P = 1, L = 0$). Jetzt hat Ihr Freund aber gelogen! $P \rightarrow L$ hat also den Wahrheitswert 0.

Überzeugen Sie sich davon, dass diese vier Fälle genau denen in der Wahrheitstabelle der Subjunktion entsprechen.

Aussprache-
vorschlag für
Subjunktion

Die Subjunktion $A \rightarrow B$ hat die bemerkenswerte Eigenschaft, dass sie wahr ist, sobald A falsch ist. Um dies auch sprachlich deutlich zu machen, empfehle ich $A \rightarrow B$ nicht als „Wenn A , dann B “ zu lesen, sondern als

„Wenn A , dann B (aber vielleicht auch sonst)“.

Der Zusatz in Klammern bringt zum Ausdruck, dass B wahr sein kann, selbst wenn A falsch ist.

Möglicherweise wäre nun ein passender Zeitpunkt, um auf das zurückzublicken, was Sie bisher gelesen haben, oder eine Pause zu machen.

2.4.5 Modellieren von Alltagssprache - Teil 2

Wir hatten in Abschnitt 2.4.2 gesehen, dass nicht jedes alltagssprachliche „und“ eine Konjunktion im Sinne der Logik ist. Ebenso ist nicht jedes „oder“ eine Disjunktion. Wenn Sie in einer Kneipe einen Wein spendiert bekommen und gefragt werden, ob Sie lieber einen roten oder einen weißen Wein wollen, geht es um folgende Aussage:

„Sie bekommen Rotwein oder Sie bekommen Weißwein.“

Dies besteht aus den beiden Teilaussagen R = „Sie bekommen Rotwein“ und W = „Sie bekommen Weißwein.“ Die logische korrekte Modellierung der Gesamtaussage ist aber nicht $R \vee W$, sondern $R \nleftrightarrow W$. Schließlich ist es unangebracht auf die Frage „Rot- oder Weißwein?“ mit „beides“ zu antworten.

Viele vermeintliche Disjunktionen in der Alltagssprache sind tatsächlich Alternationen. Trotzdem werden Sie umgangssprachlich nicht mit „entweder . . . oder“ ausgedrückt, sondern alleine mit „oder“. Das ist häufig dann der Fall, wenn das, was mit „oder“ verknüpft wird, sich gegenseitig ausschließt. „Am Wochenende fahre ich nach Hause oder ich bleibe in Wolfenbüttel“ ist ein Beispiel dafür. Ein

weiterer häufiger Fall besteht darin, dass aus dem Kontext klar ist, dass nicht beide Teilaussagen gleichzeitig wahr sein können. „Sie bekommen Rotwein oder Sie bekommen Weißwein“ ist hierfür ein Beispiel.

Wir sind es als Menschen gewohnt, Sprache nicht absolut wörtlich zu nehmen, sondern kontextabhängig zu interpretieren. Dadurch wissen wir, ohne groß nachdenken zu müssen, ob ein „oder“ für eine Disjunktion oder Alternation steht.

Genauso verhält es sich mit Wenn-dann-Sätzen. Allerdings ist die Lage hier wesentlich komplexer und auch komplizierter. Das liegt daran, dass wir Wenn-dann-Sätze für sehr viele, aus logischer Sicht unterschiedliche Formulierungen verwenden. Die damit verbundene Problematik wird sich wie ein roter Faden durch diesen Kurs ziehen. Die Angelegenheit ist so komplex, dass wir sie hier noch nicht abschließend klären können.

Hier schauen wir uns nur eine erste Ausprägung der Problematik an – anhand der folgenden beiden Wenn-dann-Aussagen:

- $A =$ „Wenn Du zu meiner Party kommst, dann lernst Du nette Leute kennen.“
- $B =$ „Wenn man mindestens siebzehn ist, darf man (von Gesetzes wegen) den Führerschein Klasse B machen.“

Wir zerlegen beides in Teilaussagen und schreiben die Teilaussage im Wenn-Satz auf die linke Seite des Subjunktionsoperators $\rightarrow$ und die Teilaussage im Dann-Satz auf die rechte Seite von $\rightarrow$, also $A = (P \rightarrow L)$ und $B = (S \rightarrow F)$. Ich hoffe die Abkürzungen für die Teilaussagen sind selbsterklärend für Sie.

Eine Frage vorweg: Kann es vorkommen, dass jemand mindestens siebzehn ist und den Führerschein nicht machen darf? (Etwa aufgrund von körperlichen Beeinträchtigungen) 

Die Wahrheitstabelle für Aussage A haben wir schon zur Genüge untersucht (siehe S. 28). Sie haben hoffentlich auch verstanden, dass und warum A nur dann falsch ist, wenn Sie zur Party kommen, aber keine netten Leute kennenlernen ($P = 1, L = 0$).

Auch die Aussage zum Führerschein, $S \rightarrow F$, wäre demnach nur falsch, wenn man mindestens siebzehn ist und nicht (von Gesetzes wegen) den Führerschein machen darf, also $S = 1, F = 0$. Ich hoffe, dass dies im Widerspruch zu dem steht, was Sie oben zu diesem Sachverhalt geschrieben haben. Tatsächlich ist Aussage B nur in dem Fall falsch, wenn man den Führerschein machen darf und jünger als siebzehn ist! (Also $S = 0, F = 1$).

Die korrekte Wahrheitstabelle für B ist

S	F	B	F	S	$F \rightarrow S$
0	0	1	0	0	1
0	1	0	0	1	1
1	0	1	1	0	0
1	1	1	1	1	1

und das ist die Subjunktion $F \rightarrow S$ (also nicht $S \rightarrow F$, wie oben fälschlich angenommen wurde – ausgehend davon, dass S im Wenn-Satz steht)! Sie können dies durch Vergleich mit der rechten Wahrheitstabelle sehen. Als Subjunktion formuliert lautet B

„Wenn man (von Gesetzes wegen) den Führerschein Klasse B machen darf, ist man mindestens siebzehn (aber vielleicht auch sonst).“

Wir müssen daraus schließen, dass der Wenn-dann-Satz

„Wenn man mindestens siebzehn ist, dann darf man (von Gesetzes wegen) den Führerschein Klasse B machen.“

keine Subjunktion ist. Subjunktionen modellieren Wenn-dann-Sätze der Form „Wenn ..., dann ... (aber vielleicht auch sonst)“. Hier haben wir allerdings einen Wenn-dann-Satz der Form „(Nur wenn ..., dann ... (aber vielleicht auch nicht))“, nämlich

„Nur wenn man mindestens siebzehn ist, dann darf man (von Gesetzes wegen) den Führerschein Klasse B machen (aber vielleicht auch nicht).“

Halten wir als Zwischenergebnis fest:

Wir müssen zwei Arten von Wenn-dann-Sätzen unterscheiden:

1. Sätze von der Form „Wenn A , dann B (aber vielleicht auch sonst)“. Diese werden durch die Subjunktion $A \rightarrow B$ modelliert.
2. Sätze von der Form „(Nur) wenn B , dann A (aber vielleicht auch nicht)“. Diese sind bedeutungsgleich mit den Sätzen der ersten Bauart, obwohl die Reihenfolge von A und B anders herum ist. Weil sie bedeutungsgleich sind, wird diese zweite Art von Sätzen ebenfalls durch die Subjunktion $A \rightarrow B$ modelliert.

Um bei der zweiten Kategorie von Sätzen die Reihenfolge der Symbole A und B so aufschreiben zu können, dass sie mit der im Satz übereinstimmt, schreiben wir in diesem Fall auch $B \leftarrow A$.

Beispiel 2.21: Die folgenden vier Sätze stellen alle Möglichkeiten dar, mit denen die Aussagen $R =$ „es regnet“ und $N =$ „Die Straße ist nass“ kombiniert werden können:

1. „Wenn es regnet, ist die Straße nass.“ (Modell $R \rightarrow N$)
2. „Wenn die Straße nass ist, regnet es.“ (Modell $N \rightarrow R$)
3. „Nur wenn es regnet, ist die Straße nass.“ (Modell $R \leftarrow N$)
4. „Nur wenn die Straße nass ist, regnet es.“ (Modell $N \leftarrow R$)

Sätze Nr. 1 und 4 sind dabei bedeutungsgleich, ebenso Sätze 2 und 3. ■

2.4.6 Die 16 binären logischen Operatoren

Bisher haben wir sechs logische Operatoren kennengelernt. Gibt es noch weitere? Um eine Antwort zu finden machen wir nochmal einen Vorausgriff in die Werkzeugkiste der Kombinatorik (Kapitel 10). Als Symbol für die noch unbekannten Operatoren verwenden wir $\diamond$. Die Wahrheitstabelle sieht dann so aus:

A	B	$A \diamond B$
0	0	
0	1	
1	0	
1	1	

In der ersten Zeile der Ergebnisspalte gibt es zwei mögliche Einträge ($0 \diamond 0$ kann entweder 0 oder 1 sein), ebenso in der zweiten Zeile (macht insgesamt $2 \cdot 2 = 4$ Möglichkeiten), ebenso in der dritten Zeile (macht nun insgesamt $2 \cdot 4 = 8$ Möglichkeiten) und auch in der vierten Zeile. Insgesamt gibt es also $2^4 = 16$ Möglichkeiten die Wahrheitstabelle auszufüllen. Also muss es genau 16 binäre logische Operatoren geben – keinen mehr und keinen weniger.

Die folgenden 16 Wahrheitstabellen zeigen jede Möglichkeit. Die sechs uns bereits bekannten Operatoren sind darin schon eingetragen:

A	B	$A \diamond_0 B$	$A \diamond_1 B$ $A \wedge B$	$A \diamond_2 B$	$A \diamond_3 B$	$A \diamond_4 B$	$A \diamond_5 B$	$A \diamond_6 B$ $A \not\rightarrow B$	$A \diamond_7 B$ $A \vee B$
0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

A	B	$A \diamond_{15} B$	$A \diamond_{14} B$	$A \diamond_{13} B$ $A \rightarrow B$	$A \diamond_{12} B$ $\neg A$	$A \diamond_{11} B$	$A \diamond_{10} B$	$A \diamond_9 B$ $A \leftrightarrow B$	$A \diamond_8 B$
0	0	1	1	1	1	1	1	1	1
0	1	1	1	1	1	0	0	0	0
1	0	1	1	0	0	1	1	0	0
1	1	1	0	1	0	1	0	1	0

Halten Sie erst einmal inne und untersuchen Sie die Struktur der Gesamtheit aller 16 Wahrheitstabellen. Dies ist eine gute Gelegenheit *diskrete Strukturen* selbst zu entdecken. Welche Strukturmerkmale fallen Ihnen auf?



- Die Operatoren, die in der oberen Tabellenhälfte stehen, sind immer die Negationen der in der unteren Tabellenhälfte stehen. Besonders deutlich ist dies bei der Operation $A \diamond_{15} B = \neg(A \leftrightarrow B)$. Allgemein gilt also $A \diamond_i B = \neg(A \diamond_{15-i} B)$.
 - Die Nummern der Operatoren nehmen nicht durchgängig von links nach rechts zu. In der oberen Tabelle wird von 0 bis 7 hochgezählt, in der unteren Tabelle wird von 15 bis 8 heruntergezählt. Diese Struktur spielt auch bei der sogenannten Komplementbildung in der Digitaltechnik eine Rolle.
 - Die 16 Operatoren sind von 0 bis 15 durchnummeriert. Die Wahrheitswerte in der Ergebnisspalte ergeben spaltenweise als Binärziffern gelesen genau die Nummer des jeweiligen Operators. Das ist unsere erste Begegnung mit einer strukturellen Verwandtschaft zwischen Logik und binären Zahlen.
- Die Gesamtheit der 16 Wahrheitstabellen weist u. a. folgende strukturelle Eigenschaften auf:

Ist Ihnen aufgefallen, dass auch die *unäre* Operation Negation eine der möglichen 16 *binären* Operationen ist? (Nämlich Operation $\diamond_{12}$). Jede unäre Operation lässt sich als binäre Operation auffassen, die einen ihrer Operanden ignoriert. In unserem Fall wird bei $A \diamond_{12} B$ der zweite Operand ignoriert und der erste negiert.³ Die binäre Operation $A \diamond_{12} B$ liefert so das Resultat $\neg A$.

Ihre Aufgabe besteht nun darin, sich die Bedeutungen der zehn verbleibenden Operatoren zu überlegen. Sie können dabei von der Alltagssprache ausgehen und sich überlegen, welche Wahrheitstabellen Sprachkonstrukte wie „... weder ... noch“, „sowohl ... als auch ...“ *etc.* beschreiben. Sie können aber auch die Struktur der Tabelle ausnutzen, um die Bedeutungen zu finden. Beispielsweise ist ja $A \diamond_8 B = \neg(A \vee B)$. Um die Bedeutung von $\diamond_8$ zu finden, können Sie sich also überlegen, welches Sprachkonstrukt der Alltagssprache durch $\neg(A \vee B)$ am besten beschrieben wird. Tragen Sie die Sprachkonstrukte dann in die folgende Tabelle ein:



³Diese Betrachtungsweise lässt sich auf andere Konzepte verallgemeinern: Eine Funktion mit einer Variablen kann als Funktion mit zwei Variablen aufgefasst werden, die eine ihrer Variablen ignoriert. Eine Aussageform mit einer Variablen kann als Aussageform mit zwei Variablen aufgefasst werden, die eine ihrer Variablen ignoriert. Eine Aussage kann als Aussageform einer Variablen aufgefasst werden, die ihre Variable ignoriert.

A	B	$\Diamond_0$	$\Diamond_1$: ... und ...	$\Diamond_2$	$\Diamond_3$	$\Diamond_4$	$\Diamond_5$	$\Diamond_6$: entweder ... oder ...	$\Diamond_7$: ... oder auch ...	$\Diamond_8$	$\Diamond_9$: ... genau dann, wenn ...	$\Diamond_{10}$	$\Diamond_{11}$	$\Diamond_{12}$: nicht die erste Teilaussage	$\Diamond_{13}$: wenn ..., dann sicher ... (aber vielleicht auch sonst)	$\Diamond_{14}$	$\Diamond_{15}$
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

Bisher haben wir nur Aussagen (bzw. Symbole, die für Aussagen stehen sollen) mittels logischer Operatoren zu neuen Aussagen verknüpft. Genauso gut können wir aber auch Aussageformen zu neuen Aussageformen verknüpfen. Dabei gelten alle bisher gefundenen Eigenschaften wie z. B. (2.2-2.4) oder (2.13-2.15) weiterhin.

Beispiel 2.22: Hier sind zwei Beispiele:

- $(x^2 = 1) \leftrightarrow (x = -1) \wedge (x = 1)$
- „Wenn dieser Ausdruck wahr ist, fresse ich einen Besen.“

Im ersten Fall werden drei, jeweils x -abhängige Aussageformen zu einer neuen Aussageform verknüpft. Im zweiten Fall entsteht eine neue Aussageform durch Verknüpfung einer Aussageform (Variable „dieser Ausdruck“) mit einer Aussage (wenn „ich“ als Konstante und nicht als Variable modelliert wird). ■

Wie steht es eigentlich mit Beziehungen wie $(A \rightarrow B) = (\neg A \vee B)$? Sind diese immer Aussagen? Sind sie immer Aussageformen? Oder ist der Sachverhalt komplexer?



2.5 Präzedenz

Bisher haben die logischen Ausdrücke, die wir verwendet haben, immer unheimlich viele Klammern enthalten. Klammern machen klar, in welcher Reihenfolge Operatoren ausgewertet werden müssen. Unterschiedliche Auswertungsreihenfolgen können zu unterschiedlichen Ergebnissen führen. $(3 \cdot 4) + 5$ und $3 \cdot (4 + 5)$ sind eben nicht dasselbe.

Um nicht allzu viele Klammern schreiben zu müssen, gibt es das Instrument der *Präzedenz* oder *Bindungsstärke*. Sie kennen das von „Punkt geht vor Strich“. Die Multiplikation hat Vorrang (Präzedenz) vor der Addition, d. h. die Multiplikation bindet stärker als die Addition – es sei denn durch Kammersetzung wird eine andere Reihenfolge erzwungen. Weil die Multiplikation stärker bindet als die Addition, können wir $3 \cdot 4 + 5$ statt $(3 \cdot 4) + 5$ schreiben.

Die Präzedenz der logischen und arithmetischen Operatoren ist (mit von links nach rechts zunehmender Bindungsstärke):

$$\left[\begin{array}{c} \text{niedrige} \\ \text{Bindungs-} \\ \text{stärke} \end{array} \right] \rightarrow \leftrightarrow \vee \nleftrightarrow \wedge \neg \quad \begin{array}{c} = \\ \neq \\ > \\ \geq \\ < \\ \leq \end{array} \quad + \cdot \text{Potenz} \left[\begin{array}{c} \text{hohe} \\ \text{Bindungs-} \\ \text{stärke} \end{array} \right] \quad (2.16)$$

Operationen, die gleich stark binden, sind dabei untereinander angeordnet. Die Präzedenz in *setlX* ist geringfügig anders.⁴

Beispiel 2.23: Aufgrund der Präzedenzregeln kann statt $(A \wedge B) \vee C$ einfach $A \wedge B \vee C$ geschrieben werden. ■

Beispiel 2.24: $\neg A \wedge B \rightarrow C \leftrightarrow E$ bedeutet $((\neg A) \wedge B) \rightarrow (C \leftrightarrow E)$. ■

Beispiel 2.25: Was vermeintlich wie die Umschreibung (2.10) der Subjunktion durch Negation und Disjunktion aussieht

$$A \rightarrow B = \neg A \vee B$$

bedeutet aufgrund der Präzedenzregeln

$$A \rightarrow ((B = \neg A) \vee B).$$

Wir kommen leider wie in (2.10) nicht umhin reichlich Klammern zu setzen, wenn wir die angesprochene Umschreibung ausdrücken wollen:

$$(A \rightarrow B) = (\neg A \vee B)$$

Dadurch wird $=$ zur zuletzt auszuführenden Operation, die die Wahrheitswerte auf der linken und rechten Seite vergleicht. ■

Beispiel 2.26: $\emptyset \wedge \emptyset = \emptyset$ bedeutet nicht einen Ausschnitt aus der Wahrheitstabelle zur Konjunktion. Dies müsste als $(\emptyset \wedge \emptyset) = \emptyset$ geschrieben werden und hat den Wahrheitswert 1, wie wir uns schnell mit *setlX* überzeugen:

```
=> (false && false) == false;
~< Result: true >~
```

Das muss natürlich so sein, weil die Wahrheitstabelle der Konjunktion die Wahrheit ausdrücken soll.

$\emptyset \wedge \emptyset = \emptyset$ bedeutet dagegen $\emptyset \wedge (\emptyset = \emptyset)$ und das hat den Wahrheitswert 0:

```
=> false && (false == false);
~< Result: false >~
=> false && false == false;
~< Result: false >~
```

⁴In *setlX* bindet der Operator der Bijunktion schwächer als der der Subjunktion. Siehe <https://github.com/herrmanntom/setlX/blob/master/interpreter/core/src/main/antlr4/org/raandoom/setlX/grammar/OperatorPrecedences.txt>

Wenn wir also „falsch und falsch ergibt falsch“ ausdrücken wollen, müssen wir $(0 \wedge 0) = 0$ schreiben, aber nicht $0 \wedge 0 = 0$. ■

Das letzte Beispiel macht die Misere deutlich, in der wir uns befinden: Wir können bei logischen Operationen das Symbol $=$ nicht mehr in der Bedeutung „ergibt“ verwenden, so wie das bei „3 plus 5 ergibt 8“ und $3 + 5 = 8$ noch funktioniert hat. Schuld daran ist die geringere Bindungsstärke der logischen Operatoren im Vergleich zum Gleichheitsoperator.

Statt die Schuld an der Misere der Präzedenz zu geben, könnten wir den Schuldigen auch darin ausmachen, dass „ergibt“ mit $=$ symbolisiert wird. Das Vertiefungsmaterial in Abschnitt 2.7.3 thematisiert dies neben der Bedeutung des Gleichheitszeichens im Allgemeinen.

Um nicht immer so viele Klammern schreiben zu müssen und ein Symbol zu haben, das als „ergibt“ dienen kann, führen wir zur Rettung ein neues Symbol ein:

Die Äquivalenz $A \Leftrightarrow B$ drückt aus, dass A und B immer und garantiert den gleichen Wahrheitswert haben. Das Symbol $\Leftrightarrow$ wird als Äquivalenzoperator bezeichnet und „gleichbedeutend mit“ oder „äquivalent zu“ ausgesprochen.

Wir geben dem Äquivalenzoperator die geringste Bindungsstärke:

$$\left[\begin{array}{c} \text{niedrige} \\ \text{Bindungs-} \\ \text{stärke} \end{array} \right] \quad \Leftrightarrow \rightarrow \leftrightarrow \vee \nrightarrow \wedge \neg \quad \begin{array}{c} = \\ \neq \\ > \\ \geq \\ < \\ \leq \end{array} \quad + \cdot \text{Potenz} \quad \left[\begin{array}{c} \text{hohe} \\ \text{Bindungs-} \\ \text{stärke} \end{array} \right] \quad (2.17)$$

Indem wir beim Vergleich zweier Aussagen oder Ausgeformen $=$ durch $\Leftrightarrow$ ersetzen, können wir nun scheinbar überflüssige Klammern auf beiden Seiten des Gleichheitszeichens weglassen.

Beispiel 2.27: Wir können nun also schreiben

$$\begin{array}{ll} A \rightarrow B \Leftrightarrow \neg A \vee B & \text{statt} \quad (A \rightarrow B) = (\neg A \vee B) \\ 0 \wedge 0 \Leftrightarrow 0 & \text{statt} \quad (0 \wedge 0) = 0 \end{array}$$

■

2.6 Negation sprachlicher Aussagen

„Wenn Du zu meiner Party kommst, lernst Du nette Leute kennen.“ Wie haben Sie diese Aussage in der Aufwärmübung auf S. 13 negiert? Viele Menschen formulieren das Gegenteil dieses Satzes so

„Wenn Du zu meiner Party kommst, lernst Du nicht nette Leute kennen.“

oder auch

„Wenn Du nicht zu meiner Party kommst, lernst Du nicht nette Leute kennen.“

Beide Formulierungen sind nicht das Gegenteil des Ausgangssatzes!

Wir können das mit Hilfe der Logik sehen, indem wir die drei Aussagen formalisieren bzw. mit Hilfsmitteln der Logik modellieren. Dazu zerlegen wir den Ausgangssatz in die Teilaussagen

$$\begin{array}{ll} P & = \text{„Du kommst zu meiner Party.“} \\ L & = \text{„Du lernst nette Leute kennen.“} \end{array}$$

und schreiben den Ausgangssatz als $P \rightarrow L$. Die beiden Vorschläge für die Formulierung des Gegenteils lassen sich dann mit $P \rightarrow \neg L$ bzw. $\neg P \rightarrow \neg L$ modellieren.

Hier sind die Wahrheitstabellen für diese drei Sätze:

P	L	$P \rightarrow L$	$P \rightarrow \neg L$	$\neg P \rightarrow \neg L$	$\neg(P \rightarrow L)$
0	0	1	1	1	0
0	1	1	1	0	0
1	0	0	1	1	1
1	1	1	0	1	0

Aus diesen Wahrheitstabellen geht deutlich hervor, dass weder $P \rightarrow \neg L$ noch $\neg P \rightarrow \neg L$ korrekte Negationen von $P \rightarrow L$ sind. Die korrekten Wahrheitswerte von $\neg(P \rightarrow L)$ sind in der letzten Spalte dargestellt.

Wir hatten schon in (2.13) festgestellt, dass

$$\neg(P \rightarrow L) \Leftrightarrow P \wedge \neg L.$$

Eine korrekte Formulierung des Gegenteils des Ausgangssatzes ist also

„Du kommst zu meiner Party und Du lernst nicht nette Leute kennen.“

oder, um den Gegensatz zwischen den beiden Satzhälften zu betonen mit „aber“ statt „und“ als Verbalisierung von $\wedge$,

„Du kommst zu meiner Party, aber lernst keine netten Leute kennen.“

Bei der sprachlichen Formulierung des Gegenteils ist also Vorsicht geboten. Allerdings bietet uns die Logik, wie wir gesehen haben, einen Prozess, mit dem wir das Gegenteil einer Aussage korrekt formulieren können. Dieser Prozess besteht aus drei Schritten:

1. Formalisieren der Aussage bzw. Aussageform
2. Formales Negieren
3. Zurückübersetzen in Alltagssprache

Der Anwendungsbereich dieses Prozesses ist nicht auf das Bilden des Gegenteils beschränkt. Es handelt sich vielmehr um eine allgemeine Problemlösungsstrategie, die sehr häufig angewandt werden kann und angewandt wird. Abb. 2.3 stellt den Prozess graphisch dar.

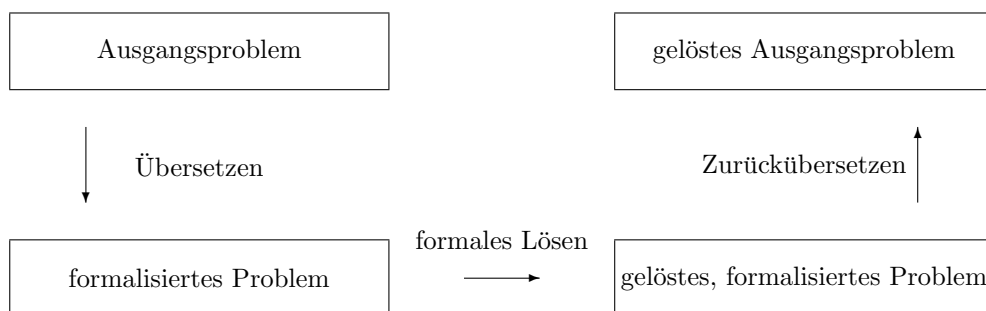


Abbildung 2.3: Vierfelderprozess zum Problemlösen durch Formalisieren: Statt dass Problem direkt zu lösen, wird es erst formalisiert, dann formal gelöst und schließlich in die Ausgangssprache zurückübersetzt.

2.7 Empfohlene Vertiefungen

2.7.1 Beziehung von Aussagenlogik und Sprache

Ich empfehle dazu den Text „Einführung in die Aussagenlogik“ von R. Golecki und J. Jungmann zu lesen: <http://www.hyfisch.de/HyFISCH/KI/GoleckiAussagenlogik.pdf>

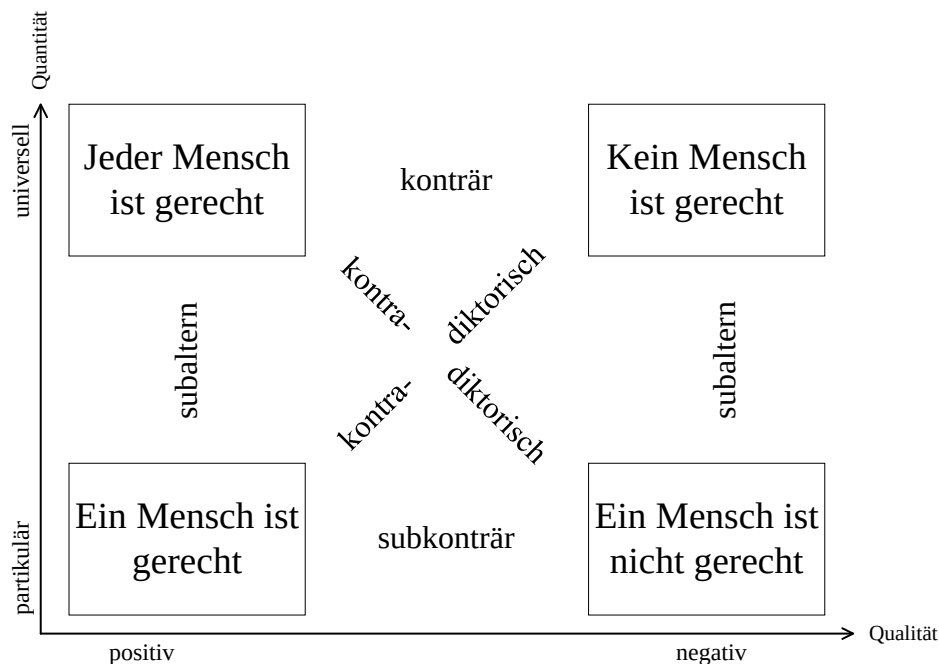


Abbildung 2.4: Logisches Quadrat zur Visualisierung des Unterschieds von konträr (gegensätzlich) und kontradiktorisch (gegenteilig).

2.7.2 Gegenteil und Gegensatz

Negieren bedeutet das Gegenteil bilden. Das Gegenteil bzw. die Negation von „Das Glas ist voll“ ist nicht „Das Glas ist leer“, sondern „Das Glas ist nicht voll“! Ebenso ist das Gegenteil von „Seine Hautfarbe ist schwarz“ nicht „Seine Hautfarbe ist weiß“, sondern „Seine Hautfarbe ist nicht schwarz.“

Wenn Sätze nicht korrekt negiert werden, liegt dies häufig daran, dass nicht sauber zwischen *Gegenteil* und *Gegensatz* unterschieden wird. Diese beiden Begriffe haben zwar einiges gemeinsam, sind aber unterscheidungswürdig und unterscheidungsbedürftig.

Zwei Aussagen oder Aussageformen sind gegenteilig bzw. *kontradiktorisch*, wenn die eine wahr und die andere falsch ist. Hier sind wir beim Kern der Logik und ihrer Zweiteilung in entweder wahr oder falsch. Das Konzept lässt sich ebenso auf Begriffe anwenden: Schwarz und nicht schwarz sind kontradiktorisch, wahr und falsch sind kontradiktorisch usw.

Bei gegensätzlichen bzw. *konträren* Aussagen bzw. Aussageformen sind nie beide wahr; beide oder eines ist unwahr. Auch hier lässt sich das Konzept entsprechend auf Begriffe anwenden: Schwarz und weiß sind konträr zueinander, gut ist der Gegensatz von böse usw.

Die Begriffe konträr und kontradiktorisch werden oft anhand des *logischen Quadrats* visualisiert. Dieses ist in Abb. 2.4 dargestellt.

2.7.3 Bedeutungen des Gleichheitszeichens

Das Symbol = hat in der Mathematik mehrere Bedeutung. Drei wichtige Bedeutungen sind:

1. „soll den Wert haben“ (Zuweisung)



Abbildung 2.5: Welche Bedeutung hat das Gleichheitszeichen hier? (Foto Ch. Kautz)

2. „ergibt“ (Vereinfachen)

3. „vergleichen“

Die erste Bedeutung tritt v. a. dann auf, wenn Ausdrücke durch Symbole ersetzt oder abgekürzt werden sollen wie z. B.

$$g = \frac{1 + \sqrt{5}}{2}.$$

Die zweite Bedeutung tritt beim „Rechnen“ im Sinne von Umformen bzw. Vereinfachen auf wie z. B. in

$$1 + 2 + 3 + 4 = 3 + 3 + 4 = 6 + 4 = 10$$

$$(x + y)^2 - (x - y)^2 = x^2 + 2xy + y^2 - (x^2 - 2xy + y^2) = x^2 + 2xy + y^2 - x^2 + 2xy - y^2 = 4xy$$

Die dritte Bedeutung vergleicht linke und rechte Seite, z. B. $2 = 4$ oder $ax^2 + bx + c = 0$.

In Programmiersprachen werden die Bedeutungen Nr. 1 und Nr. 3 gewöhnlich durch verschiedene Symbole ausgedrückt (z. B. $:=$ und $==$). Dass diese in der Mathematik (in der Regel) nicht getrennt werden, ist Fluch und Segen zugleich. Der Fluch besteht darin, dass aus dem Kontext erschlossen werden muss, welche Bedeutung des Gleichheitszeichens vorliegt. Der Segen besteht darin, dass durch einen Ausdruck mehrere Aussagen bzw. Interpretationen gleichzeitig kommuniziert oder gedacht werden können, die sinnhaftig sind. Dass dies möglich ist, kommt daher, dass sich die verschiedenen Bedeutungen des Gleichheitszeichens nicht gegenseitig ausschließen. Der Ausdruck $4 + 6 = 10$ kann eben bedeuten, dass ein neues Symbol (10) für $6+4$ eingeführt wird, dass $6+4$ die Zahl 10 ergibt oder dass die Ausdrücke auf den beiden Seiten des Gleichheitszeichens miteinander verglichen werden sollen.

Dem gegenüber stehen „missbräuchliche“ Verwendungen des Gleichheitszeichens im Alltag, meist als Abkürzungen von „ist“ wie in „Logik = wichtig“ oder in Abb. 2.5.

2.8 Lernziele

Studierende	Kennen	Können	Verstehen
fachlich	☐ erläutern Unterschied zwischen Aussage und Aussageform ☐ kennen die Definition und Wahrheitstabellen aller unären und binären logischen Operatoren ☐ wissen, dass und welche unterschiedlichen Symbole und Bezeichnungen es für die logischen Operationen gibt ☐ wissen, dass, und begründen, warum $A \rightarrow B \Leftrightarrow \neg A \vee B$ ist	☐ bestimmen Wahrheitswerte von formalen Aussagen mit Wahrheitstabellen ☐ negieren natürlichsprachliche Aussagen mittels formaler Ausdrücke ☐ übersetzen formallogische Ausdrücke in natürliche Sprache und umgekehrt ☐ drücken Subjunktion und Bijunktion durch andere logischen Operationen aus ☐ drücken Negation von Konjunktion, Disjunktion und Bijunktion durch andere logischen Operationen aus ☐ zeigen Gleichheit logischer Ausdrücke mit Hilfe von Wahrheitstabellen ☐ zeigen Gleichheit logischer Ausdrücke mit Hilfe von Computerprogrammen	☐ erkennen bzw. wägen ab, ob ein Satz geeignet als Aussage oder Aussageform modelliert werden kann ☐ begründen Sinnhaftigkeit der Definition der Subjunktion (insbesondere $(1 \rightarrow 0) = 0$)
methodisch	☐ kennen das Konzept der Präzedenz	☐ entscheiden, ob ein formaler Ausdruck syntaktisch korrekt oder falsch ist ☐ können bei vorgegebener Präzedenz Klammer weglassen ☐ bestimmen Wahrheitswerte von formalen Aussagen mit setlx	

Hinzu kommen die themenübergreifenden Lernziele aus den Kategorien „sozial“ und „persönlich“ aus Abschnitt 1.7 sowie die Lernziele aus der Kategorie „methodisch“ aller vorangegangener Kapitel. Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben.

2.9 Übungsaufgaben

- Ist die Aussage oder die Aussageform das bessere Modell für die folgenden Sätze bzw. Ausdrücke? Von welchen Umständen hängt dies ab?
 - Er hat seine Berufung gefunden.

- (b) Wolfenbüttel hat eine Bürgermeisterin.
 - (c) Neustadt hat einen Bürgermeister.
 - (d) Ein Glas ist ein Gefäß.
 - (e) Die Abb. 2.5 zeigt den Blick aus einem Hotelfenster.
 - (f) $A \wedge B$
 - (g) $n > 0 \rightarrow n^2 > 0$
2. Ist der Satz „Dieser Satz ist keine Aussage.“ eine Aussage? Ist er wahr? Erläutern Sie Ihre Antwort.
 3. Es gibt 16 binäre logische Operatoren. Wie viele unäre logischen Operatoren gibt es? Geben Sie für jeden Operator die Wahrheitstabelle an.
 4. Lösen Sie die Gleichung

$$(P \vee \neg Q) = (\neg(P \wedge Q) \vee \neg P).$$

Finden Sie also Werte für P und Q , so dass die beiden Seiten dieser Gleichung denselben Wahrheitswert haben.

5. Welchen Wahrheitswert hat $\mathbb{1} \not\leftrightarrow \mathbb{0} \vee \mathbb{1}$?
6. Finden Sie heraus, wie in Java die Präzedenz der logischen Operatoren im Vergleich zum Vergleichsoperator `==` ist. Welche Konsequenzen hat das, wenn in Java $\mathbb{0} \wedge \mathbb{0} = \mathbb{0}$ ausgewertet wird?
7. Zerlegen Sie die folgenden Aussagen in Teilaussagen und geben Sie die logischen Operationen an, mit denen diese Teilaussagen zur Ausgangsaussage verknüpft werden:
 - (a) „Bei Hannover kreuzen sich die Autobahnen A2 und A7.“
 - (b) „Falls wir diesen Text nicht lesen, können wir weder der Vorlesung folgen noch die Übungsaufgaben bearbeiten.“
 - (c) „Entweder stimmt es nicht, dass Üben beim Lernen hilft, oder ein Lerneffekt tritt nur auf, wenn man die Dinge vor dem Üben tief genug verstanden hat.“
 - (d) „Es gibt keine Hochschule in Wolfenbüttel und Wolfsburg.“
 - (e) „Eins und eins ist zwei.“
8. In einem Lehrbuch zur Mathematik für Informatiker befindet sich folgender Text⁵:

Die Verwendung von „und“ bzw. „oder“ in der Aussagenlogik stimmt in den meisten Fällen mit dem überein, was wir uns erwarten würden. Manchmal gibt es aber in der Umgangssprache Formulierungen, bei denen die Bedeutung nur aus dem Zusammenhang klar ist: Wenn zum Beispiel auf einem Schild „Rauchen und Hantieren mit offenem Feuer verboten!“ steht, dann weiß jeder, dass man hier weder Rauchen noch mit offenem Feuer hantieren darf. Vom Standpunkt der Aussagenlogik aus bedeutet das Verbot aber, dass nur *gleichzeitiges* Rauchen und Hantieren mit offenem Feuer verboten ist, es aber zum Beispiel erlaubt wäre, mit offenem Feuer zu hantieren, solange man dabei nicht raucht. Nach den Regeln der Aussagenlogik müsste das Verbot „Rauchen *oder* Hantieren mit offenem Feuer verboten“ lauten (eine Argumentation, die Ihnen aber wohl vor einem Richter nicht helfen würde, nachdem die Tankstelle abgebrannt ist).

Dieser Text weist korrekterweise auf die kontextabhängige Bedeutung von Sprache hin. Allerdings wird im Zusammenhang mit dem erläuterten Beispiel ein Modellierungsfehler gemacht. Worin besteht dieser?

⁵G. Teschl, S. Teschl: Mathematik für Informatiker, Band 1, Springer Vieweg, 4. Auflage, S. 4

9. Wenn bei einem Empfang Heißgetränke angeboten werden, was ist dann die Formulierung, welche die Situation im Sinne der Aussagenlogik korrekt beschreibt?
- (a) Die Bedienung bietet Kaffee und Tee an.
 - (b) Die Bedienung bietet Kaffee oder Tee an.
10. Mit welchem logischen Operator lässt sich „oder“ in den folgenden Sätzen am besten modellieren?
- (a) Kader ist in der Türkei oder in Deutschland geboren.
 - (b) Fritzchen darf sich ein Spielzeugauto oder ein Plüschtier nehmen.
 - (c) Die Bedienung bietet Kaffee oder Tee an.
 - (d) Die Bedienung bietet entweder Kaffee oder Tee an.
 - (e) Wir suchen einen Informatiker oder einen Elektrotechniker.
 - (f) Wir suchen eine Informatikerin oder einen Informatiker.
11. Untersuchen Sie die folgenden Ausdrücke mittels Wahrheitstabelle.
- (a) $A \wedge (A \rightarrow B) \rightarrow B$
 - (b) $\neg B \wedge (A \rightarrow B) \rightarrow \neg A$
 - (c) $A \wedge B \rightarrow A$

Welche Besonderheit haben die Ergebnisspalten der Wahrheitstabellen?

12. Untersuchen Sie die folgenden Ausdrücke mittels Wahrheitstabelle.
- (a) $((A \rightarrow B) \rightarrow C) \leftrightarrow (A \rightarrow (B \rightarrow C))$
 - (b) $((A \wedge B) \vee C) \leftrightarrow (A \vee (B \wedge C))$
13. Modellieren Sie die folgenden Sätze mit den Mitteln der Logik, d. h. zerlegen Sie sie, wenn möglich, in Teilaussagen bzw. Teilaussageformen, geben Sie diesen Symbole und verbinden Sie die Symbole mit den passenden Operatorsymbolen.
- (a) Wenn es das schon war, dann geht es nicht weiter.
 - (b) Nur wenn es nicht weitergeht, war es das schon.
 - (c) Es geht nur dann weiter, wenn es nicht regnet oder schneit.
 - (d) Es geht nur dann weiter, wenn es nicht regnet und schneit.
 - (e) Es geht nur dann weiter, wenn es nicht regnet oder nicht schneit.
 - (f) Es geht genau dann weiter, wenn es nicht schneit.
 - (g) Geht es weiter oder war's das schon?

Bearbeiten Sie auch Übungsaufgaben aus anderen Lehrtexten, z. B. die Kontrollfragen und Übungsaufgaben zu Abschnitt 1.1 im Lehrbuch von Teschl und Teschl.

Kapitel 3

Tautologien

3.1 Aufwärmübungen

1. Was haben die Wahrheitswerte der folgenden Gruppen formallogischer Ausdrücke jeweils gemeinsam?

Gruppe 1	$A \vee \neg A$ $(A \wedge \neg A) = \emptyset$ $(A \wedge B) \rightarrow B$
Gruppe 2	$A \wedge \neg A$ $(A \leftrightarrow B) = (A \nleftrightarrow B)$ $(A \wedge B) \leftrightarrow (\neg A \vee \neg B)$
Gruppe 3	$A \wedge A$ $(A \rightarrow B) \rightarrow B$ $B \rightarrow (A \wedge B)$ $A \wedge B \leftrightarrow B$



2. Eine Mutter sagt zu ihrem kleinen Kind: „Wenn Du Deinen Teller leer isst, bekommst Du Nachtisch.“ Sie wissen, dass die Mutter damit auch sagen will: „Wenn Du Deinen Teller nicht leer isst, bekommst Du keinen Nachtisch.“

Modellieren Sie diese Aussage Ihrer Mutter als logischen Ausdruck und geben Sie die Wahrheitstabelle an.



3. Leitfragen

- (a) Warum sind manche Wenn-dann-Sätze immer wahr?
- (b) Warum bedeutet wenn-dann in den folgenden Sätzen Unterschiedliches?
- Wenn es bei einer Kollektion nicht auf Mehrfachvorkommen und Reihenfolge ankommt, dann spricht man von einer Menge.
 - Wenn eine Kollektion ein Tupel ist, dann spielt Mehrfachvorkommen eine Rolle.
- (c) Was haben $=$ und $\Leftrightarrow$ gemeinsam? Was unterscheidet sie?

3.2 Tautologie und Kontradiktion

Einige logische Ausdrücke und Aussageformen, die wir bisher untersucht haben, weisen eine Besonderheit auf: Sie sind unabhängig von der Belegung der Variablen immer wahr. $(A \rightarrow B) = (\neg A \vee B)$ und $A \vee \neg A$ gehören dazu. Dies ist an den entsprechenden Wahrheitstabellen deutlich zu sehen:

A	B	$A \rightarrow B$	$\neg A \vee B$	$(A \rightarrow B) = (\neg A \vee B)$	A	$A \vee \neg A$
0	0	1	1	1	0	1
0	1	1	1	1	1	1
1	0	0	0	1		
1	1	1	1	1		

Blättern Sie an dieser Stelle zurück und identifizieren Sie alle mit einer Nummer gekennzeichneten logischen Ausdrücke, die die Eigenschaft haben, dass Sie immer wahr sind.



Schauen wir noch ein paar weitere Ausdrücke bzw. Aussageformen an, die immer wahr sind:

Beispiel 3.1: $A \wedge (A \vee B) \leftrightarrow A$ ist immer wahr, denn:

A	B	$A \vee B$	$A \wedge (A \vee B)$	$A \wedge (A \vee B) \leftrightarrow A$
0	0	0	0	1
0	1	1	0	1
1	0	1	1	1
1	1	1	1	1

■

Beispiel 3.2: Auch $x \geq 0 \leftrightarrow \neg(x < 0)$ ist immer wahr, egal mit welcher reellen Zahl wir x belegen:

x	$x \geq 0$	$\neg(x < 0)$	$x \geq 0 \leftrightarrow \neg(x < 0)$
$\vdots$	0	0	1
-1	0	0	1
$\vdots$	0	0	1
0	1	1	1
$\vdots$	1	1	1
π	1	1	1
$\vdots$	1	1	1

■

Es gibt nicht nur Ausdrücke, die immer wahr sind, sondern auch Ausdrücke, die immer falsch sind:

Beispiel 3.3: $x \geq 0 \nleftrightarrow \neg(x < 0)$ ist immer falsch. Das ergibt sich schon aus der sprachlichen Formulierung „Entweder ist $x \geq 0$ oder es gilt nicht $x < 0$.“ Sie können das wegen

$$(x \geq 0 \nleftrightarrow \neg(x < 0)) = \neg(x \geq 0 \leftrightarrow \neg(x < 0))$$

auch aus der obigen Wahrheitstabelle für $x \geq 0 \leftrightarrow \neg(x < 0)$ sehen. ■

Was immer falsch ist, ist offensichtlich das Gegenteil von dem, was immer wahr ist. Was immer wahr ist, ist offensichtlich bei der Suche nach Wahrheit wichtig. Wir geben diesen beiden wichtigen Spezialfällen der Logik erst mal Namen:

Definition 3.4: Aussageformen bzw. logische Ausdrücke, die unabhängig von der Belegung ihrer Variablen immer wahr sind, heißen *Tautologien*. Aussageformen bzw. logische Ausdrücke, die unabhängig von der Belegung ihrer Variablen immer falsch sind, heißen *Kontradiktionen*.

Wir sollten Tautologien aufgrund ihrer Wichtigkeit besonders kennzeichnen. Dann können wir sofort sehen, dass ein Ausdruck unabhängig von seinen Variablen immer wahr ist und müssen dies nicht mehr mittels Durchspielen aller Variablenkonstellationen herausfinden. Lassen Sie uns dafür das Symbol $\mathcal{T}$ verwenden. Immer wenn $\mathcal{T}$ links vor einem Ausdruck steht, soll dies signalisieren, dass eine Tautologie vorliegt. Das sieht dann bspw. so aus:

$$\mathcal{T} \quad x \geq 0 \leftrightarrow \neg(x < 0) \quad (3.1)$$

$$\mathcal{T} \quad x > 0 \rightarrow x \geq 0 \quad (3.2)$$

$$\mathcal{T} \quad A \wedge B \leftrightarrow B \wedge A \quad (3.3)$$

$$\mathcal{T} \quad B \vee \neg B \quad (3.4)$$

Dagegen ist es *nicht* in Ordnung

$$\mathcal{T} \quad A \wedge B \leftrightarrow B$$

zu schreiben, weil $A \wedge B \leftrightarrow B$ keine Tautologie ist.

Sehr viele Tautologien sind Bijunktionen oder Subjunktionen. Der am schwächsten bindende Operator im Ausdruck ist dann also ein $\leftrightarrow$ oder ein $\rightarrow$. Die Ausdrücke (3.1-3.3) sind Beispiele dafür. In diesen Fällen werden Tautologien üblicherweise statt des vorangestellten $\mathcal{T}$ dadurch gekennzeichnet, dass anstelle von $\leftrightarrow$ das Symbol $\Leftrightarrow$ und an Stelle von $\rightarrow$ das Symbol $\Rightarrow$ verwendet wird.

An Stelle von (3.1-3.3) kann also auch

$$x \geq 0 \Leftrightarrow \neg(x < 0)$$

$$x > 0 \Rightarrow x \geq 0$$

$$A \wedge B \Leftrightarrow B \wedge A$$

geschrieben werden.

Das Symbol $\Leftrightarrow$ wird – wie sie bereits aus Abschnitt 2.5 wissen – als *Äquivalenzoperator* bezeichnet und u. a. als „ist äquivalent zu“ gelesen. Das Symbol $\Rightarrow$ kannten Sie sicher schon vorher. Es wird als *Implikationsoperator* bezeichnet und u. a. „impliziert“ oder „daraus folgt“ ausgesprochen.

Definition 3.5: Tautologien der Form $A \Rightarrow B$ heißen *Implikationen*. Tautologien der Form $A \Leftrightarrow B$ heißen *Äquivalenzen*. Dabei stehen A und B für Aussagen oder Aussageformen.

Es ist durchaus üblich das Symbol $\Leftrightarrow$ als „genau dann, wenn“ und $\Rightarrow$ als „wenn . . . , dann“ auszusprechen. Dadurch geht offensichtlich die Unterscheidung zu Bijunktion und Subjunktion verloren. Man kann bspw. an einem „wenn . . . dann“ nicht erkennen, ob damit eine Implikation (also eine Tautologie) oder eine Subjunktion gemeint ist. In solchen Fällen muss die eigentliche Bedeutung aus dem Kontext erschlossen werden. Abschnitt 3.4 wird dies thematisieren.

Noch zwei Hinweise, die die Vielfalt der in der Logik verwendeten Symbole betreffen:

1. Manchmal wird auch $\equiv$ als Symbol für den Äquivalenzoperator geschrieben. Äquivalenzen haben dann entsprechend die Form $A \equiv B$.

2. Wie Sie wissen, werden in *setlX* die Subjunktion mit $\Rightarrow$ und die Bijunktion mit $\Leftrightarrow$ notiert. Auch wenn diese Symbole – leider – ähnlich zu $\Rightarrow$ bzw. $\Leftrightarrow$ aussehen, stehen sie nicht für Implikation bzw. Äquivalenz.

Vielleicht fragen Sie sich nun, was dann die Symbole für Implikation bzw. Äquivalenz in *setlX* sind. Die Antwort ist: Es gibt sie nicht, weil es in *setlX* Implikation und Äquivalenz überhaupt nicht geben kann. Ein Rechner ist zum Rechnen da, in der Logik also zum Auswerten von formallogischen Ausdrücken. Da Implikation und Äquivalenz Tautologien sind, gibt es bei ihnen nichts mehr auszuwerten. Der Wahrheitswert ist ja bereits bekannt.

Eine Implikation ist also eine Subjunktion, die immer wahr ist. Eine Äquivalenz ist eine Bijunktion, die immer wahr ist.

Äquivalenz und Implikation verbinden zwei Aussagen oder Aussageformen miteinander. Sie sind aber keine logischen Operationen! In Abschnitt 2.4.6 haben wir gesehen, dass es genau 16 binäre logische Operationen gibt. Äquivalenz und Implikation sind nicht darunter, schon von daher können sie keine logischen Operationen sein. Sie können auch keine *logischen* Operatoren sein, weil sie nicht zum Gegenstand der Logik gehören. In der Logik geht es ja um das Verknüpfen von Aussagen (bzw. Aussageformen) zu neuen Aussagen. Äquivalenz und Implikation sind nicht Elemente der Sprache der Logik. Es sind *metasprachliche* Begriffe, die wir brauchen um *über* Logik reden zu können.

3.3 Notwendige und hinreichende Bedingungen

Die Implikation $A \Rightarrow B$ kann auch als Schlussfolgerung verstanden werden: Wenn A wahr ist, *muss* auch B wahr sein.

Nochmal zur Erinnerung: Bei der Subjunktion ist das nicht so. Wenn A wahr ist, kann B zwar wahr sein, kann aber auch falsch sein. Im ersten Fall ($A = 1$, $B = 1$) ist $A \rightarrow B$ wahr. Im zweiten Fall ($A = 1$, $B = 0$) ist $A \rightarrow B$ falsch. Das unterscheidet ja gerade die Subjunktion $A \rightarrow B$ von der Implikation $A \Rightarrow B$. Erstere kann falsch sein. Letztere kann als Tautologie nie falsch sein.

Im Zusammenhang mit Schlussfolgerungen werden häufig die Worte Bedingung bzw. Voraussetzung verwendet. Angenommen es gilt $A \Rightarrow B$, welche der beiden Teilaussagen ist dann die Bedingung? A oder B ?



Beispiel 3.6: Betrachten wir die Implikation

n ist ohne Rest durch 4 teilbar $\Rightarrow$ n ist ohne Rest durch 2 teilbar.

Die Teilbarkeit durch 2 ist die Bedingung für die Teilbarkeit durch 4. Nur unter der Voraussetzung, dass eine Zahl durch 2 teilbar ist, kann sie überhaupt durch 4 geteilt werden.

Dagegen ist die Teilbarkeit durch 4 keine Voraussetzung für die Teilbarkeit durch 2. Beispielsweise ist 6 durch 2 teilbar, obwohl 6 nicht durch 4 teilbar ist. Die Bedingung der Teilbarkeit durch 4 muss also nicht erfüllt sein, damit eine Zahl durch 2 geteilt werden kann. ■

Beispiel 3.7: Betrachten wir die Implikation

Wenn man falsch parkt und dabei erwischt wird, bekommt man einen Strafzettel (aber vielleicht auch sonst).

Wenn die Bedingung Falschparken und Erwischtwerden erfüllt ist – wir nennen dies hier verkürzt „Falschparken“ –, bekommt man einen Strafzettel. Falschparken ist also die Voraussetzung dafür, dass man einen Strafzettel bekommt.

Dagegen ist der Erhalt eines Strafzettels keine Bedingung/Voraussetzung dafür, dass man falsch geparkt hat. ■

Was ist hier los? Im ersten Beispiel scheint in $A \Rightarrow B$ die rechte Seite B die Bedingung für A zu sein. Im zweiten Beispiel ist es die linke.

Die Auflösung des Rätsels besteht darin, dass in $A \Rightarrow B$ sowohl A als auch B Bedingungen bzw. Voraussetzungen sind. Allerdings gibt es zwei unterscheidbare und unterscheidungswürdige Bedingungen:

Definition 3.8: In der Implikation $A \Rightarrow B$ wird A die *hinreichende Bedingung* bzw. *hinreichende Voraussetzung* für B genannt. B wird die *notwendige Bedingung* bzw. *notwendige Voraussetzung* für A genannt.

Beispiel 3.9: In der Implikation

$$n \text{ ist durch 4 teilbar} \Rightarrow n \text{ ist durch 2 teilbar}$$

ist die Teilbarkeit durch 4 hinreichende Bedingung für die Teilbarkeit durch 2. Es reicht aus zu wissen, dass eine Zahl durch 4 teilbar ist, um daraus schließen zu können, dass die Zahl durch 2 teilbar ist.

Umgekehrt ist die Teilbarkeit durch 2 die notwendige Bedingung für die Teilbarkeit durch 4. Es ist notwendig, dass eine Zahl durch 2 geteilt werden kann, damit sie auch durch 4 geteilt werden kann. Ist die notwendige Bedingung nicht erfüllt, ist die Zahl sicher nicht durch 4 teilbar. ■

Beispiel 3.10: In der Implikation „Wenn man falsch parkt und dabei erwischt wird, bekommt man einen Strafzettel“ ist das Falschparken die hinreichende Bedingung dafür, dass man einen Strafzettel ausgestellt bekommt. Falschparken ist aber keine notwendige Bedingung dafür. Man kann nämlich auch unter anderen Voraussetzungen einen Strafzettel erhalten. ■

In $A \rightarrow B$ haben beide Seiten den Charakter einer Bedingung. Das können wir auch sehen, wenn wir nochmal zu den Operationen $A \rightarrow B$ und $B \leftarrow A$ zurückgehen. Wegen

$$A \rightarrow B \Leftrightarrow B \leftarrow A$$

sind beide gleichbedeutend.

In Abschnitt 2.4.3 haben Sie die empfohlene Aussprache für $A \rightarrow B$ kennengelernt: Wenn A , dann sicher B (aber eventuell auch sonst). Diese Formulierung macht deutlich, dass A hinreichend für B ist.

Die empfohlene Aussprache für $B \leftarrow A$ ist: Nur wenn B , dann A (aber eventuell auch nicht). Diese Formulierung macht deutlich, dass B notwendig für A ist.

Wegen

$$A \leftrightarrow B \Leftrightarrow (A \rightarrow B) \wedge (B \rightarrow A)$$

(siehe (2.9)) ist jede Tautologie der Form $A \leftrightarrow B$ gleichbedeutend mit der entsprechenden Tautologie der Form $(A \Rightarrow B) \wedge (B \Rightarrow A)$. A ist dann also sowohl notwendige als auch hinreichende Bedingung, ebenso B . Das führt zu folgendem Sprachgebrauch:

Definition 3.11: In einer Äquivalenz $A \leftrightarrow B$ wird A die *notwendige und hinreichende Bedingung* (bzw. *Voraussetzung*) für B genannt. B wird *notwendige und hinreichende Bedingung* (bzw. *Voraussetzung*) für A genannt.

Beispiel 3.12: In der Äquivalenzaussage

Eine Zahl ist dann und nur dann gerade, wenn sie durch 2 teilbar ist

ist die Teilbarkeit durch 2 notwendige und hinreichende Voraussetzung dafür dass die Zahl gerade ist. Umgekehrt ist die notwendige und hinreichende Bedingung für die Teilbarkeit durch 2, dass die Zahl gerade ist. ■

Wegen

$$A \rightarrow B \Leftrightarrow \neg B \rightarrow \neg A$$

(vgl. (2.8)) ist jede Implikation der Form $A \Rightarrow B$ gleichbedeutend mit der entsprechenden Tautologie der Form $\neg B \Rightarrow \neg A$. „ A impliziert B “ ist also gleichbedeutend mit „Das Gegenteil von B impliziert das Gegenteil von A “.

Beispiel 3.13: Die Implikation

Wenn Zahl durch 4 teilbar ist, dann ist sie auch durch 2 teilbar

kann daher auch so formuliert werden:

Wenn Zahl nicht durch 2 teilbar ist, dann ist sie auch nicht durch 4 teilbar.

■

Möglicherweise wäre nun ein passender Zeitpunkt, um auf das zurückzublicken, was Sie bisher gelesen haben, oder eine Pause zu machen.

3.4 Modellieren mit Subjunktion, Bijunktion, Implikation bzw. Äquivalenz

Wir haben jetzt drei Operatoren, die häufig als „wenn ..., dann“ ausgesprochen werden: Implikation und Subjunktion und die Operation mit dem Operator $\leftarrow$. Da ist die Verwechslungsgefahr natürlich hausgemacht. Daher ist es mehr als angebracht Formulierungen zu verwenden, die diese Mehrdeutigkeit vermeiden, also z. B.

- Für $W \rightarrow D$: Wenn W , dann sicher D (aber vielleicht auch sonst).
- Für $W \leftarrow D$: Nur wenn W , dann D (aber vielleicht auch nicht).
- Für $W \Rightarrow D$: W impliziert D .

Wie steht die Sache allerdings beim Modellieren? Woher kann man wissen, ob $W \rightarrow D$, $W \leftarrow D$ oder $W \Rightarrow D$ das angemessene Modell darstellt? Damit $W \Rightarrow D$ angemessen ist, muss definitionsgemäß eine Tautologie vorliegen. Es muss also festgestellt worden sein, dass der vorliegende Wenn-dann-Satz immer wahr ist. Wie in Kapitel 2.3 betont, ist das Feststellen von Wahrheit im Allgemeinen keine Aufgabe der Logik, sondern der wissenschaftlichen Fachdisziplinen.

Beispiel 3.14: „Wenn Zahl durch 4 teilbar ist, dann ist sie auch durch 2 teilbar“ kann erst dann als Implikation

Teilbarkeit durch 4 $\Rightarrow$ Teilbarkeit durch 2

geschrieben werden, wenn die hier zuständige Disziplin, die Zahlentheorie, festgestellt hat, dass eine Tautologie vorliegt. ■

Wenn man weiß, dass keine Tautologie vorliegt und keine klärende Begriffe wie „nur“ oder „eventuell auch sonst“ verwendet werden, hilft eigentlich nur anhand des eigenen Kontextverständnisses herauszufinden, in welchem Fall die Aussage falsch ist, um zwischen $W \rightarrow D$ und $W \leftarrow D$ unterscheiden zu können. Hat man kein Kontextverständnis, kann dies kaum gelingen.

Beispiel 3.15: „Wenn Du zu meiner Party kommst, dann lernst Du nette Leute kennen.“ Unser Kontextverständnis sagt, dass diese Aussage falsch ist, wenn die Aussage W im Wenn-Satz wahr, aber die Aussage im Dann-Satz D falsch ist. Das angemessene Modell hat hier also die Form $W \rightarrow D$. ■

Beispiel 3.16: „Wenn Traxoline gebraktert ist, dann ist sie auch montilliert.“ Das Schlüsselwort „auch“ im Dann-Satz signalisiert, dass eine Subjunktion vorliegt. Das angemessene Modell hat also die Form $W \rightarrow D$. ■

Beispiel 3.17: „Nur wenn Traxoline gebraktert ist, ist sie montilliert.“ Das Schlüsselwort „nur“ im Wenn-Satz signalisiert, dass das angemessene Modell hat die Form $W \leftarrow D$ hat. ■

Beispiel 3.18: Ihre Mutter sagt: „Wenn Du Deinen Teller leer isst, bekommst Du Nachtisch.“ Da Ihre Mutter natürlich (in Erziehungsfragen) immer die Wahrheit sagt, liegt eine Tautologie vor. Diese ist allerdings nicht von der Form $T \Rightarrow N$!

Aussage T (Teller leer essen) ist hier zwar auch hinreichende Bedingung für Nachtisch (Aussage N). Sie ist aber auch notwendige Bedingung! Denn Sie wissen, dass Ihre Mutter mit dem Satz auch folgendes meint: „Nur wenn Du Deinen Teller leer isst, bekommst Du Nachtisch“ bzw. „Wenn Du

Deinen Teller nicht leer isst, bekommst Du keinen Nachtisch.“ Das angemessene Modell ist also $T \Leftrightarrow N$. ■

Das vorausgehende Beispiel zeigt eine weitere Komplikation auf. Wenn-dann-Sätze können nicht nur Subjunktionen oder Implikationen sein, sondern auch (versteckte) Äquivalenzen! Auch hier hilft wieder nur Kontextverständnis, um zum passenden Modell zu kommen.

3.5 Definitionen und Theoreme

In wissenschaftlichen Systemen gibt es zwei wichtige Arten von Tautologien: Definitionen und Theoreme.

Definitionen dienen v. a. als Begriffsvereinbarungen. Dass Definitionen Tautologien sind, ist eigentlich keine spektakuläre Erkenntnis. Dass sie immer wahr sind, kommt auf Grund einer Vereinbarung zustande. Vereinbart wird die Bedeutung des definierten Begriffs. Definitionen sind Vereinbarung von letztendlich willkürlichen¹ Bezeichnungen.

Beispiel 3.19: „Ein rechtwinkliges Dreieck ist ein Dreieck, bei dem der Winkel zwischen zwei Seiten dieses Dreiecks 90° beträgt.“ Diese Definition vereinbart die Bedeutung von „rechtwinklig“ im Kontext Dreiecke.

Um die logische Struktur besser erkennen zu können, formulieren wir den Wortlaut etwas um, ohne die Bedeutung zu ändern: „Ein Dreieck heißt rechtwinklig genau dann, wenn der Winkel zwischen zwei Seiten dieses Dreiecks 90° beträgt.“ Anhand dieser Formulierung sehen Sie hoffentlich, dass hier eine Aussageform vorliegt und dass die Variable „Dreieck“ ist. Sie sehen hoffentlich auch, dass die logische Struktur die einer Bijunktion ist:

Dreieck heißt rechtwinklig. $\leftrightarrow$ Der Winkel zwischen zwei Seiten des Dreieck ist 90° .

Diese Aussageform ist immer wahr, egal welches konkrete Dreieck wir betrachten. Probieren Sie es aus! Fälle $1 \leftrightarrow 0$ und $0 \leftrightarrow 1$ werden nicht auftreten. Um diese Erkenntnis schriftlich zum Ausdruck zu bringen, können wir

Dreieck heißt rechtwinklig. $\Leftrightarrow$ Der Winkel zwischen zwei Seiten des Dreieck ist 90° .

schreiben. Dass eine Tautologie vorliegt, ist, wie gesagt, nicht besonders spektakulär. ■

Nicht ganz so unspektakulär ist, dass Theoreme Tautologien sind. Theoreme werden auch Sätze (wie in „Satz des Pythagoras“) oder Gesetze genannt (wie in „Kommutativgesetz“). Beim Begriff Satz besteht allerdings Verwechslungsgefahr mit dem Alltagssprachlichen bzw. sprachwissenschaftlichen Begriff.

Beispiel 3.20: „In einem rechtwinkligen Dreieck mit Kathetenlängen a und b und Hypotenusenlänge c gilt $a^2 + b^2 = c^2$.“ Das ist der Satz des Pythagoras. Um seine logische Struktur besser sehen zu können, formulieren wir den Wortlaut um zu „Wenn ein Dreieck rechtwinklig ist und die Katheten die Längen a und b haben und die Hypotenuse die Länge c hat, dann gilt $a^2 + b^2 = c^2$.“ Das ist eine Aussageform mit den Variablen a , b und c und es hat die Struktur einer Subjunktion:

Rechtwinkliges Dreieck hat Katheten a und b und Hypotenuse $c \rightarrow a^2 + b^2 = c^2$

Wie immer bei Subjunktionen ist der Fall $1 \rightarrow 0$ besonders interessant. Die Subjunktion „Satz des Pythagoras“ hat die erstaunliche Eigenschaft, dass es keine Werte für a , b , c gibt, die die linke Seite wahr und die rechte falsch machen.

¹Auch wenn Begriffsbezeichnungen willkürlich sind, ist es hilfreich, wenn sie sinnvoll gewählt werden. Natürlich könnte man $A \Leftrightarrow B$ als die Brakterisierung von A und B definieren. Das ist aber kaum sinnvoll. Zum einen hat der Begriff Brakterisierung keinen Bezug zur Idee der Gleichwertigkeit, was der lateinisch stämmige Begriff Äquivalenz treffend ausdrückt. Zum anderen steht Brakterisierung im allgemeinen deutschen Sprachverständnis wegen der Endung „-ierung“ für einen Prozess. $A \Leftrightarrow B$ ist aber kein (bzw. nicht nur) Prozess, sondern ein Objekt. Wenn schon „Brakter-“, dann „Brakterität“!

Es gibt also keine Fälle, in der diese Subjunktion als ganzes den Wahrheitswert 0 hat. Es liegt also eine Tautologie vor. Um das deutlich sichtbar zu kommunizieren, können wir

$$\text{Rechtwinkliges Dreieck hat Katheten } a \text{ und } b \text{ und Hypotenuse } c \Rightarrow a^2 + b^2 = c^2$$

schreiben. ■

Die Wahrheit von Theoremen liegt nicht auf der Hand. Um die Wahrheit eines Theorems zu sehen, ist Arbeitsaufwand nötig, nämlich ein Beweis bzw. wissenschaftlicher Nachweis. Anders bei Definitionen. Diese sind per Konstruktion wahr.

Zu beweisen, dass eine Aussageform ein Theorem ist, gelingt nur sehr selten dadurch, die Variablen der Aussageform mit allen möglichen Werten zu belegen und festzustellen, dass alle so entstehenden Aussagen wahr sind. Beispielsweise beim Satz von Pythagoras gibt es einfach zu viele mögliche Werte für a , b und c , um alle Möglichkeiten einzeln durchzugehen.

Beispiel 3.21: „Die Gleichung $ax = b$ ist genau dann eindeutig nach x lösbar, wenn $a \neq 0$ “ ist ein Theorem, das als Äquivalenz formuliert ist: $ax = b$ eindeutig lösbar $\Leftrightarrow a \neq 0$. ■

Theoreme können die Form von Implikationen oder Äquivalenzen haben. Definitionen sind dagegen immer Äquivalenzen. Sie folgen dem Muster

$$\text{Objekt ist ein } \dots \Leftrightarrow \text{Objekt erfüllt definierende Eigenschaften.}$$

Trotzdem werden sie häufig als Wenn-dann-Satz formuliert und nicht etwa als Genau-dann-wenn-Satz. Aus dem Kontext Definition ergibt sich dann, dass der Wenn-dann-Satz für eine Äquivalenz steht (vgl. Abschnitt 3.4).

Als Begriffsvereinbarungen verwenden Definitionen häufig die Worte „heißt“ oder „wird genannt“ und sind in diesen Fällen an diesen Schlüsselworten leicht als solche zu erkennen.

Beispiel 3.22: „Eine Zahl heißt gerade, wenn sie durch 2 teilbar ist“ ist eine mögliche Definition des Begriffs gerade Zahl. Sie ist eine Äquivalenzaussage, denn die Geradheit einer Zahl impliziert ihre Teilbarkeit durch 2 und umgekehrt impliziert ihre Teilbarkeit durch 2, dass sie gerade ist. ■

3.6 Äquivalenzumformungen

Besonders in der Algebra wird ein Zeichen benötigt, das den Zusammenhang zwischen aufeinanderfolgenden bzw. umgeformten Gleichungen ausdrücken soll. Gleichungen sind aus Sicht der Logik Aussagen oder Aussageformen. Wenn eine Gleichung umgeformt wird, soll natürlich der Wahrheitswert der Gleichung unverändert bleiben. Der Äquivalenzoperator ist genau das passende Symbol um die Ausgangsgleichung mit der neuen (dazu äquivalenten!) Gleichung in Bezug zu setzen.

Beispiel 3.23: Beim Lösen der folgenden Gleichung treten die darunter stehenden äquivalenten Gleichungen auf:

$$\begin{aligned} & ax^2 + bx = -c \\ \Leftrightarrow & ax^2 + bx + c = 0 \\ \Leftrightarrow & x = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \vee x = \frac{-b - \sqrt{b^2 - 4ac}}{2a} \end{aligned}$$

■

Ein häufig zu beobachtender Fehler ist, dass Studierende stereotyp den Implikationspfeil verwenden, um einen Zusammenhang zwischen zwei Aussagen auszudrücken. Stünden im obigen Beispiel statt Äquivalenzpfeilen nur Implikationspfeile, wäre das buchstäblich die halbe Wahrheit.

Beispiel 3.24: Natürlich gibt es auch Umformungen bei einer Rechnung, die nicht äquivalent sind.

$$x = 1 \Rightarrow x^2 = 1$$

ist ein Beispiel. Hier könnte nicht $x = 1 \Leftrightarrow x^2 = 1$ geschrieben werden, denn die Belegung $x = -1$ macht die linke Seite falsch, aber die rechte wahr, im Widerspruch zu behaupteten Äquivalenz. ■

3.7 Lernziele

Studierende	Kennen	Können	Verstehen
fachlich	☐ erläutern Unterschied zwischen Implikation und Subjunktion bzw. Äquivalenz und Bijunktion ☐ erläutern Unterschied zwischen notwendiger und hinreichender Bedingung	☐ bestimmen, ob ein logischer Ausdruck aus formalen Gründen eine Tautologie oder Kontradiktion ist (in Abgrenzung zu „beweisbedürftigen Tautologien“) ☐ verwenden die Begriffe notwendig und hinreichend definitionsgemäß ☐ verwenden die Symbole $\rightarrow$, $\leftrightarrow$, $\Rightarrow$ und $\Leftrightarrow$ gemäß ihrer Bedeutung	☐ entscheiden, ob ein Wenn-dann-Satz als Subjunktion, Bijunktion, Implikation oder Äquivalenz zu modellieren ist ☐ identifizieren notwendige und hinreichende Bedingungen
methodisch	☐ erläutern Gemeinsamkeiten und Unterschiede von Definitionen und Sätzen	☐ begründen Rechenschritte mittels Äquivalenzumformungen	☐ erkennen, ob eine Aussage eine Definition oder ein Theorem ist ☐ formulieren und interpretieren mathematische Aussagen, insbesondere Definitionen und Theoreme, als logische Aussagen

Hinzu kommen die themenübergreifenden Lernziele aus den Kategorien „sozial“ und „persönlich“ aus Abschnitt 1.7 sowie die Lernziele aus der Kategorie „methodisch“ aller vorangegangener Kapitel. Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben.

3.8 Übungsaufgaben

- Für welche Belegungen von x ist die Aussageform $x = \pm 1$ wahr? Modellieren Sie dazu $x = \pm 1$ als formallogischen Ausdruck.
- Modellieren Sie:
 - Wenn heute Montag ist, dann ist morgen Mittwoch.
 - `if (x<0) then x:=-x;`
 - Nur wenn das Wetter schön ist, gehen wir ins Freibad.
 - Vorausgesetzt es schneit nicht, komme ich morgen zu Dir.
 - Ein Quadrat ist ein spezielles Rechteck.
 - Wenn eine Zahl ungerade ist, bleibt bei Division durch 2 der Rest 1.
 - Wenn dies ein endlicher Automat berechnen kann, kann es auch eine Turing-Maschine berechnen.
 - Wenn dies ein deterministischer endlicher Automat berechnen kann, kann es auch ein nichtdeterministischer Automat berechnen.
- Vorausgesetzt, die folgenden Aussagen sind Tautologien, handelt es sich dann um Definitionen oder Sätze?

- (a) Eine Relation R auf M ist eine Teilmenge des kartesischen Produkts von M mit sich selbst.
- (b) Eine Funktion heißt bijektiv, wenn sie injektiv und surjektiv ist.
- (c) Eine Funktion ist dann und nur dann invertierbar, wenn Sie bijektiv ist.
- (d) Eine Sprache heißt regulär, wenn es einen deterministischen endlichen Automaten gibt, der sie erkennt.
- (e) Zu jeder regulären Sprache, gibt es einen nichtdeterministischen endlichen Automaten, der sie erkennt.
- (f) Ein deterministischer endlicher Automat ist ein Fünftupel $(Q, \Sigma, \delta, q_0, F)$ mit den Eigenschaften ... (Die Eigenschaften sind hier nicht genannt.)
- (g) Die Konjunktion zweier Aussagen ist nur dann wahr, wenn beide Aussagen wahr sind.
- (h) Ein Dreieck heißt ostfälisch, wenn die Längen a, b, c seiner drei Seiten die Gleichung $a^2 + b^2 = c^2$ erfüllen.
- (i) Wenn ein Dreieck ostfälisch ist, dann ist der Winkel zwischen seinen beiden kurzen Seiten 90° .

Kapitel 4

Aussagenlogisches Rechnen

4.1 Aufwärmübungen

1. Zeigen Sie sowohl mittels Wahrheitstabelle als auch *setlX*, dass die folgenden Aussageformen Tautologien sind:

$$\begin{aligned}A \wedge (B \wedge C) &\leftrightarrow (A \wedge B) \wedge C \\A \vee (B \wedge C) &\leftrightarrow (A \vee B) \wedge (A \vee C)\end{aligned}$$



2. Angenommen die drei Aussagen $P_1 \rightarrow P_2$, $P_2 \rightarrow P_3$, $P_3 \rightarrow P_4$ sind alle wahr. (Wir könnten also auch $P_1 \Rightarrow P_2$, $P_2 \Rightarrow P_3$, $P_3 \Rightarrow P_4$ schreiben.)
 - (a) Kann man dann eine Aussage über den Wahrheitswert von $P_1 \rightarrow P_4$ machen? Welche und warum?
 - (b) Erstellen Sie eine Wahrheitstabelle für die vier Aussagen. Streichen Sie dann alle Zeilen, in denen $P_1 \rightarrow P_2$ den Wert 0 hat, dann alle in denen $P_2 \rightarrow P_3$ den Wert 0 hat, und schließlich alle in denen $P_3 \rightarrow P_4$ den Wert 0 hat, Stimmen die übrig gebliebenen Zeilen mit Ihrer Antwort aus Teil (a) überein?
 - (c) Gibt es einen kürzeren Weg, um die Frage aus Teil (a) zu beantworten, der ohne das Aufstellen der ganzen Wahrheitstabelle auskommt?



3. Leitfragen:

- (a) Wieso kann man mit $\vee$ und $\wedge$ fast genauso rechnen wie mit $\cdot$ und $+$?
- (b) Wie kann man Faulheit in der Logik vorteilhaft nutzen?
- (c) Warum müssen Computer nicht mehr als eine einzige logische Operationen kennen?

4.2 Rechnen mit Wahrheitstabellen

Sie haben bereits mehrfach gesehen, dass man mit logischen Operatoren rechnen kann. Wir haben mittels Wahrheitstabellen mehrere Beziehungen entdeckt, die die Gleichwertigkeit verschiedener logischer Ausdrücke ausdrücken. (2.10) und (2.5) sind Beispiele.

Das Vorgehen mittels Wahrheitstabelle ist aber recht umständlich. Zum einen werden die benötigten Tabellen mit wachsender Anzahl von Variablen schnell riesig. Zum anderen braucht man eine Vermutung, um diese dann mittels Wahrheitstabelle zu verifizieren oder zu falsifizieren. Es ist nicht praktikabel, für jeden logischen Ausdruck, wie z. B.

$$A \vee (\neg A \wedge B) \vee \neg B,$$

erst eine Vermutung für einen gleichwertigen Ausdruck aufzustellen, um diese dann mittels Wahrheitstabelle zu überprüfen.

Die gute Nachricht ist, dass man mit logischen Operatoren symbolisch rechnen kann, genauso wie Sie das bspw. aus der Algebra mit Addition, Multiplikation, Potenz usw. kennen. Aufgrund entsprechender Rechenregeln lässt sich obiger Ausdruck dann auch schnell vereinfachen:

$$A \vee (\neg A \wedge B) \vee \neg B \Leftrightarrow 1 \quad (4.1)$$

Wir schauen uns zunächst diese Regeln an.

4.3 Algebraische Eigenschaften logischer Operatoren

Konjunktion, Disjunktion und Negation haben folgende Eigenschaften:

neutrales Element	$A \vee 0 \Leftrightarrow A$	(4.2)	$A \wedge 1 \Leftrightarrow A$	(4.3)
<i>tertium non datur</i> ¹	$A \vee \neg A \Leftrightarrow 1$	(4.4)	$A \wedge \neg A \Leftrightarrow 0$	(4.5)
faule Auswertung	$A \vee 1 \Leftrightarrow 1$	(4.6)	$A \wedge 0 \Leftrightarrow 0$	(4.7)
Kommutativität	$A \vee B \Leftrightarrow B \vee A$	(4.8)	$A \wedge B \Leftrightarrow B \wedge A$	(4.9)
Assoziativität	$A \vee (B \vee C) \Leftrightarrow (A \vee B) \vee C$	(4.10)	$A \wedge (B \wedge C) \Leftrightarrow (A \wedge B) \wedge C$	(4.11)
Distributivität	$A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$	(4.12)	$A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$	(4.13)
Idempotenz	$A \vee A \Leftrightarrow A$	(4.14)	$A \wedge A \Leftrightarrow A$	(4.15)

¹Lateinisch für „eine dritte Möglichkeit gibt es nicht“ – eine dritte Möglichkeit neben A und $\neg A$

Absorption

$$A \vee (A \wedge B) \Leftrightarrow A \quad (4.16)$$

$$A \wedge (A \vee B) \Leftrightarrow A \quad (4.18)$$

$$A \vee (\neg A \wedge B) \Leftrightarrow A \vee B \quad (4.17)$$

$$A \wedge (\neg A \vee B) \Leftrightarrow A \wedge B \quad (4.19)$$

de Morgan'sche Regeln

$$\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B \quad (4.20)$$

$$\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B \quad (4.21)$$

doppelte Negation

$$\neg\neg A \Leftrightarrow A \quad (4.22)$$

Die verwendeten Begriffe sind an dieser Stelle erst einmal sekundär. Einige werden erst in Kapitel 12 erläutert werden.

Welche übergeordnete Struktur fällt Ihnen auf, wenn Sie die linke und rechte Spalte dieser Auflistung analysieren?



Wenn man links Konjunktionen und Disjunktionen sowie $\mathbb{1}$ und $\mathbb{0}$ vertauscht, erhält man genau die Ausdrücke rechts (und natürlich auch umgekehrt).

An dieser Stelle ist für Sie noch mal eine gute Gelegenheit zu üben, die Gleichwertigkeit von Ausdrücken mittels Wahrheitstabelle zu zeigen. Wir tun dies gemeinsam exemplarisch für (4.16):

A	B	$A \wedge B$	$A \vee (A \wedge B)$	$A \vee (A \wedge B) \Leftrightarrow A$
$\mathbb{0}$	$\mathbb{0}$	$\mathbb{0}$	$\mathbb{0}$	$\mathbb{1}$
$\mathbb{0}$	$\mathbb{1}$	$\mathbb{0}$	$\mathbb{0}$	$\mathbb{1}$
$\mathbb{1}$	$\mathbb{0}$	$\mathbb{0}$	$\mathbb{1}$	$\mathbb{1}$
$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$

Oder mittels *setlX*:

```
=> ausdruck:=procedure(A,B){return A || (A && B) <==> A;};
~< Result: procedure(A, B) { return A || A && B <==> A; } >~
=> ausdruck(false,false);
~< Result: true >~
=> ausdruck(false,true);
~< Result: true >~
=> ausdruck(true,false);
~< Result: true >~
=> ausdruck(true,true);
~< Result: true >~
```

Wie gesagt, nutzen Sie jetzt die Gelegenheit einige der obigen Ausdrücke zu überprüfen!

Eine Besonderheit aller auf S. 53 aufgelisteten Beziehungen ist, dass nur Negation, Konjunktion und Disjunktion involviert sind. Was ist mit den anderen logischen Operatoren?

Nun, von Bijunktion und Subjunktion sollten Sie bereits wissen, dass diese alleine durch diese drei Operationen ausgedrückt werden können. Dies gilt auch für alle anderen Operatoren. Dies wird Gegenstand von Abschnitt 4.7 sein. Wenn also andere Operationen auftreten, ist eine gute Strategie für das symbolische Rechnen, diese erst einmal durch Negation, Konjunktion und Disjunktion auszudrücken.

4.4 Symbolisches Rechnen mit logischen Ausdrücken

Wir machen unsere ersten ernsthaften Gehversuche in Sachen symbolisches Rechnen mit logischen Ausdrücken, indem wir (4.1) auf diese Art nachweisen. Die verwendete Regel ist dabei immer im

Nachhinein angegeben.

Beispiel 4.1:

$$\begin{aligned}
 & A \vee (\neg A \wedge B) \vee \neg B \\
 \Leftrightarrow & A \vee B \vee \neg B \quad | \quad \text{durch Absorption} \\
 \Leftrightarrow & A \vee \mathbb{1} \quad | \quad \text{durch } \textit{tertium non datur} \\
 \Leftrightarrow & \mathbb{1} \quad | \quad \text{durch faule Auswertung}
 \end{aligned}$$

Damit ist die Korrektheit von (4.1) gezeigt. ■

Natürlich gibt es mehr als einen Weg, um ans Ziel zu kommen. Hier sind zwei weitere. Vollziehen Sie diese wieder Schritt für Schritt nach:

Beispiel 4.2:

$$\begin{aligned}
 & A \vee (\neg A \wedge B) \vee \neg B \\
 \Leftrightarrow & A \vee \neg B \vee (\neg A \wedge B) \quad | \quad \text{Kommutativität} \\
 \Leftrightarrow & \neg \neg A \vee \neg B \vee (\neg A \wedge B) \quad | \quad \text{doppelte Negation} \\
 \Leftrightarrow & \neg(\neg A \wedge B) \vee (\neg A \wedge B) \quad | \quad \text{de Morgan} \\
 \Leftrightarrow & \mathbb{1} \quad | \quad \textit{tertium non datur}
 \end{aligned}$$
■

Beispiel 4.3:

$$\begin{aligned}
 & A \vee (\neg A \wedge B) \vee \neg B \\
 \Leftrightarrow & [(A \vee \neg A) \wedge (A \vee B)] \vee \neg B \quad | \quad \text{Distributivität} \\
 \Leftrightarrow & [\mathbb{1} \wedge (A \vee B)] \vee \neg B \quad | \quad \textit{tertium non datur} \\
 \Leftrightarrow & (A \vee B) \vee \neg B \quad | \quad \text{neutrales Element} \\
 \Leftrightarrow & A \vee (B \vee \neg B) \quad | \quad \text{Assoziativität} \\
 \Leftrightarrow & A \vee \mathbb{1} \quad | \quad \textit{tertium non datur} \\
 \Leftrightarrow & \mathbb{1} \quad | \quad \text{faule Auswertung}
 \end{aligned}$$
■

Die nächsten Gehversuche machen wir mit

$$(A \rightarrow B) \wedge (B \rightarrow C) \rightarrow (A \rightarrow C)$$

und schauen, ob wir diesen Ausdruck vereinfachen können.

Beispiel 4.4: Hier sind Subjunktionen involviert. Diese werden zuerst durch Negation und Disjunktion ausgedrückt:

$$\begin{aligned}
 & (A \rightarrow B) \wedge (B \rightarrow C) \rightarrow (A \rightarrow C) \\
 \Leftrightarrow & \neg[(\neg A \vee B) \wedge (\neg B \vee C)] \vee (\neg A \vee C) \quad | \quad \text{mittels (2.10)} \\
 \Leftrightarrow & \neg(\neg A \vee B) \vee \neg(\neg B \vee C) \vee \neg A \vee C \\
 \Leftrightarrow & (A \wedge \neg B) \vee (B \wedge \neg C) \vee \neg A \vee C \\
 \Leftrightarrow & \neg A \vee (A \wedge \neg B) \vee C \vee (\neg C \wedge B) \\
 \Leftrightarrow & \neg A \vee \neg B \vee C \vee B \quad | \quad \text{durch Absorption} \\
 \Leftrightarrow & \neg A \vee (\neg B \vee B) \vee C \quad | \quad \text{durch Kommutativität und Assoziativität} \\
 \Leftrightarrow & \neg A \vee \mathbb{1} \vee C \\
 \Leftrightarrow & \mathbb{1}
 \end{aligned}$$

Hier wurde nicht mehr jeder einzelne Schritt begründet – Gelegenheit für Sie, auf S. 53 die jeweils verwendeten Beziehung zu identifizieren.

Die Erkenntnis ist also, dass

$$(A \rightarrow B) \wedge (B \rightarrow C) \rightarrow (A \rightarrow C) \Leftrightarrow \mathbb{1} \quad (4.23)$$

Der Ausgangsausdruck ist also eine Tautologie. Wir können daher auch

$$(A \rightarrow B) \wedge (B \rightarrow C) \Rightarrow (A \rightarrow C)$$

schreiben. In Worten: Aus „Wenn A , dann B “ und „Wenn B , dann C “ folgt „Wenn A , dann C “. ■

Es geht hier nicht darum, dass Sie flüssig im symbolischen Rechnen mit logischen Operatoren werden. Sie können ja die Liste auf S. 53 als Hilfsmittel verwenden. Es ist hier aber wie beim Sprachen lernen: Wer auf Dauer jedes Wort nachschlagen muss, wird nicht weit kommen.

Möglicherweise wäre nun ein passender Zeitpunkt, um auf das zurückzublicken, was Sie bisher gelesen haben, oder eine Pause zu machen.

4.5 Faule Auswertung

Wenn Variablensymbole nicht mit Werten belegt sind, aber in einem logischen Ausdruck auftreten, kann `setlX` diesen natürlich nicht auswerten. Das folgende Beispiel illustriert dies für $X \vee \mathbb{1}$:

```
=> X || true;
Error in "X || true":
Left-hand-side of 'om || true' is not a Boolean value.
Replay:
1.2: X || true FAILED
1.1: X <~> om

=> X:=false; X || true
~< Result: true >~
```

$X \vee \mathbb{1}$ kann nicht ausgewertet werden, solange X nicht mit einem Booleschen Wert belegt ist. Bei anderen Ausdrücken funktioniert dies aber:

```
=> false && X;
~< Result: false >~

=> true || X;
~< Result: true >~
```

Das sind genau die Eigenschaften (4.6) und (4.7) von Disjunktion bzw. Konjunktion. Wenn $\mathbb{0}$ in einer Konjunktion auftritt, ist aufgrund ihrer Definition „nur wahr, wenn beide Teilaussagen wahr“ sofort klar, dass die Gesamtaussage falsch sein muss. Entsprechend folgt aus der Definition der Disjunktion, dass diese wahr sein muss, wenn $\mathbb{1}$ in ihr vorkommt. Das ist die sogenannte *faule Auswertung* (*lazy evaluation*) von Konjunktion bzw. Disjunktion, von der die meisten Programmiersprachen Gebrauch machen. Die faule Auswertung erlaubt Ausdrücke der Form $\mathbb{1} \vee \text{Rest}$ und $\mathbb{0} \wedge \text{Rest}$ auszuwerten, ohne dass der Wahrheitswert von *Rest* berechnet werden muss.

Da die Booleschen Operatoren von links nach rechts ausgewertet werden², funktioniert dies aber nicht für Ausdrücke der Form $\text{Rest} \wedge \mathbb{0}$ bzw. $\text{Rest} \vee \mathbb{1}$. Hier muss der *Rest* zuerst ausgewertet werden.

4.6 Allgemeine Strukturen

Negation, Konjunktion und Disjunktion haben Eigenschaften, die auch andere Operatoren außerhalb der Logik haben. Wir haben es also mit Strukturen zu tun, die themenübergreifend sind. Dazu gehören bspw. die Assoziativität und Kommutativität.

²Diese Eigenschaft wird als *Linksassoziativität* bezeichnet.

Kommutativität bedeutet, dass die Operanden einer binären Operation vertauscht werden können, ohne dass sich der Wert des Ausdrucks ändert. Sie kennen sie auch von der Addition und Multiplikation reeller Zahlen:

$$\begin{aligned}x + y &= y + x \\x \cdot y &= y \cdot x\end{aligned}$$

Assoziativität bedeutet quasi „freie Partnerwahl“ (freie Wahl des Sozies). In

$$(A \wedge B) \wedge C \Leftrightarrow A \wedge (B \wedge C)$$

ist es egal, ob sich B erst mit dem linken Partner A verbandelt und das Ergebnis dann mit C oder ob B erst mit dem rechten Partner C anbandelt und das Ergebnis dann mit A . Wenn die „Reihenfolge der Partnerwahl“ keine Rolle spielt, kann man die Klammern, die die Reihenfolge ja erzwingen, auch weglassen:

$$(A \wedge B) \wedge C \Leftrightarrow A \wedge B \wedge C \quad \text{bzw.} \quad A \wedge (B \wedge C) \Leftrightarrow A \wedge B \wedge C$$

Sie kennen diese Eigenschaft ebenfalls von Addition und Multiplikation reeller Zahlen:

$$\begin{aligned}(x + y) + z &= x + (y + z) = x + y + z \\(x \cdot y) \cdot z &= x \cdot (y \cdot z) = x \cdot y \cdot z\end{aligned}$$

Natürlich können nicht alle Operationen assoziativ oder kommutativ sein. Dann wären es keine besonderen Eigenschaften, die jeweils einen eigenen Namen haben. Die Subjunktion ist bspw. weder kommutativ noch assoziativ. Wenn es so wäre, müssten

$$\begin{aligned}(A \rightarrow B) &\leftrightarrow (B \rightarrow A) \\((A \rightarrow B) \rightarrow C) &\leftrightarrow (A \rightarrow (B \rightarrow C))\end{aligned}$$

beides Tautologien sein, was aber nicht der Fall ist. Sie haben drei Möglichkeiten, dies zu sehen: Wahrheitstabelle, Auswerten in *setlX*, Vereinfachen durch symbolisches Rechnen. Da wir hier sehen wollen, dass etwas nicht gilt, kann ein vierter Weg allerdings schneller sein: Belegungen für die Variablen finden, für die die Ausdrücke falsch werden. Dann können es keine Tautologien sein. Entscheiden Sie, welchen Weg Sie hier gehen wollen.



Eine weitere themenübergreifende Eigenschaft, die ab Kapitel 7 eine wichtige Rolle spielen wird, ist die sogenannte Transitivität.

Eine binäre logische Operation $\diamond$ heißt transitiv, wenn gilt: Wenn $A \diamond B$ und $B \diamond C$, dann auch $A \diamond C$.

Die Struktur der Definition ist besser zu verstehen, wenn wir sie formalisieren:

$$\diamond \text{ ist transitiv} \Leftrightarrow A \diamond B \wedge B \diamond C \rightarrow A \diamond C$$

Beispiel 4.5: Die Subjunktion ist eine transitive Operation, weil $(A \rightarrow B) \wedge (B \rightarrow C) \rightarrow (A \rightarrow C)$ eine Tautologie ist, wie wir in Abschnitt 4.4 mit (4.23) bereits nachgewiesen haben. ■

Weil die Subjunktion transitiv ist, ist auch die Implikation transitiv. Die beiden Implikationen $A \Rightarrow B$ und $B \Rightarrow C$ können also zu $A \Rightarrow C$ verkürzt werden. Auch die Bijunktion ist transitiv und damit auch die Äquivalenz.

4.7 Minimalset binärer logischer Operatoren

Die 16 binären logischen Operatoren sind nicht unabhängig von einander. Sie können alle durch eine geeignete Kombination aus $\neg$, $\wedge$ und $\vee$ ausgedrückt werden. Sie kennen das bereits von der Subjunktion: $A \rightarrow B \Leftrightarrow \neg A \vee B$. Wir schauen uns hier als weiteres Beispiel die Alternation an.

Beispiel 4.6: Wir führen die Alternation erst auf die Bijunktion zurück, diese dann auf die Subjunktion und letztere schließlich auf Negation und Disjunktion:

$$\begin{aligned} A \nleftrightarrow B &\Leftrightarrow \neg(A \leftrightarrow B) \\ &\Leftrightarrow \neg((A \rightarrow B) \wedge (B \rightarrow A)) \\ &\Leftrightarrow \neg((\neg A \vee B) \wedge (\neg B \vee A)) \end{aligned}$$

Damit ist gezeigt, dass die Alternation alleine durch Negation, Konjunktion und Disjunktion ausgedrückt werden kann. Der Ausdruck hat allerdings Potential vereinfacht zu werden:

$$\begin{aligned} A \nleftrightarrow B &\Leftrightarrow \neg((\neg A \vee B) \wedge (\neg B \vee A)) \\ &\Leftrightarrow \neg(\neg A \vee B) \vee \neg(\neg B \vee A) \\ &\Leftrightarrow (A \wedge \neg B) \vee (B \wedge \neg A) \end{aligned}$$

Das ist ein recht gute Beschreibung dessen, was auch die Wahrheitstabelle der Alternation aussagt: $A \nleftrightarrow B$ ist genau dann wahr, wenn A und nicht B oder B und nicht A wahr sind. ■

Dass logische Ausdrücke alleine durch Negationen, Konjunktionen und Disjunktionen ausgedrückt werden können, erlaubt im nächsten Schritt logische Ausdrücke als Konjunktion von Disjunktionen und Negationen auszudrücken. Das führt auf die konjunktive Normalform logischer Ausdrücke. Alternativ kann ein logischer Ausdruck als Disjunktion von Konjunktionen und Negationen geschrieben werden – in der disjunktiven Normalform. Diese Normalformen spielen eine Rolle an verschiedenen Stellen der Informatik – prominent in der Digitaltechnik.

Alle binären logischen Operatoren können also auf eine geeignete Kombination der Operatoren $\neg$, $\wedge$ und $\vee$ zurückgeführt werden. Diese minimale Menge benötigter logischer Operatoren kann sogar noch kleiner gewählt werden. Alle logischen Operatoren lassen sich durch einen einzigen ausdrücken. Dies funktioniert wahlweise mit unseren Operatoren $\diamond_8$ und $\diamond_{14}$ aus Abschnitt 2.4.6. Der erstgenannte wird gewöhnlich als *nor* bezeichnet, der andere als *nand*. Wir verwenden hier *nand* und verwenden $\overline{\wedge}$ als Symbol.

A	B	$A \overline{\wedge} B$
0	0	1
0	1	1
1	0	1
1	1	0

Negation, Konjunktion und Disjunktion lassen sich jeweils alleine durch *nand* formulieren, weshalb alle logischen Operatoren alleine durch diesen ausgedrückt werden können:

$$\begin{aligned} \neg A &\Leftrightarrow A \overline{\wedge} A \\ A \wedge B &\Leftrightarrow (A \overline{\wedge} B) \overline{\wedge} (A \overline{\wedge} B) \\ A \vee B &\Leftrightarrow (A \overline{\wedge} A) \overline{\wedge} (B \overline{\wedge} B) \end{aligned}$$

Nutzen Sie auch hier die Gelegenheit das Rechnen mit logischen Ausdrücken zu üben, indem Sie diese drei Beziehungen z. B. via Wahrheitstabellen oder *setlX* verifizieren.

4.8 Lernziele

Studierende	Kennen	Können	Verstehen
fachlich	☐ benennen Bedeutung von Kommutativität, Assoziativität und Distributivität ☐ benennen Bedeutung von Transitivität ☐ kennen das Konzept der faulen Auswertung	☐ vereinfachen formallogische Ausdrücke durch symbolisches Rechnen ☐ schreiben formallogische Ausdrücke alleine unter Verwendung von Negation, Disjunktion und Konjunktion	☐ stellen fest, ob eine Operation kommutativ oder assoziativ ist ☐ stellen fest, ob eine Operation transitiv ist ☐ sagen voraus, ob ein Ausdruck faul ausgewertet werden kann

Hinzu kommen die themenübergreifenden Lernziele aus den Kategorien „sozial“ und „persönlich“ aus Abschnitt 1.7 sowie die Lernziele aus der Kategorie „methodisch“ aller vorangegangener Kapitel. Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben.

4.9 Übungsaufgaben

1. Schreiben Sie einen logischen Ausdruck, der nur die Operatoren $\neg$ und $\wedge$ verwendet und gleichwertig zu $A \vee B$ ist.
2. Welche der sechs fundamentalen logischen Operatoren aus Tabelle 2.1 sind kommutativ? Welche sind assoziativ? Beweisen Sie Ihre Antworten. Gibt es einen Zusammenhang zwischen Kommutativität und Art des Operatorsymbols? Was können Sie über Kommutativität und Assoziativität der Negation sagen?
3. Zeigen Sie sowohl mittels Wahrheitstabelle, formaler Rechnung als auch mit Hilfe von *setLX* dass $A \rightarrow B \Leftrightarrow \neg B \rightarrow \neg A$ ist.
4. Welche der 16 binären logischen Operatoren sind assoziativ, welche sind kommutativ?
5. Weisen Sie nach, dass die Bijunktion transitiv ist.
6. Welche Vergleichsoperatoren ($=$, $>$, $\leq$ etc.) sind transitiv?
7. Kann *setLX* $0 \rightarrow X$ faul auswerten? Begründen Sie Ihre Vermutung und überprüfen Sie sie anschließend in *setLX*.
8. Drücken Sie jeden der 16 binären Operationen alleine durch Negation, Konjunktion und Disjunktion aus.

Bearbeiten Sie auch Übungsaufgaben aus anderen Lehrtexten, z. B. die Kontrollfragen und Übungsaufgaben zu Abschnitt 1.3 im Lehrbuch von Teschl und Teschl.

Kapitel 5

Mengen

5.1 Aufwärmübungen

1. Welche der folgenden Mengen sind gleich? Schreiben Sie *zuerst* Ihre Antworten auf und überprüfen Sie sie *anschließend* in *setlX* via `A==B`; `A==C`; etc. Erklären Sie Ihre Antworten und eventuelle Unterschiede zwischen diesen und denen des Computers.

```
A:={n: n in {-4..10} | n%2==0};
B:={2*(n-2): n in {0..7}};
C:={(n-30)/5: n in {10,20..80}};
D:={-4,-2..10};
E:={4*x-4: x in {0, 1/10 .. 35/10} | isInteger(4*x-4) };
F:={floor( ((n+65)**2)/70 - 66): n in {1..8}};
G:={x+y: x in {-3,-1..5}, y in {-3,-1..5}};
H:={0,4,-2,10,0,8,-4,6,-2,2};
```

	=	A	B	C	D	E	F	G	H
A									
B									
C									
D									
E									
F									
G									
H									

2. Wie viele Elemente haben die folgenden Mengen? Schreiben Sie *zuerst* Ihre Antworten auf und berechnen Sie *anschließend* mit *setlX* via `#S1`; etc. die jeweilige Anzahl der Elemente. Erklären Sie Ihre Antworten und eventuelle Unterschiede zwischen diesen und denen des Computers.

```
S1 := {0,1,true}
S2 := {{10,20,30,40}}
S3 := { }
S4 := {S3,10,20,30,40}
S5 := {10,20..50}
S6 := { {1..n} : n in {2..8} }
S7 := {10, 20, [30..50]}
S8 := {{true, 'true'}, 2, 3, {2, 3}}
S9 := {S2!=S4, true, #S3==0}
S10 := {S4<=S2, S3<=S4, S3 in S4, S3 <= S2}
```

$S_{11} := \{1, \{1, \{1\}\}\}$
 $S_{12} := \{ \{1\} \leq S_{11}, \{1\} \text{ in } S_{11} \}$

	#S1	#S2	#S3	#S4	#S5	#S6	#S7	#S8	#S9	#S10	#S11	#S12
 meine Antwort												
setlX												

3. Welche der folgenden Mengen enthalten $\{1, 2\}$?

- (a) $S_1 = \{1, 2\}$
- (b) $S_2 = \{1, 2, 3, 4\}$
- (c) $S_3 = \{1, \{1, 2\}, 2\}$
- (d) $S_4 = \{\{1, 2\}\}$



4. Schreiben Sie alle Teilmengen der Menge $\{1, 2, 3\}$ auf.



5. Leitfragen:

- (a) Warum ist die leere Menge Teilmenge, aber nicht Element einer jeden Menge?
- (b) Warum ist es nicht sinnvoll zu sagen, dass „etwas in einer Menge enthalten ist“?
- (c) Warum hat die Menge $\{a, b\}$ nicht immer zwei Elemente?

5.2 Mengen und andere Kollektionen

Was eine Menge von anderen Arten von Kollektionen unterscheidet, haben Sie schon in Kapitel 1 erarbeitet.

Solange es bei Kollektionen auf Vielfachheit und Reihenfolge nicht ankommt, sind Mengen das geeignete Modellierungswerkzeug. Dabei ist es völlig unerheblich, was in der Kollektion angesammelt ist. Das muss keine Gemeinsamkeit haben. In diesem Sinne ist

$\{\text{Mobiltelefon, Feuerzeug, Tempo, Kugelschreiber, Pflaster, Kaugummi}\}$

eine mengenartige Kollektion, auch wenn es (augenscheinlich) keine Gemeinsamkeiten der Elemente gibt.¹

Die Anzahl der Elemente einer Menge muss nicht endlich sein. Viele wichtige Mengen haben eine unbegrenzte Anzahl an Elemente. Dazu gehören

- die Menge $\mathbb{N}$ der natürlichen Zahlen,
- die Menge $\mathbb{Z}$ der ganzen Zahlen,
- die Menge $\mathbb{Q}$ der rationalen Zahlen,
- die Menge $\mathbb{R}$ der reellen Zahlen.

In der Informatik ist *Typ* übrigens ein gängiges Synonym für Menge.

¹Eine tatsächliche Gemeinsamkeit könnte darin bestehen, dass sich all diese Elemente in einer bestimmten Handtasche angesammelt haben.

5.3 Element und Teilmenge

Wie in Kapitel 1 vereinbart, nennen wir die Teile, die in einer Menge zu einem Ganzen angesammelt sind, ihre *Elemente*. Aussagen der Form (also Aussageformen) „ x ist Element der Menge M “ werden üblicherweise kurz als $x \in M$ geschrieben, in *setlX* als **x in M**. Die Negation „ x ist kein Element der Menge M “ wird entsprechend $x \notin M$ und in *setlX* in der Form **x notin M** geschrieben.

Beispiel 5.1: Für die Menge $M = \{1, 2, \{3, 4\}, \text{„Flugzeug“}\}$ sind die folgenden Aussagen wahr:

$$1 \in M, \quad 2 \in M, \quad \{3, 4\} \in M, \quad \text{„Flugzeug“} \in M$$

Dagegen sind diese Aussagen falsch:

$$3 \in M, \quad \text{„Eisenbahn“} \in M$$

Das bedeutet, dass

$$3 \notin M, \quad \text{„Eisenbahn“} \notin M$$

jeweils wahr ist.

Natürlich kann auch *setlX* diese Wahrheitswerte bestimmen:

```
=> M:={1,2,{3,4},"Flugzeug"};
~< Result: {1, 2, {3, 4}, "Flugzeug"} >~
=> 1 in M;
~< Result: true >~
=> {3,4} in M;
~< Result: true >~
=> "Flugzeug" in M;
~< Result: true >~
=> 3 in M;
~< Result: false >~
=> 3 notin M;
~< Result: true >~
```

■

Wie haben Sie Aufwärmübung 3 beantwortet? Ist für Sie $\{1, 2\}$ in $\{1, 2, 3, 4\}$ enthalten? Dann meinen Sie mit „enthalten“ sicher nicht

$$\{1, 2\} \in \{1, 2, 3, 4\},$$

denn das ist eine falsche Aussage. Die Menge $\{1, 2\}$ ist kein Element von $\{1, 2, 3, 4\}$. Dagegen sind die beiden Elemente von $\{1, 2\}$ jeweils Elemente von $\{1, 2, 3, 4\}$.

Das „Enthaltensein“ von $\{1, 2\}$ in $\{1, 2, 3, 4\}$ ist von anderer Natur als das „Elementsein“. Wir brauchen für dieses „Enthaltensein“ einen neuen Begriff:

Eine Menge A heißt *Teilmenge* einer Menge B , wenn jedes Element von A auch Element von B ist. Man schreibt dann $A \subseteq B$.

Wir sind hier wieder in der Situation, dass wir für zwei verwandte, aber unterscheidungswürdige und unterscheidungsbedürftige Konzepte zwei unterschiedliche Begriffe benötigen: Teilmenge und Element.

Das bedeutet aber auch, dass der Begriff „enthalten sein“ im Kontext von Mengen ungeeignet ist. Der Grund liegt darin, dass dieser Begriff mehrdeutig ist und nicht klar ist, ob damit $\in$ oder $\subseteq$ gemeint ist.

Beispiel 5.2: $\{1, 2\}$ ist eine Teilmenge von $\{1, 2, 3, 4\}$, denn definitionsgemäß ist jedes Element von $\{1, 2\}$ auch Element von $\{1, 2, 3, 4\}$:

$$1 \in \{1, 2, 3, 4\} \wedge 2 \in \{1, 2, 3, 4\} \Leftrightarrow 1$$

Wir dürfen also $\{1, 2\} \subseteq \{1, 2, 3, 4\}$ schreiben. ■

Beispiel 5.3: Beispiele für Teilmengen der Menge $M = \{1, \{2, 3\}, 4, \{4, 5\}, 5\}$ sind

$$\{1, 4\} \subseteq M, \quad \{\{2, 3\}, 4\} \subseteq M, \quad \{1, 4, \{4, 5\}\} \subseteq M, \quad M \subseteq M.$$

$\{2, 3\}$ ist dagegen keine Teilmenge von M , aber ein Element. Die Menge $\{4, 5\}$ hat hier die Besonderheit, dass sie sowohl Teilmenge von M als auch Element von M ist

$$\{4, 5\} \in M, \quad \{4, 5\} \subseteq M,$$

aber die Symbole 4 und 5 haben als Element bzw. Teilmenge unterschiedlichen Ursprung:

$$\begin{aligned} \{4, 5\} &\subseteq \{1, \{2, 3\}, 4, \{4, 5\}, 5\} \\ \{4, 5\} &\in \{1, \{2, 3\}, 4, \{4, 5\}, 5\} \end{aligned}$$

Wir können $A \subseteq B$ auch als Aussageform verstehen, die genau dann wahr ist, wenn A Teilmenge von B ist. ■

Beispiel 5.4: Setzen wir *setlX* auf das vorangegangene Beispiel an:

```
=> M:={1,{2,3},4,{4,5},5};
~< Result: {1, {2, 3}, 4, {4, 5}, 5} >~
=> {1,4} <= M;
~< Result: true >~
=> {2,3} <= M;
~< Result: false >~
=> {2,3} in M;
~< Result: true >~
=> {4,5} <= M;
~< Result: true >~
=> {4,5} in M;
~< Result: true >~
```

Wie Sie sehen, wird in *setlX* $\subseteq$ mit $<=$ notiert. ■

Es ist mit der Definition der Teilmenge kompatibel, dass jede Menge Teilmenge von sich selbst ist. Will man explizit ausschließen, dass dieser Fall nicht vorliegt, also quasi eine echte Teilmenge vorliegt, schreibt man $\subset$. Wir definieren das Symbol $\subset$ mittels formaler Logik:

$$A \subset B \Leftrightarrow A \subseteq B \wedge A \neq B$$

In *setlX* wird für $\subset$ das Symbol $<$ verwendet.

Beispiel 5.5: Ist $\{1, 2\} \subset \{1, 2\}$?

```
=> {1,2}<{1,2};
~< Result: false >~
```

Ist $\{1\} \subset \{1, 2\}$?

```
=> {1}<{1,2};
~< Result: true >~
```

Möglicherweise wäre nun ein passender Zeitpunkt, um auf das zurückzublicken, was Sie bisher gelesen haben, oder eine Pause zu machen. ■

5.4 Mengen-Konstruktoren

Mengen können dadurch beschrieben werden, dass alle ihre Elemente aufgelistet werden wie bspw. bei $M = \{1, 2, \{3, 4\}, \text{„Flugzeug“}\}$. Das ist aber nicht immer möglich oder sinnvoll. Bei unendlichen Mengen wie den natürlichen Zahlen ist es unmöglich alle Elemente aufzulisten. In anderen Fällen, wie der Menge aller ungeraden Zahlen zwischen 0 und 100, wäre ein Auflisten zwar möglich, aber aufwändig. In solchen Situationen ist es hilfreich die Menge durch eine geeignete Notation zu beschreiben.

Beispiel 5.6: Der Ausdruck

$$\{n : n \in \mathbb{N} \mid n \bmod 2 = 1 \wedge n \leq 100\}$$

beschreibt die Mengen aller ungeraden Zahlen zwischen 0 und 100. Dazu wird jede natürliche Zahl $n \in \mathbb{N}$ darauf hin überprüft, ob sie ungerade ist ($n \bmod 2 = 1$) und ob sie kleiner als 100 ist. ■

In dieser Notation dient die Aussageform, die hinter dem *Konditionalstrich* „ $\mid$ “ steht quasi als Filter. Im obigen Beispiel werden aus den Zahlen $n \in \mathbb{N}$ genau die in die Menge übernommen, für die die Aussageform $n \bmod 2 = 1 \wedge n \leq 100$ wahr ist. Alle anderen werden herausgefiltert.

Beispiel 5.7: Die Menge der ungeraden Zahlen zwischen 0 und 100 lässt sich auf viele Arten beschreiben, z. B. auch als

$$\begin{aligned} \{n : n \in \{1..100\} \mid n \bmod 2 = 1\} \\ \{2k + 1 : k \in \{0..49\}\}. \end{aligned}$$

Der erste Ausdruck entnimmt n nicht mehr der Menge der natürlichen Zahlen, sondern sofort der Menge der natürlichen Zahlen bis 100. Damit ist die Bedingung $n \leq 100$ automatisch sichergestellt. Es muss nur noch sichergestellt werden, dass die Zahlen auch ungerade sind, was hier wieder mit der Aussageform $n \bmod 2 = 1$ erreicht wird.

Der zweite Ausdruck beruht auf der Erkenntnis, dass eine Zahl der Bauart $2k+1$ für ganzzahliges k ungerade sein muss. Dann muss nur noch k so gewählt werden, dass die entstehenden Zahlen kleiner gleich 100 sind.

In dieser Form können die beiden Mengenbeschreibungen auch von *setlX* ausgewertet werden.

```
=> {n: n in {1..100} | n%2==1 };
~< Result: {1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33,
35, 37, 39, 41, 43, 45, 47, 49, 51, 53, 55, 57, 59, 61, 63, 65, 67, 69, 71,
73, 75, 77, 79, 81, 83, 85, 87, 89, 91, 93, 95, 97, 99} >~
=> {2*k+1: k in {0..49}};
~< Result: {1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33,
35, 37, 39, 41, 43, 45, 47, 49, 51, 53, 55, 57, 59, 61, 63, 65, 67, 69, 71,
73, 75, 77, 79, 81, 83, 85, 87, 89, 91, 93, 95, 97, 99} >~
```

■

Die Syntax dieses Beschreibungsmusters für Mengen ist

$$\{\text{Ausdruck abh. von Variable} : \text{Variable} \in \text{Menge} \mid \text{Aussageform abh. von Variable}\}. \quad (5.1)$$

Es wird als *Mengen-Konstruktor* bzw. mit dem englischen Begriff *set comprehension* bezeichnet.

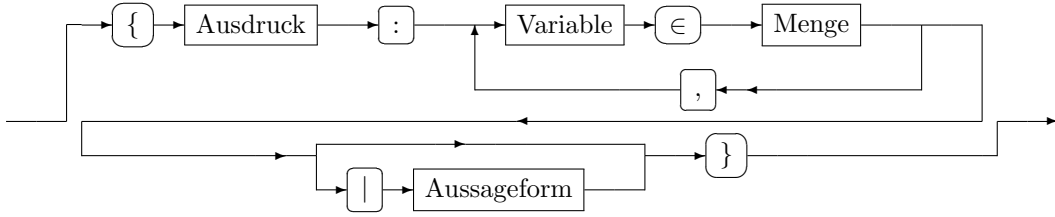
Die folgenden Abbildungen (Abb. 5.1) visualisieren die Syntax von Mengen-Konstruktoren anhand sogenannter *Railway*-Diagramme. Diese werden vom linken Rand ausgehend durchlaufen. Wenn es einen Weg (ein Gleis) gibt, der ans Ende führt, ist ein Ausdruck syntaktisch korrekt, sonst nicht. Dabei können die Wege bzw. Gleise immer nur in Pfeilrichtung durchlaufen werden und sind daher immer Einbahnstraßen.

Die Ausdrücke

$$\begin{aligned} \{n : n \in \mathbb{N} \mid\} \\ \{n \mid n > 0\} \\ \{n > 0 : n \in \mathbb{N}\} \end{aligned}$$

sind syntaktisch nicht korrekt. Im ersten Ausdruck fehlt die Aussageform hinter dem Konditionalstrich, denn wenn ein Konditionalstrich auftritt, muss er gemäß *Railway*-Diagramm von einer Aussageform gefolgt sein. Beim zweiten Ausdruck fehlt nach dem ersten n ein Doppelpunkt gefolgt von der Angabe, aus welcher Menge die Elemente „herausgefiltert“ werden sollen. Der dritte Ausdruck schließlich beginnt nach der öffnenden geschweiften Klammer mit einer Aussageform, was gemäß *Railway*-Diagramm nicht syntaktisch korrekt ist.

Auf Papier:



In *setlX*:

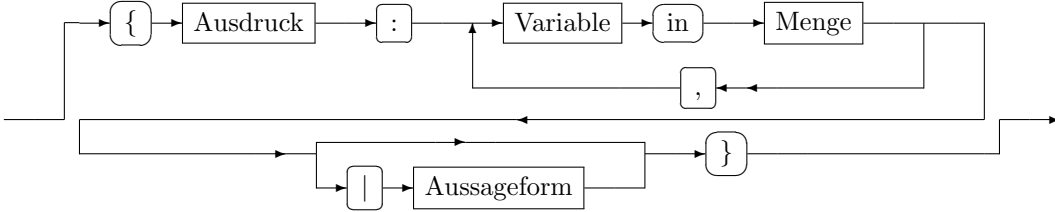


Abbildung 5.1: Syntax-Diagramm für Mengen-Konstruktoren

Der Ausdruck zwischen öffnender geschweiffter Klammer und Doppelpunkt kann so einfach sein, wie ein einzelnes Symbol oder beliebig komplex. Bei einer gängigen Beschreibung der rationalen Zahlen ist es ein Bruch:

$$\mathbb{Q} = \left\{ \frac{p}{q} : p \in \mathbb{Z}, q \in \mathbb{Z} \mid q \neq 0 \right\} \quad (5.2)$$

Dieses Beispiel zeigt auch, dass mehr als ein Variablensymbol in einem Mengen-Konstruktor beteiligt sein kann.

Die Elemente von Mengen müssen nicht atomar sein, sondern können selbst komplexere Strukturen sein, z. B. Tupel oder Mengen.

Beispiel 5.8: $\{[k, 2k] : k \in \mathbb{Z}\}$ ist die Menge von Paaren ganzer Zahlen, deren zweite Komponente doppelt so groß ist wie die erste. Elemente dieser Menge sind z. B. $[-1, -2]$, $[0, 0]$ und $[5, 10]$. ■

Beispiel 5.9: Mit der Menge der Booleschen Werte $\mathbb{B}$ ist

$$\{[a, b] : a \in \mathbb{B}, b \in \mathbb{B}\} = \{[0, 0], [0, 1], [1, 0], [1, 1]\} \quad (5.3)$$

die Menge aller Paare von Wahrheitswerten. Diese Menge kann auch als die Menge der Zahlen von 0 bis 3 in Binärdarstellung interpretiert werden. ■

Beispiel 5.10: $\{\{a, b\} : a \in \mathbb{N}, b \in \mathbb{N} \mid a \neq b\}$ ist die Menge der zweielementigen Teilmengen von $\mathbb{N}$. Elemente dieser Menge sind z. B. $\{1, 2\}$, $\{1, 42\}$ und $\{17, 42\}$. ■

Beispiel 5.11: Der Mengen-Konstruktor

$$\mathbb{Q} = \left\{ \frac{p}{q} : p \in \mathbb{Z}, q \in \mathbb{N} \right\}$$

ist eine gleichwertige Alternative zur Beschreibung (5.2) der rationalen Zahlen. ■

Abweichend von den obigen Syntax-Diagrammen haben sich in einigen Spezialfällen Kurzschreibweisen etabliert. Wenn Variablensymbole derselben Menge entnommen werden, wie etwa in (5.2), wird manchmal diese Menge nur einmal angegeben, z. B.

$$\mathbb{Q} = \left\{ \frac{p}{q} : p, q \in \mathbb{Z} \mid q \neq 0 \right\}.$$

Wenn aus dem Kontext klar hervorgeht, aus welcher Menge die Variablensymbole entnommen werden sollen, wird die Menge mitunter nicht mehr angegeben. Wenn bspw. klar ist, dass es nur um natürliche Zahlen geht, wird für die Menge der Quadratzahlen kleiner 100 durchaus $\{n^2 \mid n^2 < 100\}$ statt $\{n^2 : n \in \mathbb{N} \mid n^2 < 100\}$ geschrieben.

5.5 Operationen auf Mengen

In der Logik geht es weniger um das Feststellen des Wahrheitswerts einer Aussage, sondern darum, wie Aussagen zu neuen Aussagen verknüpft werden können. Entsprechend geht es in der Mengenlehre weniger um die Bestimmung der Elemente einer Menge, sondern darum, wie Mengen zu neuen Mengen verknüpft werden können.

Eine mögliche Verknüpfung ist, alle Elemente zweier Mengen A und B zu einer neuen Menge zu vereinen. Mittels Mengen-Konstruktor können wir diese neue Menge so schreiben:

$$\{x : x \in \mathbb{U} \mid x \in A \vee x \in B\} \quad (5.4)$$

Die Menge $\mathbb{U}$ soll dabei die Menge sein, die alle möglichen Elemente enthält. Sie wird *Universum* genannt. Der Mengen-Konstruktor übernimmt also nur die Elemente des Universums $\mathbb{U}$, die Element von A oder Element von B sind.

Abb. 5.2 visualisiert die Bildung dieser neuen Menge aus den Mengen A und B in einem sogenannten *Venn-Diagramm*. Dort sind Mengen als umgrenzte Flächen dargestellt. Die Elemente sind die umgrenzten Punkte. Die umrahmte Fläche stellt das Universum dar.

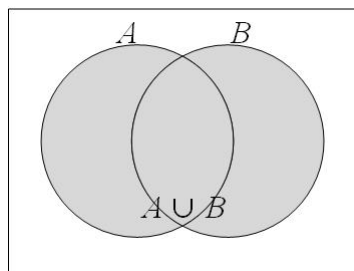


Abbildung 5.2: Venn-Diagramm zur Vereinigung zweier Mengen

Die in (5.4) gebildete Menge wird als Vereinigung der beiden Ausgangsmengen A und B bezeichnet. Weitere wichtige Verknüpfungen von Mengen und die damit verbundene Symbolik nennt die folgende Definition:

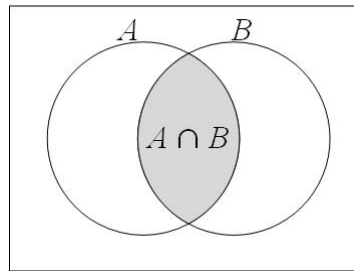


Abbildung 5.3: Venn-Diagramm zum Schnitt zweier Mengen

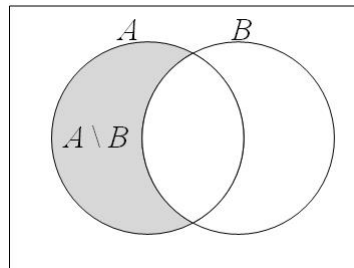


Abbildung 5.4: Venn-Diagramm zur Differenz zweier Mengen

Die Menge $A \cup B$ mit

$$A \cup B = \{x : x \in \mathbb{U} \mid x \in A \vee x \in B\} \quad (5.5)$$

nennt man die *Vereinigung* der Mengen A und B .

Die Menge $A \cap B$ mit

$$A \cap B = \{x : x \in \mathbb{U} \mid x \in A \wedge x \in B\} \quad (5.6)$$

nennt man den *Schnitt* der Mengen A und B .

Die Menge $A \setminus B$ mit

$$A \setminus B = \{x : x \in \mathbb{U} \mid x \in A \wedge x \notin B\} \quad (5.7)$$

nennt man die *Differenz* der Mengen A und B .

Die Menge $\bar{A}$ mit

$$\bar{A} = \{x : x \in \mathbb{U} \mid x \notin A\} \quad (5.8)$$

nennt man das *Komplement* der Menge A .

Die Abb. 5.2 - 5.5 zeigen die dazugehörigen Venn-Diagramme.

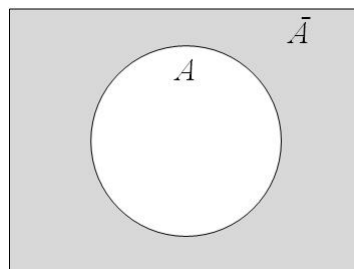


Abbildung 5.5: Venn-Diagramm zum Komplement einer Menge

Eine weitere wichtige Operation bestimmt die Anzahl der Elemente einer Menge. Sie wird *Mächtigkeit* genannt und mit $| \cdot |$ notiert. Beispielsweise ist $|\{1, 2, 3\}| = 3$.

Tabelle 5.1 stellt alle mengentheoretischen Operationen nochmals zusammen und nennt auch, wie diese in *setlX* notiert werden.

Tabelle 5.1: Mengentheoretische Symbole auf Papier und in *setlX*

Operator	Symbol auf Papier	<i>setlX</i>
Element	$\in$	in
Nicht-Element	$\notin$	notin
Teilmenge	$\subseteq$	<=
echte Teilmenge	$\subset$	<
Vereinigung	$\cup$	+
Schnitt	$\cap$	*
Differenz	$\setminus$	-
Komplement	$\overline{}$ (z. B. $\overline{S}$)	nicht möglich
Mächtigkeit	$ \cdot $ (z. B. $ S $)	# (z. B. #S)
Potenzmenge	$\mathcal{P}()$ (z. B. $\mathcal{P}(S)$)	2** (z. B. 2**S)
kart. Produkt	$\times$	><

Beispiel 5.12: Wir nutzen *setlX* um die beiden Mengen $A = \{1, 2, 3\}$ und $B = \{3, 4\}$ zu neuen Mengen zu verknüpfen:

$$\begin{aligned}
 A \cup B &= \text{✎} \\
 A \cap B &= \text{✎} \\
 A \setminus B &= \text{✎} \\
 \overline{A} &= \text{✎}
 \end{aligned}$$

Sie sollten sich allerdings vorher selbst an der Bestimmung dieser Mengen versuchen.

Die Vereinigung $A \cup B$ wird in *setlX* als **A+B** notiert, der Schnitt $A \cap B$ als **A*B** und die Differenz $A \setminus B$ als **A-B**.

```
=> A:={1,2,3};B:={3,4}; A+B;
~< Result: {1, 2, 3, 4} >~
```

```
=> A*B;
~< Result: {3} >~
```

```
=> A-B;
~< Result: {1, 2} >~
```

Das Komplement kann nicht gebildet werden, ohne dass ein Universum angegeben ist. Wäre z. B. $U = \{1..10\}$, dann ist $\overline{A}$:

```
=> U:={1..10};
~< Result: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} >~
=> Komplement_von_A:={x: x in U | x notin A};
~< Result: {4, 5, 6, 7, 8, 9, 10} >~
```



5.6 Leere Menge

Eine Menge von besonderer Bedeutung ist die *leere Menge*. Sie wird mit $\emptyset$ oder $\{\}$ symbolisiert. Da sie keine Elemente hat, ist ihre Mächtigkeit $|\emptyset| = 0$.

Beispiel 5.13: Die leere Menge entsteht bspw., wenn wir die Differenz einer Menge mit sich selbst bilden

$$\{1, 2, 3\} \setminus \{1, 2, 3\} = \emptyset$$

oder den Schnitt zweier Mengen ohne gemeinsame Elemente bilden

$$\{1, 2, 3\} \cap \{4, 5\} = \emptyset.$$

In *setlX*:

```
=> {1,2,3}-{1,2,3};
~< Result: {} >~
=> {1,2,3}*{4,5};
~< Result: {} >~
```

■

Der Fall, dass zwei Mengen keine gemeinsamen Elemente haben, ist besonders relevant. Wir geben ihm daher eine eigene Bezeichnung: Besitzen zwei Mengen kein gemeinsames Element, dann heißen sie *disjunkt*. Ihre Schnittmenge ist dann die leere Menge. Schauen Sie sich dazu noch einmal das vorausgehende Beispiel an.

Die leere Menge hat eine wichtige Eigenschaft im Zusammenhang mit der Teilmenge. Was sind die Wahrheitswerte der folgenden Aussagen bzw. Aussageformen?

$\emptyset \subseteq \{1, 2, 3\}$	
$\emptyset \subset \{1, 2, 3\}$	
$\emptyset \subseteq \emptyset$	
$\emptyset \subset \emptyset$	
$\emptyset \subseteq \mathbb{N}$	
$\emptyset \subset \mathbb{N}$	
$\emptyset \subseteq S$	
$\emptyset \subset S$	

Beginnen wir mit der ersten Frage. Um diese Frage beantworten zu können, müssen wir auf die Bedeutung, also die Definition der Teilmenge zurückgreifen: Eine Menge A heißt genau dann Teilmenge einer Menge B , wenn jedes Element von A auch Element von B ist. Hier haben wir $A = \emptyset$ und $B = \{1, 2, 3\}$ und die Definition besagt

$$\emptyset \subseteq \{1, 2, 3\} \Leftrightarrow \text{jedes Element von } \emptyset \text{ ist Element von } \{1, 2, 3\}.$$

Wir müssen also überprüfen, ob jedes Element von $\emptyset$ auch Element von $\{1, 2, 3\}$ ist. Nun hat $\emptyset$ allerdings keine Elemente!

Was tun? Manchmal hilft es über die Negation vorzugehen, so auch hier:

$$\emptyset \not\subseteq \{1, 2, 3\} \Leftrightarrow \text{nicht jedes Element von } \emptyset \text{ ist Element von } \{1, 2, 3\}.$$

Die Negation

„nicht jedes Element von $\emptyset$ ist Element von $\{1, 2, 3\}$ “

ist gleichbedeutend mit

„es gibt ein Element von $\emptyset$, das nicht Element von $\{1, 2, 3\}$ ist“ .

Das ist offensichtlich falsch, da $\emptyset$ ja keine Elemente hat. Also ist auch $\emptyset \not\subseteq \{1, 2, 3\}$ falsch und somit ist $\emptyset \subseteq \{1, 2, 3\}$ wahr! Die leere Menge ist eine Teilmenge von $\{1, 2, 3\}$.

Wir können das mit *setlX* überprüfen:

```
=> {} <= {1,2,3};
~< Result: true >~
```

Wenn wir nun die Menge $\{1, 2, 3\}$ durch eine beliebige Menge B ersetzen, ändert sich nichts an der Argumentation. Es gilt $\emptyset \subseteq B$ für jede Menge B . *Die leere Menge ist also eine Teilmenge einer jeden Menge.* In Abschnitt 6.10 werden wir dies nochmals begründen – dann ohne den Weg über die Negation.

5.7 Algebraische Eigenschaften

Komplement, Schnitt und Vereinigung haben folgende Eigenschaften:

neutrales Element	$A \cup \emptyset = A$	(5.9)	$A \cap \mathbb{U} = A$	(5.10)
Komplementarität	$A \cup \bar{A} = \mathbb{U}$	(5.11)	$A \cap \bar{A} = \emptyset$	(5.12)
Assoziativität	$A \cup (B \cup C) = (A \cup B) \cup C$	(5.13)	$A \cap (B \cap C) = (A \cap B) \cap C$	(5.14)
Kommutativität	$A \cup B = B \cup A$	(5.15)	$A \cap B = B \cap A$	(5.16)
Distributivität	$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	(5.17)	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$	(5.18)
de Morgan'sche Regeln:	$\overline{A \cup B} = \bar{A} \cap \bar{B}$	(5.19)	$\overline{A \cap B} = \bar{A} \cup \bar{B}$	(5.20)
doppelte Komplementbildung	$\overline{\bar{A}} = A$	(5.21)		

Wie schon bei den Eigenschaften der logischen Operatoren in Abschnitt 4.3 sollte Ihnen auch hier eine übergeordnete Struktur auffallen, wenn Sie die linke und rechte Spalte der Auflistung dieser Eigenschaften analysieren. Es gibt noch eine zusätzliche übergeordnete Struktur, denn die Eigenschaften-Tabelle in Abschnitt 4.3 geht aus der obigen hervor, wenn wir folgende Ersetzungen vornehmen:

$$\begin{aligned}
 \cup &\mapsto \vee \\
 \cap &\mapsto \wedge \\
 \bar{} &\mapsto \neg \\
 \emptyset &\mapsto 0 \\
 \mathbb{U} &\mapsto 1
 \end{aligned}$$

Die ersten drei dieser Ersetzungen sind auch in den Definitionen (5.5) - (5.8) sichtbar, denn die Vereinigung ist über eine Disjunktion definiert, der Durchschnitt über eine Konjunktion und das Komplement über die Negation. Wir haben es hier mit einer Struktur zu tun, welche die Logik und die Mengenlehre vereinigt. Sie können diese in Abschnitt 5.10.1 vertiefen.

5.8 Potenzmenge

Wie viele Teilmengen haben Sie in Aufwärmübung 4 bei der Frage nach den Teilmengen der Menge $\{1, 2, 3\}$ aufgelistet?

Diese Anzahl lässt sich z.B. kombinatorisch finden (mehr dazu in Kapitel 10): Das Element 1 kann oder kann nicht Element einer Teilmenge sein. Das ergibt zwei Fälle. In jedem dieser Fälle kann 2 Element oder kein Element einer Teilmenge sein. Das macht insgesamt $2 \cdot 2 = 4$ Fälle. In jedem dieser Fälle kann 3 Element oder kein Element einer Teilmenge sein. Das macht insgesamt $4 \cdot 2 = 2^3$ viele Fälle.

Entsprechend hat eine Menge mit n Elementen 2^n Teilmengen. Die Menge aller Teilmengen ist ein häufig benötigtes Werkzeug.

Die Menge aller Teilmengen einer Ausgangsmenge S wird als *Potenzmenge* von S bezeichnet und mit $\mathcal{P}(S)$ symbolisiert.

`setlX` verwendet statt $\mathcal{P}(S)$ die Potenzschreibweise `2**S`, die darauf hinweist, dass die Potenzmenge $2^{|S|}$ viele Teilmengen als Elemente hat.

Für die Mächtigkeit der Potenzmenge einer Menge S gilt $|\mathcal{P}(S)| = 2^{|S|}$.

Beispiel 5.14: Wir berechnen die Potenzmenge von $\{1, 2, 3\}$ mit `setlX`:

```
=> 2**{1,2,3};
~< Result: {}, {1}, {1, 2}, {1, 2, 3}, {1, 3}, {2}, {2, 3}, {3} >~
```

Eine der Teilmengen muss natürlich die leere Menge sein, wie wir in Abschnitt 5.6 schon erarbeitet haben. ■

5.9 Kartesisches Produkt

Im vorausgegangenen Teilkapitel haben wir mit der Potenzmenge die Menge aller Teilmengen einer Menge angeschaut. In diesem Kapitel steht die Menge aller Paare im Zentrum. Um Paare zu bilden, brauchen wir allerdings zwei Mengen: Eine Menge, aus deren Elementen wir die erste Komponente des Paares bilden, und eine Menge für die zweite Komponente. Die beiden Mengen können unter Umständen identisch sein, dann sind die beiden Paarkomponenten aus derselben Menge.

Nennen wir die beiden involvierten Mengen A und B . Die Menge aller Paare, die aus A und B gebildet werden können, kann dann mit Hilfe der Mengen-Konstruktor-Schreibweise als

$$\{[a, b] : a \in A, b \in B\} \quad (5.22)$$

geschrieben werden bzw. in `setlX` als `{[a,b] : a in A, b in B}`. Auf diese Art hatten wir in (5.3) bereits eine Mengen aller Paare gebildet.

Beispiel 5.15: Die Menge aller Paare, in denen auf einen der Buchstaben x oder y eine der Ziffern 1, 2 oder 3 folgt, kann als

$$\{[a, b] : a \in \{x, y\}, b \in \{1, 2, 3\}\}$$

geschrieben werden. Für die erste Komponenten eines Paares gibt es zwei Möglichkeiten (x bzw. y). Für die zweite Komponente eines Paares gibt es jeweils drei Möglichkeiten (1, 2 bzw. 3). Insgesamt gibt es also $2 \cdot 3 = 6$ mögliche Paare, nämlich $[x, 1], [x, 2], [x, 3], [y, 1], [y, 2], [y, 3]$. Daher ist

$$\{[a, b] : a \in \{x, y\}, b \in \{1, 2, 3\}\} = \{[x, 1], [x, 2], [x, 3], [y, 1], [y, 2], [y, 3]\}.$$

Das Bilden von Paaren über den Mengen-Konstruktor (5.22) ist eine wichtige Operation, die eine eigene Bezeichnung verdient: ■

Die Menge aller geordneten Paare zweier Mengen A und B wird *kartesisches Produkt von A und B* genannt und mit $A \times B$ symbolisiert:

$$A \times B = \{[a, b] : a \in A, b \in B\} \quad (5.23)$$

Eine weitere gebräuchliche Bezeichnung für das kartesische Produkt ist *Kreuzprodukt*. Die Menge $A \times B$ wird „A Kreuz B“ gelesen. In *setlX* wird $\times$ als $><$ notiert.

Beispiel 5.16: Die Menge aller Paare, in denen auf einem Buchstaben x oder y eine der Ziffern 1, 2 oder 3 folgt, kann in *setlX* so berechnet werden:

```
=> {'x','y'} >< {1,2,3};
~< Result: [{"x", 1}, [{"x", 2}, [{"x", 3}, [{"y", 1}, [{"y", 2}, [{"y", 3}]] >~
```

■

Im Spezialfall können die beiden an der Bildung eines kartesischen Produkts beteiligten Mengen identisch sein. Das kartesische Produkt ist dann von der Form $A \times A$. Dafür wird dann auch A^2 geschrieben.

$\mathbb{N}^2 = \mathbb{N} \times \mathbb{N}$ ist entsprechend die Menge aller Paare bestehend aus natürlichen Zahlen. $\mathbb{R}^2 = \mathbb{R} \times \mathbb{R}$ ist die Menge aller Paare reeller Zahlen. Die Menge $\mathbb{R}^2$ beschreibt z. B. die Kollektion aller Punkte einer Ebene.

Das kartesische Produkt bildet Paare. Man kann es verallgemeinern, so dass Tupel der beliebigen Länge n gebildet werden:

$$A_1 \times A_2 \times \dots \times A_n = \{[a_1, a_2, \dots, a_n] : a_1 \in A_1, a_2 \in A_2, \dots, a_n \in A_n\}$$

$\mathbb{R}^n$ bezeichnet dann die Menge aller n -Tupel reeller Zahlen.

5.10 Empfohlene Vertiefungen

5.10.1 Boolesche Algebra

Die Struktur, die die Eigenschaften der logischen Operatoren $\neg$, $\wedge$ und $\vee$ und der mengentheoretischen Operatoren $\overline{}$, $\cap$ und $\cup$ verallgemeinert, wird *Boolesche Algebra* genannt.

Eine boolesche Algebra ist eine Menge B mit mindestens zwei Elementen und mit zwei binären Operatoren $\oplus$ und $\otimes$ und einem unären Operator $\ominus$ mit folgenden Eigenschaften:

1. Abgeschlossenheit: Für alle $a \in B$, $b \in B$ sind auch immer $a \oplus b \in B$, $a \otimes b \in B$, $\ominus a \in B$.
2. Kommutativität: Für alle $a \in B$, $b \in B$ gilt:

$$\begin{aligned} a \oplus b &= b \oplus a \\ a \otimes b &= b \otimes a \end{aligned}$$

3. Assoziativität: Für alle $a \in B$, $b \in B$, $c \in B$ gilt:

$$\begin{aligned} a \oplus (b \oplus c) &= (a \oplus b) \oplus c \\ a \otimes (b \otimes c) &= (a \otimes b) \otimes c \end{aligned}$$

4. Distributivität: Für alle $a \in B$, $b \in B$, $c \in B$ gilt:

$$\begin{aligned} a \oplus (b \otimes c) &= (a \oplus b) \otimes (a \oplus c) \\ a \otimes (b \oplus c) &= (a \otimes b) \oplus (a \otimes c) \end{aligned}$$

5. Neutrale Elemente: Es gibt zwei Elemente 0 und 1, so dass für alle $a \in B$ gilt:

$$\begin{aligned} a \oplus 0 &= a \\ a \otimes 1 &= a \end{aligned}$$

6. Extremalität: Für alle $a \in B$ gilt:

$$\begin{aligned} a \oplus (\ominus a) &= 1 \\ a \otimes (\ominus a) &= 0 \end{aligned}$$

Die folgende Tabelle zeigt die Beziehungen zur Logik und Mengenlehre:

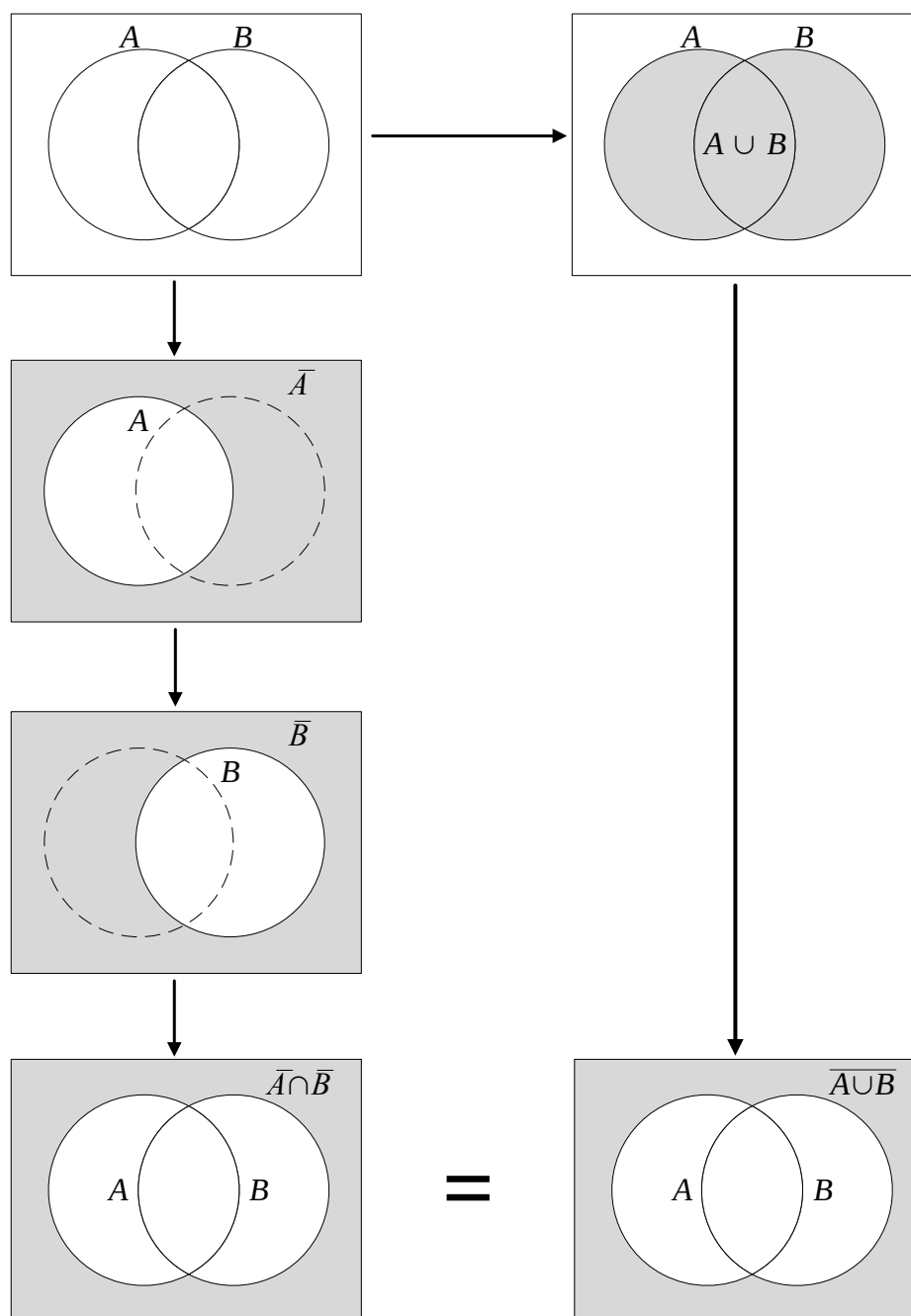
	Logik	Mengenlehre
$\oplus$	$\vee$	$\cup$
$\otimes$	$\wedge$	$\cap$
$\ominus$	$\neg$	$-$
0	0	$\emptyset$
1	1	$\mathcal{U}$

Beispiel 5.17: Für jede Menge M bildet die Potenzmenge $\mathcal{P}(M)$ mit den Operationen Vereinigung, Schnitt und Komplement eine Boolesche Algebra, wenn $\mathcal{U} = \mathcal{P}(M)$ gewählt wird. Das 0-Element ist $\emptyset$, das 1-Element ist $\mathcal{P}(M)$. ■

Beispiel 5.18: Die Menge $M = \{-1, 0, 1\}$ mit den arithmetischen Operatoren $+$, $\cdot$ und $-$ ist keine Boolesche Algebra. Schon die Abgeschlossenheit ist nicht erfüllt, z. B. weil $1 + 1 \notin M$. ■

5.10.2 Beweis mengentheoretischer Beziehungen

Mengenoperationen lassen sich gut durch Venn-Diagramme veranschaulichen. Da ist es naheliegend auch die Korrektheit mengentheoretischer Beziehungen mit Venn-Diagrammen zu zeigen. Versuchen wir das am Beispiel der de Morgan'schen Regel $\overline{A \cup B} = \overline{A} \cap \overline{B}$ aus (5.19). Abb. 5.6 zeigt diese Gleichheit mit Hilfe von Venn-Diagrammen.

Abbildung 5.6: Versuch einer Begründung von $\overline{A \cup B} = \overline{A} \cap \overline{B}$ mittels Venn-Diagrammen

Das Problem besteht dabei darin, dass die Gleichheit nur in dem Spezialfall gezeigt wird, in dem A und B nicht disjunkt sind. Was ist mit dem Fall, dass der Schnitt von A und B leer ist? Was ist, wenn $A = \emptyset$ oder $B = \emptyset$? All diese Fälle werden durch die Venn-Diagramme nicht (explizit) erfasst.

Ausgehend von den Definitionen der beteiligten Operatoren kann ein Beweis allerdings alleine und recht leicht mit Mitteln der Logik geführt werden:

Nach Definition der Vereinigung (5.5)

$$A \cup B = \{x : x \in \mathbb{U} \mid x \in A \vee x \in B\}$$

und des Komplements (5.8)

$$\overline{A} = \{x : x \in \mathbb{U} \mid x \notin A\} = \{x : x \in \mathbb{U} \mid \neg(x \in A)\}$$

ist

$$\begin{aligned} \overline{A \cup B} &= \{x : x \in \mathbb{U} \mid \neg(x \in A \vee x \in B)\} \\ &= \{x : x \in \mathbb{U} \mid x \notin A \wedge x \notin B\}. \end{aligned}$$

Dabei wurde im letzten Schritt die Aussageform mittels der Logikregel (4.20) umgeformt. Nach Definition (5.6) haben wir rechts den Schnitt zweier Mengen stehen

$$\begin{aligned} \overline{A \cup B} &= \{x : x \in \mathbb{U} \mid x \notin A \wedge x \notin B\} \\ &= \{x : x \in \mathbb{U} \mid x \notin A\} \cap \{x : x \in \mathbb{U} \mid x \notin B\} \end{aligned}$$

und diese beiden Mengen sind wegen Definition (5.8) Komplemente:

$$\begin{aligned} \overline{A \cup B} &= \{x : x \in \mathbb{U} \mid x \notin A\} \cap \{x : x \in \mathbb{U} \mid x \notin B\} \\ &= \overline{A} \cap \overline{B} \end{aligned}$$

Damit ist die de Morgan'sche Regel bewiesen. Alle Spezialfälle wie $A \cap B = \emptyset$ oder $A = \emptyset$ sind dabei mit abgedeckt, denn die Variablen A und B wurden nie mit konkreten Mengen belegt.

5.11 Lernziele

Studierende	Kennen	Können	Verstehen
fachlich	☐ kennen die Definitionen der mengentheoretischen Operationen ☐ erläutern Gemeinsamkeiten und Unterschiede von Element- und Teilmengen-Relation ☐ benennen strukturelle Ähnlichkeiten von Mengenlehre und Aussagenlogik ☐ erläutern Bedeutung von Mengen für Mathematik und Informatik ☐ kennen formale Regeln der Umformung von mengentheoretischen Ausdrücken auswendig	☐ beschreiben sprachlich beschriebene Mengen mittels Mengen-Konstruktoren ☐ werten mittels Mengen-Konstruktoren beschriebene Mengen aus ☐ werten Verknüpfungen von Mengen aus ☐ berechnen Wahrheitswerte von Aussagen über Mengen ☐ bestimmen Potenzmengen ☐ bestimmen kartesische Produkte ☐ stellen mengentheoretische Zusammenhänge durch Venn-Diagramme dar ☐ stellen fest, ob Mengen disjunkt sind ☐ führen Operationen Teilmenge, Vereinigung, Schnitt, Differenz, Komplement per Venn-Diagramm aus	☐ interpretieren Definition der Teilmenge als formallogischen Ausdruck und werten Teilmengenbeziehungen formallogisch aus ☐ stellen fest und begründen in welcher Beziehung (Teilmenge oder Element) die leere Menge zu einer anderen Menge steht ☐ unterscheiden Teilmengen- und Element-Relation
methodisch		☐ stellen fest, ob ein Mengen-Konstruktor syntaktisch korrekt formuliert ist ☐ werten Mengen und deren Verknüpfung mit $setLX$ aus	

Hinzu kommen die themenübergreifenden Lernziele aus den Kategorien „sozial“ und „persönlich“ aus Abschnitt 1.7 sowie die Lernziele aus der Kategorie „methodisch“ aller vorangegangener Kapitel. Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben.

5.12 Übungsaufgaben

- Schreiben Sie für $M = \{1, 2, 3\}$ die folgenden Mengen explizit auf, indem Sie alle Elemente angeben:

(a) $\{x : x \in M \mid x \leq 0\}$

(b) $\{x : x \in M \mid 1 < 2\}$

(c) $\{x : x \in M \mid 1 > 2\}$

2. Wenn Sie die algebraischen Eigenschaften der logischen Operatoren auf S. 52 mit denen der mengentheoretischen Operatoren auf S. 69 vergleichen, wird Ihnen auffallen, dass bei den logischen Operatoren mehr Eigenschaften aufgeführt sind. Welche dieser Eigenschaften haben eine Entsprechung in der Mengenlehre und wie lauten diese?
3. Ist das kartesische Produkt kommutativ? Ist es assoziativ?
4. Bestimmen Sie die Potenzmengen der folgenden Mengen und überprüfen Sie Ihre Ergebnisse anschließend mit $\text{setl}X$.
 - (a) $\{1, 2\}$
 - (b) $\{1, 1\}$
 - (c) $\emptyset$
 - (d) $\{\clubsuit, \diamond, \heartsuit, \spadesuit\}$
5. Die Vereinigung $A \cup B$ hat alle Elemente von A *und* alle Elemente von B als Elemente. Warum steht dann in der Definition der Vereinigung (5.5) der logische Operator $\vee$ (und nicht etwa $\wedge$)?

Bearbeiten Sie auch Übungsaufgaben aus anderen Lehrtexten, z. B. die Kontrollfragen und Übungsaufgaben zu Abschnitt 1.2 im Lehrbuch von Teschl und Teschl.

Kapitel 6

Prädikatenlogik

Quite seriously, however, it seems that if we want to be able to communicate at all, we have to adopt some common base, and it pretty well has to include logic.
(Douglas R. Hofstadter)

Hinweis: Dieses Kapitel ist teilweise in Farbe gesetzt. Es wird empfohlen zumindest die Seiten 80-84 farbig auszudrucken.

6.1 Aufwärmübungen

1. Handelt es sich bei den folgenden Sätzen um Aussagen, Aussageformen oder keines von beidem?
 - (a) Jede natürliche Zahl ist positiv.
 - (b) Für alle $n \in \mathbb{N}$ gilt $n > 0$.
 - (c) Wenn $n \in \mathbb{N}$, dann ist $n > 0$.
 - (d) Zu jeder natürlichen Zahl gibt es eine weitere natürliche Zahl, die größer als diese ist.
 - (e) Für jedes $k \in \mathbb{N}$ gibt es ein $n \in \mathbb{N}$, so dass $n > k$.
 - (f) Es gibt eine reelle Zahl x , die größer als y ist.
 - (g) Zu jedem Ehemann gibt es eine Ehefrau.
 - (h) Es gibt eine Ehefrau ohne Ehemann.
2. Modellieren Sie die folgenden Ausdrücke bzw. Berechnungsvorschriften *jeweils als Ganzes* als Funktionenmaschine. Geben Sie also an, welche Größen bzw. Objekte von Außen als Variablen an die Funktionenmaschine übergeben werden müssen, damit diese die Berechnung bzw. Auswertung durchführen kann. Geben Sie auch an, welche Art von Größe bzw. Objekte die Funktionenmaschine zurückgibt.
 - (a)

```
produkt := 1;
for ( int i = 1; i <= obergrenze; i++ ) {
    produkt := produkt * i;
}
```
 - (b)

```
for ( int faktor = 1; faktor <= 9; faktor ++ ) {
    System.out.println("3 x " + faktor + " = " + 3*faktor );
}
```
 - (c) $\sum_{i=0}^{100} i$

$$(d) \sum_{i=0}^{100} i^2$$

$$(e) \sum_{i=0}^{100} i^n$$

$$(f) x > 0 \wedge x < \pi$$

(g) Für alle reelle Zahlen x gilt $x > 0 \vee x < \pi$.

(h) Es gibt eine natürliche Zahl n , für die $n^2 = n$ gilt.

(i) Für alle natürliche Zahlen n gibt es eine natürliche Zahl m , so dass $m = n + 1$.

(j) Für jedes $k \in \mathbb{N}$ gibt es ein $n \in \mathbb{N}$, so dass $n > k$.

3. Negieren Sie folgende Aussagen:

(a) Alle verstehen Logik.

(b) Keiner mag mich.

(c) Für jedes Auto auf dem Parkplatz gibt es einen Besitzer, der alle Fahrzeugpapiere hat.

4. Leitfragen:

(a) Warum ist „keiner“ nicht das Gegenteil von „alle“?

(b) Wie bildet man das Gegenteil von Sätzen mit „alle“, „keiner“ etc.?

(c) Warum muss man unterschiedliche Arten von Variablen unterscheiden?

6.2 Prädikate

Aussageformen haben zwei Bestandteile: Variablen und den Rest. Dieser Rest wird in der Logik als Prädikat bezeichnet.

Beispiel 6.1: Die Aussageform „ x ist größer als 1“ hat die Variable x und das Prädikat „ist größer als 1“. ■

Beispiel 6.2: Die Aussageform „ x ist größer als y “ hat die Variablen x und y und das Prädikat „ist größer als“. ■

Beispiel 6.3: „Wenn eine Zahl durch 4 teilbar ist, ist sie durch 2 teilbar“ kann als Subjunktion zweier Aussageformen modelliert werden. Das Prädikat der ersten Aussageform lautet „ist durch 4 teilbar“, das der zweiten „ist durch 2 teilbar“. Beide Aussageformen haben die Variable „Zahl“. ■

Der aussagenlogische Begriff Prädikat hat wenig mit dem gleichnamigen Konstrukt in der deutschen Grammatik zu tun. Er ist mehr mit den Prädikatsbegriffen in der englischen Grammatik und der Linguistik verwandt, unterscheidet sich von diesen aber auch geringfügig.

Da Prädikate die Bedeutung von Aussageformen und damit ihren Kern ausmachen – die Variablen sind ja nur Symbole –, wird in der Logik häufig nicht streng zwischen Aussageform und Prädikat unterschieden.

Der Teil der Logik, der sich mit Aussageformen und insbesondere mit Wegen befasst, Aussageformen zu Aussagen zu machen, wird als *Prädikatenlogik* bezeichnet. Ein Weg, eine Aussageform zu einer Aussage zu machen, besteht darin, ihre Variable(n) mit Werten zu belegen. Eine andere wichtige Form werden Sie im folgenden Abschnitt kennenlernen.

6.3 Quantifizierte Aussagen

Eine weitere Möglichkeit, Aussageformen zu Aussagen zu machen, ist die Quantifizierung. Dabei geht es darum, ob die Aussageform für alle möglichen Variablenwerte wahr ist, oder auch darum, ob die Aussageform für mindestens einen möglichen Variablenwert wahr ist.

Aussagen der Art

„Für alle x aus der Menge M gilt $P(x)$ “

sind genau dann wahr, wenn die Aussageform $P(x)$ für alle Belegungen von x mit Elementen aus der zugrundegelegten Menge M wahr ist. Man nennt solche Aussagen *All-Aussagen*. Die formale Notation ist

$$\forall(x \in M \mid P(x)). \quad (6.1)$$

Aussagen der Art

„Es gibt ein x aus der Menge M , so dass $P(x)$ gilt“

sind genau dann wahr, wenn die Aussageform $P(x)$ *mindestens* für eine Belegung $x \in M$ wahr ist. Man nennt solche Aussagen *Existenz-Aussagen*. Die formale Notation ist

$$\exists(x \in M \mid P(x)). \quad (6.2)$$

Die Symbole $\forall$ und $\exists$ werden als *All-Quantor* bzw. *Existenz-Quantor* bezeichnet. Das Symbol $\forall$ wird üblicherweise als „für alle“ oder „für jedes“ gelesen. In *setlX* wird es mit `forall` notiert. Für das Symbol $\exists$ sind übliche Lesarten: „es gibt mindestens ein“, „es existiert mindestens ein“ bzw. „für mindestens ein“. Oft wird dabei „mindestens“ auch weggelassen, also „es gibt ein“ usw. In *setlX* wird $\exists$ als `exists` notiert.

Wie die Definition klar sagt, geht es beim Existenz-Quantor um „mindestens ein“, aber nicht um „genau ein“.

Beispiel 6.4: Die Aussage „Es gibt eine natürliche Zahl, deren Quadrat gleich der Zahl selbst ist“, formal notiert als

$$\exists(n \in \mathbb{N} \mid n^2 = n),$$

ist wahr, weil es mindestens eine Belegung $x \in \mathbb{N}$ gibt, die $n^2 = n$ erfüllt. Hier ist es allerdings eine einzige, nämlich $n = 1$. Daher ist die Aussage „Es gibt genau eine natürliche Zahl, deren Quadrat gleich der Zahl selbst ist“ ebenfalls wahr. ■

Beispiel 6.5: Die Aussage „Es gibt eine ganze Zahl, deren Quadrat gleich der Zahl selbst ist“, formal notiert als

$$\exists(n \in \mathbb{Z} \mid n^2 = n),$$

ist wahr, weil es mindestens eine Belegung $n \in \mathbb{Z}$ gibt, die $n^2 = n$ erfüllt. Hier sind es sogar zwei, nämlich $n = 0$ und $n = 1$. Die Aussage „Es gibt genau eine ganze Zahl, deren Quadrat gleich der Zahl selbst ist“ ist hier falsch. ■

6.4 Modellieren mit Quantoren

Jede quantifizierte Aussage hat drei wesentliche Bestandteile: (Erstens) den Quantor und in der quantifizierten Aussageformen (zweitens) die Variablen mit Mengenangabe und (drittens) Prädikate. Um quantifizierte Aussagen mittels Logik modellieren zu können, also formal aufschreiben zu können, müssen in einer Aussage diese drei Bestandteile identifiziert werden. Das ist wie beim Übersetzen: Auch dort müssen charakteristische Bestandteile identifiziert werden (Subjekt, Verb, Objekt *etc.*). Danach ist die Formalisierung ziemlich einfach: Die Quantoren werden passend durch $\forall$ oder $\exists$ ersetzt, die Variablen mit Mengenangabe durch Ausdrücke der Form $x \in M$. Die Aussageform des Prädikats bekommt ggf. ein abkürzendes Symbol. Das Ganze muss dann noch in die syntaktisch richtige Reihenfolge gebracht werden. (Auch das ist alles wie beim Übersetzen: Die charakteristischen Bestandteile werden in die Zielsprache übersetzt.) In diesem Abschnitt konzentrieren wir uns auf das Identifizieren der Bausteine und deren formales Aufschreiben. Die Syntax thematisieren wir in Abschnitt 6.8.

Wir werden die einzelnen Bestandteile in einer Aussage unterschiedlich kennzeichnen: Quantoren werden unterstrichen, Variablen und auch deren Mengenangaben werden unterstrichelt. Prädikate werden unterringelt.

Beispiel 6.6: Wir analysieren den ersten Satz dieses Abschnitts:

Jede quantifizierte Aussage hat drei wesentliche Bestandteile.

Es liegt eine All-quantifizierte Aussage vor ($\forall$). Quantifiziert wird die Variable „Aussage“ aus der Menge aller quantifizierten Aussagen. Symbolisieren wir die Variable mit a und die Menge mit Q , können wir $a \in Q$ schreiben. Die zugrundeliegende Aussageform, nennen wir sie B , ist $B(a) = „a$ hat drei wesentliche Bestandteile“.

Zusammensetzen ergibt das $\forall(a \in Q \mid B(a))$. Fertig ist die Formalisierung! ■

Beispiel 6.7: In „Eine natürliche Zahl ist immer positiv“ ist „immer“ ein versteckter All-Quantor. Man könnte stattdessen auch „Jede natürliche Zahl ist positiv“ sagen. Wir haben hier also eine All-quantifizierte Aussage, mit der Variablen „natürliche Zahl“ und dem Prädikat „ist positiv“. Wenn wir für die Variable das Symbol n verwenden und das Prädikat zweckmäßig mit „ > 0 “ symbolisieren, erhalten wir als logische Formalisierung des Ausgangssatzes:

$$\forall(n \in \mathbb{N} \mid n > 0)$$

Diesen Ausdruck können wir nun in *setlX* auswerten – mit der kleinen Einschränkung, dass wir die unendliche Menge der natürlichen Zahlen geeignet durch eine endliche Menge nähern müssen:

```
=> N:={1..10**2}; forall(n in N|n>0);
~< Result: true >~
```

Das zeigt leider nicht, dass wirklich *alle* natürlichen Zahlen positiv sind, sondern nur die ersten Hundert. Aber es ist ein Anfang. ■

Wir wagen uns zu komplexeren Sätzen vor:

Beispiel 6.8: „Für jede Zahl gibt es eine Zahl, die größer als diese ist.“ Hier haben wir zwei Quantoren: „für jede“ und „es gibt eine“. Daher verwenden wir jetzt zusätzlich Farben, um die beiden Quantoren und die dazugehörigen Variablen auseinanderhalten zu können:

Für jede natürliche Zahl gibt es eine weitere natürliche Zahl, die größer als diese ist.

Wir brauchen hier also zwei Quantoren: $\forall$ und $\exists$. Außerdem zwei Variablen: Nennen wir die erste k und die zweite m . Beide beziehen ihre Werte aus der Menge der natürlichen Zahlen $\mathbb{N}$. Das Prädikat lautet „ist größer als“ – mit dem dazu passenden Symbol $>$.

Setzen wir die formalisierten Bausteine zusammen:

$$\forall(k \in \mathbb{N} \mid \exists(m \in \mathbb{N} \mid m > k))$$

Diesen Ausdruck können wir nun wieder – näherungsweise – in *setlX* auswerten:

```
=> N:={1..10**2}; forall(k in N|exists(m in N|m>k));
~< Result: false >~
```

Hier haben wir nun ein echtes Rechnerproblem. Obwohl die Aussage wahr ist, sagt uns der Rechner, dass sie falsch ist. Das liegt daran, dass es für die größte Zahl in der Menge N keine Zahl in dieser Menge gibt, die größer als diese ist. Bei $\mathbb{N}$ ist die Sachlage anders, denn diese Menge hat kein größtes Element.

Diese Problematik soll uns an dieser Stelle aber nicht grämen. Es geht uns hier ja um korrektes Modellieren, nicht so sehr um korrektes Berechnen. ■

6.5 Negation quantifizierter Aussagen

Die Negation von

„Jeder versteht Logik.“ – formal: $\forall(m \in \text{Menschen} \mid m \text{ versteht Logik})$ –

ist nicht

„Keiner versteht Logik.“ – formal: $\neg \exists(m \in \text{Menschen} \mid m \text{ versteht Logik})$,

sondern

„Jemand versteht Logik nicht.“ – formal: $\exists(m \in \text{Menschen} \mid \neg(m \text{ versteht Logik}))$

Das Gegenteil von „alle“ ist nicht „keiner“, sondern „nicht alle“. „Alle“ und „keiner“ sind Gegensätze, keine Gegenteile! Zwischen den Polen „alle“ und „keine“ gibt es viel Raum für „jemanden“ bzw. „einige“.

Es ist hier so, dass die Negation einer All-Aussage eine Existenz-Aussage ist, bei der die Negation „nach hinten“ vor die quantifizierte Aussageform „durchgereicht“ wird. Dies gilt allgemein so, und entsprechend auch für den Existenzquantor:

Negation quantifizierter Aussagen

$$\neg \forall(x \in M \mid P(x)) \Leftrightarrow \exists(x \in M \mid \neg P(x)) \quad (6.3)$$

$$\neg \exists(x \in M \mid P(x)) \Leftrightarrow \forall(x \in M \mid \neg P(x)) \quad (6.4)$$

Für $\neg \forall \dots$ ist auch die Notation $\nexists \dots$ üblich, entsprechend $\nexists \dots$ statt $\neg \exists \dots$.

Der wohl sicherste Weg, Aussagen wie „Jeder versteht Logik“ korrekt zu negieren, stellt der Vierfelderprozess aus Abb. 2.3 dar: 1. Aussage formalisieren – 2. formalisierte Aussage negieren – 3. negierte formalisierte Aussage versprachlichen.

Den ersten Schritt sind wir oben schon gegangen: $\forall(m \in \text{Menschen} \mid m \text{ versteht Logik})$. Nun kommt die Negation:

$$\neg \forall(m \in \text{Menschen} \mid m \text{ versteht Logik}) \Leftrightarrow \exists(m \in \text{Menschen} \mid \neg(m \text{ versteht Logik}))$$

Schließlich die Versprachlichung: „Es gibt einen Menschen, der nicht Logik versteht“ – oder etwas geschliffener: „Jemand versteht Logik nicht.“

Beispiel 6.9: „Jemand läuft am schnellsten“ kann formal als

$$\exists(m \in \text{Menschen} \mid m \text{ läuft am schnellsten})$$

geschrieben werden. Formale Negation ergibt

$$\neg \exists(m \in \text{Menschen} \mid m \text{ läuft am schnellsten}) \Leftrightarrow \forall(m \in \text{Menschen} \mid \neg(m \text{ läuft am schnellsten}))$$

Also: „Für alle Menschen gilt, dass sie nicht am schnellsten laufen“ bzw. „Alle Menschen laufen nicht am schnellsten.“ ■

Natürlich gibt es immer eine „billige Methode“ quantifizierte Aussagen zu negieren: Einfach ein Nicht an den Anfang des Satzes stellen. Wenn wir Logik als Modellierungswerkzeug verwenden wollen, sind wir an solchen auf einfache Art verneinten Sätzen natürlich nicht interessiert, sondern an gleichwertigen Ausdrucksformen, bei denen das Nicht eben nicht ziemlich am Anfang steht.

6.6 Gebundene und freie Variablen

Ist $\exists(n \in \mathbb{N} \mid n^2 = n)$ Aussage oder Aussageform? 

Gehen wir zurück auf S. 16 zur Definition der Aussageform: „Aussageformen sind Sätze oder Ausdrücke, die variable Elemente enthalten und erst durch Belegung dieser variablen Elemente mit konkreten Werten zu einer Aussage werden.“ Der Ausdruck $\exists(n \in \mathbb{N} \mid n^2 = n)$ erfüllt die erste definierende Bedingung einer Aussageform. Er enthält das variable Element n . Aber er erfüllt nicht die zweite definierende Bedingung, denn $\exists(n \in \mathbb{N} \mid n^2 = n)$ ist wahr, *ohne* dass wir n mit einem Wert belegen müssen. Es liegt hier also eine Aussage und keine Aussageform vor.

Das ist auch deutlich in *setlX* zu sehen. Dort wird der Ausdruck problemlos ausgewertet (streng genommen eine Näherung, denn wir können die unendliche Menge $\mathbb{N}$ nicht in *setlX* repräsentieren):

```
=> exists(n in {1..10**6}|n**2==n);
~< Result: true >~
```

Hier wurde n nicht mit einem Wert belegt! Oder sehen Sie irgendwo einen Zuweisungsoperator $:=$ in diesem *setlX*-Ausdruck?

Anders ist die Angelegenheit bei $\exists(n \in N \mid n < y)$. Hier gibt es zwei Variablensymbole: n und y . Im Lichte der Definition von Aussageform ist y die relevante Variable. Erst wenn y mit einem konkreten Wert belegt wird, wird $\exists(n \in N \mid n < y)$ zu einer Aussage. Das ist wieder deutlich in *setlX* zu sehen:

```
=> exists(n in {1..10**6}|n<y);
Error in "exists(n in {1 .. 10 ** 6} | n < y)":
[...]
1.1: exists(n in {1 .. 10 ** 6} | n < y) FAILED

=> y:=3/4; exists(n in {1..10**6}|n<y);
~< Result: false >~
```

Solange y nicht mit einem Wert belegt wurde, muss uns *setlX* den Ausdruck „um die Ohren hauen“.

Es gibt offenbar zwei Arten von Variablen, die unterscheidbar und unterscheidungswürdig sind. Der Unterschied besteht in den obigen Beispielen darin, dass wir im Falle von n nicht die Freiheit haben, n mit einem Wert zu belegen. Das macht der Quantor für uns, indem er sich die Aussageform für alle möglichen Belegungen von n (hier jede natürliche Zahl) anschaut und überprüft, ob die Aussageform für eine dieser möglichen Belegungen von n wahr ist.

Wir können also nicht einfach hergehen und n frei mit irgendeinem Wert belegen, z. B. weder mit π noch mit 42. Im ersten Fall wäre der Wert, mit dem n belegt ist, keine natürliche Zahl. Das ist aber gar nicht das Schlimme. Das Schlimme tritt bei einer Belegung $n = 42$ auf, denn dann würden wir bspw. statt $\exists(n \in \mathbb{N} \mid n^2 = n)$ nur $42^2 = 42$ auswerten. Die erste Aussage ist aber wahr, während die zweite falsch ist.

Dagegen sind wir in der Wahl von y völlig frei. Wir können $y = 3/4$ belegen oder $y = -42$ oder auch $y = \text{„Auto“}$.

Es gibt also zwei Arten von Variablen:

Sind alle Vorkommen einer Variable innerhalb eines Ausdrucks an große Operatoren wie $\forall$, $\exists$, $\sum$ gebunden, bezeichnet man die Variable als *gebunden*, andernfalls als *frei*.

Was große Operatoren sind, werden wir in Abschnitt 6.9 klären. Bis dahin sollen die drei genannten als Beispiele dienen.

Beispiel 6.10: In

$$\forall(k \in S \mid \exists(n \in S \mid n > k))$$

sind k und n gebunden: k an $\forall$, n an $\exists$. Dagegen ist S eine freie Variable. Es liegt also eine Aussageform mit der Variablen S vor. Erst wenn diese mit einer konkreten Menge belegt ist, liegt eine Aussage vor. ■

Beispiel 6.11: In

$$\exists\left(n \in \mathbb{N} \mid n = \sum_{i=1}^{100} i\right)$$

sind i und n gebunden: i an $\sum$, n an $\exists$. Es liegt also eine Aussage vor. ■

Die Symbole gebundener Variablen können durch andere (vorher nicht vorkommende) Variablensymbole ersetzt werden. Dadurch entsteht ein äquivalenter Ausdruck.

Beispiel 6.12: In $\exists(n \in \mathbb{N} \mid n < k)$ kann die gebundene Variable n gefahrlos durch m ersetzt werden, denn m kommt bisher nicht vor:

$$\exists(m \in \mathbb{N} \mid m < k) \Leftrightarrow \exists(n \in \mathbb{N} \mid n < k)$$

Egal mit welchem Wert man k belegt, beide Ausdrücke haben immer denselben (k -abhängigen) Wahrheitswert. ■

Beispiel 6.13: In $\exists(n \in \mathbb{N} \mid n < k)$ kann n nicht durch k ersetzt werden, da k bereits verwendet wird. Aus der k -abhängigen Aussageform würde so eine (zudem falsche) Aussage werden: $\exists(k \in \mathbb{N} \mid k < k)$ ■

Bisher haben wir den Unterschied von freien und gebundenen Variablen nur an formalisierten Beispielen untersucht. Wagen wir uns nun an ein sprachliches Beispiel:

Beispiel 6.14: „Es gibt einen Mann, der mit ihr verheiratet ist.“ Wenn der Kontext keinen Hinweis darauf gibt, welche konkrete Person mit „ihr“ gemeint ist, müssen wir „ihr“ als variabel betrachten.

Wir haben in diesem Fall einen Existenz-Quantor, zwei Variablen („Mann“ und „ihr“) und das Prädikat „ist verheiratet mit“.

Es gibt einen Mann, der mit ihr verheiratet ist.

Hier ist deutlich zu sehen, dass die Variable „Mann“ an den Existenz-Quantor gebunden ist. Es ist also eine gebundene Variable. Dagegen ist „ihr“ nicht gebunden, also eine freie Variable. Der Ausgangssatz hat also eine freie Variable und ist daher keine Aussage, sondern eine Aussageform.

Zum Formalisieren dieser Aussageform können wir „Mann“ mit m symbolisieren, „ihr“ mit f . Dann haben wir:

$$\exists(m \in \text{Männer} \mid m \text{ ist verheiratet mit } f)$$

In dieser Form ist der Ausgangssatz nun recht deutlich als Aussageform mit der Variablen f zu erkennen. ■

Beispiel 6.15: $\sqrt{2} < \pi$ ist eine Aussage. $\forall(n \in \mathbb{N} \mid \sqrt{2} < \pi)$ ist eine quantifizierte Aussage von der Form $\forall(n \in \mathbb{N} \mid P(n))$ mit der Aussageform P . Eine Besonderheit besteht hier allerdings darin, dass $P(n) = (\sqrt{2} < \pi)$ für jede Belegung von n den Wahrheitswert $\mathbb{1}$ hat. Deshalb ist $\forall(n \in \mathbb{N} \mid \sqrt{2} < \pi)$ wahr.

Nur was ist nun $P(n) = (\sqrt{2} < \pi)$ – Aussage oder Aussageform? Beides!

$\sqrt{2} < \pi$ kann als Aussage modelliert werden, denn es hat keine freien Variablen. Es kann auch als Aussageform mit der Variablen n und dem konstanten Rückgabewert $\sqrt{2} < \pi$ modelliert werden.¹ In diesem Sinne „sind“ Aussagen spezielle Aussageformen, d. h. Aussagen können auch immer als Aussageformen modelliert werden. ■

6.7 Geschachtelte quantifizierte Aussagen

Quantoren können geschachtelt werden.

„Es gibt eine natürliche Zahl, die um 1 größer ist als jede natürliche Zahl“

enthält zwei Quantoren und kann formal so geschrieben werden:

$$\exists(k \in \mathbb{N} \mid \forall(n \in \mathbb{N} \mid k = n + 1)) \quad (6.5)$$

Die Schachtelung der Quantifizierungen ist in der formalen Form deutlich.

¹Vgl. Fußnote auf S. 31.

In (6.5) steht innen die Aussageform $\forall(n \in \mathbb{N} \mid k = n + 1)$ mit der nun freien Variable k . Kapseln wir diese Aussageform im Symbol S , also $S(k) = \forall(n \in \mathbb{N} \mid k = n + 1)$, können wir statt (6.5) auch

$$\exists(k \in \mathbb{N} \mid S(k))$$

schreiben.

Der Wahrheitswert der Aussage (6.5) ist übrigens 0, denn S ist eine Kontradiktion. Wie wir auch k wählen, es wird die geforderte Bedingung nie für alle n und damit unabhängig von n erfüllen: Denn für gegebenes k erfüllt *nur ein spezielles* n die Bedingung $k = n + 1$ (nämlich $n = k - 1$), aber eben *nicht alle* n , wie es eigentlich gefordert ist.

Bei solchen geschachtelten Sätzen ist die Reihenfolge der Quantoren i. A. wichtig. Vertauschen wir die Reihenfolge der Quantoren in unserer obigen Aussage

„Es gibt eine natürliche Zahl, die um 1 größer ist als jede natürliche Zahl“,

erhalten wir

„Zu jeder natürlichen Zahl gibt es eine natürliche Zahl, die um 1 größer als diese ist.“

Diese „gedrehte Aussage“ lässt sich formal als

$$\forall(n \in \mathbb{N} \mid \exists(k \in \mathbb{N} \mid k = n + 1)) \quad (6.6)$$

schreiben.

Auch hier kapseln wir wieder den inneren Teil $\exists(k \in \mathbb{N} \mid k = n + 1)$ um deutlich zu machen, dass dies eine Aussageform mit der Variablen n ist. Lassen Sie uns diese Aussageform P nennen. Dann lässt sich die Aussage (6.6) verkürzt als

$$\forall(n \in \mathbb{N} \mid P(n))$$

schreiben. Ihr Wahrheitswert ist 1, denn $P(1) = 1$ (die um 1 größere Zahl zu 1 ist 2), ebenso $P(2) = 1$ (die um 1 größere Zahl ist dann 3) usw. Die beiden Aussagen (6.5) und (6.6) haben also unterschiedliche Wahrheitswerte und können daher nicht gleichwertig sein.

Halten wir zur Sicherheit noch mal fest:

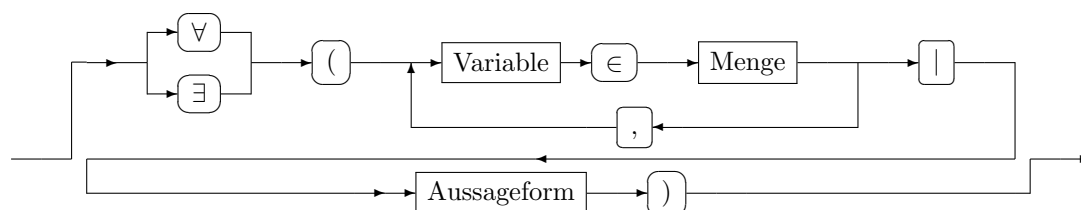
Bei Sätzen mit geschachtelten Quantoren ist die Reihenfolge der Quantoren i. A. wichtig. Das bedeutet, dass i. A.

$$\forall(x \in M \mid \exists(y \in N \mid P(x, y))) \not\equiv \exists(y \in N \mid \forall(x \in M \mid P(x, y))).$$

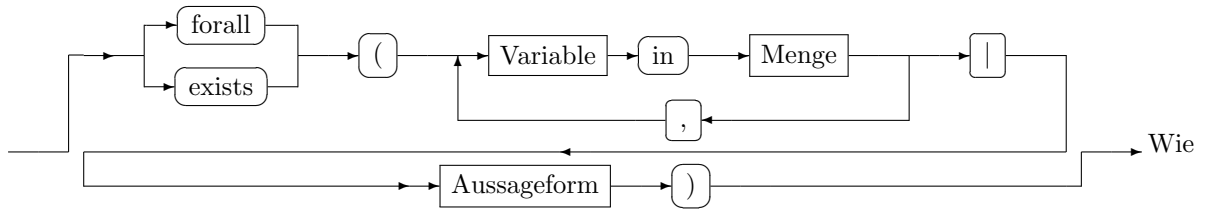
Beispiel 6.16: „Für alle Hemden gibt es einen Käufer“ und „Es gibt einen Käufer für alle Hemden“ sind zwei doppeltquantifizierte Aussagen. Die eine geht jeweils aus der anderen durch Vertauschen der Reihenfolge der Quantifizierungen hervor. Die beiden Aussagen sind nicht gleichbedeutend. ■

6.8 Syntax

Die folgenden *Railway*-Diagramme visualisieren die Syntax-Regeln für quantifizierte Aussagen. Auf Papier:



In *setlX*:



Sie sehen können, ist die Syntax in *setlX* zu der auf Papier identisch – abgesehen von der Verwendung der Symbole $\forall$, $\exists$ und $\in$, die es auf der Tastatur ja nicht gibt.

Beispiel 6.17: Aufgrund dieser Regeln sind Ausdrücke der Art

$$x^2 \geq 0 \forall x \in \mathbb{R}$$

für „ x^2 ist größer oder gleich 0 für jede reelle Zahl“ syntaktisch falsch. Es müsste heißen:

$$\forall(x \in \mathbb{R} \mid x^2 \geq 0)$$

■

Abweichend vom obigen Syntaxdiagramm haben sich in einigen Spezialfällen Kurzschreibweisen etabliert. Tritt derselbe Quantor mehrfach hintereinander auf, wird er häufig nur einmal geschrieben, z. B.

$$\forall(x \in M, y \in N \mid P(x, y)) \quad \text{statt} \quad \forall(x \in M \mid \forall(y \in N \mid P(x, y))). \quad (6.7)$$

Wenn zusätzlich die Mengen gleich sind, wird die Menge nur einmal genannt, z. B.

$$\forall(x, y \in M \mid P(x, y)) \quad \text{statt} \quad \forall(x \in M, y \in M \mid P(x, y)). \quad (6.8)$$

Wenn aus dem Kontext klar ist, welche Menge M zugrunde liegt, dann wird diese oft nicht mehr genannt, z. B.

$$\forall(x \mid P(x)) \quad \text{statt} \quad \forall(x \in M \mid P(x)) \quad (6.9)$$

Entsprechend jeweils für den Existenzquantor.

Beispiel 6.18:

- (a) Das Kommutativgesetz „Für alle reellen Zahlen a, b gilt $a+b = b+a$ “ hat die Form $\forall(a \in \mathbb{R} \mid \forall(b \in \mathbb{R} \mid a+b = b+a))$. Es kann aber auch in der Form $\forall(a, b \in \mathbb{R} \mid a+b = b+a)$ geschrieben sein.
- (b) „Das Quadrat jeder reellen Zahl ist nicht negativ“ kann durchaus in der Form $\forall(x \mid x^2 \geq 0)$ geschrieben sein, wenn aus dem Kontext klar wird, dass $x \in \mathbb{R}$ gemeint ist.

■

Wie so häufig sind auch bei Quantoren andere Notationsformen üblich, z. B.

$$\forall x \in S : P(x) \quad \text{statt} \quad \forall(x \in S \mid P(x)).$$

6.9 Quantoren als große Operatoren

Große Operatoren bezeichnen die Mehrfachausführung eines kommutativen und assoziativen binären Operators. Vermutlich ist Ihnen bereits der Summenoperator Σ vertraut, der die Mehrfachausführung der Addition notiert:

$$\sum_{i=1}^N a_i = a_1 + a_2 + \dots + a_{N-1} + a_N$$

Entsprechend steht der Produktoperator $\prod$ für die Mehrfachausführung der Multiplikation

$$\prod_{i=1}^N a_i = a_1 \cdot a_2 \cdot \dots \cdot a_{N-1} \cdot a_N.$$

In beiden Fällen kommen die Indizes aus der Menge der Zahlen $\{1, \dots, N\}$. Im Prinzip sind auch nicht fortlaufende Zahlenmengen von Indizes denkbar. Dann schreibt man

$$\sum_{i \in M} \quad \text{bzw.} \quad \prod_{i \in M}$$

und die Indizes werden einer Menge M entnommen. In jedem Fall sind die Indexsymbole (hier i) gebundene Variablen.

Beispiel 6.19:

$$\begin{aligned} \sum_{i \in \{1, 3, 9\}} i^2 &= 1^2 + 3^2 + 9^2 = 91 \\ \prod_{i \in \{1, 3, 9\}} i &= 1 \cdot 3 \cdot 9 = 27 \end{aligned}$$

■

Die Indexmenge muss dabei nicht einmal natürliche Zahlen oder generell Zahlen umfassen.

Beispiel 6.20:

$$\begin{aligned} \sum_{i \in \{1/2, 2, \pi\}} i^2 &= \left(\frac{1}{2}\right)^2 + 2^2 + \pi^2 \\ \sum_{i \in \{x, y, z\}} i^2 &= x^2 + y^2 + z^2 \end{aligned}$$

■

Auch die Operatoren der Konjunktion und Disjunktion sind assoziativ und kommutativ. Entsprechend können wir folgende Symbole einführen:

$$\begin{aligned} \bigwedge_{i=1}^N A_i &= A_1 \wedge A_2 \wedge \dots \wedge A_{N-1} \wedge A_N \\ \bigvee_{i=1}^N A_i &= A_1 \vee A_2 \vee \dots \vee A_{N-1} \vee A_N \end{aligned}$$

Der erste Ausdruck ist nur dann wahr, wenn alle Aussagen A_i wahr sind. Der zweite Ausdruck ist nur dann falsch, wenn alle Aussagen A_i falsch sind. Sie sollten diese beiden großen Operatoren bereits unter anderen Namen kennen. Welche sind das?



Wenn wir keine fortlaufenden Indizes haben, können wir diese wieder aus einer beliebigen Menge nehmen.

Beispiel 6.21: Für $M = \{2, 4, 6\}$ und $A(i) = „i \text{ ist Primzahl}“$ ist

$$\begin{aligned} \bigwedge_{i \in M} A(i) &= A(2) \wedge A(4) \wedge A(6) = \mathbb{1} \wedge \mathbb{0} \wedge \mathbb{0} = \mathbb{0} \\ \bigvee_{i \in M} A(i) &= A(2) \vee A(4) \vee A(6) = \mathbb{1} \vee \mathbb{0} \vee \mathbb{0} = \mathbb{1} \end{aligned}$$

Nicht alle Elemente von M sind Primzahlen, aber es gibt ein Element von M , das eine Primzahl ist. ■

Der große Operator zur Konjunktion ist natürlich $\forall$ und der große Operator zur Disjunktion ist $\exists$.

Dass $\forall$ der große Operator zu $\wedge$ ist und $\exists$ der zu $\vee$, erklärt auch, warum die Reihenfolge der Quantoren von Bedeutung ist (siehe Abschnitt 6.7). Schon bei Konjunktion und Disjunktion kommt es auf die Reihenfolge der Operatoren an. $A \wedge (B \vee C)$ und $(A \wedge B) \vee C$ haben unterschiedliche Bedeutung (siehe auch (4.12-4.13)).

Es bleibt noch zu klären wie die Bindungsstärke der großen Operatoren im Vergleich zu ihren „kleinen Geschwistern“ ist. Generell binden große Operatoren immer stärker als die Operatoren, die sie mehrfach ausführen. Die Präzedenz ist also mit von links nach rechts zunehmender Bindungsstärke:

$$\Leftrightarrow \rightarrow \leftrightarrow \vee \nrightarrow \wedge \neg \quad \begin{array}{c} \exists \\ \forall \end{array} \quad \begin{array}{c} = \\ \neq \\ > \\ \geq \\ < \\ \leq \end{array} \quad + \quad \sum \quad \cdot \quad \prod \quad \text{Potenz} \quad (6.10)$$

Beispiel 6.22:

$$\sum_{i=1}^N a_i + b \text{ bedeutet } \left(\sum_{i=1}^N a_i \right) + b, \text{ aber nicht } \sum_{i=1}^N (a_i + b).$$

■

6.10 Tautologien und quantifizierte Aussagen

Wir hatten in Abschnitt 3.3 festgestellt, dass

„ n ist durch 4 teilbar $\Rightarrow$ n ist durch 2 teilbar“

eine Tautologie ist. Formal lässt sie sich in der Form

$$T_4(n) \Rightarrow T_2(n) \quad (6.11)$$

schreiben, wobei hier die Symbole T_4 und T_2 für die Teilbarkeit durch 4 bzw. 2 stehen sollen.

Eine alternative Formulierung der Ausgangsaussage ist

„Für jede natürliche Zahl gilt: wenn sie durch 4 teilbar ist, dann ist sie auch durch 2 teilbar.“

Ausgehend von dieser Formulierung ist die Formalisierung

$$\forall (n \in \mathbb{N} \mid T_4(n) \rightarrow T_2(n)). \quad (6.12)$$

Wenn wir noch explizit ausdrücken wollen, dass dies eine Tautologie ist, müssen wir

$$\forall (n \in \mathbb{N} \mid T_4(n) \rightarrow T_2(n)) \Leftrightarrow \mathbb{1} \quad (6.13)$$

schreiben.

Wegen der gleichen Bedeutung der beiden Formulierungen müssen die Ausdrücke (6.11) und (6.13) äquivalent sein:

$$(T_4(n) \Rightarrow T_2(n)) \Leftrightarrow (\forall (n \in \mathbb{N} \mid T_4(n) \rightarrow T_2(n)) \Leftrightarrow \mathbb{1}) \quad (6.14)$$

Das sieht auf den ersten Blick seltsam aus, weil auf der linken Seite die Variable n scheinbar ungebunden auftritt, aber auf der rechten Seite gebunden ist. Tatsächlich steckt in der Formulierung der Ausgangstautologie aber ein versteckter, *impliziter All-Quantor*, weshalb die beiden Formulierungen ja gleichwertig sind.

Genauso gut kann die Ausgangsaussage auch als

„Jede natürliche Zahl, die durch 4 teilbar ist, ist durch 2 teilbar.“

formuliert werden. Es liegt wieder eine All-quantifizierte Aussage vor. Die gebundene Variable ist aus der Menge der natürlichen Zahlen, die durch 4 teilbar sind. Symbolisieren wir die Variable mit n und die Menge mit V_4 , erhalten wir als formallogischen Ausdruck der dritten Formulationsvariante

$$\forall(n \in V_4 \mid T_2(n)) \Leftrightarrow 1 \quad (6.15)$$

Dabei wurde wieder explizit vermerkt, dass es sich um eine Tautologie handelt.

Alle drei formallogischen Ausdrücke müssen äquivalent sein, insbesondere die letzten beiden:

$$\forall(n \in V_4 \mid T_2(n)) \Leftrightarrow \forall(n \in \mathbb{N} \mid T_4(n) \rightarrow T_2(n))$$

Wir können diese beiden formallogischen Ausdrücke noch gleichartiger machen, indem wir die Aussageform $T_4(n)$ – sprich: n ist durch 4 teilbar – als $n \in V_4$ – sprich: n ist Element der durch 4 teilbaren natürlichen Zahlen – schreiben. Damit erhalten wir

$$\forall(n \in \mathbb{N} \mid n \in V_4 \rightarrow T_2(n)) \Leftrightarrow \forall(n \in V_4 \mid T_2(n)). \quad (6.16)$$

Die beiden Beziehungen (6.14) und (6.16), die wir gerade anhand der Ausgangsaussage hergeleitet haben, gelten auch allgemein:

Implikationen $A(x) \Rightarrow B(x)$ mit $x \in \mathcal{U}$ lassen sich auf zwei verschiedene Weisen als All-Aussagen schreiben, die Tautologien sind:

$$(A(x) \Rightarrow B(x)) \Leftrightarrow (\forall(x \in \mathcal{U} \mid A(x) \rightarrow B(x)) \Leftrightarrow 1) \quad (6.17)$$

$$\forall(x \in \mathcal{U} \mid P(x)) \Leftrightarrow \forall(x \in \mathcal{U} \mid x \in \mathcal{U} \rightarrow P(x)) \quad (6.18)$$

Von rechts nach links gelesen ist (6.18) eine Vereinfachung. Da $x \in \mathcal{U}$ aber immer wahr ist – es werden bei der All-Quantifizierung nur solche x -Werte gewählt – ist $x \in \mathcal{U} \rightarrow P(x) \Leftrightarrow 1 \rightarrow P(x) \Leftrightarrow P(x)$.

Von links nach rechts gelesen sagt (6.18) aus, dass in jedem All-quantifizierten Ausdruck der Form $\forall(x \in \mathcal{U} \mid P(x))$ implizit über eine Subjunktion $x \in \mathcal{U} \rightarrow P(x)$ quantifiziert wird.

Die Beziehung (6.17) für die Implikation gilt entsprechend für die Äquivalenz:

Äquivalenzen $A(x) \Leftrightarrow B(x)$ mit $x \in \mathcal{U}$ lassen sich auf folgende Weise als All-Aussagen schreiben, die Tautologien sind:

$$(A(x) \Leftrightarrow B(x)) \Leftrightarrow (\forall(x \in \mathcal{U} \mid A(x) \leftrightarrow B(x)) \Leftrightarrow 1) \quad (6.19)$$

Beispiel 6.23: Die Definition der Geradheit kann als

$$n \text{ ist gerade} \Leftrightarrow n \bmod 2 = 0$$

geschrieben werden oder auch als

$$\forall(n \in \mathbb{N} \mid n \text{ ist gerade} \leftrightarrow n \bmod 2 = 0) \Leftrightarrow 1.$$

■

Definitionen und Theoreme als Tautologien lassen sich immer zu All-quantifizierten Aussagen umformulieren. Manchmal ist der All-Quantor direkt im Wortlaut erkennbar – etwa im Theorem

„Jede bijektive Funktion ist invertierbar.“ Manchmal ist die All-Quantifizierung auch nur implizit enthalten – etwa in „Eine Funktion ist genau dann invertierbar, wenn sie bijektiv ist.“

Auch Rechengesetze sind implizit All-quantifiziert. So bedeutet beispielsweise das Kommutativgesetz $a + b = b + a$ der Addition reeller Zahlen

$$\forall(a, b \in \mathbb{R} \mid a + b = b + a).$$

Das Kommutativgesetz der Disjunktion $A \vee B \Leftrightarrow B \vee A$ in (4.8) bedeutet

$$\forall(A, B \in \mathbb{B} \mid A \vee B \leftrightarrow B \vee A),$$

wobei $\mathbb{B} = \{0, 1\}$ für die Menge der Booleschen Werte steht.

Beispiel 6.24: Die Definition von Teilmenge aus Abschnitt 5.3 lautet formal aufgeschrieben

$$A \subseteq B \Leftrightarrow \forall(x \in A \mid x \in B)$$

oder aufgrund von (6.18)

$$A \subseteq B \Leftrightarrow \forall(x \in A \mid x \in A \rightarrow x \in B).$$

Diese formale Form der Definition wenden wir nun auf $A = \emptyset$ an, um zu untersuchen, ob die leere Menge Teilmenge einer beliebigen Menge B ist:

$$\emptyset \subseteq B \Leftrightarrow \forall(x \in \emptyset \mid x \in \emptyset \rightarrow x \in B) \quad (6.20)$$

Dazu müssen wir den Ausdruck $x \in \emptyset \rightarrow x \in B$ auswerten:

$$\begin{aligned} & x \in \emptyset \rightarrow x \in B \\ \Leftrightarrow & 0 \rightarrow x \in B, \text{ weil } \emptyset \text{ keine Elemente hat} \\ \Leftrightarrow & 1 \end{aligned}$$

Damit wird (6.20) zu

$$\begin{aligned} \emptyset \subseteq B & \Leftrightarrow \forall(x \in \emptyset \mid x \in \emptyset \rightarrow x \in B) \\ & \Leftrightarrow \forall(x \in \emptyset \mid 1) \\ & \Leftrightarrow 1. \end{aligned}$$

Es gilt also immer $\emptyset \subseteq B$ – die leere Menge ist Teilmenge einer jeden Menge. ■

6.11 Empfohlene Vertiefungen

6.11.1 Sichtbarkeitsbereich von gebundenen Variablen

Gebundene Variable werden außerhalb der Struktur, an die sie gebunden sind, gewöhnlich als unsichtbar behandelt.

Beispiel 6.25: Das n in $\forall(n \in \{1, 2, 3\} \mid n > 0)$ ist außerhalb der durch den All-Quantor geklammerten Struktur nicht sichtbar. Daher kann n außerhalb keinen zugewiesenen Wert haben. Die folgende Berechnung in *setlX* zeigt dies deutlich:

```
=> forall(n in {1,2,3} | n > 0);
~< Result: true >~
=> n;
~< Result: om >~
```

Nach der Berechnung von $\forall(n \in \{1, 2, 3\} \mid n > 0)$ hat n keinen zugewiesenen Wert (*om* ist das *setlX*-Symbol für „hat keinen zugewiesenen Wert“). ■

Auch in den meisten Programmiersprachen sind gebundene Variable außerhalb der sie bindenden Struktur nicht sichtbar. Dort wird der Sichtbarkeitsbereich auch (lexikalischer) *Scope* genannt.

Beispiel 6.26: Die an die *for*-Schleife gebundene Variable *n* hat außerhalb von

```
forall(n=1,n<=3,n++){
  ...
}
```

keinen zugewiesenen Wert. Der *Scope* von `n` ist auf die `for`-Schleife beschränkt. ■

Auch in *setlX* haben gebundene Variablen lexikalischen *Scope*. Allerdings gibt es eine durchaus sinnvolle Ausnahme: Wenn eine Existenz-Aussage wahr ist, wird die gebundene Variable außerhalb der Existenz-Aussage sichtbar gemacht.

Beispiel 6.27: Die folgende Berechnung zeigt, dass nicht nur der Wahrheitswert einer Existenz-Aussage berechnet werden, sondern auch *ein* Wert bestimmt wird, für den die Existenz-Aussage wahr ist:

```
=> exists(n in {1,2,3} | n>1);
~< Result: true >~
=> n;
~< Result: 2 >~
```

■

6.11.2 Rekursive Definition

Die Schreibweise

$$\sum_{i=1}^N a_i = a_1 + a_2 + \dots + a_{N-1} + a_N$$

ist in der Regel für Menschen verständlich, aber kaum formal. Ein Rechner würde Pünktchen nicht verstehen. Wir brauchen also eine andere Form, um die Bedeutung des Summenzeichens $\sum$ zu beschreiben. Das geeignete Hilfsmittel ist hier die Rekursion:

$$\begin{aligned} \sum_{i=1}^1 a_i &= a_1 \\ \sum_{i=1}^N a_i &= a_N + \sum_{i=1}^{N-1} a_i \end{aligned}$$

Dadurch wird das $\sum$ -Symbol *rekursiv* definiert.

6.11.3 Große Operatoren der Mengenlehre

Auch $\cup$ und $\cap$ sind binäre, kommutative, assoziative Operatoren. Entsprechend bedeuten

$$\begin{aligned} \bigcup_{i=1}^N S_i &= S_1 \cup S_2 \cup \dots \cup S_{N-1} \cup S_N \\ \bigcap_{i=1}^N S_i &= S_1 \cap S_2 \cap \dots \cap S_{N-1} \cap S_N \end{aligned}$$

oder rekursiv definiert:

$$\begin{aligned} \bigcup_{i=1}^1 S_i &= S_1, & \bigcup_{i=1}^N S_i &= S_N \cup \bigcup_{i=1}^{N-1} S_i \\ \bigcap_{i=1}^1 S_i &= S_1, & \bigcap_{i=1}^N S_i &= S_N \cap \bigcap_{i=1}^{N-1} S_i \end{aligned}$$

Auch hier binden die großen Operatoren stärker als die entsprechenden „kleinen“.

6.11.4 Große Operatoren in *setlX*

In *setlX* wird die Idee der großen Operatoren konsequent verwendet, indem statt Σ die Zeichenkette `+/` verwendet wird und `*/` statt Π . Das, was summiert bzw. multipliziert wird, ist geordnet – es hat ja einen eigenen Index – und wird daher als Tupel (*Array*) notiert. Summe bzw. Produkt der ersten 10 natürlichen Zahlen werden dann bspw. so berechnet:

```
=> +/[1..10];
~< Result: 55 >~
=> */[1..10];
~< Result: 3628800 >~
```

Da `+` bzw. `*` in *setlX* auch für $\cup$ bzw. $\cap$ stehen, haben `+/` bzw. `*/` im Kontext von Mengen die Bedeutung der entsprechenden großen Operatoren:

```
=> +/[{1,2},{2,3},{4,5}];
~< Result: {1, 2, 3, 4, 5} >~
=> */[{1,2},{2,3},{4,5}];
~< Result: {} >~
```

6.12 Lernziele

Studierende	Kennen	Können	Verstehen
fachlich	☐ erläutern den Zusammenhang von Quantoren zu „großen Operatoren“	☐ negieren quantifizierte natürlichsprachliche Aussagen mittels formaler Ausdrücke ☐ übersetzen formallogische quantifizierte Aussagen in natürliche Sprache ☐ können formallogische Ausdrücke in verschiedenen Notationssystemen lesen	☐ modellieren quantifizierte, natürlichsprachliche Aussagen durch formale Ausdrücke ☐ entscheiden, ob eine Variable frei oder gebunden ist ☐ erkennen Quantoren und Prädikate in natürlichsprachlichen Sätzen
methodisch		☐ werten quantifizierte Aussagen mit <i>setlX</i> aus ☐ stellen fest, ob ein quantifizierter formallogischer Ausdruck syntaktisch korrekt formuliert ist	

Hinzu kommen die themenübergreifenden Lernziele aus den Kategorien „sozial“ und „persönlich“ aus Abschnitt 1.7 sowie die Lernziele aus der Kategorie „methodisch“ aller vorangegangener Kapitel. Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben.

6.13 Übungsaufgaben

1. Modellieren Sie die folgenden Sätze als formallogische Aussagen:

- „Jedes Töpfchen hat sein Deckelchen.“
- „Jemand wird besser als der Rest sein.“

- (c) „Zu jedem Element der Definitionsmenge gibt es maximal einen Wert der Wertemenge.“
2. Was bedeutet die Präzedenz von $\sum$ relativ zu $+$ für den Wert von $\sum_{i=1}^{100} i - 5050$? Ist er negativ?
3. Geben Sie jeweils an, ob es sich bei den verwendeten Variablensymbolen um freie oder gebundene Variablen handelt.
- (a) $\{n : n \in \mathbb{N} | n^2 = n\}$
- (b) $\{n : n \in A | n^2 = n\}$
- (c) `groesser_y := procedure(y){ return {x: x in {1,2,3} | x > y}; };`
- (d) $\sum_{i=N}^M a \cdot i^2$
4. Sind „nicht für alle gilt ...“, „für alle gilt ... nicht“ und „für alle gilt nicht ...“ gleichwertige Aussagen?
5. Welche Quantoren sind in den folgenden Sätzen bzw. Ausdrücken versteckt? Schreiben Sie die Sätze jeweils formallogisch auf und machen Sie dabei die implizite Quantifizierung explizit.
- (a) „Ein Tiger ist eine Katze.“
- (b) $1.19 \cdot x = 4$ (Im Kontext: Was ist der Preis ohne Mehrwertsteuer, wenn das Produkt 4 EUR kostet?)
- (c) $x + 3 = 3 + x$
6. Finden Sie Beispiele von Sätzen, die zwei Quantifizierungen enthalten und bei denen ein Vertauschen der Quantoren zu einer Änderung des Wahrheitswerts und damit der Bedeutung führt.
7. Welchen Wahrheitswert hat $\exists(n \in \mathbb{N} | \sqrt{2} < \pi)$ und warum? Wie können Sie den Wahrheitswert überprüfen?

Kapitel 7

Relationen

7.1 Aufwärmübungen

1. Schreiben Sie in *setlX* eine **procedure**, die als Eingaben drei Mengen S , A und B entgegen nimmt und $\mathbb{1}$ zurückgibt, wenn S eine Teilmenge des kartesischen Produkts von A und B ist.
2. Welche der folgenden Mengen sind Teilmengen des kartesischen Produkts von $A = \{0..9\}$ und $b = \{0..100\}$? Verifizieren Sie Ihre Antworten mit dem Programm aus Aufgabe 1.
 - (a) $\{[0, 0], [1, 1], [2, 4], [3, 9], [4, 16], [5, 25], [6, 36], [7, 49], [8, 64], [9, 81]\}$
 - (b) $\{1, [2, 3]\}$
 - (c) $\{[a, b] : a \in A, b \in B \mid a \bmod 7 = b \bmod 7\}$
3. Was ist ein geeignetes Modell (Tupel, Menge *etc.*), um Zusammenhänge wie „... ist Vater von ...“ oder „... studiert in ...“ zu modellieren?
4. Schreiben Sie Mengen-Konstruktoren für die folgenden Mengen:
 - (a) Die Menge der Paare $[x, y]$, für die $x + y < 8$ und x, y aus der Menge $\{1..6\}$ kommen.
 - (b) Die Menge der Paare von Zahlen aus $\{1..20\}$, die nach Division durch 5 denselben Rest haben.
 - (c) Die Menge von Paaren $[x, y]$, für die $x \subseteq y$ und x, y Elemente der Potenzmenge von $\{ 'a', 'b', 'c' \}$ sind.
5. Welche strukturellen Gemeinsamkeiten haben die folgenden Mengen?
 - $\{[1, 1], [7, 7], [13, 13]\}$
 - $\{[0, 0], [0, 5], [5, 0], [5, 5]\}$
 - $\{[\text{FORTRAN}, \text{FORTRAN}], [\text{C}, \text{C}], [\text{C}, \text{Java}], [\text{Java}, \text{C}], [\text{Java}, \text{Java}]\}$
6. Welche strukturellen Gemeinsamkeiten haben die folgenden Mengen?
 - $\{[0, 1], [0, 2], [0, 3], [1, 2], [1, 3], [2, 3]\}$
 - $\{[\text{nicht, oder}], [\text{nicht, und}], [\text{oder, und}]\}$
 - $\{['d', 'c'], ['d', 'b'], ['d', 'a'], ['c', 'b'], ['c', 'a'], ['b', 'a']\}$

Schreiben Sie außerdem Mengen-Konstruktoren für diese drei Mengen.

7. Schreiben Sie als formallogischen Ausdruck: Eine Menge R von Paaren natürlicher Zahlen heißt transitiv, wenn für alle $a, b, c \in \mathbb{N}$ gilt: Wenn $[a, b]$ und $[b, c]$ Elemente von R sind, dann ist auch $[a, c]$ Element von R .

8. Leitfragen:

- (a) Wie stellt man fest, ob eine Relation eine bestimmte Eigenschaft hat?
- (b) Was ist der Unterschied zwischen Relation und Beziehung?
- (c) Wie kann man Graphen symbolisch beschreiben?

7.2 Modellieren von Beziehungsmustern

Die Teile eines Paares stehen in einer Beziehung zueinander. Bei [Berlin, Deutschland] ist diese Beziehung „ist Hauptstadt von“. Es gibt viele weitere Paare mit derselben Beziehung: [Rom, Italien], [Hannover, Niedersachsen] usw. Bei $[1, 2]$ ist eine mögliche Beziehung „ist kleiner als“. Auch hier gibt es weitere Paare mit dieser Beziehung, z. B. $[-2, 1]$. Wir können also Einzelbeziehungen in den Blick nehmen oder die Menge aller (gleichartigen) Beziehungen als Struktur. Diese Struktur wird in der Informatik *Relation* genannt.

Relationen sind also Mengen von Paaren und beschreiben Beziehungsmuster. Sie treten immer auf, wenn die Elemente zweier Mengen in Beziehung gesetzt werden – so wie bei der obigen Beziehung zwischen der Menge von Hauptstädten und der Menge von Ländern. Sie treten auch auf, wenn wir die Elemente einer Menge miteinander vergleichen, anordnen oder nach gemeinsamen Eigenschaften gruppieren wollen – so wie bei der Beziehung „kleiner als“.

Die Definition der Struktur Relation muss also zwei Mengen beinhalten, die miteinander in Bezug gesetzt werden (wobei die Mengen auch identisch sein können) und aussagen, dass es sich bei der Relation um eine Menge von Paaren von Elementen dieser zwei Mengen handelt:

Eine Relation R zwischen den Mengen A und B ist eine Teilmenge des kartesischen Produkts $A \times B$, also $R \subseteq A \times B$.

Streng genommen wird nicht der Begriff Relation definiert, sondern „Relation zwischen zwei Mengen“. „Ist Hauptstadt von“ ist für sich also keine Relation; zwischen der Menge aller Städte und der Menge aller Länder jedoch schon.

Aus Perspektive der Logik betrachtet, besagt die Definition der Relation:

$$R \text{ ist Relation zwischen den Mengen } A \text{ und } B \Leftrightarrow R \subseteq A \times B$$

Beispiel 7.1: Die Menge

$$H_S = \{[\text{Oslo, Norwegen}], [\text{Stockholm, Schweden}], [\text{Kopenhagen, Dänemark}]\}$$

ist eine Relation zwischen der Menge aller Städte und der Menge aller Länder. Eine mögliche Bedeutung dieser Relation könnte „... ist Hauptstadt von ... und liegt in Skandinavien“ sein. ■

Beispiel 7.2: Die Menge

$$R_7 = \{[9, 0], [9, 1], [9, 2], [8, 0], [8, 1], [7, 0]\}$$

ist eine Relation zwischen der Menge D aller Dezimalziffern und D selbst, denn $R_7 \subseteq D \times D$. Eine mögliche Bedeutung dieser Relation könnte „ist mindestens um sieben größer als“ sein. ■

Sie können an diesen beiden Beispielen sehen, dass die Bedeutung einer Relation keine Rolle spielt. Es geht nur um die Struktur. Die Bedeutung ist wegabstrahiert.

Häufig sind Relationen nicht zwischen zwei unterschiedlichen Mengen, sondern zwischen identischen Mengen, so wie im zweiten Beispiel. In diesen Fällen ist es üblich statt „ R ist Relation zwischen A und A “ kurz „ R ist Relation auf A “ oder „ R ist Relation über A “ zu sagen.

Um auszudrücken, dass ein Paar zu einer Relation gehört, leistet $\in$ gute Dienste, z. B. $[9, 0] \in R_7$ oder $[\text{Oslo, Norwegen}] \in H_S$. Es ist auch üblich zu diesem Zweck das Relationssymbol zwischen die Komponenten des Paares zu schreiben, z. B. $9R_70$ oder $\text{Oslo}H_S\text{Norwegen}$. Allgemein formuliert:

$$aRb \Leftrightarrow [a, b] \in R$$

Sie sind diese, *Infix-Notation* genannte Schreibweise u. a. von der „ist kleiner als“-Relation gewohnt, wo es üblicher ist $1 < 2$ statt $[1, 2] \in <$ zu schreiben.

Aus Sichtweise der Logik sind aRb und $[a, b] \in R$ Aussageformen bzw. nach Belegung der Symbole mit konkreten Objekten Aussagen.

7.3 Eigenschaften von Relationen

Sie haben im Laufe dieses Kurses gesehen, dass so unterschiedliche binäre Operationen wie Konjunktion, Summe und Vereinigung strukturelle Gemeinsamkeiten haben, z. B. Kommutativität und Assoziativität. Auch völlig unterschiedliche Relationen können strukturelle Gemeinsamkeiten haben. Die wichtigsten unter diesen sind Gegenstand dieses Abschnitts.

Beispiel 7.3: Die Relationen

- „hat denselben Familiennamen wie“ auf der Menge aller Menschen
- $\leq$ auf der Menge der natürlichen Zahlen
- $\subseteq$ auf der Menge aller Mengen

haben die Eigenschaft, dass jedes Element a der zugrunde gelegten Menge mit sich selbst in Relation steht, z. B. $1 \leq 1$ oder $\{1, 2\} \subseteq \{1, 2\}$. Formal ausgedrückt haben diese Relationen die Eigenschaft $\forall(a \in A \mid aRa)$.

Dagegen haben die Relationen

- „ist Kind von“ auf der Menge aller Menschen
- $<$ auf der Menge der natürlichen Zahlen
- $\subset$ auf der Menge aller Mengen

diese Eigenschaft nicht. Beispielsweise gilt nicht $1 < 1$ und auch nicht $\{1, 2\} \subset \{1, 2\}$. ■

Beispiel 7.4: Die Relationen

- „hat denselben Familiennamen wie“ auf der Menge aller Menschen
- „ist verheiratet mit“ auf der Menge aller Menschen
- „hat bei Division durch 5 denselben Rest wie“ auf der Menge der ganzen Zahlen

haben die Eigenschaft, dass wenn a mit b in Relation steht, umgekehrt auch immer b mit a in Relation steht. Beispielsweise gilt „7 hat den selben Rest nach Division durch 5 wie 12“ ebenso wie „12 hat den selben Rest nach Division durch 5 wie 7“. Oder wenn Hans mit Hanna verheiratet ist, ist Hanna auch mit Hans verheiratet.

Formulieren Sie diese besondere Eigenschaft einer Relation R auf einer Menge A als formallogischen Ausdruck:



Dagegen haben die Relationen

- „ist Kind von“ auf der Menge aller Menschen
- $<$ auf der Menge der natürlichen Zahlen

diese Eigenschaft nicht. Beispielsweise ist $1 < 2$, aber nicht $2 < 1$, und zwei Personen können nicht wechselseitig Kinder voneinander sein. ■

Die folgende Definition gibt den beiden gerade diskutierten, besonderen Eigenschaften sowie weiteren jeweils eine Bezeichnung:

Eine Relation R auf A heißt

- *reflexiv*, wenn $[a, a] \in R$ für alle $a \in A$.
- *symmetrisch*, wenn $[a, b] \in R \leftrightarrow [b, a] \in R$ für alle $a, b \in A$ gilt.
- *transitiv*, wenn für alle $a, b, c \in A$ gilt: $[a, b] \in R \wedge [b, c] \in R \rightarrow [a, c] \in R$.
- *antisymmetrisch*, wenn für alle $a, b \in A$ gilt: $[a, b] \in R \wedge [b, a] \in R \rightarrow a = b$.
- *asymmetrisch*, wenn für alle $a, b \in A$ gilt: $[a, b] \in R \rightarrow [b, a] \notin R$.

Logisch prägnant formuliert bedeutet das:

R ist reflexiv	$\Leftrightarrow$	$R \subseteq A \times A \wedge \forall(a \in A \mid [a, a] \in R)$
R ist symmetrisch	$\Leftrightarrow$	
R ist transitiv	$\Leftrightarrow$	$R \subseteq A \times A \wedge \forall(a, b, c \in A \mid [a, b] \in R \wedge [b, c] \in R \rightarrow [a, c] \in R)$
R ist antisymmetrisch	$\Leftrightarrow$	
R ist asymmetrisch	$\Leftrightarrow$	

Nutzen Sie an dieser Stelle die Gelegenheit, logisches Formulieren zu üben, und füllen Sie die Lücken.

Die formallogischen Formulierungen der Definitionen lassen sich leicht in *setLX* programmieren und ermöglichen so per Rechner Relationen auf Eigenschaften zu überprüfen.

Beispiel 7.5: Die folgende Funktion ist ein direkter Aufschrieb der Definition der Transitivität:

```
is_transitive_relation := procedure(rel, set){
  return rel <= set ><set &&
    forall(a in set, b in set, c in set |
      [a,b] in rel && [b,c] in rel => [a,c] in rel);
};
```

Damit lässt sich maschinell überprüfen, ob die Behauptung von S. 56, dass die Subjunktion transitiv ist, korrekt ist. Dazu wird die Subjunktion zuerst als Relation, also als Menge von Paaren geschrieben:

```
=> Boolean:={true,false};
~< Result: {true, false} >~
=> subjunktion:={[a,b]: a in Boolean, b in Boolean | a => b};
~< Result: {[false, false], [false, true], [true, true]} >~
=> is_transitive_relation(subjunktion, Boolean);
~< Result: true >~
```

Die Subjunktion ist also tatsächlich transitiv. ■

Beispiel 7.6: Wir untersuchen, ob die Relation $\tilde{R} = \{[3, 1], [1, 2], [2, 3]\}$ auf der Menge $\{1, 2, 3\}$ transitiv ist. Zunächst sind Sie dran:

nach Definition die Bestimmung des Wahrheitswerts von

$$\forall(a, b \in \mathbb{N} \mid a < b \wedge b < a \rightarrow a = b).$$

Die linke Seite der Subjunktion, $a < b \wedge b < a$ ist hier immer falsch und damit die Subjunktion immer wahr.

Asymmetrie erfordert nach Definition die Bestimmung des Wahrheitswerts von

$$\forall(a, b \in \mathbb{N} \mid a < b \rightarrow \neg(b < a)) = \forall(a, b \in \mathbb{N} \mid a < b \rightarrow a \leq b).$$

Dieser ist hier wahr, wie an der Subjunktion nach der Umformung von $\neg(b < a)$ in $b \geq a$ zu sehen ist. ■

Beispiel 7.9: Die Relation $\leq$ auf $\mathbb{N}$ ist antisymmetrisch, aber nicht asymmetrisch. Antisymmetrie erfordert nach Definition die Bestimmung des Wahrheitswerts von

$$\forall(a, b \in \mathbb{N} \mid a \leq b \wedge b \leq a \rightarrow a = b).$$

Wenn die linke Seite der Subjunktion falsch ist, ist die Subjunktion ohnehin wahr. Die linke Seite ist dann und nur dann wahr, wenn $a = b$. Somit ist dann auch die rechte Seite wahr und die Subjunktion insgesamt wahr. Der $\forall$ -Ausdruck ist also wahr und somit die Relation $\leq$ auf $\mathbb{N}$ antisymmetrisch.

Asymmetrie erfordert nach Definition die Bestimmung des Wahrheitswerts von

$$\forall(a, b \in \mathbb{N} \mid a \leq b \rightarrow \neg(b \leq a)) = \forall(a, b \in \mathbb{N} \mid a \leq b \rightarrow a < b).$$

Hier gibt es bei der Subjunktion Fälle mit $1 \rightarrow 0$, z. B. für $a = b = 1$. Damit ist die Bedingung für Asymmetrie nicht erfüllt. ■

7.4 Graphen

Relationen sind das Mittel der Wahl, um Beziehungen zu modellieren. Gleichzeitig erlauben Relationen Beziehungen zu beschreiben und schriftlich darzustellen. Eine graphische Darstellung erlauben sogenannte *Graphen*.

Graphen sind Netzwerke aus Punkten und Linien. Diese sehen aus wie Straßenkarten, nur dass die Lage der Punkte keine Bedeutung hat, sondern nur ob und mit wem sie verbunden sind. Abb. 7.1 zeigt einige Beispiele.

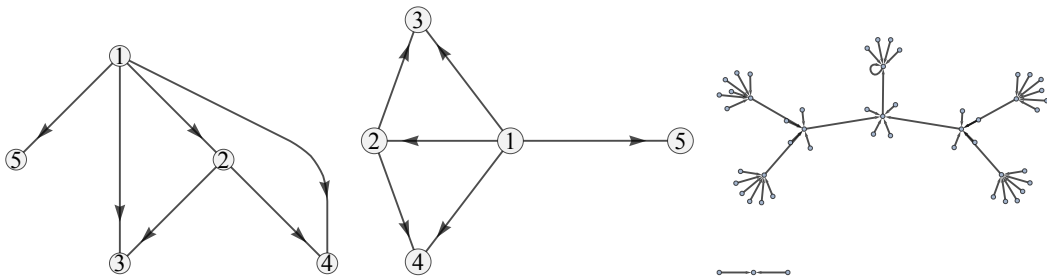
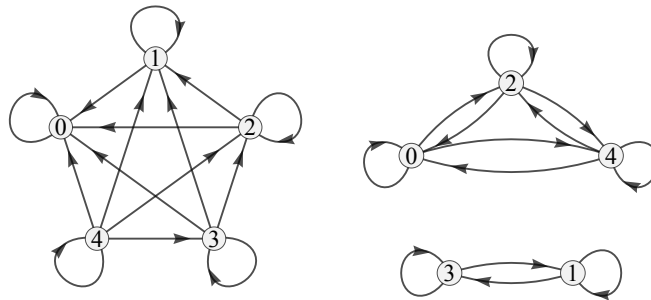


Abbildung 7.1: Beispiele für Graphen. Die linken beiden Graphen sind gleichwertig, da die Position der Knoten keine Rolle spielt.

Sie sind bereits mit der Idee von Graphen zur Darstellung von Funktionen vertraut. Das Wort Graph wird auch verwendet, um allgemeinere Strukturen bildlich darzustellen, wie sie in vielen Situationen auftreten. In der Informatik können dies Flussdiagramme, Bäume, ER-Diagramme und vieles andere mehr sein. Immer sind Punkte mit Linien verbunden. Wenn die Linien eine Richtung haben sollen, werden Pfeile verwendet.

In der Sprache der Graphentheorie heißen die Punkte *Knoten* und die Linien *Kanten*. Die Menge ist das geeignete Denkzeug, um die Knoten zu beschreiben. Tupel sind das geeignete Denkzeug um Kanten zu beschreiben. So können wir die gerichtete Kante von 3 nach 1 im linken Graph in Abb. 7.2 durch $[3, 1]$ beschreiben. $[1, 3]$ würde dagegen für eine Kante von 1 nach 3 stehen, die in diesem Graphen nicht vorhanden ist. Mittels der Menge aller Knoten V und der Menge aller Kanten E kann ein Graph dann vollständig beschrieben werden.¹



Abbildungung 7.2: Links: Graph der Relation $\geq$ auf $\{0, 1, 2, 3, 4\}$. Rechts: Graph der Relation „hat gleichen Rest nach Division durch 2“ auf der selben Menge.

Beispiel 7.10: Der linke Graph in Abb. 7.2 lässt sich also so beschreiben:

$$\begin{aligned} V &= \{0, 1, 2, 3, 4\} \\ E &= \{[0, 0], \\ &\quad [1, 0], [1, 1] \\ &\quad [2, 0], [2, 1], [2, 2] \\ &\quad [3, 0], [3, 1], [3, 2], [3, 3] \\ &\quad [4, 0], [4, 1], [4, 2], [4, 3], [4, 4]\} \end{aligned}$$

Diese Beschreibung ist aber nichts anderes als eine Relation $E \subseteq V \times V$. ■

Was in dem Beispiel funktioniert hat, geht auch allgemein: Jeder Graph lässt sich als Relation beschreiben und umgekehrt lässt sich jede Relation als Graph darstellen. So stellt der rechte Graph in Abb. 7.2 die Relation „hat gleichen Rest nach Division durch 2“ auf derselben Menge V dar.

Beide Relationen in Abb. 7.2 sind reflexive Relationen. Graphisch ist dies an den Schleifen (also Kanten, die im selben Knoten enden wie sie beginnen) an allen Knoten zu erkennen. Die Relation „hat gleichen Rest nach Division durch 2“ ist symmetrisch. Am Graphen ist dies daran zu erkennen, dass es zu jeder Kante eine Kante in umgekehrter Richtung gibt. Im linken Graphen ist dies nicht der Fall. Die Relation $\geq$ ist nicht symmetrisch.

Während der Graph einer symmetrischen Relation zu jeder Kante eine Kante mit umgekehrter Richtung hat, ist dies bei einer antisymmetrischen Relation genau nicht der Fall – mit der Ausnahme von Schleifen. Der linke Graph in Abb. 7.2 ist ein Beispiel. Hinsichtlich Asymmetrie wissen wir schon aus Abschnitt 7.3, dass die Relation $\geq$ antisymmetrisch ist.

Das Schöne ist, dass wir die graphische Bedeutung von Reflexivität, Symmetrie usw. gar nicht brauchen, um diese Eigenschaften festzustellen. Dafür genügen alleine die definierenden Eigenschaften aus Abschnitt 7.3, die sogar ein Rechner überprüfen kann.

¹Das Symbol V für die Mengen aller Knoten kommt von der englischen Bezeichnung *vertex* für Knoten. Das Symbol E für die Mengen aller Kanten kommt von der englischen Bezeichnung *edge* für Kante.

7.5 Äquivalenzrelationen

Manche Relationen drücken eine Ähnlichkeitsbeziehung oder Gleichwertigkeit aus. Dazu gehören „arbeitet in derselben Abteilung wie“, „ist genauso alt wie“ (jeweils auf der Menge aller Menschen) und auch die mit $=$ symbolisierte Gleichheitsrelation. Solche Relationen sind immer reflexiv, denn ein Element ist immer zu sich selbst ähnlich. Sie sind immer symmetrisch, denn wenn a zu b ähnlich ist, ist umgekehrt auch b zu a ähnlich. Und sie sind transitiv, denn wenn a zu b ähnlich ist und b zu c , dann ist auch a zu c ähnlich. Solche Relationen heißen Äquivalenzrelationen:

Eine Relation R auf einer Menge A heißt *Äquivalenzrelation*, wenn R reflexiv, symmetrisch und transitiv ist. Für aRb sagt man dann „ a ist äquivalent zu b “.

Die bekanntesten Äquivalenzrelationen sind sicher $=$ (auf Zahlenmengen) und $\Leftrightarrow$ (auf Aussageformen).

Äquivalenzrelationen haben die Eigenschaft, dass sie in disjunkte Teilmengen unterteilt werden können. In der Darstellung von Relationen als Graphen sind diese Teilmengen deutlich erkennbar.

Beispiel 7.11: Die Äquivalenzrelation „hat gleichen Rest modulo 4 wie“ auf der Menge der ganzen Zahlen ist in Abb. 7.3 als Graph dargestellt. Dieser Graph besteht aus vier Teilgraphen, je einer für Rest 0, 1, 2 bzw. 3. ■

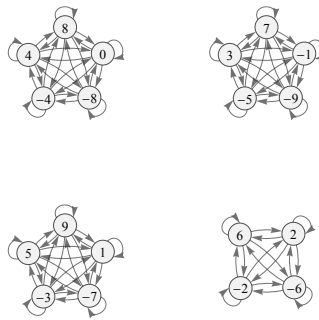


Abbildung 7.3: Graph der Äquivalenzrelation „hat gleichen Rest modulo 4 wie“ auf der Menge $\{n : n \in \mathbb{N} \mid -9 \leq n \leq 9\}$.

Beispiel 7.12: Die Äquivalenzrelation „hat gleichen Anfangsbuchstaben wie“ auf der Menge einiger Konzepte dieser Lehrveranstaltung ist in Abb. 7.4 als Graph dargestellt. Dieser Graph besteht aus Teilgraphen, je einen für den jeweiligen Anfangsbuchstaben. ■

Vereinbaren wir zunächst einen Begriff für die Teilmengen, in die eine Äquivalenzrelation zerfällt:

Die Menge $[a]$ aller Elemente von A zu einer Äquivalenzrelation R auf A , die äquivalent zu $a \in A$ sind,

$$[a] = \{x : x \in A \mid xRa\},$$

heißt *Äquivalenzklasse* von a .

Alle Elemente einer Äquivalenzklasse sind im Sinne der Äquivalenzrelation gleichwertig.

Beispiel 7.13: Die Äquivalenzrelation „hat gleichen Rest modulo 4 wie“ auf der Menge der ganzen Zahlen hat die folgenden Äquivalenzklassen:

$$\begin{aligned} [0] &= \{z : z \in \mathbb{Z} \mid z \bmod 4 = 0\} = \{\dots, -8, -4, 0, 4, 8, \dots\} \\ [1] &= \{z : z \in \mathbb{Z} \mid z \bmod 4 = 1\} = \{\dots, -7, -3, 1, 5, 9, \dots\} \\ [2] &= \{z : z \in \mathbb{Z} \mid z \bmod 4 = 2\} = \{\dots, -6, -2, 2, 6, \dots\} \end{aligned}$$

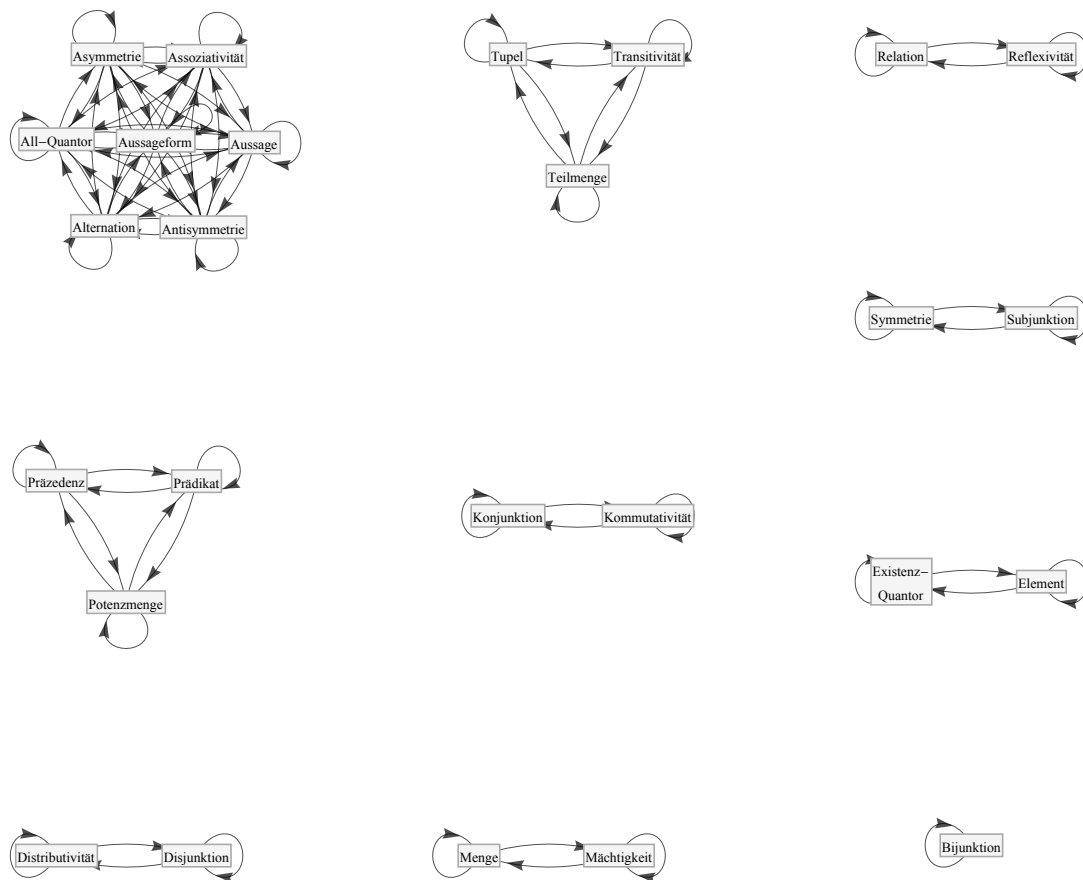


Abbildung 7.4: Graph der Äquivalenzrelation „hat gleichen Anfangsbuchstaben wie“ auf der Menge einiger Konzepte dieser Lehrveranstaltung.

$$[3] = \{z : z \in \mathbb{Z} \mid z \bmod 4 = 3\} = \{\dots, -9, -5, -1, 3, 7, \dots\}$$

Zur Äquivalenzklasse $[0]$ gehören also alle Vielfachen von 4; zu der von $[1]$, alle Zahlen, die bei Division durch 4 den Rest 1 haben; usw. Alle Elemente jeder Äquivalenzklasse sind gleichwertig in dem Sinne, dass sie denselben Rest nach Division durch 4 haben.

Alle weiteren Äquivalenzklassen sind identisch zu einer der oben genannten, z. B. $[4] = [0]$ oder $[-1] = [3]$.

Die Vereinigung aller Äquivalenzklassen ergibt wieder die Ausgangsmenge

$$[0] \cup [1] \cup [2] \cup [3] = \mathbb{Z},$$

über die die Relation gebildet ist. ■

Natürlich sind zwei Äquivalenzklassen identisch, d. h. $[a] = [b]$, wenn $b \in [a]$. Was in $[a]$ zwischen den eckigen Klammern steht, ist nur ein *Repräsentant* der Äquivalenzklasse.

Wie bereits oben diskutiert, haben Äquivalenzklassen besondere Eigenschaften:

Jede Äquivalenzrelation über einer Menge A hat die folgenden Eigenschaften:

- Je zwei verschiedene Äquivalenzklassen sind disjunkt.
- Die Vereinigung aller Äquivalenzklassen ist gleich A .

7.6 Ordnungsrelationen

Neben Äquivalenzrelationen sind Ordnungsrelationen eine weitere wichtige Klasse von Relationen. Sie zeichnen sich dadurch aus, dass die Relation die Elemente der Menge, über die sie gebildet ist, anordnet. Die Relationen $<$, $\leq$, $>$ und $\geq$ ordnen bspw. die Elemente aus $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ und $\mathbb{R}$. Die Relation „... steht im Wörterbuch vor ...“ ordnet die Menge der Worte einer Sprache.

Überlegen Sie sich, welche der Eigenschaften aus Abschnitt 7.3 solche Ordnungsrelationen haben müssen:

Reflexivität	 ☐ Ja ☐ Nein
Symmetrie	 ☐ Ja ☐ Nein
Transitivität	 ☐ Ja ☐ Nein
Antisymmetrie	 ☐ Ja ☐ Nein
Asymmetrie	 ☐ Ja ☐ Nein

Beispiel 7.14: Der linke Graph in Abb. 7.2 zeigt die Relation $\geq$ auf $\{0, 1, 2, 3, 4\}$. Die Relation ist reflexiv, antisymmetrisch und transitiv. Sie ist nicht symmetrisch und auch nicht asymmetrisch. ■

Beispiel 7.15: Die Relation $>$ auf $\{0, 1, 2, 3, 4\}$

$$\{[1, 0], [2, 0], [2, 1], [3, 0], [3, 1], [3, 2], [4, 0], [4, 1], [4, 2], [4, 3]\}$$

ist dagegen nicht reflexiv. Sie ist auch nicht symmetrisch. Sie ist antisymmetrisch, asymmetrisch und transitiv. ■

Offenbar gibt es zwei Arten von Ordnungsrelationen: Zum einen solche, die Gleichheit zulassen, so wie $\geq$. Sie sind reflexiv, transitiv und nur antisymmetrisch. Zum anderen solche, die Gleichheit nicht zulassen, so wie $>$. Sie sind transitiv und asymmetrisch.

Eine Relation R auf einer Menge A heißt *Ordnungsrelation*, wenn R reflexiv, antisymmetrisch und transitiv ist.

Eine Relation R auf einer Menge A heißt *strikte Ordnungsrelation*, wenn R asymmetrisch und transitiv ist.

Beispiel 7.16: Abb. 7.5 zeigt links den Graphen der Relation „teilt“ auf der Menge der natürlichen Zahlen von 1 bis 9. Diese Relation hat u. a. die Elemente $[7, 7]$, $[2, 6]$ und $[3, 6]$. Verwenden wir für diese Relation das Symbol $\models$, können wir dafür $7 \models 7$, $2 \models 6$ bzw. $3 \models 6$ schreiben.

Ordnet $\models$ die Ziffern in irgendeiner Weise? Jeder Knoten hat eine Schleife, also ist die Relation reflexiv. Transitivität, Antisymmetrie und Asymmetrie untersuchen wir mit *setlX*:

```
=> N:={1..9};
~< Result: {1, 2, 3, 4, 5, 6, 7, 8, 9} >~
=> teilt:={[k,l]:k in N, l in N | 1%k==0};
~< Result: {[1, 1], [1, 2], [1, 3], [1, 4], [1, 5], [1, 6], [1, 7], [1, 8],
[1, 9], [2, 2], [2, 4], [2, 6], [2, 8], [3, 3], [3, 6], [3, 9], [4, 4], [4, 8],
[5, 5], [6, 6], [7, 7], [8, 8], [9, 9]} >~
=> is_transitive_relation(teilt,N);
~< Result: true >~
=> forall(a in N, b in N | [a,b] in teilt && [b,a] in teilt => a==b);
~< Result: true >~
=> forall(a in N, b in N | [a,b] in teilt => [b,a] notin teilt);
~< Result: false >~
```

Damit hat die Relation alle Eigenschaften einer (nicht strikten) Ordnungsrelation. Wir können daher Ordnungsketten bilden, z. B. $2 \models 4 \models 8 \models 8$. ■

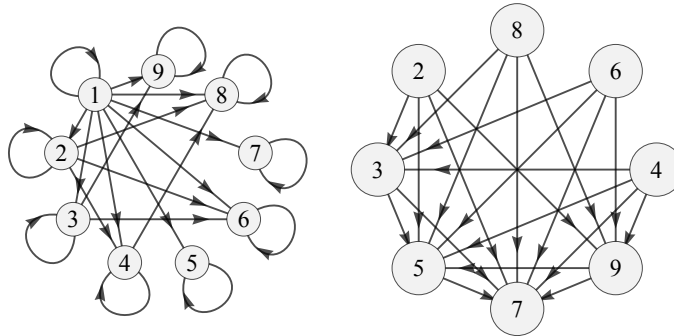


Abbildung 7.5: Links: Graph der Ordnungsrelation „teilt“ auf der Menge der Dezimalziffern. Rechts: Graph der Ordnungsrelation „kleinster Primfaktor ist kleiner als der von“ ebenfalls auf der Menge der Dezimalziffern.

Beispiel 7.17: Schließen wir bei der teilt-Relation die Gleichheit aus, also „teilt und ist nicht gleich“, erhalten wir eine strikte Ordnung mit z. B. $2 \vdash 4$, aber $7 \not\vdash 7$ im Gegensatz zu $7 \models 7$. Nun ist die Asymmetrie erfüllt:

```
=> teiltstrikt:={[k,l]:k in N, l in N | 1%k==0 && k!=l};
~< Result: {[1, 2], [1, 3], [1, 4], [1, 5], [1, 6], [1, 7], [1, 8], [1, 9],
[2, 4], [2, 6], [2, 8], [3, 6], [3, 9], [4, 8]} >~
=> forall(a in N, b in N | [a,b] in teiltstrikt => [b,a] notin teiltstrikt);
~< Result: true >~
```

Wieder können wir Ziffern anordnen, z. B. $2 \vdash 4 \vdash 8$. ■

Beispiel 7.18: Abb. 7.5 zeigt rechts den Graphen der Relation

$$\prec = \{[2, 3], [2, 5], [2, 7], [2, 9], [3, 5], [3, 7], [4, 3], [4, 5], [4, 7], [4, 9], [5, 7], [6, 3], [6, 5], [6, 7], [6, 9], [8, 3], [8, 5], [8, 7], [8, 9], [9, 5], [9, 7]\}$$

auf der Menge der Dezimalziffern von 1 bis 9.

Nutzen Sie dieses Beispiel um nachzuweisen, dass diese Relation transitiv und asymmetrisch ist, also eine strikte Ordnungsrelation.



Wegen der strikten Ordnung können wir z. B. folgende Ordnungsketten aufschreiben:

$$\begin{aligned} 8 &\prec 3 \prec 7 \\ 9 &\prec 5 \prec 7 \end{aligned}$$

Eine mögliche Bedeutung von $a \prec b$ ist, dass der kleinste Primfaktor von a kleiner als der kleinste Primfaktor von b ist. ■

7.7 Operationen auf Relationen

Relationen sind – wenn auch besondere – Mengen. Als solche können alle Mengenoperationen auf Relationen angewandt werden. Zusätzlich sind zwei weitere Operationen für Relationen bedeutsam: Umkehrung und Verkettung.

Relationen haben eine Richtung. Wir lesen aRb oder $[a, b] \in R$ von links nach rechts als „ a steht in Relation R mit b “. Das bedeutet i. A. nicht „ b steht in Relation R mit a “, es sei denn R ist symmetrisch. Trotzdem kann aRb auch sinnvoll von rechts nach links gelesen werden.

Beispiel 7.19: $a < b$ bedeutet von links nach rechts gelesen „ a kleiner b “. Von rechts nach links gelesen, ist die Bedeutung $b > a$, also „ b größer a “. ■

Beispiel 7.20: Die Relation „ist Hauptstadt von“ bedeutet von rechts nach links gelesen „hat als Hauptstadt“. ■

Die Relation $R^{-1} \subseteq B \times A$ mit

$$R^{-1} = \{[b, a] : a \in A, b \in B \mid [a, b] \in R\}$$

heißt *Umkehrrelation* oder *inverse Relation* zu $R \subseteq A \times B$.

Die Umkehrrelation ist also über einen Mengen-Konstruktor definiert und lässt sich daher wörtlich in *setlX* programmieren:

```
inverse:=procedure(R,A,B){return {[b,a]: a in A, b in B|[a,b] in R};};
```

Beispiel 7.21: Die kleiner-Relation auf der Menge der Dezimalziffern ist

```
=> D:={0..9};
~< Result: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9} >~
=> kleiner:={[a,b]: a in D, b in D|a<b};
~< Result: {[0, 1], [0, 2], [0, 3], [0, 4], [0, 5], [0, 6], [0, 7], [0, 8],
[0, 9], [1, 2], [1, 3], [1, 4], [1, 5], [1, 6], [1, 7], [1, 8], [1, 9],
[2, 3], [2, 4], [2, 5], [2, 6], [2, 7], [2, 8], [2, 9], [3, 4], [3, 5],
[3, 6], [3, 7], [3, 8], [3, 9], [4, 5], [4, 6], [4, 7], [4, 8], [4, 9],
[5, 6], [5, 7], [5, 8], [5, 9], [6, 7], [6, 8], [6, 9], [7, 8], [7, 9],
[8, 9]} >~
```

Die Umkehrrelation ist

```
=> inverse(kleiner,D,D);
~< Result: {[1, 0], [2, 0], [2, 1], [3, 0], [3, 1], [3, 2], [4, 0], [4, 1],
[4, 2], [4, 3], [5, 0], [5, 1], [5, 2], [5, 3], [5, 4], [6, 0], [6, 1],
[6, 2], [6, 3], [6, 4], [6, 5], [7, 0], [7, 1], [7, 2], [7, 3], [7, 4],
[7, 5], [7, 6], [8, 0], [8, 1], [8, 2], [8, 3], [8, 4], [8, 5], [8, 6],
[8, 7], [9, 0], [9, 1], [9, 2], [9, 3], [9, 4], [9, 5], [9, 6], [9, 7],
[9, 8]} >~
```

Sie sollten erkennen, dass diese Relation die Bedeutung „ist größer als“ auf D hat. ■

Beispiel 7.22: Die folgenden beiden Tabellen lassen sich jeweils als Relation modellieren:

Ort	Verkehrsweg
BS	A2
BS	B79
H	A2
H	A7
H	B65
WF	B79

Verkehrsweg	Netz
A2	Autobahn
A7	Autobahn
B65	Bundesstraße
B79	Bundesstraße

„liegt an“ = $\{[BS, A2], [BS, B79], [H, A2], [H, A7], [H, B65], [WF, B79]\}$

„ist eine“ = $\{[A2, Autobahn], [A7, Autobahn], [B65, Bundesstraße], [B79, Bundesstraße]\}$

Die Inversen sind:

$$\begin{aligned}\text{„führt durch“} &= \text{„liegt an“}^{-1} = \{[A2, BS], [A2, H], [A7, H], [B65, H], [B79, BS], [B79, WF]\} \\ \text{„ist z. B.“} &= \{[Autobahn, A2], [Autobahn, A7], [Bundesstraße, B65], [Bundesstraße, B79]\}\end{aligned}$$

■

Beispiel 7.23: Im vorangegangenen Beispiel können die Relationen „liegt an“ und „ist eine“ zu einer neuen Relation „hat Anbindung an“ verknüpft werden:

$$\text{„hat Anbindung an“} = \{[BS, Autobahn], [BS, Bundesstraße], [H, Autobahn], [H, Bundesstraße], [WF, Bundesstraße]\}$$

Dabei wurde beim ersten Element der Relation „BS liegt an A2“ und „A2 ist eine Autobahn“ zu „BS hat Anbindung an Autobahn“ verknüpft. Analog bei den anderen Elementen. ■

Diese Art der Verknüpfung wird *Verkettung* genannt. Das ist die *Bezeichnung*. Wichtiger ist natürlich, *wie* die Verkettung gebildet wird, d. h. ihre *formale Beschreibung*:

Die Relation $S \circ R \subseteq A \times C$ mit

$$S \circ R = \{[a, c] : a \in A, c \in C \mid \exists(b \in B \mid [a, b] \in R \wedge [b, c] \in S)\}$$

heißt *Verkettung* oder *Komposition* der Relationen $R \subseteq A \times B$ und $S \subseteq B \times C$.

Beachten Sie, dass aus aRb und bSc Folgendes wird: $a(S \circ R)c$. Die „erste“ Relation R steht rechts in der Verkettung $S \circ R$, nicht etwa links. Anders formuliert $S \circ R$ ist von rechts nach links zu lesen! $S \circ R$ meint also „erst R , dann S “. Natürlich könne man das auch anders herum als „erst S , dann R “ lesen. Manche Bücher sehen das so. Dort ist die Bedeutung des Symbols „ $S \circ R$ “ dann eben anders definiert. In jedem Fall ist durch die Definition eindeutig festgelegt, wie es zu lesen ist.

Beispiel 7.24: Die Menge

$$R_1 = \{[1, x], [1, y], [2, y], [3, z]\}$$

kann als Relation zwischen den Mengen $A = \{1, 2, 3\}$ und $B = \{x, y, z\}$ interpretiert werden. Die Menge

$$R_2 = \{[x, 1], [x, 0], [y, 1]\}$$

kann als Relation zwischen den Mengen B und $C = \{0, 1\}$ interpretiert werden. Wir haben also $R_1 \subseteq A \times B$ und $R_2 \subseteq B \times C$. Die Menge B „steht in der Mitte“, daher können R_1 und R_2 zu $R_2 \circ R_1$ verkettet werden. Tun Sie das zunächst selbst per Hand, bevor wir anschließend gemeinsam Ihr Ergebnis mit Hilfe von *setlX* überprüfen.

$$\pencil R_2 \circ R_1 =$$

```
=> A:={1,2,3};B:={"x","y","z"};C:={true,false};
=> R1:={ [1,"x"], [1,"y"], [2,"y"], [3,"z"] };
=> R2:={ ["x",true], ["x",false], ["y",true] };
=> {[a,c]: a in A, c in C | exists(b in B| [a,b] in R1 && [b,c] in R2) };
```

■

Da die Verkettung eine binäre Operation ist, die zwei Relationen zu einer neuen Relation verknüpft, stellen sich Strukturfragen:

- Ist die Verkettung assoziativ? 
- Ist die Verkettung kommutativ? 
- gibt es eine neutrale Relation I , so dass immer $R \circ I = R$ bzw. $I \circ R = R$ gilt? 

Sie sollten inzwischen genügend Strukturwissen und -verständnis aufgebaut haben, um diese Fragen alleine zu beantworten.

Die Verkettung $S \circ R$ ist so aufgebaut, dass nicht beliebige Relationen verkettet werden können. Wegen $R \subseteq A \times B$ und $S \subseteq B \times C$ muss die in der „Mitte stehende Menge B “ die Verbindung zwischen den beiden Relationen „vermitteln“. Daher kann i. A. $R \circ S$ nicht gebildet werden, es sei denn $A = C$.

Einen Spezialfall in dieser Hinsicht bilden jede Relation $R \subseteq A \times B$ und ihre Inverse $R^{-1} \subseteq B \times A$. Bei $R^{-1} \circ R$ „steht B in der Mitte“, bei $R \circ R^{-1}$ „steht A in der Mitte“. Die Verkettung kann also in beide Richtungen gebildet werden.

Beispiel 7.25: Die Relation $R = \{[1, 2], [1, 3]\}$ hat die Inverse $R^{-1} = \{[2, 1], [3, 1]\}$. Die beiden Verkettungen sind

$$\begin{aligned} R^{-1} \circ R &= \{[1, 1]\}, \\ R \circ R^{-1} &= \{[2, 2], [2, 3], [3, 3], [3, 2]\}. \end{aligned}$$

Das erste Resultat ist von der Form $\{[a, a] : a \in A\}$, also eine Menge von Tupeln identischer Elemente, das zweite aber nicht. ■

Relationen der Art $\{[a, a] : a \in A\}$ haben eine besondere strukturelle Bedeutung für die Verkettung von Relationen. Wir geben Relationen von dieser Art zunächst einen Namen:

Die Relation I_A über einer Menge A mit

$$I_A = \{[a, a] : a \in A\}$$

wird *Identität* genannt.

Die Bedeutung der Identität besteht darin, dass sie die oben angesprochene neutrale Relation der Verkettung darstellt.

7.8 Empfohlene Vertiefungen

7.8.1 Verallgemeinerung

Relationen sind Mengen von Paaren. Dieses Konzept lässt sich auf Mengen von Tupeln verallgemeinern: Eine n -stellige Relation ist eine Menge von n -Tupeln. In diesem Sinne fokussierte dieses Kapitel auf zweistellige Relationen.

Ein Beispiel für eine dreistellige Relation ist

$$\{[k, m, v] : k \in M, m \in M, v \in M \mid k \text{ ist Kind von } m \text{ und } v\}$$

über der Menge M aller Menschen.

Ein Beispiel für vierstellige Relationsen ist

$$\{[a, b, c, d] : a, b, c, d \in \mathbb{N} \mid a = b + c + d\}.$$

7.8.2 Graphentheorie

Graphen haben ihre eigenen strukturellen Eigenschaften. Diese stehen in Verbindung zu Fragestellungen wie z. B. der Frage nach der Existenz geschlossener Wege in Graphen, der Frage nach dem kürzesten Weg zwischen zwei Knoten eines Graphen oder der Art des Graphen. Besonders Bäume sind eine Art von Graphen, ohne die die Informatik nicht auskommt.

Die strukturellen Eigenschaften von Graphen werden in der Graphentheorie untersucht. Lesen Sie dazu die Kapitel 15 und 16 im Lehrbuch von Teschl und Teschl.

7.8.3 Relationales Datenmodell

Lesen Sie dazu Abschnitt 5.1.1 im Lehrbuch von Teschl und Teschl.

Gerade bei Datenbankabfragen will man wissen, wer mit wem in Beziehung steht. So steht in der Relation $R = \{[1, 2], [1, 3], [2, 4]\}$ die 2 nur mit der 4 in Beziehung, während die 1 sowohl mit der 2 als auch mit der 3 in Beziehung steht. In *setlX* kann die Menge der Elemente, mit denen ein Element a in der Relation R in Beziehung steht, mittels $R\{a\}$ abgefragt werden:

Beispiel 7.26:

```
=> R:={ [1,2] , [1,3] , [2,4] };
~< Result: { [1, 2] , [1, 3] , [2, 4] } >~
=> R{1};
~< Result: {2, 3} >~
=> R{2};
~< Result: {4} >~
=> R{5};
~< Result: {} >~
```

■

7.9 Lernziele

Studierende	Kennen	Können	Verstehen
fachlich	☐ geben die Definition von Relation und besonderer Eigenschaften (reflexiv, symmetrisch usw.) wieder ☐ erläutern Bedeutung von Relationen für Mathematik und Informatik ☐ benennen Besonderheit von Relationen in Abgrenzung zu Mengen	☐ stellen Relationen symbolisch, graphisch und auf Rechner dar und übersetzen zwischen diesen Repräsentationsformen ☐ können alle im Text eingeführten Operationen auf Relationen berechnen	☐ analysieren, ob eine Relation (allgemein: ein Objekt) eine bestimmte Eigenschaft hat ☐ entscheiden, ob etwas als Relation modelliert werden kann ☐ konstruieren Beispiele für Relationen mit bestimmten gegebenen Eigenschaften
methodisch	☐ kennen Programmstrukturen zum Repräsentieren von Relationen	☐ schreiben Programm, das im Text eingeführte Operation auf Relationen berechnet ☐ schreiben Programm, das überprüft, ob eine Relation eine bestimmte Eigenschaft hat	☐ formulieren Definitionen als logische Aussagen/ Aussageformen alleine an Hand der Definitionen

Hinzu kommen die themenübergreifenden Lernziele aus den Kategorien „sozial“ und „persönlich“ aus Abschnitt 1.7 sowie die Lernziele aus der Kategorie „methodisch“ aller vorangegangener Kapitel. Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben.

7.10 Übungsaufgaben

1. Welche der folgenden Symbole

$<, \leq, =, \neq, \approx, \perp, \parallel, \in, \notin, \subset, \subseteq, \Leftrightarrow, \Rightarrow$

stehen für Relationen und zwischen welchen Mengen sind diese Relationen?

2. Was ist das Gegenteil von symmetrisch? Antisymmetrisch, asymmetrisch oder keines von beiden?
3. In welcher Beziehung stehen Asymmetrie und Antisymmetrie zueinander? Impliziert das eine das andere?
4. Das Verb „sein“ hat u. a. die Bedeutung „zur Klasse gehören“, z. B. in „Ein Hund ist ein Tier“ oder „Ein Spitz ist ein Hund“. Lässt sich dieses „ist“ als Relation modellieren? Wenn ja, welche Eigenschaften hat diese Relation (reflexiv, symmetrisch *etc.*)?
5. Das Verb „sein“ hat u. a. die Bedeutung „die Eigenschaft haben“, z. B. in „Wasser ist nass“ oder „Wolfenbüttel ist schön“. Lässt sich dieses „ist“ als Relation modellieren? Wenn ja, welche Eigenschaften hat diese Relation (reflexiv, symmetrisch *etc.*)?
6. Der folgende Code zeigt einen typischen Versuch die Transitivität in *setlX* zu implementieren:

```
is_transitive_relation := procedure(rel) {
  for (a in rel) {
    for (b in rel) {
      if (a[2] == b[1]) {
        if (!([a[1], b[2]] in rel)) {
          return false;
        }
      }
    }
  }
  return true;
};
```

- (a) Untersuchen Sie, ob dieser Code funktional korrekt ist.
- (b) Vergleichen Sie den Code mit dem von S. 96. Welcher Code hat welche Vorteile?
7. In welcher Beziehung stehen Ordnungsrelation und strikte Ordnungsrelation zu einander? Ist eine ein Spezialfall der anderen?
8. Welche eingerahmten und grau hinterlegten Texte in diesem Kapitel sind Definitionen, welche sind Theoreme?
9. In Abschnitt 7.5 steht der Satz „Äquivalenzrelationen haben die Eigenschaft, dass sie in disjunkte Teilmengen unterteilt werden können.“ Dieser Satz ist gleichbedeutend mit dem folgenden: „Wenn eine Relation eine Äquivalenzrelation ist, dann besteht sie aus disjunkten Teilrelationen.“ Ist dieser Satz eine Subjunktion, Bijunktion, Implikation oder Äquivalenz? Was sagt dieser Satz über eine Relation aus, die disjunkte Teilrelationen hat?
10. Welche besonderen Eigenschaften (reflexiv, symmetrisch *etc.*) hat jeweils eine Relation R auf der Menge A mit der folgenden Eigenschaft?
 - (a) $R^{-1} = R$
 - (b) $R^{-1} \cap R \subseteq I_A$
 - (c) $R^{-1} \cap R = \emptyset$
 - (d) $R \circ R \subseteq R$

Bearbeiten Sie auch Übungsaufgaben aus anderen Lehrtexten, z. B. die Kontrollfragen und Übungsaufgaben zu Abschnitt 5.1 im Lehrbuch von Teschl und Teschl.

Kapitel 8

Funktionen

aliis linearum in figura data functiones facientium generibus assumtis
(Gottfried Wilhelm Leibniz)

8.1 Aufwärmübungen

1. Welche der folgenden Situationen lässt sich als Funktionenmaschine modellieren?

- (a) Der Zusammenhang zwischen Person und ihrer Staatsbürgerschaft
- (b) Der Zusammenhang zwischen einem Wort und der Anzahl seiner Buchstaben
- (c) Die Relation $\{[x, \sin(x)] : x \in \mathbb{R}\}$
- (d) Die Relation $\{[x^2, x] : x \in \mathbb{R}\}$
- (e) Die Addition ganzer Zahlen

(f)

Name	Matrikelnummer
Thomas Müller	123456
Eva Meyer	123457
Thomas Müller	123458

2. Untersuchen Sie die folgenden Relationen mit der *setlX*-Funktion **isMap**, d.h. lassen Sie **isMap(R)**, **isMap(S)** etc. berechnen.

- (a) $R = \{[1, 1], [1, 2], [2, 2]\}$
- (b) $S = \{[1, 1], [2, 1], [2, 2]\}$
- (c) $T = \{[n - 1, n + 1] : n \in \{-10..10\}\}$
- (d) $U = \{[n^2, n] : n \in \{-10..10\}\}$

Welche Eigenschaft einer Relation berechnet **isMap**? Wenden Sie ggf. **isMap** auf weitere Relationen an, um dies herauszufinden.

3. Leitfragen:

- (a) Wie erkennt man, dass sich etwas als Funktion modellieren lässt?
- (b) Wie kann man mittels Computer entscheiden, ob etwas eine Funktion ist?
- (c) Warum ist jede Definition eine Funktion?

8.2 Perspektiven auf Funktionen

Die Funktion ist eines der mächtigsten Konzepte und Modellierungsdenkzeuge. In der Informatik kommt keine Programmiersprache ohne sie aus. Funktionen sind wie Gesichter: Von unterschiedlichen Perspektiven betrachtet sieht ein Gesicht unterschiedlich aus, obwohl es sich um dasselbe Gesicht handelt. Jede Perspektive hat dabei seine Eigenart: z. B. ist im Profil die Nase am Rand, während sie in der Frontansicht in der Mitte ist.

Das bedeutet für Sie: Sie müssen Funktionen als solche erkennen können, egal ob Sie sie von vorne, von oben oder im Profil sehen. Damit nicht genug: Sie müssen Funktionen denken können. Sie müssen denken können, wie eine Funktion, die sie von vorne sehen, im Profil aussieht.

8.2.1 Funktion als Transformationsprozess

Diese Perspektive kennen Sie bereits von der Funktionenmaschine aus Abschnitt 1.5: Eine Funktion ist ein Prozess, der Eingaben in Ausgaben transformiert. Dies ist besonders in der Informatik eine verbreitete Sichtweise. Das Konzept *procedure* in *setlX* trägt diese Prozess-Sichtweise sogar im Namen.

Diese Perspektive ist allerdings unvollständig. Sie sagt nur aus, *dass* transformiert wird. Aber sie sagt nichts darüber aus, *was* transformiert wird und *worauf* es transformiert wird.

Vielleicht wollen Sie hinzufügen, dass diese Perspektive auch nichts darüber aussagt, *wie* transformiert wird. Die Frage nach dem *Wie* ist für eine Funktion nicht von herausragender Bedeutung, u. a. weil es häufig mehr als ein Wie, also mehr als einen Weg gibt, eine Funktion zu implementieren. Dieser deklarative Charakter von Funktionen ist erfahrungsgemäß gerade für Informatikstudierende schwer greifbar, weil diese in der Programmierausbildung sinnvollerweise zunächst imperativ zu arbeiten lernen. Gehen Sie ggf. nochmals zurück zu Kapitel 1, um die Abstraktion von imperativer Herangehensweise zu einer deklarativen zu erarbeiten.

Die Kuchenmaschine in Abb. 1.4 ist durchaus eingeschränkt darin, *was* sie transformiert. Sie transformiert lediglich Kuchenzutaten, aber z. B. keine Autos oder Mineraleerze. Sie verwandelt diese Zutaten lediglich in Kuchen, aber z. B. nicht in Menschen.

Die Eingaben in den Transformationsprozess einer Funktion können beliebig komplex sein. Im Fall der Kuchenmaschine ist dies eine Liste von Zutaten, die am besten als Tupel modelliert wird. Bei der Sinus-Funktion ist die Eingabe einfach eine reelle Zahl. Wir wollen uns an dieser Stelle zunächst nicht um die passende Struktur für *eine* Eingabe kümmern, sondern um die passende Struktur für die Kollektion *aller* Eingaben. Dabei stellt sich die Frage, welche Eigenschaften diese Kollektion hat: Ist ein eventuelles Mehrfachvorkommen von Eingaben in der Kollektion relevant? Ist die Reihenfolge der Eingaben, die die Funktion jeweils zu einer Ausgabe transformiert, relevant? Was ist also das geeignete Modell für die Kollektion aller möglichen Eingaben?



Entsprechend verhält es sich mit den Ausgaben. Sie können ebenfalls zu einer Kollektion zusammengefasst werden. Ist die Reihenfolge möglicher Ausgaben in dieser Kollektion von Bedeutung? Ist ein Mehrfachvorkommen von Bedeutung? Was ist also das geeignete Modell für die Kollektion aller möglichen Ausgaben?



Fassen wir zusammen: Als Transformationsprozess betrachtet hat eine Funktion drei Bestandteile: Den Prozess selbst und die Kollektionen möglicher Eingaben und Ausgaben. Dazu kommt optional ein Name. Ohne Namen ist es schwer Dinge, hier Funktionen, anzusprechen. Dies führt auf folgende Definition:

Eine Funktion $f : \mathbb{D} \rightarrow \mathbb{W}$ ist ein Prozess, der jedes Element x einer Menge $\mathbb{D}$ (die sogenannte *Definitionsmenge* - englisch *domain*) eindeutig in ein Element $f(x)$ einer Menge $\mathbb{W}$ (die sogenannte *Wertemenge* - englisch *range*) transformiert.

Alternativ wird die Definitionsmenge auch als *Urbildmenge* bezeichnet. Das Objekt $f(x)$, in das die Funktion $x \in \mathbb{D}$ transformiert, wird als *Funktionswert an der Stelle x* , als *Bild von x* oder auch als *Rückgabewert für x* bezeichnet.

Schlüsselbegriffe in dieser Definition sind „jede“ und „eindeutig“. *Jede* Eingabe in die Funktionenmaschine wird von dieser verarbeitet und in eine Ausgabe transformiert - aber nur Eingaben aus der Definitionsmenge. Die Transformation ist *eindeutig*, d. h. es kann nicht passieren, dass beim selben Input unterschiedliche Outputs generiert werden.

Die Schreibweise $f : \mathbb{D} \rightarrow \mathbb{W}$ beinhaltet kompakt die wesentlichen Bausteine einer Funktion: Definitionsmenge $\mathbb{D}$, Wertemenge $\mathbb{W}$, die Transformation $\mathbb{D} \rightarrow \mathbb{W}$, der jedes Element der Definitionsmenge in ein Element der Wertemenge transformiert, und den Namen f .

„Werte“ in Wertemenge wird häufig missverstanden als Menge der „Eingabewerte“. Tatsächlich bezeichnet es die Menge der *Ausgabewerte*. Denken Sie an die Kuchenmaschine aus Abb. 1.4: Die „Werte“, die interessieren, sind die produzierten Kuchen, nicht die dafür verarbeiteten Eingaben (Zutaten).

Beachten Sie, dass das Symbol $\rightarrow$ nun kontextabhängig zwei unterschiedliche Bedeutungen hat. Verknüpft es Aussageformen miteinander, steht $\rightarrow$ für die Subjunktion. Verknüpft es Mengen miteinander wie in $\mathbb{D} \rightarrow \mathbb{W}$, steht es für die Abbildung von Mengen aufeinander durch eine Funktion.

Eine Funktion besteht also aus vier elementaren Bausteinen: Ihrem Namen f , ihrer Definitionsmenge $\mathbb{D}$, ihrer Wertemenge $\mathbb{W}$ und dem Prozess, der jedes Element von $\mathbb{D}$ eindeutig auf ein Element von $\mathbb{W}$ abbildet. Eine häufig verwendete Darstellung, um diese Perspektive zu visualisieren, bedient sich in einer Art Venn-Diagramm zweier „Mengenblasen“ und Pfeile, die den Prozess der Transformation der Elemente der Definitionsmenge in Elemente der Wertemenge darstellen (natürlich ohne zu sagen, *wie* dieser Prozess im Detail abläuft).

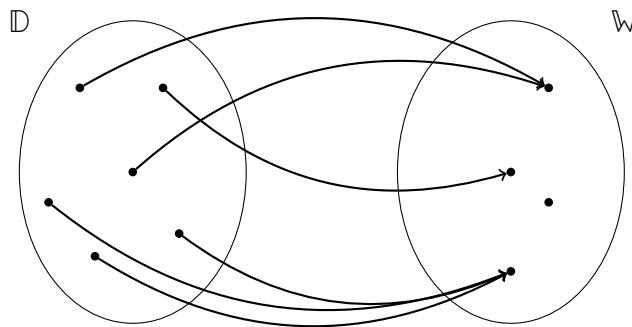


Abbildung 8.1: Funktion als *Prozess*, die jedes Element der Definitionsmenge eindeutig in ein Element der Wertemenge transformiert.

Diese Art von Visualisierung des Funktionenkonzepts macht die Transformation *aller* möglichen Eingaben in eine Funktion besonders deutlich. Ein gängiges Synonym für Transformation ist *Zuordnung*: Die Funktion *ordnet* jedem Element der Definitionsmenge eindeutig ein Element der Wertemenge *zu*. Man kann auch sagen: Die Funktion *bildet* jedes Element der Definitionsmenge eindeutig auf ein Element der Wertemenge *ab*. Die Begriffe Funktion und *Abbildung* (und der entsprechende englische Begriff *map*) sind in der Mathematik synonym.¹ Manchmal werden Funktionen auch als Operationen bezeichnet. Sie kennen dies u. a. von der Addition. Diese Operation ist eine Funktion, die zwei gleichartige Objekte (z. B. reelle Zahlen) in ein neues Objekt von dieser Art transformiert.

Um das Besondere an Funktionen zu begreifen, ist es instruktiv, entsprechende Diagramme zu zeichnen, bei denen jeweils eine der definierenden Eigenschaften von Funktionen nicht erfüllt ist. Sie sind also dran:

¹Die Namensvielfalt in der Informatik ist noch viel größer: Unterroutine, Funktion, *map*, Methode. All dies sind spezielle Beschreibungsformen von Funktionen. Beispielsweise sind Methoden Beschreibungsformen von Funktionen, die das Verhalten von Objekten beschreiben und implementieren. Entgegen einer in der Informatik häufig kolportierten Meinung sind Methoden kein mächtigeres Konzept als Funktionen, sondern ein Spezialfall, der mit einer speziellen Darstellungsform einhergeht.

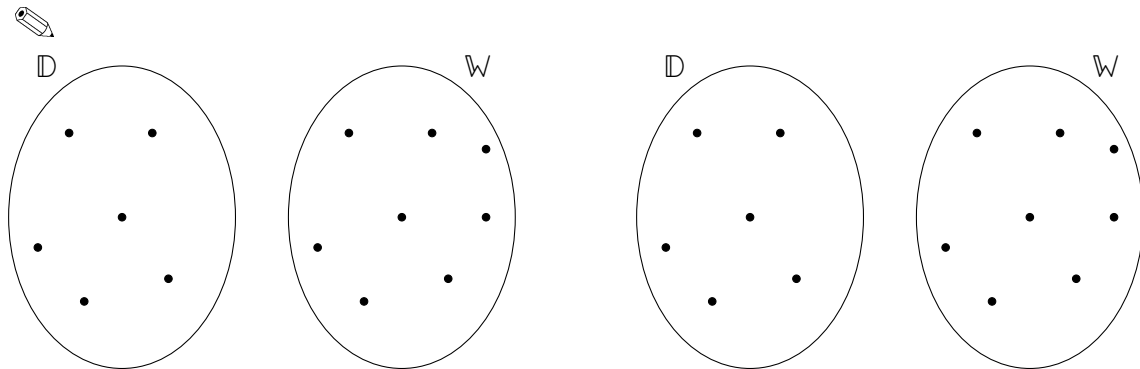


Abbildung 8.2: Links: Beziehung zwischen zwei Mengen $\mathbb{D}$ und $\mathbb{W}$, die die definierende Bedingung an eine Funktion verletzt, dass jedes Element von $\mathbb{D}$ in ein Element von $\mathbb{W}$ transformiert wird. Rechts: Beziehung zwischen zwei Mengen $\mathbb{D}$ und $\mathbb{W}$, die die definierende Bedingung an eine Funktion verletzt, dass die Elemente von $\mathbb{D}$ eindeutig in Elemente von $\mathbb{W}$ transformiert werden.

8.2.2 Funktion als spezielle Relation

Aus der Perspektive dieses Kurses betrachtet sind Relationen die nächsten Verwandten von Funktionen. Eine Funktion mit Definitionsmenge $\mathbb{D}$ und Wertemenge $\mathbb{W}$ ist eine Relation zwischen $\mathbb{D}$ und $\mathbb{W}$ mit der zusätzlichen Eigenschaft der Eindeutigkeit. Jedem Element aus $\mathbb{D}$ ist also eindeutig ein Element aus $\mathbb{W}$ zugeordnet. Die Beziehungen, also Relationen, die Sie in Abb. 8.2 graphisch dargestellt haben, können (mit den dort verwendeten Mengen $\mathbb{D}$ und $\mathbb{W}$) dagegen nicht als Funktionen der Art $\mathbb{D} \rightarrow \mathbb{W}$ beschrieben werden, weil sie jeweils eine definierende Eigenschaft nicht erfüllen.

Beispiel 8.1: Die Menge

$$A = \{[\clubsuit, 0], [\clubsuit, 42], [\spadesuit, 42], [\heartsuit, 42], [\diamondsuit, 42]\}$$

kann als Relation zwischen den Mengen $\mathbb{D} = \{\clubsuit, \diamondsuit, \heartsuit, \spadesuit\}$ und $\mathbb{W} = \mathbb{N}$ modelliert werden, denn $A \subseteq \mathbb{D} \times \mathbb{W}$. Ebenso können die Mengen

$$\begin{aligned} B &= \{[\heartsuit, 42], [\diamondsuit, 42]\}, \\ F &= \{[\clubsuit, 42], [\spadesuit, 42], [\heartsuit, 42], [\diamondsuit, 42]\} \end{aligned}$$

als solche Relationen modelliert werden, denn $B \subseteq \mathbb{D} \times \mathbb{W}$ und $F \subseteq \mathbb{D} \times \mathbb{W}$. Allerdings kann nur F als Funktion mit der Definitionsmenge $\mathbb{D}$ modelliert werden. Bei F wird *jedes* Element von $\mathbb{D}$ in ein Element von $\mathbb{W}$ transformiert. Bei B ist dies nicht für *jedes* Element von $\mathbb{D}$ der Fall. A erfüllt nicht die Eigenschaft der Eindeutigkeit des Transformationsprozesses einer Funktion, denn $\clubsuit$ wird sowohl in 0 als auch 42 transformiert. ■

Die Sichtweise auf Funktionen als spezielle Relationen führt zu folgender Definition des Begriffs Funktion:

Eine Funktion $f : \mathbb{D} \rightarrow \mathbb{W}$ ist eine Menge von Paaren $[x, y]$ mit $x \in \mathbb{D}$, $y \in \mathbb{W}$, so dass

1. keine zwei Paare dieselbe erste, aber unterschiedliche zweite Komponenten haben und
2. es für jedes $x \in \mathbb{D}$ genau ein Paar $[x, y] \in f$ gibt.

Auch diese Definition nimmt natürlich wieder Bezug auf die vier elementaren Bausteine einer Funktion: Name, Definitions- und Wertemenge und Prozess.

Das Verb „ist“ bedeutet auch hier wieder keine Seinseigenschaft, sondern „kann modelliert werden als“.

Mittels *setIX* kann ausgehend von der obigen Definition leicht überprüft werden, ob eine Menge von Paaren als Funktion modelliert werden kann. Dazu werden die Mengen $\mathbb{D}$ und $\mathbb{W}$ als die kleinstmöglichen Mengen bestimmt, so dass die Menge von Paaren eine Relation zwischen $\mathbb{D}$ und $\mathbb{W}$ ist. Die geschieht mit

den Befehlen `domain` und `range`, was zugleich die englischen Begriffe für Definitions- bzw. Wertemenge sind. Der Befehl `isMap` überprüft, ob Eindeutigkeit in der Transformation von $\mathbb{D}$ nach $\mathbb{W}$ vorliegt.

Beispiel 8.2: Die Relation A

```
=> A:={"Kreuz",0},{"Kreuz",42},{"Pik",42},{"Herz",42},{"Karo",42};;
~< Result: {"Herz", 42}, {"Karo", 42}, {"Kreuz", 0}, {"Kreuz", 42}, {"Pik", 42} >~
```

aus dem obigen Beispiel ist eine Relation zwischen den Mengen

```
=> D:=domain(A);
~< Result: {"Kreuz", "Herz", "Karo", "Pik"} >~
```

und

```
=> W:=range(A);
~< Result: {0, 42} >~
```

denn $A \subseteq \mathbb{D} \times \mathbb{W}$. Diese Relation lässt sich jedoch bei der obigen Wahl von $\mathbb{D}$ und $\mathbb{W}$ nicht als Abbildung der Form $\mathbb{D} \rightarrow \mathbb{W}$ modellieren:

```
=> isMap(A);
~< Result: false >~
```

Dagegen ist die Relation F eine Abbildung:

```
=> F:={"Pik",42},{"Kreuz",42},{"Herz",42},{"Karo",42};;
~< Result: {"Herz", 42}, {"Karo", 42}, {"Kreuz", 42}, {"Pik", 42} >~
=> isMap(F);
~< Result: true >~
```

Hier ist die Transformation von $\mathbb{D}$ auf $\mathbb{W}$ eindeutig. Damit kann F als Funktion mit $\mathbb{D} = \{\clubsuit, \diamond, \heartsuit, \spadesuit\}$, $\mathbb{W} = \{42\}$ und einem Prozess, der jedes Symbol in $\mathbb{D}$ auf 42 transformiert, modelliert werden. ■

8.2.3 Funktion als Kovariation

Ein in sehr vielen Wissenschaftsdisziplinen häufig verwendetes Modellierungswerkzeug ist die *Variable*. Variablen werden zur Beschreibung von Größen verwendet, deren Werte sich ändern können, also variabel sind, z. B. Lebensalter, Geschwindigkeit, Höhe über Meeresspiegel. Um Variablen anzugeben, benötigt man eine Menge von möglichen Werten, die die Variable annehmen kann. Zudem ist es zweckmäßig, die Variable mit einem Symbol zu adressieren.

Beispiel 8.3: Im Satz

Wenn heute Montag ist, dann ist morgen Mittwoch

können „heute“ und „morgen“ als Variable modelliert werden. Die Menge der möglichen Werte ist für beide Variablen die Menge der Wochentage. Geeignete Variablensymbole sind „heute“ und „morgen“ oder auch h und m . ■

Häufig variieren zwei Größen gemeinsam, z. B. Rohölpreis und Heizölpreis oder heute und morgen. Es kann also sein, dass zwei oder auch mehr Variablen irgendwie „zusammenhängen“. Die beiden Variablen müssen dabei nicht von derselben Art sein, also ihre Werte nicht Elemente derselben Menge sein. Der Drehwinkel, um den ein Heizungsthermostat verdreht wurde, und die Raumtemperatur sind Variablen, die nicht von derselben Art sind, aber zusammenhängen.

Solche Zusammenhänge sind für uns Menschen besonders interessant, weil sie viele hilfreiche Aktivitäten erlauben. Wenn zwei variable Größen zusammenhängen, können wir unsere Kenntnis des Wertes einer Variablen benutzen, um auf den Wert der anderen zu schließen. Wir können die eine Größe benutzen, um die andere zu steuern. Und vieles mehr.

Um solche Zusammenhänge nutzen zu können, ist es wichtig, dass sie *eindeutig* sind. Wir wollen die Kenntnis des Wertes einer Variablen benutzen, um *eindeutig* auf den Wert der anderen zu schließen. Das ist genau das Konzept der Eindeutigkeit aus der Funktionendefinition in Abschnitt 8.2.1.

Das passende mathematische Konzept, um solche eindeutigen Zusammenhänge zwischen zwei Variablen zu beschreiben, ist die Funktion:

Eine Funktion $f : x \mapsto y$ ist ein Zusammenhang zwischen zwei Variablen $x \in \mathbb{D}$ und $y \in \mathbb{W}$, bei dem es immer möglich ist, eindeutig aus dem Wert von x auf den Wert von y zu schließen.

Diese Definition ist natürlich äquivalent zu der aus Abschnitt 8.2.1, nur die Sichtweise ist eine etwas andere: Dort werden Mengen eindeutig transformiert: $\mathbb{D} \rightarrow \mathbb{W}$. Hier werden Mengen als Variablen gedacht und die Variablenwerte transformiert: $x \mapsto y$. Beachten Sie die unterschiedlichen Symbole $\rightarrow$ und $\mapsto$, die für die beiden Sichtweisen verwendet werden: $\rightarrow$ für die Mengentransformation, $\mapsto$ für die Variablentransformation.

Die bei einer Funktion involvierten Variablen haben unterschiedliche Rollen, die begrifflich unterschieden werden. Die Variable, die transformiert wird (oben mit x symbolisiert), heißt *unabhängige Variable*. Die Variable, auf die transformiert wird (oben mit y symbolisiert), wird *abhängige Variable* genannt.

Beispiel 8.4: Bei der gemeinsamen Variation von „heute“ und „morgen“ kann „heute“ als unabhängige Variable angesehen werden, aus der sich der Wert der abhängigen Variablen „morgen“ eindeutig ergibt.

Auch die umgekehrte Sichtweise ist hier möglich: Die unabhängige Variable ist „morgen“. Aus ihrem Wert ergibt sich eindeutig der Wert der abhängigen Variablen „heute“. ■

Beispiel 8.5: Da der Heizölpreis vom Rohölpreis abhängt, ist es naheliegend den Heizölpreis als abhängige Variable und den Rohölpreis als unabhängige Variable zu betrachten. Die gemeinsamen Variation der beiden Preise kann allerdings in der Regel nicht als Funktion beschrieben werden. Es ist möglich, dass zu unterschiedlichen Zeiten bei gleichem Rohölpreis der Heizölpreis unterschiedlich ist. Damit ist das Kriterium der Eindeutigkeit verletzt.

Selbst wenn man den Heizölpreis als die unabhängige Variable und entsprechend den Rohölpreis als abhängige Variable betrachten würde, wäre das Kriterium der Eindeutigkeit verletzt. Es ist möglich, dass zu unterschiedlichen Zeiten bei gleichem Heizölpreis der Rohölpreis unterschiedlich ist. ■

Anstelle von unabhängige Variable ist auch die Bezeichnung *Argument der Funktion* üblich.²

8.3 Formale Notation

Um zu beschreiben, *was* eine Funktion tut, muss man eigentlich nur Definitions- und Wertemenge angeben. Nimmt man zur Beschreibung noch den Namen dazu, kann man die Funktion auch direkt ansprechen. Die folgenden Sätze beschreiben in diesem Sinne klar, was die einzelnen Funktionen tun: Die Sinus-Funktion bildet reelle Zahlen auf reelle Zahlen ab. Die Funktion F aus Abschnitt 8.2.2 bildet Spielkartenfarben auf die Zahl 42 ab. Die modulo-Funktion bildet Paare bestehend aus ganzen und natürlichen Zahlen auf Zahlen aus $\mathbb{N}_0$ ab.

In keinem dieser Beispiel sagt die Funktionenbeschreibung, *wie* die Funktionen jeweils tun, was sie tun. Die vier Bausteine einer Funktion (Name, $\mathbb{D}$, $\mathbb{W}$, Transformationsprozess) lassen sich daher sinnvoll in zwei Gruppen aufteilen: Die eine Gruppe beschreibt, *was* die Funktion tut. Zu ihr gehören Name, Definitions- und Wertemenge. Dieses Was wird mit der *Signatur* einer Funktion beschrieben. Darüber hinaus muss man nichts wissen, um zu entscheiden, ob eine Situation oder ein Objekt als Funktion modelliert werden kann. Dazu reicht alleine die Signatur aus.

Die zweite Gruppe beschreibt, *wie* die Funktion tut, was sie tut. Das ist die *algorithmische* Beschreibung des Transformationsprozesses bzw. Algorithmus. In der Programmierung erfolgt die Beschreibung des Algorithmus des Transformationsprozesses im Körper der Funktion.

Formal wird die Signatur einer Funktion in der Form

$$f : \mathbb{D} \rightarrow \mathbb{W}$$

²In der Informatik ist inzwischen der Begriff „Parameter“ für Argument üblich geworden. Diese Begriffsverwendung entspricht jedoch nicht der mathematischen Bedeutung von Parameter. Vermutlich liegt hier eine Bedeutungserweiterung vor: Die korrekte Bezeichnung von Symbolen im Funktionenkörper als Parameter wurde ausgeweitet auf die Bezeichnung von Variablen in der Signatur der Funktion.

aufgeschrieben. Die Signatur benennt die drei konstituierenden Elementen Name (hier f), Definitionsmenge (hier $\mathbb{D}$) und Wertemenge (hier $\mathbb{W}$). Manchmal werden zusätzlich die Symbole für unabhängige und abhängige Variable (nachfolgend x bzw. y) genannt. Die Signatur hat dann die Form:

$$f : \mathbb{D} \rightarrow \mathbb{W}, x \mapsto y$$

Beachten Sie dabei die unterschiedlichen Pfeilsymbole. Das Symbol $\rightarrow$ verknüpft Mengen. Das Symbol $\mapsto$ (lies: wird abgebildet auf) verknüpft Variablen bzw. deren Werte. Bei der Sinus-Funktion treten z. B. u. a. folgende Werteverknüpfungen auf: $0 \mapsto 0$, $\pi/2 \mapsto 1$, $\pi \mapsto 0$.

Beispiel 8.6: Die Signatur der Sinus-Funktion ist $\sin : \mathbb{R} \rightarrow \mathbb{R}$ bzw., wenn man unabhängige und abhängige Variable als φ und s bezeichnet, $\sin : \mathbb{R} \rightarrow \mathbb{R}, \varphi \mapsto s$. ■

Will man sich ein Symbol für die abhängige Variable sparen, schreibt man die Signatur in der Form

$$f : \mathbb{D} \rightarrow \mathbb{W}, x \mapsto f(x). \quad (8.1)$$

Darin bezeichnet $f(x) \in \mathbb{W}$ den Wert, in den die Funktion f die unabhängige Variable $x \in \mathbb{D}$ transformiert. In der Form (8.1) enthält die Signatur alle Bausteine, mit der sie in vielen Programmiersprachen aufgeschrieben wird:

`W name(D unabhaengigeVariable)`

Statt $\sin : \text{float} \rightarrow \text{float}, \varphi \mapsto \sin(\varphi)$ wird bspw. `float sin(float phi)` geschrieben. Die Information ist dieselbe, nur die Reihenfolge der Symbole ist eine andere.

Die Schreibweise $x \mapsto f(x)$ in (8.1) ist übrigens genau die Syntax, mit der in einigen Programmiersprachen (v. a. den funktionalen) konkrete Funktionen definiert werden können, so auch in *setlX*:

`funktionsname := unabhaengigeVariable |-> Rueckgabewert;`

Beispiel 8.7: Anstelle von

```
=> sqr:=procedure(x){return x*x;};
~< Result: procedure(x) { return x * x; } >~
```

kann die Quadratfunktion in *setlX* auch „platzsparend“ mittels

```
=> sqr:=x|->x*x;
~< Result: x |-> x * x >~
```

implementiert werden. ■

8.4 Bild- und Wertemenge

Vielleicht halten Sie es für seltsam, dass das Konzept der Funktion fordert, dass *alle* Elemente der Definitionsmenge (eindeutig) auf ein Element der Wertemenge abgebildet werden müssen, aber nicht, dass *auf alle* Elemente der Wertemenge abgebildet werden muss. Während die Definitionsmenge genau passend sein muss, darf die Wertemenge „zu groß“ sein, d. h. sie darf Elemente enthalten, auf die niemals abgebildet werden wird. Warum ist das so? Warum diese scheinbare Ungleichbehandlung von Definitions- und Wertemenge?

Zunächst einmal schauen wir uns die mögliche Alternative zur Wertemenge an. Das ist die Menge, die „genau richtig dimensioniert ist“, also nur Elemente enthält, auf die durch die Funktion auch tatsächlich abgebildet wird. Sie wird als *Bildmenge* der Abbildung bezeichnet.

Die Menge

$$f(\mathbb{D}) = \{y : y \in \mathbb{W} \mid \exists(x \in \mathbb{D} \mid f(x) = y)\}$$

der Elemente der Wertemenge, auf die eine Funktion tatsächlich abbildet, wird als *Bildmenge* der Funktion f bezeichnet und mit $f(\mathbb{D})$ symbolisiert.

Beispiel 8.8: Die Sinus-Funktion hat $\mathbb{W} = \mathbb{R}$ und $\sin(\mathbb{D}) = \{x : x \in \mathbb{R} \mid x \leq 1 \wedge x \geq -1\}$. ■

Würde man für die Definition einer Funktion die Bild- statt der Wertemenge verwenden, würde dies schnell zu kaum bewältigbaren Komplikationen führen.

Beispiel 8.9: Die Funktion $f_1 : \mathbb{Z} \rightarrow \mathbb{Z}, x \mapsto (x+1)^2$ hat als Bildmenge die Menge der Quadratzahlen $f_1(\mathbb{D}) = \{x^2 : x \in \mathbb{N}\}$. In Java dürfte man dann nicht

```
int f1(int x) {return (x+1)*(x+1);}
```

schreiben, denn `int` ist ja nicht die Bildmenge. Man müsste stattdessen vorher eine Klasse `quad` für die Menge $f_1(\mathbb{D})$ der Quadratzahlen erzeugen, um dann

```
quad f1(int x) {return (x+1)*(x+1);}
```

schreiben zu können.

Diese Komplikation tritt potentiell bei jeder weiteren Funktion auf. Für $f_2 : \mathbb{Z} \rightarrow \mathbb{Z}, x \mapsto -x^2$ müsste als Bildmenge eine weitere Klasse `negquad` als Menge der negativen Quadratzahlen erzeugt werden, um die Funktion als

```
negquad f2(int x) {return -x*x;}
```

überhaupt definieren zu können. Da ist es schon angenehmer einfach die Wertemenge statt der Bildmenge angeben zu können und damit

```
int f2(int x) {return -x*x;}
```

schreiben zu können. ■

8.5 Formale Definition

Wir haben nun alles zusammen, um Funktionen formal mit den Mitteln der Logik zu definieren. Eine solche Definition kann sofort in ein Programm übersetzt werden, mit dessen Hilfe wir mittels Rechner überprüfen können, ob ein vorliegendes Objekt als Funktion beschreibbar ist.

Eine Funktion $f : \mathbb{D} \rightarrow \mathbb{W}, x \mapsto f(x)$ ist ein Prozess, der jedes Element x einer Menge $\mathbb{D}$ (die sogenannte *Definitionsmenge*) eindeutig in ein Element $f(x)$ einer Menge $\mathbb{W}$ (die sogenannte *Wertemenge*) transformiert.

Statt „eindeutig in ein Element $f(x)$ transformiert“ könnte man auch „in genau ein Element $f(x)$ transformiert“ sagen. Das ist eine der wenigen Stellen, wo „es gibt genau ein“ statt „es gibt (mindestens) ein“ (also $\exists$) eine Rolle spielt. Das eine lässt sich aber durch das andere ausdrücken, denn „genau ein“ bedeutet „es gibt (mindestens) ein, aber nicht zwei unterschiedliche“.

Beispiel 8.10: Für „es gibt ein Element von A , für das die Aussageform $A(x)$ gilt“ schreiben wir bekanntlich $\exists(a \in A \mid A(a))$. Damit können wir die quantifizierte Aussage „es gibt *genau* ein Element von A , für das die Aussageform $A(x)$ gilt“ so ausdrücken:

$$\exists(a \in A \mid A(a)) \wedge \neg \exists(a_1, a_2 \in A \mid a_1 \neq a_2 \wedge A(a_1) \wedge A(a_2))$$

Durch $\neg \exists(a_1, a_2 \in A \mid a_1 \neq a_2 \wedge A(a_1) \wedge A(a_2))$ wird sichergestellt, dass es keine zwei verschiedenen Elemente aus A gibt, die die Aussageform beide erfüllen (d. h. wahr machen). ■

In der obigen Definition bedeutet „ x wird eindeutig in ein Element $f(x)$ der Menge $\mathbb{W}$ transformiert“ also

$$\begin{aligned} &\exists(f(x) \in \mathbb{W} \mid x \mapsto f(x)) \wedge \\ &\neg \exists(f(x_1), f(x_2) \in \mathbb{W} \mid f(x_1) \neq f(x_2) \wedge x \mapsto f(x_1) \wedge x \mapsto f(x_2)) \end{aligned}$$

Die Definition fordert, dass dies für alle x gilt. Daher lässt sich die Definition einer Funktion $f : \mathbb{D} \rightarrow \mathbb{W}$ formal als

$$f : \mathbb{D} \rightarrow \mathbb{W} \Leftrightarrow \forall (x \in \mathbb{D} \mid \exists (f(x) \in \mathbb{W} \mid x \mapsto f(x)) \wedge \neg \exists (f(x_1), f(x_2) \in \mathbb{W} \mid f(x_1) \neq f(x_2) \wedge x \mapsto f(x_1) \wedge x \mapsto f(x_2))) \quad (8.2)$$

schreiben. Der negierte Existenz-Quantor lässt sich als All-Quantor schreiben:

$$f : \mathbb{D} \rightarrow \mathbb{W} \Leftrightarrow \forall (x \in \mathbb{D} \mid \exists (f(x) \in \mathbb{W} \mid x \mapsto f(x)) \wedge \forall (f(x_1), f(x_2) \in \mathbb{W} \mid x \mapsto f(x_1) \wedge x \mapsto f(x_2) \rightarrow f(x_1) = f(x_2))) \quad (8.3)$$

Sie sollten zu diesem Zeitpunkt genügend formale Logik beherrschen, um die einzelnen Schritte, die zu dieser Umformung geführt haben, nachvollziehen zu können. Beweisen Sie sich, dass Sie das können!



In Worten bedeutet (8.3): Zu jedem x gibt es mindestens ein $f(x)$ und, wenn x auf mehrere f -Werte abgebildet wird, dann müssen diese gleich sein.

Beachten Sie, dass in (8.3) das Symbol $\rightarrow$ eine doppelte Bedeutung hat. Links vom Äquivalenzpfeil steht es für die Transformation von Definitions- in Wertemenge. Rechts steht es für die Subjunktion von Aussageformen.

Wenn es Ihnen lieber ist, können Sie natürlich an Stelle von $x \mapsto f(x)$ mit $x \mapsto y$ arbeiten. Dann kann der Wahrheitswert von $x \mapsto f(x)$ auch mittels $y = f(x)$ überprüft werden. Die formale Definition (8.3) ist dann frei vom Symbol $\mapsto$:

$$f : \mathbb{D} \rightarrow \mathbb{W} \Leftrightarrow \forall (x \in \mathbb{D} \mid \exists (y \in \mathbb{W} \mid y = f(x)) \wedge \forall (y_1, y_2 \in \mathbb{W} \mid y_1 = f(x) \wedge y_2 = f(x) \rightarrow y_1 = y_2))) \quad (8.4)$$

Die rechte Seite von (8.4) ist ein formallogischer Ausdruck, mit dem ein Rechner überprüfen kann, ob ein Tupel $[f, \mathbb{D}, \mathbb{W}]$ als Funktion modelliert werden kann. Eine Implementation in *setlX* erfordert also nichts anderes als bspw. (8.4) direkt abzutippen:

```
forall(x in D | exists(y in W|y==f(x)) &&
    forall(y1 in W, y2 in W | y1==f(x) && y2==f(x) => y1==y2 )
);
```

Das ist eine Aussageform mit den ungebundenen Variablen f , $\mathbb{D}$ und $\mathbb{W}$. Wir können also eine Funktion mit diesen Variablen schreiben, die wahr zurückgibt, wenn f eine Funktion $\mathbb{D} \rightarrow \mathbb{W}$ ist, und andernfalls falsch zurückgibt!

Beispiel 8.11: Nennen wir die Funktion, mit deren Hilfe wir überprüfen wollen, ob ein Objekt $[f, \mathbb{D}, \mathbb{W}]$ als Funktion beschrieben werden kann, `isFunction`:

```
isFunction:=closure(f,D,W){
    return forall(x in D |
        exists(y in W|y==f(x)) &&
        forall(y1 in W, y2 in W | y1==f(x) && y2==f(x) => y1==y2 )
    );
};
```

Vermutlich haben Sie in diesem Code an der Stelle von `closure` das Schlüsselwort `procedure` erwartet. Aus technischen Gründen, die in Kapitel 9 genannt werden, ist dies nicht möglich. Für Ihr Verständnis können Sie an dieser Stelle jedoch so tun, als stünde dort `procedure`.

Wir überprüfen mit `isFunction` ob die `procedure` `sqr` aus Abschnitt 8.3 eine Funktion von $\{1, 2\}$ nach $\{1..5\}$ ist:

```
=> isFunction(sqr,{1,2},{1..5});
~< Result: true >~
```

Dagegen ist `sqr` keine Funktion von $\{1, 2, 3\}$ nach $\{1..5\}$

```
=> isFunction(sqr,{1,2,3},{1..5});
~< Result: false >~
```

denn es gibt kein Element, auf das die 3 abgebildet wird. ■

Betrachten wir im Sinne von Abschnitt 8.2.2 Relationen als mögliche Funktion, können wir den Wahrheitswert von $y = f(x)$ in (8.4) für eine gegebene Relation f mittels $[x, y] \in f$ überprüfen. Dies führt auf den formalen Ausdruck:

$$f : \mathbb{D} \rightarrow \mathbb{W} \Leftrightarrow \forall(x \in \mathbb{D} \mid \exists(y \in \mathbb{W} \mid [x, y] \in f) \wedge \forall(y_1, y_2 \in \mathbb{W} \mid [x, y_1] \in f \wedge [x, y_2] \in f \rightarrow y_1 = y_2))) \quad (8.5)$$

Auch die rechte Seite von (8.5) lässt sich unmittelbar so in *setlX* schreiben:

```
forall(x in D | exists(y in W | [x,y] in f) &&
    forall(y1 in W, y2 in W | [x,y1] in f && [x,y2] in f => y1==y2));
```

Beispiel 8.12: Wir bezeichnen die Funktion, mit deren Hilfe wir überprüfen wollen, ob eine Relation $f \subseteq \mathbb{D} \times \mathbb{W}$ als Funktion beschrieben werden kann, mit `isRelationFunction`

```
isRelationFunction:=procedure(f,D,W){
    return forall(x in D |
        exists(y in W | [x,y] in f) &&
        forall(y1 in W, y2 in W |
            [x,y1] in f && [x,y2] in f => y1==y2)
    );
};
```

und überprüfen erneut, ob die Relationen

$$\begin{aligned} A &= \{[\clubsuit, 0], [\clubsuit, 42], [\spadesuit, 42], [\heartsuit, 42], [\diamondsuit, 42]\}, \\ F &= \{[\clubsuit, 42], [\spadesuit, 42], [\heartsuit, 42], [\diamondsuit, 42]\} \end{aligned}$$

aus Abschnitt 8.2.2 als Funktionen beschrieben werden können:

```
=> A:={"Kreuz",0}, {"Kreuz",42}, {"Pik",42}, {"Herz",42}, {"Karo",42}};
~< Result: {"Herz", 42}, {"Karo", 42}, {"Kreuz", 0}, {"Kreuz", 42}, {"Pik", 42}} >~
=> F:={"Pik",42}, {"Kreuz",42}, {"Herz",42}, {"Karo",42}};
~< Result: {"Herz", 42}, {"Karo", 42}, {"Kreuz", 42}, {"Pik", 42}} >~
=> isRelationFunction(A,domain(A),range(A));
~< Result: false >~
=> isRelationFunction(F,domain(F),range(F));
~< Result: true >~
```

F kann als Funktion modelliert werden. Bei A ist dies nicht der Fall, weil es das Eindeutigkeitskriterium für $\clubsuit$ verletzt. ■

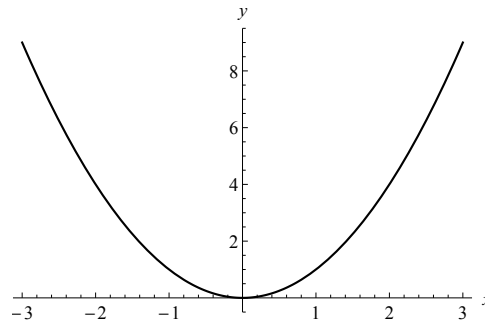
Wir sind nun auch in der Lage das *setlX*-Kommando `isMap` selbst zu implementieren, das für eine gegebenen Relation überprüft, ob sie als Funktion betrachtet werden kann:

```
myisMap:=procedure(f){
    D:= domain(f);
    W:= range(f);
    return isRelationFunction(f,D,W);
};
```

Beispiel 8.13: Wir wenden `myisMap` auf die beiden Relationen aus dem vorangegangenen Beispiel an:

```
=> myisMap(A);
~< Result: false >~
=> isMap(A);
~< Result: false >~
=> myisMap(F);
~< Result: true >~
```

`myisMap` führt natürlich zu denselben Ergebnissen wie das *setlX*-interne `isMap`. ■

Abbildung 8.3: Darstellung der Quadratfunktion $sqr : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto x^2$ als Punktgraph.

8.6 Repräsentationen von Funktionen

Wie für alle mathematischen Objekte gibt es auch für Abbildungen unterschiedliche Darstellungsformen. Denken Sie an natürliche Zahlen als Objekte. Auch diese lassen sich auf verschiedene Arten darstellen: Die Vier z.B. als Buchstabenfolge (vier), im Dezimalsystem als 4, im Dualsystem als 100, als römische Zahl als IV, als |||| und vieles mehr. Ebenso können Funktionen auf verschiedene Weisen dargestellt werden.

Die Darstellung als Relation kennen Sie bereits, ebenso die Darstellung durch Graphen wie in Abb. 8.1. Diese Art von Graphen werden *bipartite* Graphen genannt, weil die Menge der dargestellten Knoten in zwei Teile zerfällt: Solche, von denen die Pfeile ausgehen ($\mathbb{D}$), und solche, bei denen Sie enden können ($\mathbb{W}$).

Eine Darstellungsform, die Ihnen sicher auch geläufig ist, sind Punktgraphen. Abb. 8.3 zeigt ein Beispiel für die Quadratfunktion. Die Darstellung durch Tabellen bzw. Wertetabellen kennen Sie u. a. von der Tabelle aus Aufwärmübung 1f und den Wahrheitstabellen in Kapitel 2. Als ziemlich abstrakte Darstellungsform kennen Sie das Blockdiagramm der Funktionenmaschine.

Eher konkrete Darstellungsformen werden durch Ausdrücke und Programmcode realisiert. Der Ausdruck

$$c(x) = 1 - \frac{x^2}{2} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!}$$

in $c : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto c(x)$ beschreibt eine Funktion, die $\cos$ für kleine Argumentwerte näherungsweise ganz gut berechnet. Entsprechendes leistet das folgende Programm:

```
float c (float x){
    x2 := x*x;
    x4 := x2*x2;
    x6 := x4*x2;
    x8 := x4*x4;
    return 1 - x2/2 + x4/4! - x6/6! + x8/8!;
}
```

8.7 Modellieren mit Funktionen

Informatikerinnen und Informatiker sind regelmäßig gefordert zu entscheiden, ob eine gegebene Situation, ein Sachverhalt oder ein Vorgehen als Funktion modelliert werden kann. Nur wenn die Antwort Ja ist, ist klar, wie die Modellierung in einer gegebenen Programmiersprache zu erfolgen hat. Dann genügt es zunächst, die Signatur der Funktion aufzuschreiben, also Definitions- und Wertemenge anzugeben und der Funktion ggf. einen Namen zu geben.

Beispiel 8.14: Wie haben Sie sich beispielsweise in Aufwärmübung 1b hinsichtlich der Frage entschieden, ob der Zusammenhang zwischen einem Wort und der Anzahl seiner Buchstaben als Funktion beschrieben werden kann?

Dieser Zusammenhang erfüllt die beiden definierenden Eigenschaften einer Funktion: Zu jedem Wort gibt es eine Anzahl an Buchstaben und die Anzahl der Buchstaben ist für ein gegebenes Wort eindeutig. Die Elemente der Definitionsmenge sind hier Worte, also $\mathbb{D} = \{w | w \text{ ist Wort}\}$. Die Elemente der Wertemenge sind natürliche Zahlen einschließlich der Null (die Länge der leeren Zeichenkette), also $\mathbb{W} = \mathbb{N}_0$. Ein zweckmäßiger Name für diese Funktion ist *laenge*. Wir haben also als Signatur $laenge : \{w | w \text{ ist Wort}\} \rightarrow \mathbb{N}_0$.

Die tatsächliche Implementierung dieser Länge ist in *setlX* übrigens besonders einfach, da dies genau die Prozedur mit der Bezeichnung `#` ist. Beispielsweise ist

```
=> #("Konstantinopolitanischerdudelsackpfeifenkopfmachergeselle");
~< Result: 57 >~
```

■

Definitions- und Wertemengen können natürlich komplexer sein, z. B. Menge von Tupeln oder Mengen von Mengen.

Beispiel 8.15: Die folgende Tabelle zeigt das (vereinfachte) Angebot eines Snack-Automaten, bei dem neben der Entrichtung des Kaufpreises die Eingabe einer Tastenkombination nötig ist, um das gewünschte Produkt zu erhalten:

Tabelle 8.1: Snack-Automat

Preis in Cent	Tastenkombination	Produkt
50	12	Mars
60	18	Twix
50	21	Snickers
150	23	Ritter Sport
70	26	Bounty
50	29	KitKat

Lässt sich der durch die diese Tabelle beschriebene Snack-Automat mit Hilfe einer Funktion beschreiben? Sie sind dran! Sie wissen, was Sie zu tun haben: mögliche Definitions- und Wertemenge identifizieren und überprüfen, ob von jedem Element von $\mathbb{D}$ eindeutig auf ein Element von $\mathbb{W}$ geschlossen werden kann.



Bei dem Snack-Automaten in Tabelle 8.1 ist der Zusammenhang zwischen Preis und Produkt nicht eindeutig, denn es gibt mehrere Produkte, die 50 Cent kosten. Daher eignet sich die Menge der Preise nicht als Definitionsmenge einer möglichen Funktion.

Der Funktionalität des Snack-Automaten könnte als Funktion modelliert werden, die die Menge der Tastenkombinationen auf die Menge der Produkte abbildet. Dieser Zusammenhang ist eindeutig! Technisch gesehen ist er auch nötig, damit der Automat bei einem Preis von 50 Cent „weiß“ welches Produkt der Kunde will.

Die Modellierung des Snack-Automaten als Abbildung von Tastenkombination auf Produkt ist jedoch kein besonders zweckmäßiges Modell, denn es ignoriert den Preis, der für viele Nutzer wesentlich sein dürfte. Ein Modell, dass den Preis berücksichtigt, ist die Abbildung der Menge der

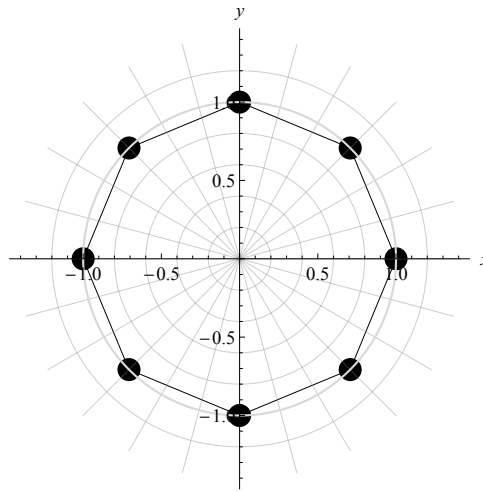


Abbildung 8.4: Eckpunkte eines Achtecks

Preis-Tastenkombination-Paare auf die Menge der Produkte. Die Definitionsmenge ist also

$$\mathbb{D} = \{[50, 12], [60, 18], [50, 21], [150, 23], [70, 26], [50, 29]\}.$$

Eine Möglichkeit diese Funktion auf einem Rechner darzustellen bietet die Relation. Wir tun dies in *setlX* und „geben“ zum Test 50 Cent und die Tastenkombination 21 ein:

```
=> snackautomat:=[[50,12], "Mars"], [[60,18], "Twix"], [[50,21], "Snickers"],
[[150,23], "Ritter Sport"], [[70,26], "Bounty"], [[50,29], "KitKat"]];

=> isMap(snackautomat);
~< Result: true >~
=> snackautomat[[50,21]];
~< Result: "Snickers" >~
=> snackautomat[[50,22]];
~< Result: om >~
```

Bei der „Eingabe“ 50 Cent mit der Tastenkombination 22 liefert dieses Modell des Snack-Automaten dagegen kein Produkt aus der Wertemenge, sondern das *setlX*-Symbol *om* für einen undefinierten Wert. ■

Manche Situationen scheinen sich dagegen nicht als Funktion modellieren zu lassen.

Beispiel 8.16: Abb. 8.4 zeigt die Eckpunkte eines Achtecks im Einheitskreis. Lässt sich die Menge der Eckpunkte als Funktion modellieren und warum (nicht)?



Repräsentiert man jeden Eckpunkt als kartesisches $[x, y]$ -Koordinatenpaar, erhält man für die Menge der Eckpunkte

$$E_8 = \left\{ [-1, 0], \left[-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}} \right], \left[-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right], [0, -1], [0, 1], \left[\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}} \right], \left[\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right], [1, 0] \right\}.$$

Vielleicht haben Sie geantwortet, dass diese Menge nicht als Funktion beschreibbar ist, weil es für einen x -Wert (z. B. $x = 0$) mehrere y -Werte gibt ($y = -1$ und $y = 1$). Dieses Argument ist durchaus richtig, übersieht aber, dass das Achteck und nicht *eine spezielle Repräsentation* des Achtecks durch die Menge E_8 als Funktion beschrieben werden soll. Dies ist hier durchaus möglich, wenn man eine andere Repräsentation wählt, z. B.

$$\tilde{E}_8 = \left\{ [0, 1], \left[\frac{\pi}{4}, 1\right], \left[\frac{\pi}{2}, 1\right], \left[\frac{3\pi}{4}, 1\right], [\pi, 1], \left[\frac{5\pi}{4}, 1\right], \left[\frac{3\pi}{2}, 1\right], \left[\frac{7\pi}{4}, 1\right] \right\}.$$

Mit $\tilde{E}_8$ werden die Eckpunkte als Paare³ repräsentiert, deren ersten Komponente der Winkel ist, auf dem der Punkt vom Ursprung aus gesehen gegen die x -Achse liegt. Die zweite Komponente ist der Abstand des Eckpunktes vom Ursprung. Mit der Definitionsmenge (die Menge der Winkel) $\mathbb{D} = \{k \cdot \pi/4 : k \in \{0, \dots, 7\}\}$ kann die Menge der Eckpunkte nun als Funktion beschrieben werden, die den Winkel des Eckpunktes eindeutig auf den konstanten Wert 1 abbildet. $\mathbb{W} = \{1\}$ oder auch $\mathbb{W} = \mathbb{R}$ eignen sich als Wertemenge. ■

Mitunter ist also etwas Kreativität erforderlich, um eine Perspektive zu finden, aus der betrachtet sich eine Situation oder ein Sachverhalt als Funktion beschreiben lässt. In der Informatik ist es eine durchaus gängige Praxis, Situationen, die sich wegen fehlender Eindeutigkeit (scheinbar) nicht als Funktionen beschreiben lassen, dadurch eindeutig zu machen, dass als Elemente der Wertemenge Mengen gewählt werden.

Beispiel 8.17: Wir wenden den „Trick“ als Wertemenge eine Menge von Mengen zu wählen auf die Eckpunkte des Achtecks an.

Bei der kartesischen Beschreibung der Eckpunkte durch die Menge E_8 liegt keine Eindeutigkeit vor, weil bspw. 0 sowohl mit -1 als auch mit 1 in Beziehung gesetzt werden. 0 kann nicht sowohl auf -1 als auch auf 1 abgebildet werden. Aber 0 kann auf die Menge $\{-1, 1\}$ abgebildet werden. Diese Sichtweise führt auf eine Funktionenbeschreibung mit

$$\begin{aligned} \mathbb{D} &= \left\{ -1, -\frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}}, 1 \right\} \\ \mathbb{W} &= \left\{ \{0\}, \{-1, 1\}, \left\{ -\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right\} \right\} \end{aligned}$$

und den Zuordnungen

$$-1 \mapsto \{0\}, \quad -\frac{1}{\sqrt{2}} \mapsto \left\{ -\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right\}, \quad 0 \mapsto \{-1, 1\}, \quad \frac{1}{\sqrt{2}} \mapsto \left\{ -\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right\}, \quad 1 \mapsto \{0\}.$$

Jedes Element der Definitionsmenge wird also eindeutig auf ein Element der Wertemenge abgebildet. Ein Element der Wertemenge ist jeweils die Menge aller Werte, mit denen ein Element der Definitionsmenge in Beziehung stehen kann. ■

Das letzte Beispiel zeigt, dass eine Modellierung als Funktion prinzipiell immer möglich ist. Eine solche Modellierung garantiert jedoch nicht, dass das resultierende Modell zweckmäßig oder brauchbar ist.

Beispiel 8.18: Lässt sich der folgende Programmcode als Funktion modellieren, wenn x nur reelle Werte annehmen kann?

```
if (x<0) {return -x;} else {return x;}
```

Wenn dies möglich ist, würde man diese Funktion - nennen wir sie *abs* - auf Papier so schreiben:

$$abs(x) = \begin{cases} -x & , \text{ wenn } x < 0 \\ x & , \text{ wenn } x \geq 0 \end{cases} \quad (8.6)$$

Ist der Code aus Ihrer Sicht als Funktion modellierbar? Welcher der folgenden Positionen stimmen Sie zu?

³Das sind so genannte *Polarkoordinaten*.

Leo: Der Code kann nicht als Funktion modelliert werden. Jedem x werden ja zwei Rückgabewerte zugewiesen, einmal $-x$ und einmal x . Damit ist die Bedingung der Eindeutigkeit nicht erfüllt.

Eduard: Ich sehe kein Problem, den Code als Funktion zu modellieren. Jedes x aus der Definitionsmenge wird ja *entweder* $-x$ *oder* x zugeordnet, aber nicht beiden Rückgabewerten gleichzeitig. Die Eindeutigkeit ist also gegeben.

Nicola: Ich meine auch, dass die Eindeutigkeit gegeben ist, allerdings benötigt man zwei Funktionen, um den Code zu modellieren, weil es ja zwei unterschiedlich berechnete Rückgabewerte gibt. Wir brauchen eine Funktion für den Fall $x < 0$ und eine für $x \geq 0$.

Halten Sie hier Ihre Gedanken fest: 

Wählen wir als Definitionsmenge $\mathbb{D} = \mathbb{R}$, dann wird durch den Programmcode bzw. durch (8.6) jedem $x \in \mathbb{D}$ eindeutig ein Element $abs(x) \in \mathbb{W} = \mathbb{R}$ zugewiesen. Mit $\mathbb{D} = \mathbb{R}$ ist (8.6) nichts anderes als die Definition der Betragsfunktion für reelle Zahlen. ■

Zuletzt schauen wir uns Definitionen an. Bisher haben wir Definitionen als Tautologien modelliert gemäß dem Muster

Ein $\langle \text{Objekt} \rangle$ heißt $\dots \Leftrightarrow \langle \text{Objekt} \rangle$ hat die folgenden Eigenschaften: $\dots$ (8.7)

Außerdem haben wir Definitionen als Aussageformen mit der Variablen $\langle \text{Objekt} \rangle$ modelliert. Zusammengefasst können wir Definitionen also als tautologische Aussageformen betrachten, die unabhängig von der Belegung ihrer Variablen immer den Wahrheitswert 1 haben.

Beispiel 8.19: Eine mögliche Formulierung der Definition von „gerade“ ist

Ein ganze Zahl z heißt *gerade*, wenn z sich ohne Rest durch 2 teilen lässt.

Eine mögliche Formalisierung ist

$$z \text{ heißt gerade} \Leftrightarrow z \in \mathbb{Z} \wedge z \bmod 2 = 0.$$

Der Wahrheitswert ist unabhängig von der Belegung von z immer 1, bspw. für $z = 42$, $z = -5$ oder auch $z = \pi$. ■

Die rechte Seite in (8.7) kann ebenfalls als Aussageform modelliert werden. Ihr Wahrheitswert gibt an, ob das betrachtete Objekt die definierenden Eigenschaften erfüllt, also mit dem definierten Begriff bezeichnet werden darf. Diese Art von Aussageform lässt sich als Funktion modellieren, die jedem Element aus der Menge aller Objekte ein Element aus $\{0, 1\}$ zuordnet und 1 genau dann, wenn das betrachtete Objekt alle definierenden Eigenschaften erfüllt. Die *setlX*-Funktionalität *isMap* leistet dies bspw. für Relationen, die den definierenden Eigenschaften einer Funktion genügen.

Beispiel 8.20: Die rechte Seite der Definition von „gerade“ aus dem vorangegangenen Beispiel kann als Funktion *isEven*: $\mathbb{R} \rightarrow \{0, 1\}$ modelliert werden. Der Körper dieser Funktion kann in *setlX* so implementiert werden:

```
=> isEven:= z|-> isInteger(z) && z%2==0;
~< Result: z |-> isInteger(z) && z % 2 == 0 >~
=> isEven(42);
~< Result: true >~
=> isEven(-5);
~< Result: false >~
=> isEven(3.14);
~< Result: false >~
=> isEven("zwei");
~< Result: false >~
```

Diese *procedure* nutzt *isInteger* zur (näherungsweisen) Realisierung von $\in \mathbb{Z}$. ■

8.8 Lernziele

Studierende	Kennen	Können	Verstehen
fachlich	☐ kennen Terminologie zu Funktionen und die Bedeutung der Begriffe ☐ benennen Gemeinsamkeiten und Unterschiede von Funktionen und Relationen ☐ benennen unterschiedliche Perspektiven auf Funktionen ☐ benennen unterschiedliche Repräsentationsformen für Funktionen ☐ kennen Synonyme zum Begriff Funktion	☐ beschreiben Funktion mittels Signatur ☐ bestimmen Definitions-, Werte- und Bildmenge einer Funktion ☐ formulieren die Definition von Funktion formallogisch ☐ bestimmen Bildmengen	☐ nehmen unterschiedliche Perspektiven auf Funktionen ein und wechseln zwischen diesen ☐ entscheiden, ob ein Sachverhalt als Funktion modelliert werden kann ☐ modellieren einen Sachverhalt als Funktion ☐ modellieren Definitionen als Funktionen ☐ identifizieren Argumente, unabhängige und abhängige Variablen von Funktionen
methodisch		☐ repräsentieren Funktionen als Computerprogramme	

Hinzu kommen die themenübergreifenden Lernziele aus den Kategorien „sozial“ und „persönlich“ aus Abschnitt 1.7 sowie die Lernziele aus der Kategorie „methodisch“ aller vorangegangener Kapitel. Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben.

8.9 Übungsaufgaben

- Schreiben Sie die Signaturen der folgenden Funktionen:
 - Addition, Subtraktion, Multiplikation und Division reeller Zahlen
 - trigonometrische Funktionen ($\sin$, $\cos$, $\tan$, $\cot$)
 - `isMap`
 - `isInteger`
- Kann die leere Menge als Funktion modelliert werden? Welche Perspektive auf das Funktionskonzept erlaubt es diese Frage besonders leicht zu beantworten?
- Die Funktion `isFunction` aus Abschnitt 8.5 ist selbst wiederum eine Funktion. Daher müsste mittels `isFunction` festgestellt werden können, dass `isFunction` eine Funktion ist. Ist dies tatsächlich möglich, und wenn ja, wie? Wenn nein, warum ist dies nicht möglich, und wäre es nach einer Modifikation des Codes möglich?
- Erläutern Sie den Unterschied zwischen f und $f(x)$.

Kapitel 9

Funktionen als Objekte

The keynote of Western culture is the function concept, a notion not even remotely hinted at by any earlier culture. And the function concept is anything but an extension or elaboration of previous number concepts – it is rather a complete emancipation from such notions.
(W.L. Schaaf)

9.1 Aufwärmübungen

1. Die folgenden Tabellen können jeweils als Funktionen $x \mapsto y$ modelliert werden. Können sie auch als Funktionen $y \mapsto x$ modelliert werden?

x	y
-2	4
-1	1
0	0
1	1
2	4

x	y
0	0
1	1
4	2
9	3
16	4

2. Welche Teilmengen-Beziehungen können zwischen Wertemenge $\mathbb{W}$ und Bildmenge $f(\mathbb{D})$ einer Funktion f mit Definitionsmenge $\mathbb{D}$ bestehen?

- (a) $\mathbb{W} \subseteq f(\mathbb{D})$
- (b) $f(\mathbb{D}) \subseteq \mathbb{W}$

3. Geben Sie den folgenden Code in *setlX* ein

```
sqr := closure(n){ return n**2; };
```

und berechnen Sie:

```
{[n,sqr(n)]: n in {-2..2}}
```

Was berechnet `sqr`?

4. Geben Sie den folgenden Code in *setlX* ein:

```
potenz := closure(n){  
    return closure(x){  
        return x**n;  
    };  
};
```

- (a) Was bedeutet die zweite Zeile dieser `closure`? Was bedeutet die dritte Zeile?
- (b) Was wird der folgende Code berechnen und was wird jeweils das Ergebnis sein? Schreiben Sie Ihre Vorhersagen auf und führen Sie dann den Code aus.
 - i. `potenz(5);`
 - ii. `potenz(5)(2);`
 - iii. `potenz(2)(5);`

5. Leitfragen:

- (a) Wie stellt man fest, ob eine Funktion umkehrbar ist?
- (b) Welche Funktionen haben Funktionen als Elemente ihrer Definitionsmenge?
- (c) Warum wird das Konzept der partiellen Funktion benötigt?

9.2 Umkehrfunktion

Bisher haben wir den Transformationsprozess immer „von Eingabe Richtung Ausgabe“ gedacht. Stellen wir uns eine Funktion als Funktionenmaschine wie in Abb. 9.1 vor, dann haben wir Fragen der folgenden Art gestellt: In welche Ausgabe transformiert eine vorliegende Funktion eine gegebene Eingabe?

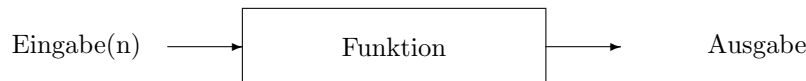


Abbildung 9.1: Funktionenmaschine

Wenn wir uns vorstellen, dass die Ausgabe das angestrebte Ziel ist, dann ist die *umgekehrte* Frage die interessantere: Was müssen wir tun, um unser Ziel zu erreichen? Was benötigen wir in Abb. 9.1 als Eingabe, damit diese in eine gewünschte Ausgabe transformiert wird? Am Beispiel der Kuchenmaschine aus Kapitel 1: Was muss man in die Kuchenmaschine hineinstecken, damit die Ausgabe Schwarzwälder Kirschtorte ist? Was muss man hineinstecken, damit die Kuchenmaschine Butterkuchen produziert?

Hier geht es also darum den Transformationsprozess einer Funktion umgekehrt zu denken: von der Ausgabe zur Eingabe. Diese Art zu denken sollte Ihnen bereits aus Abschnitt 7.7 im Zusammenhang mit Relationen bekannt sein. Dort haben wir Relationen „rückwärts gedacht“.

Beispiel 9.1: Welche Eingaben transformiert die Quadrat-Funktion $\text{sqr} : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto \text{sqr}(x) = x^2$ in die Ausgabe 4? Dies sind die Eingaben -2 und 2.

Allgemein: Welche Eingaben transformiert die Quadrat-Funktion in die Ausgabe y ? Dies sind die Eingaben $-\sqrt{y}$ und $\sqrt{y}$. ■

Beispiel 9.2: Bei der Wurzelfunktion $\sqrt{\cdot} : \mathbb{R}_0^+ \rightarrow \mathbb{R}, x \mapsto \sqrt{x}$ gibt es für die Fragestellung „Welche Eingabe führt zu einer gegebenen Ausgabe?“ zwei Arten von Antworten: Entweder ist die Antwort ein einziges Element von $\mathbb{D}$ oder sie lautet „keine“. Zur Ausgabe 2 führt die Eingabe 4. Zur Ausgabe -2 führt keine Eingabe. ■

Beispiel 9.3: Die Modulo-Operation kann als Funktion $\text{mod} : \mathbb{Z} \times \mathbb{N} \rightarrow \mathbb{N}_0, [x, m] \mapsto x \bmod m$ modelliert werden. Sie bildet jedes Tupel $[x, m] \in \mathbb{Z} \times \mathbb{N}$ auf den Rest $x \bmod m$ von x nach Division durch m ab.

Die Fragestellung „Was wird auf 1 abgebildet“ hat viele Antwortmöglichkeiten. $[5, 4], [9, 4], [7, 6]$ sind einige davon. ■

Beispiel 9.4: Die Relation „hat Matrikelnummer“ kann als Abbildung der Menge der vergebenen Matrikelnummern auf die Menge der immatrikulierten Studierenden modelliert werden. Sie „transformiert“ Matrikelnummern in Studierendennamen. Dieser Transformationsprozess erlaubt die Fragestellung „Wer hat die Matrikelnummer ...?“ zu beantworten. Die umgekehrte Fragestellung ist „Welche Matrikelnummern gehören zu einem gegebenen Studierendennamen?“ Dies können natürlich mehrere sein, weil die Matrikelnummer eindeutig auf eine Person verweist, aber nicht der Name der Person eindeutig für eine Person steht. ■

Die meisten wissenschaftlichen Probleme fragen „nach der Eingabe für eine gegebene Ausgabe“. Fragestellungen dieser Art werden als *inverse Probleme* bezeichnet.

Modellieren wir z. B. das Weltklima als Funktion, die eine komplexe Eingabestruktur abbildet, die u. a. menschliche CO₂-Produktion und das in der Arktis und in Gletschern als Eis gebundene Wasservolumen umfasst. Abgebildet wird auf die klimatischen Gegebenheiten in 50 Jahren. Die Ausgabe dieser Funktion ist ebenfalls eine komplexe Struktur. Sie umfasst u. a. die zu erwartende mittlere Temperaturdifferenz zu heute.

Die wissenschaftlich interessante Frage ist weniger „Was ist die zu erwartende mittlere Temperaturdifferenz für eine gegebene Umweltsituation als Eingabe?“, sondern die inverse Fragestellung „Wie muss die Umweltsituation aussehen, damit die zu erwartende Temperaturerhöhung auf 1 Grad beschränkt bleibt?“

Überlegen Sie sich selbst einige Situationen aus Ihrem Alltag, die als Funktionen modelliert werden können und bei denen die damit verbundenen Fragestellungen „von der Ausgabe zu den Eingaben“ gehen.



Sobald Sie die Transformationsprozesse von Funktionen „umgekehrt denken“ können, bringen wir wieder den Modellierungsaspekt ins Spiel: Sind die umgekehrten Transformationsprozesse selbst wieder als Funktion modellierbar?

Bei der Quadratfunktion aus dem obigen Beispiel ist dies nicht der Fall, denn der umgekehrte Transformationsprozess setzt die Eingabe 4 in diesem Umkehrprozess mit den Zahlen -2 und 2 in Verbindung. Die für eine Funktion definierende Eigenschaft der Eindeutigkeit ist also nicht erfüllt. Mit Ausnahme der Wurzelfunktion ist dies bei allen obigen Beispielen der Grund, weshalb der jeweilige umgekehrte Transformationsprozess nicht als Funktion modelliert werden kann.

Bei der Wurzelfunktion ist der umgekehrte Transformationsprozess zwar eindeutig (z. B. verknüpft diese Umkehrung die 2 mit der 4 und die 3 mit der 9), aber die zweite definierende Eigenschaft einer Funktion ist hier nicht erfüllt: *Jede* mögliche Eingabe in den Umkehrprozess muss durch diesen transformiert werden. Die Wurzelfunktion bildet aber z. B. nicht auf -1 ab, d. h. es gibt nichts, worauf der Umkehrprozess abbildet.

Die Situation sähe anders aus, wenn wir bei der Beschreibung der Wurzelfunktion als Wertemenge die Bildmenge verwendet hätten:

Beispiel 9.5: Bei der Wurzelfunktion $\sqrt{\cdot} : \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+$, $x \mapsto \sqrt{x}$ gibt es für die Fragestellung „Welche Eingabe führt zu einer gegebenen Ausgabe?“ immer eine eindeutige Antwort. Zu jeder Ausgabe $\sqrt{x} \in \mathbb{R}_0^+$ (die Wertemenge und zugleich Bildmenge von $\sqrt{\cdot}$) gibt es nur eine einzig mögliche Eingabe $x \in \mathbb{R}_0^+$ (die Definitionsmenge von $\sqrt{\cdot}$). ■

Schauen wir uns den Sachverhalt formal und abstrakt an. Der Umkehrprozess zu einer Funktion

$$f : \mathbb{D}_f \rightarrow \mathbb{W}_f, x \mapsto f(x)$$

kann selbst wiederum als Funktion beschrieben werden, wenn erstens dieser Umkehrprozess eindeutig ist und zweitens jedes $f(x) \in \mathbb{W}_f$ auf ein $x \in \mathbb{D}_f$ zurückgeführt werden kann. Das zweite Kriterium ist gleichbedeutend damit, dass $\mathbb{W}_f = f(\mathbb{D}_f)$ ist, die Wertemenge also gleich der Bildmenge ist. Wir können beide Kriterien auch in einem zusammenfassen: Der Umkehrprozess zu einer Funktion $f : \mathbb{D}_f \rightarrow \mathbb{W}_f$, $x \mapsto f(x)$ kann selbst wiederum als Funktion beschrieben werden, wenn es zu jedem $f(x) \in \mathbb{W}_f$ genau ein $x \in \mathbb{D}$ gibt. Der Umkehrprozess kann dann also als Funktion beschrieben

werden, dessen Definitionsmenge die Wertemenge der Ausgangsfunktion ist und dessen Wertemenge die Definitionsmenge der Ausgangsfunktion ist.

Nennen wir die Funktion, die den Umkehrprozess zu f beschreibt, f^{-1} , erhalten wir folgende formale Beschreibung:

$$f^{-1} : \mathbb{W}_f \rightarrow \mathbb{D}_f, f(x) \mapsto x \quad (9.1)$$

Wir müssen die Argumente von f^{-1} nicht $f(x)$ nennen und können genauso gut schreiben

$$f^{-1} : \mathbb{W}_f \rightarrow \mathbb{D}_f, z \mapsto f^{-1}(z)$$

oder

$$f^{-1} : \mathbb{W}_f \rightarrow \mathbb{D}_f, x \mapsto f^{-1}(x).$$

Im Vergleich zu (9.1) machen diese beiden formalen Beschreibungen jedoch kaum mehr den Bezug zur Ausgangsabbildung f deutlich. Wenn wir noch einen Schritt weitergehen und ausdrücken, dass f^{-1} *seine* Definitionsmenge auf *seiner* Wertemenge abbildet

$$f^{-1} : \mathbb{D}_{f^{-1}} \rightarrow \mathbb{W}_{f^{-1}}, x \mapsto f^{-1}(x),$$

dann ist der Umkehrcharakter nur noch aus dem Funktionsnamen f^{-1} ersichtlich.

Verwechseln Sie übrigens die hochgestellte -1 in f^{-1} nicht mit dem Kehrwert, wie er z. B. mit $x^{-1} = 1/x$ ausgedrückt wird. Der Kehrwert x^{-1} notiert die Zahl, die mit x multipliziert 1 ergibt: $x^{-1} \cdot x = 1$. Das Symbol f^{-1} notiert die Umkehrfunktion zu einer Funktion f .

Die „doppelte, kontextabhängige Bedeutung“ der hochgestellten -1 ist dennoch sinnvoll, wie wir bald sehen werden. Denn es gibt eine Verknüpfung von Funktionen (nehmen wir $\circ$ als das entsprechende Symbol), so dass $f^{-1} \circ f = \text{„Einsfunktion“}$. Was diese „Einsfunktion“ ist, werden wir dann auch noch klären. Der Punkt hier ist eine strukturelle Verwandtschaft von $x^{-1} \cdot x = 1$ und $f^{-1} \circ f = \text{„Einsfunktion“}$, die auch für die Bedeutung der hochgestellten -1 gilt.

Ein weiterer Aspekt dieser strukturellen Verwandtschaft besteht darin, dass es nicht für jede mögliche Zahl x einen Kehrwert gibt, ebenso wenig wie es für jede Funktion f eine Umkehrfunktion gibt.

9.3 Besondere Eigenschaften

9.3.1 Surjektivität

In Abschnitt 8.4 hatten wir den Unterschied zwischen Bild- und Wertemenge einer Funktion thematisiert. Im Allgemeinen sind diese beide Mengen nicht gleich, in Spezialfällen können sie es aber sein. Anders formuliert: Nicht jedes Element der Wertemenge muss ein Funktionswert der Funktion sein (aber sicher Element der Bildmenge). Bei einigen Funktionen ist dies jedoch der Fall:

Beispiel 9.6: Bei der Funktion $g : \mathbb{Z} \rightarrow \{0, 1\}$, $x \mapsto x \bmod 2$ gibt es für jedes $y \in \mathbb{W} = \{0, 1\}$ (mindestens) ein $x \in \mathbb{D} = \mathbb{Z}$, so dass $y = x \bmod 2$. Auf 0 werden z. B. 42 und -2 abgebildet, auf 1 werden -5 und 13 abgebildet. Die Bildmenge $g(\mathbb{D}) = \{0, 1\}$ ist bei der Funktion g gleich der Wertemenge. ■

Beispiel 9.7: Auch bei der *setlX*-Funktion **char**: $\{0..127\} \rightarrow \text{ASCII}$ gibt es für jedes $c \in \mathbb{W} = \text{ASCII}$ ein $n \in \mathbb{D} = \{0..127\}$, so dass $c = \text{char}(n)$. So ist z. B. das Symbol **a** das Bild von 97 und **b** das Bild von 98. ■

Beispiel 9.8: Dagegen liegt bei der Funktion $\text{sqr} : \mathbb{R} \rightarrow \mathbb{R}$, $x \mapsto x^2$ nicht die besondere Eigenschaft vor, dass Bild- und Wertemenge identisch sind. Denn es gibt nicht für jedes $y \in \mathbb{W} = \mathbb{R}$ ein $x \in \mathbb{D} = \mathbb{R}$, so dass $y = x^2$. Diese Problematik liegt bspw. für $y = -1$ vor. ■

Funktionen, bei denen Bild- und Wertemenge identisch sind, heißen *surjektiv*. Wir formulieren die Definition hier so:

Eine Funktion $f : \mathbb{D} \rightarrow \mathbb{W}$ heißt *surjektiv*, wenn jedes Element von $\mathbb{W}$ das Bild eines Elements von $\mathbb{D}$ ist.

Formallogisch formuliert bedeutet dies

$$f : \mathbb{D} \rightarrow \mathbb{W} \text{ ist surjektiv} \Leftrightarrow \forall (y \in \mathbb{W} \mid \exists (x \in \mathbb{D} \mid y = f(x))).$$

Abb. 9.2 zeigt typische Fälle surjektiver Funktionen in der abstrakten Form von Venn-Diagrammen.

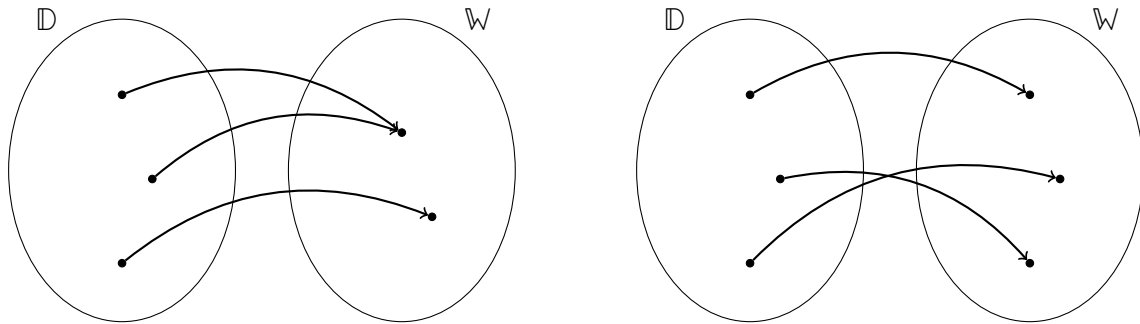


Abbildung 9.2: Bei surjektiven Funktionen wird auf jedes Element der Wertemenge abgebildet.

9.3.2 Injektivität

Funktionen müssen jedem Argument eindeutig einen Funktionswert zuordnen. Das bedeutet nicht, dass umgekehrt jeder Funktionswert in einer eindeutigen Beziehung mit nur einem Argument steht. Bei einigen Funktionen ist dies jedoch der Fall.

Beispiel 9.9: Bei der Funktion *cubic* : $\mathbb{R} \rightarrow \mathbb{R}$ mit $x \mapsto x^3$ gibt es für jeden Funktionswert x^3 *genau* ein Argument x , das auf x^3 abgebildet wird. So wird z. B. auf den Funktionswert -8 nur das Argument -2 abgebildet. ■

Beispiel 9.10: Auch bei der *setlX*-Funktion *char*: $\{0..127\} \rightarrow \text{ASCII}$ gibt es für jedes $c \in \mathbb{W} = \text{ASCII}$ genau ein $n \in \mathbb{D} = \{0..127\}$, so dass $c = \text{char}(n)$. So ist z. B. nur das Argument 97 auf das Symbol *a* abgebildet. ■

Beispiel 9.11: Bei der Quadrat-Funktion *sqr* : $\mathbb{R} \rightarrow \mathbb{R}$ mit $x \mapsto \text{sqr}(x) = x^2$ gibt es dagegen für den Funktionswert 4 zwei Elemente der Definitionsmenge, die darauf abgebildet werden: -2 und 2. Diese Funktion hat nicht die hier diskutierte Eigenschaft. ■

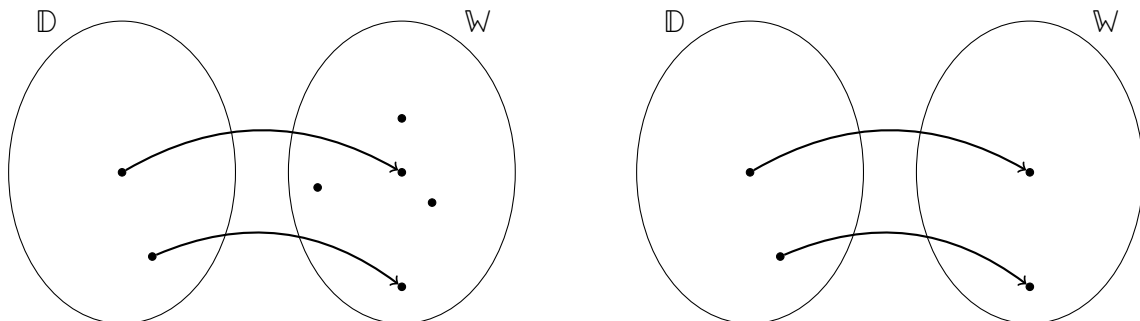


Abbildung 9.3: Bei injektiven Funktionen hat jedes Bild ein eindeutiges Urbild, d. h. verschiedene Elemente der Definitionsmenge werden immer auf verschiedene Elemente der Wertemenge abgebildet.

Abb. 9.3 zeigt typische Fälle solcher Funktionen in der abstrakten Form von Venn-Diagrammen. Wir definieren diese besondere Eigenschaft so:

Eine Funktion $f : \mathbb{D} \rightarrow \mathbb{W}$ heißt *injektiv*, genau dann wenn verschiedene Elemente von $\mathbb{D}$ auf verschiedene Elemente von $\mathbb{W}$ abgebildet werden.

Formulieren Sie diese Definition als formallogischen Ausdruck:

$$\text{✎ } f : \mathbb{D} \rightarrow \mathbb{W} \text{ ist injektiv} \Leftrightarrow$$

9.3.3 Bijektivität

Eine dritte wichtige Eigenschaft von Funktionen ist die Bijektivität:

Eine Funktion $f : \mathbb{D} \rightarrow \mathbb{W}$ heißt *bijektiv* oder *umkehrbar*, genau dann wenn f sowohl injektiv als auch surjektiv ist.

Beispiel 9.12: Die *setlX*-Funktion $\text{char} : \{0..127\} \rightarrow \text{ASCII}$ ist sowohl surjektiv als auch injektiv, also bijektiv. ■

Bijektive Funktionen sind also sowohl besondere injektive als auch besondere surjektive Funktionen. Die rechten Seiten der Abbildungen 9.2 und 9.3 zeigen jeweils bijektive Abbildungen.

Bijektive Funktionen werden auch als *eindeutig* oder *eins-zu-eins-Abbildungen* bezeichnet. Beide Bezeichnungen bringen zum Ausdruck, dass die Elemente von Definitions- und Wertemenge in einer besonderen Beziehung stehen, wie sie durch die rechten Seiten der Abbildungen 9.2 und 9.3 treffend visualisiert wird. Die dargestellten Situationen können sowohl in Pfeilrichtung als auch in umgekehrter Richtung als Funktionen modelliert werden. Die zugrunde liegenden Funktionen sind also im Sinne von Abschnitt 9.2 umkehrbar.

Ist eine Funktion $f : \mathbb{D} \rightarrow \mathbb{W}$ bijektiv, dann wird die Funktion, die jedem $y \in \mathbb{W}$ genau ein $x \in \mathbb{D}$ zuordnet, als *Umkehrfunktion* von f bezeichnet und mit f^{-1} symbolisiert.

Eine alternative Bezeichnung für Umkehrfunktion ist *inverse Funktion*.

9.3.4 Bilden der Umkehrfunktion

Gedanklich ist die Umkehrfunktion recht einfach zu bilden. Es geht ja nur darum, die Ausgaben der Ausgangsfunktion auf deren Eingaben abzubilden. Sind Funktionen als Tabellen repräsentiert, bedeutet das Bilden der Umkehrfunktion (natürlich nur, wenn diese existiert): Die Spalte, die der abhängigen Variable der Ausgangsfunktion entspricht wird zur Spalte, die der unabhängigen Variable der Umkehrfunktion entspricht und entsprechend für die anderen beiden Spalten. Sind Funktionen als (zweidimensionale) Graphen repräsentiert, wird die Achse der abhängigen Variablen der Ausgangsfunktion zur Achse der unabhängigen Variable der Umkehrfunktion und entsprechend für die anderen beiden Achsen.

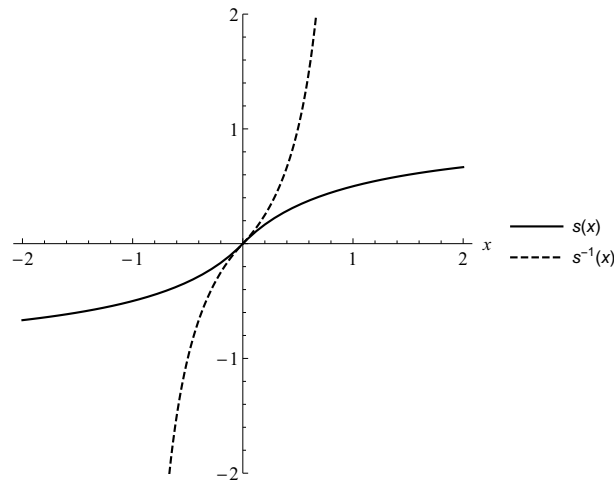
Nicht ganz so einfach gestaltet sich das Bilden der Umkehrfunktion, wenn die Ausgangsfunktion symbolisch repräsentiert ist, also als Ausdruck oder Programm. In solchen Fällen kann es mitunter aufwändig sein, einen Ausdruck für die Umkehrfunktion zu bestimmen.

Beispiel 9.13: Um die Umkehrfunktion von $s : \mathbb{R} \rightarrow \mathbb{R}$ mit

$$s(x) = \frac{x}{1 + |x|} \tag{9.2}$$

zu bestimmen, prüfen wir uns zunächst, dass s tatsächlich umkehrbar ist. Wir tun dies hier nicht formal (dazu wären Hilfsmittel der Analysis notwendig), sondern werfen in Abb. 9.4 einen Blick auf den Graphen der Funktion. Zumindest im dort dargestellten Bereich ist s eineindeutig, denn jedem Wert der unabhängigen Variable entspricht genau ein Wert der abhängigen Variable.

Als nächstes bestimmen wir die Bildmenge von s , denn diese wird die Definitionsmenge der Umkehrfunktion s^{-1} sein. Auch hierzu wären Hilfsmittel der Analysis nötig. Abb. 9.4 gibt die

Abbildung 9.4: Graphische Darstellung der Funktion s aus (9.2) und ihrer Umkehrfunktion s^{-1} .

Idee, dass dies die Menge der reellen Zahlen von -1 bis 1 ist, was tatsächlich der Fall ist, also $\mathbb{D}_{s^{-1}} = s(\mathbb{R}) =]-1, 1[$. Die Signatur der Umkehrfunktion ist also $s^{-1} :]-1, 1[\rightarrow \mathbb{R}$.

Nun können wir damit beginnen einen formalen Ausdruck für die Umkehrfunktion s^{-1} zu bestimmen. Um nicht immer $s(x)$ für den Rückgabewert von s schreiben zu müssen, geben wir der abhängigen Variablen von s zunächst ein kurzes Symbol als Namen, z. B. y :

$$y = \frac{x}{1 + |x|} \quad (9.3)$$

Die Umkehrfunktion bildet dann y auf x ab. Um einen formalen Ausdruck dieser Abbildung zu bekommen, lösen wir (9.3) nach der nun abhängigen Variablen x auf. Dies wird uns nicht gelingen, solange wir nicht die Betragstriche von $|x|$ loswerden. Wir erreichen dies, indem wir die Definition des Betrags

$$|x| = \begin{cases} x & , x \geq 0 \\ -x & , x < 0 \end{cases}$$

anwenden. Damit wird (9.3) zu

$$y = \begin{cases} \frac{x}{1+x} & , x \geq 0 \\ \frac{x}{1-x} & , x < 0 \end{cases}$$

Nun kann in beiden Fällen nach x aufgelöst werden. Sie sollten dies selbst tun und sich dabei überlegen, welche Werte y in den einzelnen Fällen annimmt:



Fassen wir wieder beide Fälle zu einem Ausdruck zusammen, erhalten wir

$$x = \begin{cases} \frac{y}{1-y} & , y \geq 0 \\ \frac{y}{1+y} & , y < 0 \end{cases} . \quad (9.4)$$

Das können wir noch kompakter schreiben als

$$x = \frac{y}{1 - |y|}. \quad (9.5)$$

(Wenn Sie nicht sehen, warum dies möglich ist, leiten Sie ausgehend von (9.5) unter Verwendung der Definition des Betrags den Ausdruck (9.4) ab.) Damit erhalten wir als Beschreibung der Umkehrfunktion

$$s^{-1} :] - 1, 1[\rightarrow \mathbb{R}, y \mapsto \frac{y}{1 - |y|}$$

oder, wenn Sie x als Symbol für die unabhängige Variable bevorzugen,

$$s^{-1} :] - 1, 1[\rightarrow \mathbb{R}, x \mapsto \frac{x}{1 - |x|}.$$

Abb. 9.4 zeigt den Graphen von s^{-1} zusammen mit dem von s . ■

9.4 Operationen auf Funktionen

Bisher haben wir Funktionen als Transformationsprozesse verstanden, die Objekte in Objekte transformieren. Nun betrachten wir Funktionen selbst als Objekte. Das erlaubt uns eine oder mehrere Funktionen zu transformieren, also gleichsam Funktionen auf Funktionen anzuwenden.

Wir können bspw. zwei Funktionen vergleichen. Das ist ein Prozess, der zwei Funktionen entgegen nimmt und in einen Wahrheitswert transformiert. Dieser ist genau dann $\mathbb{1}$, wenn die beiden Funktionen gleich sind.

Wir können auch Funktionen in Funktionen transformieren. Bei der Umkehrfunktion haben wir das schon getan. Die Funktion $^{-1}$ transformiert Funktionen in ihre Umkehrfunktion. Die Definitionsmenge von $^{-1}$ ist die Menge aller umkehrbaren Funktionen. Als Wertemenge ist ebenfalls die Menge aller umkehrbaren Funktionen geeignet.

Wir können auch aus zwei Funktionen eine neue Funktion machen, indem wir die beiden Ausgangsfunktionen bspw. addieren, subtrahieren oder multiplizieren. Dies schauen wir uns zuerst an.

9.4.1 Addition von Funktionen

Betrachten wir zum Einstieg die Addition reeller Zahlen. Diese hat die Signatur

$$+ : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}, [x, y] \mapsto x + y.$$

Die Addition macht aus zwei reellen Zahlen x und y eine neue reelle Zahl $x + y$. Sie transformiert das Paar reeller Zahlen $[x, y]$ in eine neue reelle Zahl $x + y$. Nochmals zur Sicherheit: Diese Beschreibung sagt, *was* die Addition tut, aber nicht *wie* sie es tut. Sie sagt nichts darüber, wie die Summe $x + y$ algorithmisch berechnet wird.

Bezeichnen wir die Menge aller Funktionen mit $\mathbb{F}$, können wir die Signatur der Addition von Funktionen entsprechend aufschreiben. Als Symbol verwenden wir dabei zunächst $\oplus$:

$$\oplus : \mathbb{F} \times \mathbb{F} \rightarrow \mathbb{F}, [f, g] \mapsto f \oplus g$$

Im folgenden beschränken wir uns der Einfachheit halber auf Funktionen, die reelle Zahlen auf reelle Zahlen abbilden. Diese haben die Signatur $\langle \text{Funktionsname} \rangle : \mathbb{R} \rightarrow \mathbb{R}$. Daher ist es üblich, die Menge aller dieser Funktionen als $(\mathbb{R} \rightarrow \mathbb{R})$ zu schreiben, eben all die Funktionen, die reelle Zahlen auf reelle Zahlen abbilden.

Die Signatur der Addition solcher Funktionen sieht nun so aus:

$$\oplus : (\mathbb{R} \rightarrow \mathbb{R}) \times (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow (\mathbb{R} \rightarrow \mathbb{R}), [f, g] \mapsto f \oplus g \quad (9.6)$$

Das sieht kompliziert aus, aber wenn Sie die Signatur ein paar Mal gelesen haben, sollte es Ihnen gelingen $(\mathbb{R} \rightarrow \mathbb{R})$ zu „die Menge aller Funktionen, die reelle Zahlen auf reelle Zahlen abbilden“ zu kapseln.

Zugegeben: Diese „additive“ Transformation zweier Funktionen in eine neue ist langweilig, wenn wir nicht sagen, *wie* die Transformation von Statten geht. Hier ist, was mit der neu gebildeten Funktion $f \oplus g$ gemeint sein soll:

$$f \oplus g : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto (f \oplus g)(x) = f(x) + g(x) \quad (9.7)$$

Die Funktionswerte $(f \oplus g)(x)$ der neu gebildeten Funktion $f \oplus g$ sind also nichts anderes als die Summe $f(x) + g(x)$ der Funktionswerte $f(x)$ und $g(x)$ der Ausgangsfunktionen f und g .

Halten Sie all das für eine komplizierte Darstellung einer Trivialität? Dann haben Sie möglicherweise den springenden Punkt völlig übersehen! Wir addieren hier Funktionen, nicht Funktionswerte! Am Ende berechnet zwar auch die Summenfunktion die Summe der Funktionswerte, aber die Denk- und Rechenprozesse sind vollkommen andere.

Vielleicht hilft es Ihnen den neuen Denkprozess zu verstehen, wenn Sie alles in **Java**-ähnlichem Code aufschreiben. Mit der Funktion $f \oplus g$ ist *nicht* das Folgende gemeint:

```
float fplusg (float x){
    return f(x)+g(x);
}
```

In diesem Code sind f und g nicht Variable. Die beiden Symbole stehen nicht für beliebige Funktionen. Die Symbole stehen vielmehr für Konstanten. Der Code ist nur ausführbar für zwei vorher deklarierte Funktionen mit den konkreten Namen f und g . Der Code erlaubt nur die Summenfunktion für zwei ganz bestimmte Funktionen zu bilden.

Beispiel 9.14: Wir implementieren diesen Code in *setlX*. Der Unterschied zum obigen Code beläuft sich im Wesentlichen darauf, dass *setlX* als „ungetypte Programmiersprache“ auf die Angabe von Definitions- und Wertemenge verzichtet:

```
=> fplusg:=procedure(x){return f(x)+g(x);};
~< Result: procedure(x) { return f(x) + g(x); } >~

=> fplusg(1);
Error in "fplusg(1)":
Error in "f(x) + g(x)":
Error in "g(x)":
Error in "f(x)":
Left hand side is undefined (om).

Replay:
2.3: f(x) FAILED
2.2: x <~> 1
2.1: f <~> om
1.3: fplusg(1) FAILED
1.2: 1 <~> 1
1.1: fplusg <~> procedure(x) { return f(x) + g(x); }
```

Die neu definierte Funktion **fplusg** kann nicht an der Stelle 1 ausgewertet werden, weil die Bedeutung von **f** undefiniert ist (siehe Replay Zeile 2.1). ■

Was (9.6, 9.7) wirklich meinen, könnte in **Java**-artigem Pseudocode so geschrieben werden:¹

```
function plus (function f, g){
    return float newfunction (float x){return f(x)+g(x);};
}
```

¹Da **Java** keine funktionale Programmiersprache ist, ist eine direkte Implementierung von (9.6, 9.7) nicht möglich.

Wie die Signatur sagt, macht `plus` aus zwei Funktionen eine neue Funktion und der Körper von `plus` sagt, wie diese neue Funktion aussieht.

Beispiel 9.15: Wir implementieren auch diesen Code in *setlX*:

```
=> plus:=closure(f,g){
      return closure(x){
        return f(x)+g(x);
      };
}
~< Result: closure(f, g) { return closure(x) { return f(x) + g(x); }; } >~
```

Stören Sie sich nicht daran, dass hier das Schlüsselwort `closure` an Stelle von `procedure` verwendet wird. Aus technischen Gründen ist dies notwendig.² Wenn Sie mögen, können Sie ab sofort statt `procedure` immer `closure` schreiben.

Nun können zwei Funktionen addiert werden (im Folgenden `sin` und `cos`) und Funktionswerte der neuen Funktion berechnet werden:

```
=> plus(sin,cos)(0);
~< Result: 1.0 >~
```

Nach dem neuen Objekt `plus(sin,cos)` befragt, antwortet *setlX*, was Sache ist:

```
=> plus(sin,cos);
~< Result: closure(x) { /* f := procedure(x) { /* predefined procedure
'sin' */ }; g := procedure(x) { /* predefined procedure 'cos' */ }; */
return f(x) + g(x); } >~
```

Das Objekt `plus(sin,cos)` ist eine Funktion (`closure`), aber eben kein Funktionswert. ■

Wir haben in (9.6) mit $\oplus$ ein eigenes Symbol für die Addition von Funktionen eingeführt, um diese auch visuell deutlich von der „normalen“ Addition $+$ von Zahlen zu unterscheiden. Wir geben diese symbolische Unterscheidung im folgenden auf (d. h. wir überladen $+$ mit $\oplus$) und schreiben von nun an $f + g$ statt $f \oplus g$.

Manchmal ist es nötig ganze „Funktionenfamilien“ zu beschreiben, z. B. die Familie der Potenzfunktionen mit dem Rückgabewert x^n (bei der unabhängigen Variable x) wie in Aufwärmübung 4. Diese Funktionenfamilie kann modelliert werden als Funktion mit der Definitionsmenge $\mathbb{N}$. Für jedes $n \in \mathbb{N}$ gibt diese Funktion eine Funktion zurück, die die n -te Potenz ihres Arguments berechnet.

Beispiel 9.16: Auch der Ausdruck

$$s_n(x) := \sin(2\pi nx)$$

beschreibt eine Funktionenfamilie: Die Sinus-Funktionen mit unterschiedlicher Frequenz n .

$$s : \mathbb{N} \rightarrow (\mathbb{R} \rightarrow \mathbb{R}), n \mapsto s_n$$

Für einen gegebenen Wert von n bedeutet s_n dabei:

$$s_n : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto s_n(x) = \sin(2\pi nx)$$

Eine andere Funktion ist dagegen

$$\tilde{s} : \mathbb{N} \times \mathbb{R} \rightarrow \mathbb{R}, [n, x] \mapsto \sin(2\pi nx), \quad (9.8)$$

²Eine `closure` ist ein Konzept aus der funktionalen Programmierung und berücksichtigt, dass Objekte in der Programmierung nur eingeschränkten Sichtbarkeitsbereich (*scope*) haben. Eine `closure` beschreibt eine Funktion, die Zugriffe auf ihren Erstellungskontext enthält. Im obigen Beispiel erhält die `closure plus` Zugriff auf den Implementierungscode der Funktionen, die in den Variablen `f` und `g` übergeben werden. Die Definitionen dieser Funktionen werden zum Erstellungskontext der `closure` hinzugefügt.

auch wenn die Rückgabewerte von $\tilde{s}(n, x)$ und $s_n(x)$ dieselben sind. Die Funktion $\tilde{s}$ bildet ein Tupel aus natürlicher Zahl und reeller Zahl auf eine reelle Zahl ab, während s_n eine natürliche Zahl auf eine Funktion einer Variablen abbildet. ■

Beispiel 9.17: Wir implementieren die Funktionenfamilie aus dem vorangegangenen Beispiel in *setlX*. Um Tipparbeit zu sparen, verwenden wir `|=>` an Stelle von `closure`, analog zur Verwendung von `|->` an Stelle von `procedure`:

```
=> pi:=4*atan(1);
~< Result: 3.141592653589793 >~
=> s:= n |=> (x |=> sin(2*pi*n*x));
~< Result: /* pi := 3.141592653589793; */ n |=> x |=> sin(2 * pi * n * x) >~
=> s(2)(1/8);
~< Result: 1.0 >~
=> s(2);
~< Result: /* n := 2; pi := 3.141592653589793; */ x |=> sin(2 * pi * n * x) >~
```

Die Implementierung von (9.8) sähe dagegen so aus:

```
=> stilde:= [n,x] |=> sin(2*pi*n*x);
~< Result: /* pi := 3.141592653589793; */ [n, x] |=> sin(2 * pi * n * x) >~
=> stilde(2,1/8);
~< Result: 1.0 >~
```

Sie entspricht eher der üblichen Implementierung als „Funktion mit mehreren Variablen“ in imperativen Programmiersprachen wie **Java**. ■

9.4.2 Gleichheit von Funktionen

Die beiden Funktionen

$$g : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto g(x) = x^2 - 1 \quad (9.9)$$

$$h : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto h(x) = (x - 1)(x + 1) \quad (9.10)$$

haben einiges gemeinsam, unterscheiden sich aber auch in manchen Aspekten. Gemeinsam ist ihnen, dass beide reelle Zahlen in reelle Zahlen transformieren und dass beide Transformationen die gleiche Wirkung haben. Sie haben die gleiche Wirkung, weil für alle $x \in \mathbb{R}$ die Gleichung

$$x^2 - 1 = (x - 1)(x + 1)$$

immer wahr ist, d. h. es gilt immer $g(x) = h(x)$. Andererseits unterscheiden sich die Funktionen g und h durch ihre Namen (eben g bzw. h) und durch die Berechnungsvorschrift.

Unterschiedliche Namen d. h. Bezeichnungen schließen aber nicht aus, dass diese Namen dasselbe bezeichnen. Weiterhin hatten wir in Abschnitt 8.2.1 diskutiert, dass Berechnungsvorschriften d. h. Algorithmen für den Begriff der Funktion nicht relevant sind. Funktionen sind lediglich eindeutige Zusammenhänge. Es geht darum zu beschreiben, *was* die Funktion macht, nicht *wie* sie es macht. Daher können und sollten wir zwei Funktionen, die dieselbe Transformation realisieren, aber dies auf unterschiedliche Weise tun, als gleich betrachten:

Zwei Funktionen

$$f_1 : \mathbb{D}_1 \rightarrow \mathbb{W}_1, x \mapsto f_1(x)$$

und

$$f_2 : \mathbb{D}_2 \rightarrow \mathbb{W}_2, x \mapsto f_2(x)$$

sind genau dann gleich (notiert als $f_1 = f_2$), wenn

1. die Definitionsmengen gleich sind ($\mathbb{D}_1 = \mathbb{D}_2$) und
2. beide Funktionen denselben eindeutigen Zusammenhang beschreiben, d.h. für alle $x \in \mathbb{D}_1 = \mathbb{D}_2$ gilt $f_1(x) = f_2(x)$.

Beispiel 9.18: Die beiden Funktionen

$$\tilde{g} : \{-2, -1, 0, 1, 2\} \rightarrow \mathbb{R}, x \mapsto \tilde{g}(x) = x^2 - 1$$

$$\tilde{h} : \{-2, -1, 0, 1, 2\} \rightarrow \mathbb{R}, x \mapsto \tilde{h}(x) = (x - 1)(x + 1)$$

unterscheiden sich von den in (9.9,9.10) definierten alleine durch die Definitionsmengen. Daher gilt sowohl $\tilde{g} \neq g$ als auch $\tilde{h} \neq h$, denn die erste Bedingung für Gleichheit von Funktionen – die Gleichheit der Definitionsmengen – ist nicht erfüllt. Dass die Funktionen $\tilde{g}$ und g identische Algorithmen verwenden, reicht alleine nicht für die Gleichheit der beiden Funktionen aus.

Dagegen gilt hier $\tilde{g} = \tilde{h}$, denn beide Bedingungen für die Gleichheit von Funktionen sind erfüllt. Repräsentieren wir die Funktionen wie in Abschnitt 8.2.2 als Relationen, ist die Gleichheit der beiden Funktionen auch ohne Anwenden der Definition einsichtig: Wenn wir die Menge der Paare $[x, \tilde{g}(x)]$ und $[x, \tilde{h}(x)]$ aufschreiben, erhalten wir jeweils die Menge

$$\{[-2, 3], [-1, 0], [0, -1], [1, 0], [2, 3]\}$$

als Darstellung der Funktionen $\tilde{g}$ und $\tilde{h}$. ■

Beispiel 9.19: Die drei Funktionen

```
double viertePotenz(double x) {
    return Math.pow(x,4);
}
double quadratquadrat(double x) {
    double y=x*x;
    return y*y;
}
double hoch4(double x) {
    return x*x*x*x;
}
```

sind gleich. Die Definitionsmenge ist in allen Fällen gleich (hier `double`) und für alle $x \in \text{double}$ gilt³

$$\text{viertePotenz}(x) = \text{quadratquadrat}(x) = \text{hoch4}(x).$$

Die drei Funktionen unterscheiden sich jedoch in den verwendeten Algorithmen mit durchaus erheblichen Auswirkungen. Die Auswirkungen betreffen Zeit und Speicher, die für die Berechnung der Funktionswerte benötigt werden. Dies ist eine Fragestellung, die im Informatikstudium in Algorithmen und Datenstrukturen behandelt wird. ■

³Aus numerischer Sicht ist dies nicht ganz korrekt, sondern $\text{viertePotenz}(x) \simeq \text{quadratquadrat}(x) \simeq \text{hoch4}(x)$ aufgrund von Rundungsfehlern.

Obwohl die Wertemenge ein definitionsgemäßer Bestandteil einer Funktion ist, verlangt die Definition der Gleichheit von Funktionen nicht die Gleichheit der Wertemengen. Wäre dies der Fall, müsste man z. B.

$$\begin{aligned} abs_1 : \mathbb{R} &\rightarrow \mathbb{R}, x \mapsto |x| \\ abs_2 : \mathbb{R} &\rightarrow \mathbb{R}_0^+, x \mapsto |x| \end{aligned}$$

als unterschiedliche Funktionen betrachten, obwohl sie das gleiche tun und die gleiche Definitionsmenge haben. Sie haben sogar die gleiche Bildmenge.

Die Definition der Gleichheit $f_1 = f_2$ stellt implizit sicher, dass die Bildmengen der beiden Funktionen gleich sind, denn wenn $\mathbb{D}_1 = \mathbb{D}_2$ und $\forall(x \in \mathbb{D}_1 | f_1(x) = f_2(x))$ gilt, muss auch $f_1(\mathbb{D}_1) = f_2(\mathbb{D}_2)$ gelten. Bei der Gleichheit der Funktionen wird also auf Gleichheit der Bildmengen, aber nicht auf Gleichheit der Wertemengen geachtet.

9.4.3 Umkehrfunktion

Die Definition der Umkehrfunktion zu einer Funktion f in Abschnitt 9.3.3 legt das Augenmerk auf die *entstehende* Funktion f^{-1} und definiert dieses Objekt. Wir können die Umkehrung aber auch als *Operation* sehen, also als Funktion, die eine gegebene (umkehrbare) Funktion in eine neue Funktion transformiert. Symbolisieren wir die Operation der Umkehrung (Inversion) mit $\mathcal{I}$, dann hat diese die Signatur

$$\mathcal{I} : \mathbb{F} \rightarrow \mathbb{F}, f \mapsto f^{-1}.$$

Darin soll hier $\mathbb{F}$ die Menge aller umkehrbaren Funktionen bezeichnen.

Mit der neuen Notation können wir also $f^{-1} = \mathcal{I}(f)$ schreiben. Was passiert, wenn wir die Umkehroperation $\mathcal{I}$ zweimal auf eine Funktion f anwenden?

$$\mathcal{I}(\mathcal{I}(f)) = \mathcal{I}(f^{-1})$$

Wir suchen also die Umkehrfunktion von f^{-1} . Wir müssen uns also fragen: Welche Eingabe wird durch f^{-1} in eine gegebene Ausgabe y transformiert. Die folgende Visualisierung dürfte klar machen, dass die Antwort $f(y)$ ist:

$$\begin{array}{ccc} y & \xrightarrow{f} & f(y) \\ y & \xleftarrow{f^{-1}} & f(y) \end{array}$$

Die Umkehrfunktion von f^{-1} ist also f , d. h.

$$\mathcal{I}(\mathcal{I}(f)) = \mathcal{I}(f^{-1}) = f$$

bzw.

$$(f^{-1})^{-1} = f.$$

Beispiel 9.20: Die Umkehrfunktion zu $\text{sqr}t : \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+, x \mapsto \sqrt{x}$ ist $\text{sqr}t^{-1} : \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+, x \mapsto x^2$. Deren Umkehrfunktion $(\text{sqr}t^{-1})^{-1} : \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+, x \mapsto \sqrt{x}$ ist identisch zur Ausgangsfunktion $\text{sqr}t$. ■

9.5 Komposition von Funktionen

Eine besondere Abbildung, die aus zwei Funktionen eine neue Funktion macht, ist die *Verkettung* oder *Komposition*. Sie kennen diese im Prinzip schon aus Abschnitt 7.7 von der Komposition von Relationen. Sie führt zwei Transformationsprozesse hintereinander aus. Die Idee ist, ausgehend von zwei Funktionen f und g eine neue Funktion zu bilden mit dem Rückgabewert $f(g(x))$, wenn mit x das Argument der Funktion bezeichnet wird. Die neue Funktion wird mit $f \circ g$ symbolisiert (sprich: „ f verkettet mit g “). Für ihre Funktionswerte $(f \circ g)(x)$ gilt also

$$(f \circ g)(x) = f(g(x)). \quad (9.11)$$

Die Funktion heit $f \circ g$, ihre Funktionswerte $(f \circ g)(x)$ werden gem (9.11) gebildet.

Am Rckgabewert $f(g(x))$ ist zu sehen, dass die „hintere“ Funktion g in $f \circ g$ zuerst zur Anwendung kommt. Sie wird auch als *innere Funktion* bezeichnet und entsprechend die „vordere“ Funktion als *uere Funktion*. Die Auswertereihenfolge ist von rechts nach links. Das kommt sowohl in der Schreibweise fr den Rckgabewert $f(g(x))$ zum Ausdruck als auch in der Position von g in $f \circ g$.

Beispiel 9.21: Die folgenden beiden Tabellen knnen als Funktionen modelliert werden.

Ort	Land	Land	Kontinent
Aachen	Deutschland	Albanien	Europa
Breslau	Polen	Brasilien	Sdamerika
Colombo	Sri Lanka	China	Asien
Denver	USA	Deutschland	Europa
Edmonton	Kanada	Eritrea	Afrika
Florianopolis	Brasilien	Frankreich	Europa
Gouda	Niederlande	Kanada	Nordamerika
Houston	USA	Niederlande	Europa
Indianapolis	USA	Polen	Europa
Jena	Deutschland	Sri Lanka	Asien
Kassel	Deutschland	USA	Nordamerika

Wir geben der durch die linke Tabelle dargestellten Funktion die Signatur $\ell : \text{Orte} \rightarrow \text{Lnder}_1$. Die Definitionsmenge „Orte“ hat dabei als Elemente genau alle in der Spalte „Ort“ genannten Orte. Fr die Wertemenge „Lnder₁“ whlen wir die Bildmenge der Abbildung ℓ , also die Menge aller in der rechten Spalte der linken Tabelle genannten Lnder. Die Funktion ℓ bildet (bestimmte) Orte auf die Lnder ab, in denen sie liegen.

Der durch die rechte Tabelle dargestellten Funktion geben wir die Signatur $k : \text{Lnder}_2 \rightarrow \text{Kontinente}$. Die Definitionsmenge ist die Menge aller in der linken Spalte der rechten Tabelle genannten Lnder. Entsprechend fr die Wertemenge „Kontinente“. Die Funktion k bildet (bestimmte) Lnder auf die Kontinente ab, in denen sie liegen.

Die Verkettung $k \circ \ell$ bildet (bestimmte) Orte auf die Kontinente ab, in denen sie liegen, z. B.

$$\begin{aligned}(k \circ \ell)(\text{Aachen}) &= k(\ell(\text{Aachen})) = k(\text{Deutschland}) = \text{Europa} \\ (k \circ \ell)(\text{Colombo}) &= k(\ell(\text{Colombo})) = k(\text{Sri Lanka}) = \text{Asien}\end{aligned}$$

Die umgekehrte Verkettung $\ell \circ k$ kann jedoch nicht gebildet werden, denn k bildet Lnder auf Kontinente ab, aber die nachfolgende Funktion ℓ bildet keine Kontinente ab, sondern Orte. ■

Beispiel 9.22: Die Verkettung der beiden Funktionen

$$\begin{aligned}\text{sqr}t : \mathbb{R}_0^+ &\rightarrow \mathbb{R}_0^+, x \mapsto \sqrt{x} \\ \text{sqr} : \mathbb{R} &\rightarrow \mathbb{R}_0^+, x \mapsto x^2\end{aligned}$$

ergibt in der Reihenfolge „erst Wurzel, dann Quadrat“:

$$\text{sqr} \circ \text{sqr}t : \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+, x \mapsto \sqrt{x^2}$$

Da fr $x \in \mathbb{R}_0^+$ (d. h. der Definitionsmenge der Verkettung $\text{sqr} \circ \text{sqr}t$) immer $\sqrt{x^2} = x$ gilt, ist die Funktion $\text{sqr} \circ \text{sqr}t$ gleichbedeutend mit der folgenden:

$$\text{id} : \mathbb{R}_0^+ \rightarrow \mathbb{R}_0^+, x \mapsto x \tag{9.12}$$

Die Funktion id macht nichts anderes als ihre Eingabe unverndert in ihre Ausgabe zu transformieren. Diese Transformation mag ihnen langweilig vorkommen, sie werden jedoch bald sehen, dass die Funktion id eine besondere strukturelle Bedeutung hat.

Die Verkettung in der Reihenfolge „erst Quadrat, dann Wurzel“ ergibt dagegen

$$\text{sqrt} \circ \text{sqr} : \mathbb{R} \rightarrow \mathbb{R}_0^+, x \mapsto \sqrt{x^2}.$$

Wegen $\sqrt{x^2} = |x|$ ist diese Funktion identisch zur Betragsfunktion. ■

Beide Beispiele machen deutlich, dass es bei der Verkettung von Funktionen auf die Reihenfolge ankommt. Im Allgemeinen sind $f \circ g$ und $g \circ f$ unterschiedliche Funktionen.

Die beiden Beispiele zeigen auch, dass bei der Verkettung $f \circ g$ die Bildmenge $g(\mathbb{D}_g)$ der inneren Funktion g und die Definitionsmenge $\mathbb{D}_f$ der äußeren Funktion f in einem ganz bestimmten Zusammenhang stehen müssen. Da jedes Bild von g durch f transformiert wird, muss die Bildmenge der inneren Funktion eine Teilmenge der Definitionsmenge der äußeren Funktion sein: $g(\mathbb{D}_g) \subseteq \mathbb{D}_f$. Abb. 9.5 visualisiert, was passieren würde, wenn dies nicht der Fall wäre.

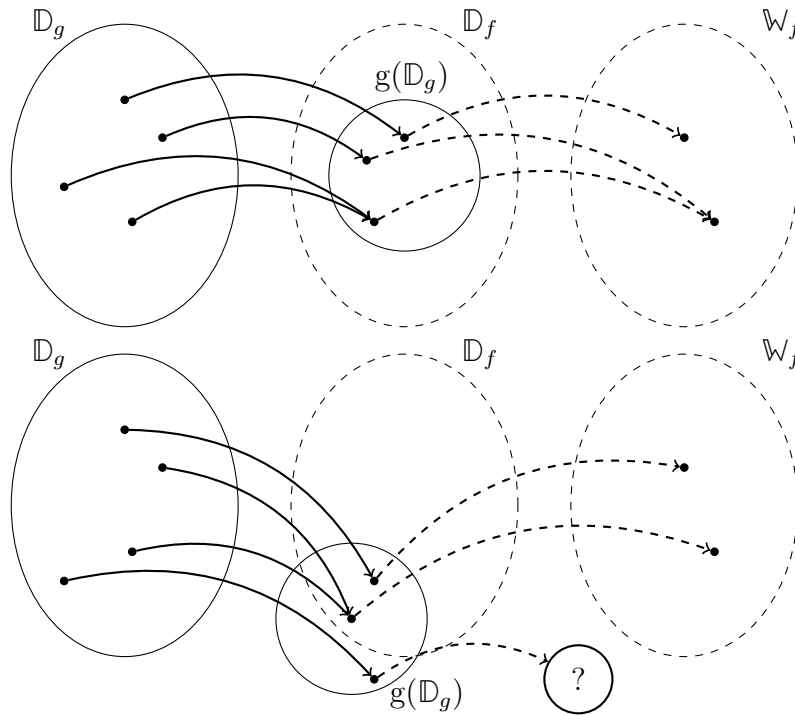


Abbildung 9.5: Bei der Verkettung $f \circ g$ muss $g(\mathbb{D}_g) \subseteq \mathbb{D}_f$ erfüllt sein (oben). Andernfalls kann nicht jedes $x \in \mathbb{D}_g$ auf ein Element $(f \circ g)(x) \in \mathbb{W}_f$ abgebildet werden (unten).

Wir schließen unsere einführenden Betrachtungen zur Verkettung mit einer formalen Definition ab:

Für die Funktionen $f : \mathbb{D}_f \rightarrow \mathbb{W}_f$ und $g : \mathbb{D}_g \rightarrow \mathbb{W}_g$ heißt die Funktion

$$f \circ g : \mathbb{D}_g \rightarrow \mathbb{W}_f, x \mapsto (f \circ g)(x) = f(g(x))$$

die *Komposition* oder *Verkettung* von f mit g .

Die Funktion legt das Augenmerk auf die *entstehende* Funktion $f \circ g$ und definiert dieses Objekt. Dadurch kommt der *operative Charakter* von $\circ$ nicht unbedingt zur Geltung. Wir können alternativ auch die Operation $\circ$ direkt definieren:

Die Funktion

$$\circ : \mathbb{F} \times \mathbb{F} \rightarrow \mathbb{F}, [f, g] \mapsto f \circ g,$$

die zwei Funktionen aus der Menge aller Funktionen $\mathbb{F}$ zu einer neuen Funktion verknüpft, so dass

$$(f \circ g)(x) = f(g(x))$$

gilt, heißt *Komposition* oder *Verkettung*.

Beispiel 9.23: In dieser Form können wir in *setlX* die Komposition direkt als Funktion definieren, die Funktionen verknüpft:

```
=> komposition:=closure(f,g){return closure(x){ return f(g(x)); }; };
~< Result: closure(f, g) { return closure(x) { return f(g(x)); }; } >~
=> komposition(sqrt,x|->x**2)(-2);
~< Result: 2.0 >~
```

Da die Quadratfunktion in *setlX* keinen eigenen Namen hat, wurde sie in der letzten Eingabe mit $x \mapsto x^2$ beschrieben. Alternativ hätte man die Quadratfunktion vorher unter Festlegen eines Namens, z. B. *sqr*, definiert werden können (wie in Aufwärmübung 3). An Stelle der letzten Eingabe könnte man dann `komposition(sqrt,sqr)(-2);` schreiben. ■

Komposition mit der Inversen

Eine interessante und wichtige Verkettung ist die Verkettung einer Funktion mit ihrer Umkehrfunktion. Dabei wird die Identitätsfunktion aus dem vorangegangenen Abschnitt eine wichtige Rolle spielen. Wir definieren sie daher zunächst formal:

Die Funktion $\text{id} : M \rightarrow M$, $x \mapsto x$ wird *Identitätsfunktion auf der Menge M* genannt.

Die Identitätsfunktion in (9.12) ist nichts anderes als die Identitätsfunktion auf $\mathbb{R}_0^+$.

Wenden wir uns nun der Verkettung von Funktion und Umkehrfunktion zu:

Beispiel 9.24: Die Funktion $g : \mathbb{R} \rightarrow \mathbb{R}$, $x \mapsto x+1$ hat als Umkehrfunktion $g^{-1} : \mathbb{R} \rightarrow \mathbb{R}$, $x \mapsto x-1$. (Prüfen Sie das nach!) Diese beiden Funktionen lassen sich auf zwei Weisen verketteten: $g^{-1} \circ g$ und $g \circ g^{-1}$. Schauen wir uns an, was diese beiden Funktionen konkret tun:

$$\begin{aligned} (g^{-1} \circ g)(x) &= g^{-1}(g(x)) = g^{-1}(x+1) = (x+1) - 1 = x \\ (g \circ g^{-1})(x) &= g(g^{-1}(x)) = g(x-1) = (x-1) + 1 = x \end{aligned}$$

Es gilt hier also $g \circ g^{-1} = g^{-1} \circ g = \text{id}$. ■

Beispiel 9.25: Als nächstes verketteten wir die Funktionen s und s^{-1} mit

$$\begin{aligned} s(x) &= \frac{x}{1+|x|} \\ s^{-1}(x) &= \frac{x}{1-|x|} \end{aligned}$$

aus Beispiel 9.13 in Abschnitt 9.3.4:

$$(s^{-1} \circ s)(x) = s^{-1}\left(\frac{x}{1+|x|}\right) = \frac{\frac{x}{1+|x|}}{1 - \left|\frac{x}{1+|x|}\right|} \stackrel{*}{=} \frac{\frac{x}{1+|x|}}{1 - \frac{|x|}{1+|x|}} = \frac{x}{1+|x| - |x|} = x$$

Dabei wurde im Schritt $\star$ ausgenutzt, dass $|a/b| = |a|/|b|$ und dass $|1+|x|| = 1+|x|$, weil $1+|x| \geq 1$ nie negativ sein kann.

Entsprechend verläuft die Rechnung für die andere Komposition. Sie sind dran!

$$\pencil (s \circ s^{-1})(x) = \qquad \qquad \qquad = x$$

Es gilt also auch hier $s \circ s^{-1} = s^{-1} \circ s = \text{id}$. ■

Was in den beiden Beispielen im Speziellen gilt, gilt auch allgemein für jede umkehrbare Funktion f :

$$f^{-1} \circ f = f \circ f^{-1} = \text{id} \quad (9.13)$$

Die Begründung sollte ihnen leicht fallen, wenn Sie überlegen, was passiert, wenn zuerst x durch die Funktion f transformiert wird und das Resultat $f(x)$ anschließend durch die Funktion f^{-1} transformiert wird.

. $x = (f^{-1}(f(x)))$ gilt. Also ist genau x . Das Zwischenresultat $f(x)$ wird durch f^{-1} in das Objekt transformiert, das f in $f(x)$ transformiert hat.

Beachten Sie, dass es gemäß (9.13) bei der Verkettung von Funktion und Umkehrfunktion nicht auf die Reihenfolge der Verkettung ankommt, während i. A. $f \circ g \neq g \circ f$ gilt.

Inverse der Komposition

Wenn man zwei Schritte macht und anschließend den zweiten Schritt rückgängig macht und dann den ersten, kommt man an den Anfang zurück. Wenn Sie ein Geschenk in einen Karton tun und dieses in Geschenkpapier verpacken, kommt man wieder ans Geschenk, indem man zuerst das Geschenkpapier entfernt und dann den Karton öffnet. Entsprechend ist es bei der Hintereinanderausführung, d. h. der Verkettung zweier Funktionen:

$$(f \circ g)^{-1} = g^{-1} \circ f^{-1}$$

Wie bereits beschrieben, steht dahinter ein allgemeingültiges Prinzip bei der Umkehrung aufeinander folgender Prozessschritte:

$$\begin{array}{ccccc} x & \xrightarrow{g} & g(x) & \xrightarrow{f} & f(g(x)) \\ x & \xleftarrow{g^{-1}} & g(x) & \xleftarrow{f^{-1}} & f(g(x)) \end{array}$$

Umkehrung der Komposition

Die zentrale Fragestellung im Zusammenhang mit der Umkehrfunktion ist: Welche Eingabe wurde von f in die Ausgabe $f(x)$ transformiert? Betrachten wir die Addition reeller Zahlen als Funktion $+: \mathbb{R} \rightarrow \mathbb{R}$, $[x, y] \mapsto x + y$, dann ist eine entsprechende Frage: Die Summe welcher Zahlen ist 7? Natürlich gibt es mehr als eine Antwort, eben weil die Addition nicht umkehrbar ist.

Entsprechend können wir bei der Komposition von Funktionen, d. h. bei $\circ: \mathbb{F} \times \mathbb{F} \rightarrow \mathbb{F}$, $[f, g] \mapsto f \circ g$ fragen, welche zwei Funktionen verkettet eine gegebene Funktion h ergeben.

Beispiel 9.26: Welche Funktionen ergeben verkettet $h: \mathbb{R} \rightarrow \mathbb{R}$, $x \mapsto \sqrt{1+x^2}$?

Auch hier gibt es viele Antwortmöglichkeiten. Zwei vermutlich offensichtliche Funktionen f, g , die $f \circ g = h$ erfüllen, sind

$$f: \mathbb{R} \rightarrow \mathbb{R}, x \mapsto \sqrt{x} \quad , \quad g: \mathbb{R} \rightarrow \mathbb{R}, x \mapsto 1+x^2.$$

Aber u. a. auch die folgenden Funktionenpaare erfüllen die Anforderung:

$$\begin{array}{l} f: \mathbb{R} \rightarrow \mathbb{R}, x \mapsto \sqrt{1+x} \quad , \quad g: \mathbb{R} \rightarrow \mathbb{R}, x \mapsto x^2 \quad , \\ f = \text{id}: \mathbb{R} \rightarrow \mathbb{R} \quad , \quad g = h \end{array}$$

Wenn Sie sich unsicher sind, warum dies so ist, bilden Sie $f \circ g$! ■

Beispiel 9.27: Wenn $h = f \circ g$, $h: \mathbb{R} \rightarrow \mathbb{R}$, $x \mapsto \sqrt{1+x^2}$ und $f: \mathbb{R}_0^+ \rightarrow \mathbb{R}$, $x \mapsto \sqrt{1+4x}$, was ist dann g ?

Es muss also gelten:

$$\begin{aligned} h(x) &= f(g(x)) \\ \Leftrightarrow \sqrt{1+x^2} &= \sqrt{1+4g(x)} \end{aligned}$$

Wir können g bestimmen, indem wir diese Gleichung nach $g(x)$ auflösen. Aber vermutlich sehen Sie schon, dass

$$g(x) = \frac{1}{4}x^2$$

gelten muss. Die Funktion $g : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto x^2/4$ erfüllt also $f \circ g = h$. ■

Eigenschaften der Komposition

Natürlich ist es auch möglich mehr als zwei Funktionen zu verketteten. Wir hatten dies eigentlich schon in (9.13) getan, denn wir können die Umkehrfunktion $f^{-1} = \mathcal{I}(f) = \mathcal{I} \circ f$ als Verkettung von f mit dem Umkehroperator $\mathcal{I}$ aus Abschnitt 9.4.3 betrachten. Damit wird (9.13) zu

$$(\mathcal{I} \circ f) \circ f = f \circ (\mathcal{I} \circ f) = \text{id}$$

und die Verkettung dreier Funktionen wird deutlich sichtbar.

Die Komposition $\circ$ ist eine binäre Operation, so wie z. B. auch Konjunktion $\wedge$, Disjunktion $\vee$ und Schnittbildung $\cap$. Die letzt genannten Operationen haben gewisse strukturelle Eigenschaften gemeinsam, z. B. sind sie alle kommutativ und assoziativ (vgl. Abschnitte 4.3 und 5.7). Ist auch die Komposition assoziativ, d. h. gilt

$$f \circ (g \circ h) = (f \circ g) \circ h?$$

Ist sie kommutativ, d. h. gilt

$$f \circ g = g \circ f?$$

Gibt es weitere besondere Eigenschaften?

Wir werden diese Fragen in Kapitel 12 wieder aufgreifen, was nicht bedeutet, dass Sie bis dahin nicht über die Antworten nachdenken sollten.

9.6 Totale und partielle Funktionen

Funktionen ordnen *jedem* Element der Definitionsmenge eindeutig ein Element der Wertemenge zu. Aus Perspektive der Informatik ist dies eine überaus wichtige Eigenschaft, weil dadurch sichergestellt wird, dass Funktionen die Berechnungen, die sie durchführen sollen, auch durchführen können.

Beispiel 9.28: Bei der Funktion „Inverse“

$$\text{inv} : \mathbb{R} \setminus \{0\} \rightarrow \mathbb{R}, x \mapsto 1/x$$

wird durch die Definitionsmenge $\mathbb{D} = \mathbb{R} \setminus \{0\}$ sichergestellt, dass *jede* mögliche Eingabe von der Funktion verarbeitet werden kann.

Bei der Definitionsmenge $\mathbb{R}$ wäre dies nicht der Fall. Für die Eingabe $x = 0$ könnte die Berechnung des Rückgabewerts $1/x$ nicht durchgeführt werden. ■

Der Preis, der für diesen Sicherheitsmechanismus zu zahlen ist, ist allerdings beachtlich: Programmiersprachen bräuchten mehr als nur eine Handvoll „Typenbezeichnungen“ (`boolean`, `int`, `float` etc.) - also mehr als nur eine Handvoll grundlegender Mengenbezeichnungen.

Beispiel 9.29: Die Menge `float` ist als Definitionsmenge für die Funktion „Inverse“ aus dem vorangegangenen Beispiel ungeeignet, da

```
float inv (float x) {return 1./x}
```

ja nicht jedes Element aus $\mathbb{D} = \text{float}$ auf ein Element aus $\mathbb{W} = \text{float}$ abbildet. Vielmehr müsste erst ein neuer Datentyp, etwa mit dem Namen `floatwithoutzero` definiert werden, um dann mittels

```
float inv (floatwithoutzero x) {return 1./x}
```

sicherzustellen, dass tatsächlich zu jeder möglichen Eingabe auch eine Ausgabe berechnet werden kann. ■

In der Informatik wird dieser Aufwand der Definition geeigneter Typen für die Definitionsmenge durch eine Erweiterung des Funktionenkonzepts auf das Konzept der *partiellen Funktion* umgangen⁴:

Eine *partielle Funktion* f ist ein Prozess, der jedes Element x einer Menge $\mathbb{D}$ in *höchstens* ein Element $f(x)$ einer Menge $\mathbb{W}$ transformiert.

„Normale“ Funktionen, also solche, die tatsächlich *jedes* Element von $\mathbb{D}$ transformieren, werden in der Informatik *totale Funktionen* genannt. Totale Funktionen transformieren die Definitionsmenge also in Gänze, während partielle Funktionen dies u. U. nur zum Teil tun.

Die Idee der partiellen Funktion folgt einem ähnlichen Pragmatismus wie die Idee, statt der Bildmenge $f(\mathbb{D})$ eine Menge $\mathbb{W} \supseteq f(\mathbb{D})$ zur Definition der totalen Funktion zu verwenden. Bei partiellen Funktionen kann als Definitionsmenge eine Menge $\mathbb{D}_{\text{partiell}}$ angegeben werden, die umfassender als die eigentliche Definitionsmenge $\mathbb{D}$ der zugrundeliegenden totalen Funktion ist: $\mathbb{D}_{\text{partiell}} \supseteq \mathbb{D}$.

Beispiel 9.30: Bei der partiellen Funktion `inv` aus obigem Beispiel ist $\mathbb{D}_{\text{partiell}} = \text{float} \supseteq \mathbb{D} = \text{floatwithoutzero}$. ■

9.7 Empfohlene Vertiefungen

9.7.1 Currying

In funktionalen Programmiersprachen werden Funktionen konsequent als Funktionen einer einzigen Variablen betrachtet. Betrachten wir als Beispiel die Addition natürlicher Zahlen. Mit der Signatur

$$+ : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}, \quad [a, b] \mapsto a + b$$

ist sie eine Funktion von zwei Variablen.

Dagegen ist sie mit der Signatur

$$+ : \mathbb{N} \rightarrow (\mathbb{N} \rightarrow \mathbb{N}), \quad a \mapsto (b \mapsto a + b)$$

eine Funktion einer Variablen, die diese Variable auf eine Funktion abbildet. $+(5)$ ist dann die Funktion, die zu ihrem Argument 5 (von links) hinzuaddiert, also z. B. $+(5)(7)$ wird dann zu 12 ausgewertet.

Beispiel 9.31: Schauen wir uns die Angelegenheit in *setLX* an:

```
=> add:= a |=> b |=> a+b;
~< Result: a |=> b |=> a + b >~
```

Der Ausdruck `add(5)` steht dann für die Funktion, die zu ihrem Argument 5 (von links) hinzuaddiert, also z. B.

```
=> add(5);
~< Result: /* a := 5; */ b |=> a + b >~
=> add(5)(7);
~< Result: 12 >~
```

Die Umwandlung einer Funktion mit mehreren Argumenten in eine Funktion mit einem Argument wird *Currying* genannt.⁵

⁴Man könnte auch sagen, dass der Anfangsaufwand der sauberen Beschreibung der Definitionsmenge auf ein späteres *Exception Handling* verschoben wird.

⁵Der Begriff Currying leitet sich vom Nachnamen des Logikers Haskell Brooks Curry ab, nach dessen Vornamen die funktionale Programmiersprache **Haskell** benannt ist.

9.7.2 Operatorpositionierung

Bei $+(x, y)$ oder allgemein $f(x, y)$ handelt es sich um die *Präfixnotation*, bei $x + y$ bzw. $x f y$ um die *Infixnotation*. Bei der Präfixnotation steht der Name der Abbildung vor den Argumenten, bei der Infixnotation zwischen den Argumenten. Beispielsweise verwenden funktionale Programmiersprachen und besonders Computeralgebrasysteme in der Regel und konsequent die Präfixnotation, auch in Fällen, in denen die Infixnotation im Alltag üblicher ist.

Ein dritte Notationsform ist die *Postfixnotation*, bei der der Name der Abbildung hinter den Argumenten steht. Üblicherweise wird die Fakultät in Postfixnotation geschrieben ($n!$, siehe Kapitel 10). Die Addition würde als $(x, y) +$ geschrieben werden.

Die Postfixnotation wird konsequent in der sogenannten *umgekehrten polnischen Notation* (UPN) verwendet, wie sie u. a. bei den legendären HP-Taschenrechnern zum Einsatz kommt. In der Informatik ist die UPN von Interesse, weil sie eine stapelbasierte Abarbeitung von Ausdrücken ermöglicht.

Etwas kurios ist die *Zirkumfixnotation*, bei der das Abbildungssymbol um das Argument herum steht. Ein typisches Beispiel ist die Wurzel $\sqrt{x}$.

9.7.3 Besondere Funktionen

In Anwendungen spielen einige Funktionenklassen eine besondere Rolle, u. a. Polynome, trigonometrische Funktionen, Exponentialfunktionen und Logarithmen. Machen Sie sich mit diesen speziellen Funktionen vertraut, z. B. mit Hilfe von Kapitel 5 des Lehrbuchs von Teschl und Teschl.

9.8 Lernziele

Studierende	Kennen	Können	Verstehen
fachlich	☐	☐ bestimmen, ob eine Funktion surjektiv, injektiv oder bijektiv ist ☐ bestimmen die Umkehrfunktion einer bijektiven Funktion ☐ bilden die Komposition von Funktionen ☐ denken Transformationsprozess einer Abbildung rückwärts, d. h. bestimmen Urbilder zu einem gegeben Bild	☐ betrachten Funktionen als Objekte, die wiederum durch Funktionen modifiziert oder erzeugt werden können ☐ entscheiden, ob eine Funktion als totale oder partielle Funktion beschrieben ist
methodisch	☐	☐ programmieren Funktionen, die Funktionen abbilden oder auf Funktionen abbilden	☐

Hinzu kommen die themenübergreifenden Lernziele aus den Kategorien „sozial“ und „persönlich“ aus Abschnitt 1.7 sowie die Lernziele aus der Kategorie „methodisch“ aller vorangegangener Kapitel. Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben.

9.9 Übungsaufgaben

1. Kann das Objekt $f^{-1}(x)$ als Verkettung von erstens f und zweitens dem Bilden der Umkehrfunktion auf x aufgefasst werden?

2. Bestimmen Sie die Komposition $f \circ g$, wobei

$$\begin{aligned} f(x) &= \begin{cases} 2x - 1 & , x \leq 10 \\ 2x - 20 & , x > 10 \end{cases} \\ g(x) &= \begin{cases} -2x & , x < 5 \\ \frac{x+1}{2} & , x \geq 5 \end{cases} . \end{aligned}$$

Schreiben Sie einen Ausdruck für $(f \circ g)(x)$. Die Definitionsmenge aller Funktionen ist $\mathbb{R}$.

3. Welche Funktionen f haben die Eigenschaft $f^{-1} = f$?
4. Treffen Sie eine allgemeingültige Aussage über die Komposition mit der Identitätsfunktion, d. h. welche Wirkungen haben $\text{id} \circ f$ und $f \circ \text{id}$?
5. Überlegen und begründen Sie, ob die Komposition assoziativ und kommutativ ist.
6. Programmieren Sie in *setLX* die Subtraktion, Multiplikation und Division von Funktionen.
7. Analysieren Sie die Funktionen, die sie kürzlich in **Java** programmiert haben. Sind diese total oder partiell?

Kapitel 10

Kombinatorik

In diesem Kapitel geht es um Zählmethoden. Zählen meint dabei nicht notwendiger Weise Abzählen. Abzählen ist i. A. nicht wirtschaftlich. Sie werden sicherlich nicht die Anzahl der Kästchen eines karierten Blattes durch Abzählen der Kästchen bestimmen. Auch bei der Frage, wie viele Elemente die Menge $\{6 \dots 20\}$ hat, werden Sie kaum die einzelnen Elemente abzählen wollen.

Die meisten Zählprobleme in der Informatik können nicht durch Abzählen gelöst werden. Häufig ist nicht einmal die exakte Anzahl interessant. Oft reicht es die ungefähre Zahl abzuschätzen, z. B. bei der Fragestellung, wie viele Rechenschritte ein Programm benötigt.

10.1 Aufwärmübungen

1. Das Produkt der ersten n natürlichen Zahlen wird als *Fakultät* bezeichnet und mit $n!$ notiert.

$$n! = \prod_{i=1}^n i = 1 \cdot 2 \cdot \dots \cdot n \quad (10.1)$$

Beispielsweise ist $4!$ (sprich „vier Fakultät“) $4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24$.

Für 0 (was ja keine natürliche Zahl ist) ist $0!$ so definiert:

$$0! = 1 \quad (10.2)$$

Der Operator der Fakultät bindet stärker als das Produkt. $3 \cdot 2!$ ist also 6 und nicht etwa $(3 \cdot 2)! = 6! = 720$.

In *setlX* wird die Fakultät genauso notiert. Hier einige Beispiele:

```
=> 4!;  
~< Result: 24 >~  
=> 0!;  
~< Result: 1 >~  
=> 42!;  
~< Result: 1405006117752879898543142606244511569936384000000000 >~
```

In diesem Kapitel werden wir häufig Operationen der Art

$$\frac{n!}{k!}$$

für natürliche Zahlen $k \leq n$ benötigen, z. B. $5!/3! = 5 \cdot 4 \cdot 3!/3! = 20$. Beschreiben Sie in Worten, das Produkt welcher Zahlen $n!/k!$ effektiv berechnet (analog zur Beschreibung, dass $n!$ effektiv das Produkt der ersten n natürlichen Zahlen berechnet).

2. Schreiben Sie kürzere Ausdrücke für

- (a) $5 \cdot 4!$
- (b) $6 \cdot 5 \cdot 4!$
- (c) $n \cdot (n-1)!$
- (d) $(n+1) \cdot n!$

3. Welche Art von Kollektion ist das geeignete Modell?

- Passwort
- Kollektion aller Passworte, die aus genau sechs Zeichen bestehen
- Kollektion aller Parteimitglieder
- Kollektion aller möglichen Parteivorstände

4. Wie viele natürliche Zahlen kann man mittels einer zwölfstelligen Binärzahl darstellen?

5. Mr. X hat 10 Hemden und 5 Krawatten. Wie viele Kleidungsstücke hat er? Wie viele Hemden-Krawatten-Varianten hat er?

6. Eine Partei hat m männliche Mitglieder und w weibliche. Der Parteivorsitz soll paritätisch besetzt werden, d. h. er soll aus einer Frau und einem Mann bestehen. Wie viele Möglichkeiten gibt es, den Vorstand zu besetzen? Wie viele Mitglieder hat die Partei?

7. Die Verzeichnisse V1 mit den Dateien

```
C:\>dir V1
Verzeichnis von C:\V1
02.03.2019  14:43    <DIR>          .
02.03.2019  14:43    <DIR>          ..
02.03.2019  14:42                178 datei1.docx
02.03.2019  14:43                416 datei2.docx
02.03.2019  14:41                328 datei3.xls
02.03.2019  14:42                722 datei4.ppt
02.03.2019  14:41                82 datei5.txt
                    5 Datei(en),          1'726 Bytes
```

und V2 mit den Dateien

```
C:\>dir V2
Verzeichnis von C:\V2
02.03.2019  14:50    <DIR>          .
02.03.2019  14:50    <DIR>          ..
02.03.2019  14:42                178 datei1.docx
02.03.2019  14:41                328 datei3.xls
02.03.2019  14:41                82 datei6.txt
                    3 Datei(en),          588 Bytes
```

werden zu einem neuen Verzeichnis V3 fusioniert. Wie viele Dateien enthält das neue Verzeichnis?

8. Wie viele Elemente hat die Menge $\{1 \dots 11\} \times \{6 \dots 20\}$? Überprüfen Sie Ihre Antwort mit *setlX*.

9. Stellen Sie eine Regel auf, mit der man die Anzahl der Elemente der Menge $\{m \dots n\}$ bestimmen kann (also $\#\{m \dots n\}$ berechnen kann).

10. Wie viele 4-Tupel können mittels der Elemente der Menge $S = \{1, 2, 3, 4\}$ gebildet werden, so dass jedes Tupel vier unterschiedliche Komponenten hat? Überprüfen Sie Ihr Ergebnis mit *setlX*, indem Sie einen Mengen-Konstruktor für die Menge aller dieser 4-Tupel schreiben.
11. Leitfragen:
- (a) Wie zählt man?
 - (b) Warum gibt es einen Zusammenhang zwischen Kombinatorik und den elementaren Kollektionen aus Kapitel 1?
 - (c) Worin besteht der Unterschied zwischen Ordnung und Reihenfolge?

10.2 Additives und multiplikatives Zählen

Zählen kann immer als die Bestimmung der Mächtigkeit einer Menge modelliert werden (z. B. die Menge aller Kästchen eines karierten Blattes oder die Menge $\{6 \dots 20\}$). Die Kernoperationen beim Zählen sind meistens Multiplikation (so wie bei der Anzahl der Kästchen eines karierten Blattes) oder Addition (bzw. Subtraktion, so wie bei der Bestimmung der Anzahl der Elemente von $\{6 \dots 20\}$).

10.2.1 Additives Zählen

Aus Modellierungssicht werden beim additiven Zählen zwei Mengen A und B vereinigt. Die Anzahl der Möglichkeiten ist dann die Anzahl der Elemente der Vereinigungsmenge $A \cup B$, also $|A \cup B|$. In Aufwärmübung 5 verfügte Mr. X über eine Menge H von 10 Hemden und eine Menge K von 5 Krawatten. Insgesamt hatte er $|H \cup K| = 15$ Oberbekleidungsstücke. In Aufwärmübung 6 hatte die Partei $|F|$ weibliche und $|M|$ männlichen Mitglieder, insgesamt also $|F \cup M|$ viele Parteimitglieder. In beiden Fällen ist die Mächtigkeit der Vereinigungsmenge gleich der Summe der Mächtigkeiten der Einzelmengen:

$$|A \cup B| = |A| + |B| \quad (10.3)$$

In Aufwärmübung 7 können die beiden Verzeichnisse als Mengen V_1 bzw. V_2 modelliert werden. Die beiden ursprünglichen Verzeichnisse enthielten $|V_1| = 5$ bzw. $|V_2| = 3$ viele Dateien. Das neue Verzeichnis umfasste die Dateien $V_1 \cup V_2$ und beinhaltete $|V_1 \cup V_2| = 6$ viele Dateien. Im Gegensatz zu (10.3) ist hier

$$|V_1 \cup V_2| < |V_1| + |V_2|$$

Was ist an den Mengen H und K in Aufwärmübung 5 bzw. F und M in Aufwärmübung 6 anders als bei V_1 und V_2 , so dass dort (10.3) gilt, hier aber nicht?



In den beiden ersten Fällen sind die beteiligten Mengen disjunkt, im dritten Fall sind sie nicht.

Für disjunkte Mengen A und B ist definitionsgemäß $|A \cap B| = 0$. A und B haben keine gemeinsamen Elemente. Daher hat die Vereinigungsmenge genau so viele Elemente wie die beiden einzelnen Mengen zusammen:

$$|A \cup B| = |A| + |B|.$$

Für nicht disjunkte Mengen ist $|A \cap B| > 0$. Nun haben A und B gemeinsame Elemente, nämlich $|A \cap B|$ viele. Die Vereinigungsmenge hat entsprechend weniger Elemente als die beiden einzelnen Mengen zusammen:

$$|A \cup B| = |A| + |B| - |A \cap B| \quad (10.4)$$

Möglicherweise hilft es Ihnen sich diesen Sachverhalt zusätzlich mittels Venn-Diagrammen zu veranschaulichen (z. B. Abb. 5.2 und 5.3).

Im Spezialfall disjunkter Mengen wird aus (10.4) natürlich wieder (10.3).

Beispiel 10.1: Die Mengen $A = \{1 \dots 5\}$ (also $|A| = 5$) und $B = \{4 \dots 10\}$ (also $|B| = 7$) haben zwei Elemente gemeinsam, denn $A \cap B = \{4, 5\}$ und $|A \cap B| = 2$. Daher haben die beiden Mengen zehn Elemente gemeinsam, denn

$$\begin{aligned} |A \cup B| &= |A| + |B| - |A \cap B| \\ &= 5 + 7 - 2 = 10. \end{aligned}$$

Zum selben Ergebnis gelangt man natürlich auch, indem man erst die Vereinigungsmenge $A \cup B = \{1 \dots 10\}$ bildet und dann die Anzahl ihrer Elemente bestimmt. ■

10.2.2 Multiplikatives Zählen

Aus Modellierungssicht wird beim multiplikativen Zählen aus zwei Mengen A und B das kartesische Produkt $A \times B$ gebildet und die Anzahl $|A \times B|$ bestimmt. In Aufwärmübung 5 ist die Menge der Hemd-Krawatten-Kombinationen $H \times K$; gebildet aus der Menge der Hemden H und der Menge der Krawatten K . Insgesamt kann Mr. X sich also auf $|H \times K|$ vielen unterschiedlichen Arten kleiden.

Analog zum additiven Zählen in (10.3) bzw. (10.4) müssen wir nun nur noch $|A \times B|$ durch $|A|$ und $|B|$ ausdrücken. Das sollte Ihnen nicht schwer fallen, wenn Sie das kartesische Produkt verstanden haben:

$$\text{✎} \quad |A \times B| = \quad (10.5)$$

Häufig ist es so, dass beim multiplikativen Zählen die beiden beteiligten Mengen identisch sind.

Beispiel 10.2: Wie viele Zeichenketten der Länge 2 können aus den 26 Buchstaben des Alphabets gebildet werden?

Wenn wir mit $A = \{a, b, \dots, z\}$ die Menge der Alphabetsymbole bezeichnen, dann ist $A \times A$ die Menge der Zeichenketten der Länge 2.

$$A \times A = \{[a,a], [a,b], [a,c], \dots, [z,x], [z,y], [z,z]\}$$

Hier geht es darum, die Anzahl der Element dieser Menge zu bestimmen, also $|A \times A|$. Gemäß (10.5) gilt

$$|A \times A| = |A| \cdot |A| = |A|^2 = 26^2.$$

■

Beispiel 10.3: In Aufwärmübung 4 wird aus der Menge der Binärzahlen $\mathbb{B} = \{0, 1\}$ durch wiederholtes kartesisches Produkt die Menge der Zwölftupel gebildet. Für die Anzahl dieser Menge gilt gemäß (10.5):

$$\begin{aligned} &|\mathbb{B} \times \mathbb{B} \times \mathbb{B} \times \mathbb{B} \times \mathbb{B} \times \mathbb{B} \times \mathbb{B} \times \mathbb{B} \times \mathbb{B} \times \mathbb{B} \times \mathbb{B} \times \mathbb{B}| \\ &= |\mathbb{B}| \cdot |\mathbb{B}| \cdot |\mathbb{B}| \cdot |\mathbb{B}| \cdot |\mathbb{B}| \cdot |\mathbb{B}| \cdot |\mathbb{B}| \cdot |\mathbb{B}| \cdot |\mathbb{B}| \cdot |\mathbb{B}| \cdot |\mathbb{B}| \cdot |\mathbb{B}| \\ &= |\mathbb{B}|^{12} = 2^{12} \end{aligned}$$

■

10.3 Elementares Zählen

10.3.1 Umkehraufgaben

Nicht bei allen Zählaufgaben geht es darum eine Gesamtanzahl zu bestimmen. Manchmal muss von einer gegebenen Gesamtanzahl auf eine Teilanzahl zurückgeschlossen werden.

Beispiel 10.4: Ein kariertes Blatt Papier hat 108 Kästchen in 12 Reihen. Wie viele Spalten hat das Blatt?

Hier liegt eine multiplikative Fragestellung vor, denn die Anzahl der Kästchen ist das Produkt aus Anzahl von Reihen und Spalten. Gesucht ist jedoch nicht die Anzahl der Kästchen (also das

Produkt), sondern die Anzahl der Spalten. Daher ist eine Division, d. h. die Umkehroperation zur Multiplikation notwendig: Das Blatt hat $108/12 = 9$ Spalten. ■

Im vorangegangenen Beispiel wird die Gesamtanzahl durch multiplikatives Zählen aus den Teilanzahlen bestimmt. Allerdings geht es dort darum, aus der gegebenen Gesamtanzahl auf eine Teilanzahl zurückzuschließen. Es wird also die Umkehrung der Multiplikation, die Division, zum Zählen benötigt.

Entsprechend wird beim additiven Zählen die Subtraktion als Umkehrung der Addition benötigt, wenn von einer bekannten Gesamtanzahl auf eine Teilanzahl zurückgeschlossen werden soll.

Beispiel 10.5: In einem Raum sind 10 Studierende, unter ihnen sind vier keine Frauen. Wie viele Studierende sind Frauen?

Hier geht es um zwei disjunkte Mengen (Frauen und Nicht-Frauen) und die Gesamtanzahl von Personen ist die Mächtigkeit der Vereinigungsmenge dieser beiden Mengen. Es liegt also eine additive Zähl Aufgabe vor. Da hier von der Gesamtanzahl auf eine Teilanzahl zurückgeschlossen werden muss, ist die Subtraktion die benötigte Operation. Es sind $10 - 4 = 6$ Frauen im Raum. ■

10.3.2 Kleiner werdende Mengen

Oft haben Zählprobleme eher multiplikativen Charakter, sind also von der Form $|A \times B|$. Dabei gibt es drei wichtige Spezialfälle:

1. Die beiden involvierten Mengen sind disjunkt (so wie in Aufwärmübung 5).
2. Die beiden involvierten Mengen sind identisch (so wie in den Beispielen in Abschnitt 10.2.2).
3. Die eine involvierte Menge geht aus der anderen hervor.

Der dritte Spezialfall bildet die Grundlage für eine Reihe von Zählverfahren. Typischerweise unterscheiden sich die beteiligten Mengen dabei genau um ein Element.

Beispiel 10.6: Wie viele zweistellige Zahlen können aus der Menge $M = \{1 \dots 5\}$ gebildet werden, so dass eine Zahl nicht aus identischen Ziffern besteht?

„Schnapszahlen“ wie 11, 22 *etc.* sind also ausgeschlossen. Zuerst stellt sich die Frage, was das geeignete Modell für eine zweistellige Zahl ist: Tupel, Menge usw.? 

Diese Antwort auf die Modellierungsfrage sagt uns sofort, dass es um multiplikatives Zählen geht. Nun ist zu klären, welche Mengen involviert sind. Das ist zum einen die Menge M . Die Elemente dieser Mengen können verwendet werden, um die erste Ziffer der zweistelligen Zahl zu bilden. Nennen wir diese Ziffer k . Sie ist offensichtlich variabel, weil sie $|M| = 5$ mögliche Werte annehmen kann. Für die zweite Ziffer der zweistelligen Zahl steht k wegen der Anforderung „keine identischen Ziffern“ nicht mehr zur Verfügung. Die zweite Ziffer wird dann aus der Menge $N(k) = M \setminus \{k\}$ entnommen. Beachten Sie, dass diese Menge variabel ist. Für $k = 1$ ist $N(1) = \{2 \dots 5\}$, für $k = 2$ ist $N(2) = \{1, 3, 4, 5\}$ usw. In jedem Fall haben diese Mengen vier mögliche Elemente: $|N(k)| = 4$

Welchen Wert k auch immer hat, es gibt $|M \times N(k)|$ mögliche Ziffernpaare (also 2-Tupeln von Ziffern). Konkret sind es $|M \times N(k)| = |M| \cdot |N(k)| = 5 \cdot 4 = 20$ mögliche Ziffern.

Natürlich kann man den Bildungsprozess auch so durchführen, dass nicht die erste Ziffer der zweistelligen Zahl aus den Elementen von M gebildet wird, sondern die zweite Ziffer. Denken Sie die Zähl Aufgabe auch unter diesem Prozess durch. ■

Das abstrakte Zählmuster ist hier: Ausgehend von einer Menge M geht es um die Menge von Paaren mit nicht identischen Elementen. Dazu gibt es $|M| \cdot (|M| - 1)$ viele Möglichkeiten.

Beispiel 10.7: Lassen Sie uns mittels *setlX* die Menge S der möglichen Ziffern aus dem vorangegangenen Beispiel berechnen und anschließend zählen:

```
=> M:={1..5};
~< Result: {1, 2, 3, 4, 5} >~
=> S:={[k,1]: k in M, 1 in M-{k} };
~< Result: {[1, 2], [1, 3], [1, 4], [1, 5], [2, 1], [2, 3], [2, 4], [2, 5],
```

```
[3, 1], [3, 2], [3, 4], [3, 5], [4, 1], [4, 2], [4, 3], [4, 5], [5, 1],
[5, 2], [5, 3], [5, 4]} >~
=> #S;
~< Result: 20 >~
```

Es sind also tatsächlich 20 Ziffern - wie oben berechnet, dort allerdings *ohne* die Menge vorher aufzustellen.

Wenn Sie statt Ziffernpaare lieber zweistellige Zahlen sehen wollen, müssen Sie stattdessen Zahlen im Mengenkonstruktor „herstellen“ lassen:

```
=> S:={10*k+l: k in M, l in M-{k} };
```

Wie immer gibt es auch hier mehr als einen Weg zum Ziel. Mögliche alternative Mengenkonstrukturen sind z.B. $\{[k, l] : k \in M, l \in M \mid k \neq l\}$ und $\{z : z \in M \times M \mid z_1 \neq z_2\}$. ■

10.3.3 Permutationen

Setzt man das Zählmuster aus dem vorangegangenen Abschnitt fort (im ersten Schritt ist jedes Element einer Menge M möglich, also $|M|$ Möglichkeiten, im zweiten Schritt gibt es eine Möglichkeit weniger, also $|M| - 1$ Möglichkeiten) kommt man zu Situationen, bei denen es im dritten Schritt wieder eine Möglichkeit weniger gibt, also $|M| - 2$, im vierten Schritt wieder eine weniger usw., bis es am Ende nur noch eine Möglichkeit gibt.

Beispiel 10.8: Auf wie viele Arten kann man die Ziffern von 1 bis 5 hintereinander schreiben?

Ein geeignetes Modell für die entstehenden Ziffernfolgen 12345, 12354, 12534 usw. sind 5-Tupel. Überlegen Sie sich, warum dieses Modell geeignet ist!



Es geht also um multiplikatives Zählen: Für die erste Komponente solcher 5-Tupel gibt es fünf mögliche Ziffern, für die zweite nur noch vier, für die dritte nur noch drei, für die vorletzte zwei und für die letzte nur noch eine mögliche Ziffer. Es gibt also $5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 5! = 120$ viele Möglichkeiten fünf unterschiedliche Ziffern hintereinander zu schreiben. ■

Das allgemeine Zählmuster ist „multiplikativ und in jedem Schritt eine Möglichkeit weniger“ ausgehend von einer Menge M . Die gesuchte Anzahl bei diesem Zählmuster ist

$$|M| \cdot (|M| - 1) \cdot (|M| - 2) \cdot \dots \cdot 2 \cdot 1 = |M|!, \quad (10.6)$$

was bequem mit Hilfe der Fakultät aus (10.1) ausgedrückt werden kann.

Die Datenstruktur, um die es hier geht, ist die *Permutation*. Eine Permutation (lateinisch für Vertauschung) ist eine Anordnung aller Elemente einer Menge in einer bestimmten Reihenfolge. Im obigen Beispiel wurden alle Elemente der Menge $\{1 \dots 5\}$ zu einer fünfstelligen Zahl aneinander gereiht.

Beispiel 10.9: In *setlX* kann für eine Menge M die Menge aller Permutationen mit `permutations(M)` berechnet werden. Wir tun dies für die Menge $\{„a“, „b“, „c“, „d“\}$, erzeugen also alle möglichen Zeichenketten der Länge vier, die aus genau diesen vier Buchstaben bestehen:

```
=> M:={„a“, „b“, „c“, „d“};
~< Result: {„a“, „b“, „c“, „d“} >~
=> P:=permutations(M);
~< Result: {[„a“, „b“, „c“, „d“], [„a“, „b“, „d“, „c“], [„a“, „c“, „b“, „d“],
[„a“, „c“, „d“, „b“], [„a“, „d“, „b“, „c“], [„a“, „d“, „c“, „b“], [„b“, „a“, „c“, „d“],
[„b“, „a“, „d“, „c“], [„b“, „c“, „a“, „d“], [„b“, „c“, „d“, „a“], [„b“, „d“, „a“, „c“],
[„b“, „d“, „c“, „a“], [„c“, „a“, „b“, „d“], [„c“, „a“, „d“, „b“], [„c“, „b“, „a“, „d“],
[„c“, „b“, „d“, „a“], [„c“, „d“, „a“, „b“], [„c“, „d“, „b“, „a“], [„d“, „a“, „b“, „c“],
[„d“, „a“, „c“, „b“], [„d“, „b“, „a“, „c“], [„d“, „b“, „c“, „a“], [„d“, „c“, „a“, „b“],
[„d“, „c“, „b“, „a“]} >~
=> #P;
~< Result: 24 >~
```

Wegen $|M| = 4$ gibt es $4! = 24$ mögliche Permutationen. ■

10.4 Kombinatorisches Modellieren

Beim additiven Zählen wird die Anzahl der Elemente in einer Menge bestimmt. Ebenso beim multiplikativen Zählen, dort sind die Elemente der Mengen allerdings Kollektionen. Bei Mr. X war eine Kollektion eine Hemd-Krawatten-Kombination. In Beispiel 10.8 in Abschnitt 10.3.3 war eine Kollektion eine Permutation von fünf unterschiedlichen Ziffern.

Es lohnt sich einen modellierenden Blick auf diese Situationen zu werfen und zwar im Lichte der Kategorien aus Abschnitt 1.4: Kollektionen, bei denen die Reihenfolge der Bestandteile (k) eine Rolle spielt; Kollektionen, bei denen ein Vielfachvorkommen der Bestandteile (k) eine Rolle spielt. Gehen Sie dazu alle bisherigen Beispiele zum multiplikativen Zählen in diesem Kapitel durch und klassifizieren Sie sie anhand des Schemas aus Abschnitt 1.4. Nutzen Sie ggf. die Kategorie „Andere“ für Beispiele, die nicht in dieses Klassifikationsschema passen.

geeignetes Modell	Reihenfolge ist relevant	Reihenfolge ist nicht relevant
Mehrfachvorkommen ist möglich oder relevant		
Mehrfachvorkommen ist nicht möglich oder relevant	 Beispiel 10.8	

Andere: 

Bei den Zählproblemen, die in die Kategorien dieses Modellierungsschemas aus Abschnitt 1.4 passen, werden die Kollektionen immer dadurch gebildet, dass aus einer *einzigen* Menge mit n Objekten k ausgewählt werden. Beispielsweise werden bei den Permutationen in Abschnitt 10.3.3 aus einer Menge von $n = 5$ Ziffern $k = 5$ von diesen ausgewählt und in eine Kollektion gepackt, bei der die Reihenfolge der Ziffern relevant ist, aber kein Mehrfachvorkommen möglich ist.

Dieser Bildungsprozess einer Kollektion wird oft als *Urnenmodell* bezeichnet: Aus einer Urne mit n Objekten werden k ausgewählt und in der Kollektion platziert. Dabei kann die Reihenfolge wichtig sein oder nicht. Dabei kann ein Objekt mehrfach ausgewählt werden oder nicht. Mehrfachauswahl können Sie sich so vorstellen, dass das Objekt aus der Urne entnommen wird, eine Kopie erstellt wird, diese in der Kollektion platziert wird und anschließend das Original in die Urne zurück gelegt wird, so dass es wieder ausgewählt werden kann.

Für Zählaufgaben gibt es aufgrund unseres Kategorienschemas also vier unterscheidbare Möglichkeiten, um k Objekte aus n möglichen Objekten auszuwählen, die in der folgenden Tabelle aufgelistet sind. Dabei ist auch jeweils angegeben, wie das Zählergebnis berechnet werden kann.

Anzahl der Möglichkeiten, um k Objekte aus n möglichen Objekten auszuwählen	Reihenfolge der ausgewählten Objekte ist relevant	Reihenfolge der ausgewählten ist nicht relevant
Mehrfachvorkommen der ausgewählten Objekte ist möglich oder relevant („mit Zurücklegen“)	n^k	$\binom{n+k-1}{k}$
Mehrfachvorkommen der ausgewählten Objekte ist nicht möglich oder relevant („ohne Zurücklegen“)	$\frac{n!}{(n-k)!}$	$\binom{n}{k}$

Im folgenden schauen wir uns jeden dieser Fälle einzeln an und leiten das Zählergebnis her. Zuvor muss noch betont werden, dass dieses Schema natürlich nicht alle denkbaren multiplikativen Zählverfahren berücksichtigt. Es ist ja gerade durch das Urnenmodell gekennzeichnet, wo Objekte aus *einer* Urne entnommen werden. Schon das Kleidungsproblem von Mr. X ist nicht mehr (alleine) mittels dieses Modells beschreibbar, denn dort gibt es quasi zwei Urnen: eine für die Hemden, eine für die Krawatten.

10.4.1 Reihenfolge und Mehrfachvorkommen relevant

Aufwärmübung 4 ist ein prototypisches Beispiel für diese Kategorie. Dort sind die Kollektionen 12-Tupel. Mittels Urnenmodell betrachtet, lagen dort $n = 2$ Elemente in der Urne und diese wurden mehrfach - insgesamt zwölfmal, also $k = 12$ - entnommen. Für die erste Tupelkomponente gibt es $n = 2$ Möglichkeiten, diese zu belegen, für die zweite ebenfalls. Damit gibt es insgesamt schon n^2 viele Möglichkeiten die ersten beiden Tupelkomponenten zu belegen. Für die dritte Tupelkomponente gibt es wieder n Möglichkeiten und so insgesamt n^3 viele Möglichkeiten für die ersten drei Komponenten zusammen. So geht es weiter bis zur letzten, der k -ten Komponente. Insgesamt gibt es als n^k viele Möglichkeiten, k aus n Objekten auszuwählen, wenn Reihenfolge und Mehrfachvorkommen relevant sind.

Beispiel 10.10: Wie viele Möglichkeiten gibt es aus den zehn Dezimalziffern (maximal) dreistellige Zahlen zu bilden?

Intuitiv ist Ihnen vermutlich klar, dass es 1000 Möglichkeiten gibt (die Zahlen von 0 bis 999). Die Kombinatorik führt zum selben Resultat: Bei den gebildeten Kollektionen handelt es sich um Zahlen, die am geeignetsten als Tupel von Ziffern modelliert werden. In der Urne liegen $n = 10$ Ziffern, von denen $k = 3$ unter Beachtung der Reihenfolge und mit möglicher Mehrfachauswahl ausgewählt werden. Es gibt also $n^k = 10^3 = 1000$ mögliche dreistellige Dezimalzahlen. ■

Beispiel 10.11: Wie viele Elemente hat die Potenzmenge von $\{\circ, \triangle, \square\}$?

Hier sind die Kollektionen, deren Anzahl gezählt wird, Teilmengen von $\{\circ, \triangle, \square\}$. Jede Teilmenge kann quasi als Tupel kodiert werden. Beispielsweise kodiert das Tupel $[1, 1, 0]$ die Menge $\{\circ, \triangle\}$, das Tupel $[0, 0, 1]$ die Menge $\{\square\}$ und $[0, 0, 0]$ die leere Menge. Die Position im Tupel besagt, ob es sich um $\circ$, $\triangle$ oder $\square$ handelt, und 0 bzw. 1 kodieren, ob die Aussageform „Symbol ist Element der Teilmenge“ wahr ist.

In der Urne liegen $n = 2$ Objekte (0 und 1) und hier werden $k = 3$ dieser Objekte entnommen um zu kodieren, welche Elemente die zu bildende Teilmenge hat. Es gibt also $n^k = 2^3$ Möglichkeiten. Die Potenzmenge von $\{\circ, \triangle, \square\}$ hat also acht Elemente.

Im Allgemeinen hat eine Menge S nicht nur drei, sondern $|S|$ viele Elemente. Dann ist $k = |S|$ und es gibt $2^{|S|}$ mögliche Teilmengen von S - in Übereinstimmung mit Abschnitt 5.8. ■

10.4.2 Reihenfolge relevant, Mehrfachvorkommen nicht relevant

Da Mehrfachvorkommen nicht möglich ist, können aus der Urne natürlich maximal so viele Objekte entnommen werden, wie anfangs da sind. Es muss also $k \leq n$ gelten.

Der Spezialfall $k = n$, wenn also alle Objekte entnommen werden, führt zu den Permutationen der n Objekte. Aus Abschnitt 10.3.3 wissen Sie bereits, dass es dafür $n!$ viele Möglichkeiten gibt.

Wenn $k < n$, also nicht alle Objekte aus der Urne genommen werden, können wir dennoch das Zählverfahren aus Abschnitt 10.3.3 übernehmen: Um das erste Objekt zu entnehmen, gibt es n Möglichkeiten, für das zweite nur noch $n - 1$, für das dritte nur noch $n - 2$ usw. Für das k -te Objekt gibt es schließlich nur noch $n - k + 1$ viele Möglichkeiten. Insgesamt haben wir daher

$$n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot (n - k + 1) \quad (10.7)$$

Möglichkeiten, k aus n Objekten mit Beachtung der Reihenfolge, aber ohne Mehrfachauswahl auszuwählen.

Der Ausdruck (10.7) ist recht sperrig. Um ihn kompakter zu schreiben, erweitern wir ihn mit $(n - k)!$:

$$\begin{aligned} & n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot (n - k + 1) \\ = & n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot (n - k + 1) \cdot \frac{(n - k)!}{(n - k)!} \\ = & \frac{n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot (n - k + 1) \cdot (n - k)!}{(n - k)!} \\ = & \frac{n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot (n - k + 1) \cdot (n - k) \cdot (n - k - 1) \cdot \dots \cdot 2 \cdot 1}{(n - k)!} \\ = & \frac{n!}{(n - k)!} \end{aligned} \quad (10.8)$$

Sinn und Zweck dieser Erweiterung ist, im Zähler $n!$ zu erzeugen. Damit erhalten wir das kompakte Ergebnis, dass es $n!/(n-k)!$ Möglichkeiten gibt, k aus n Objekten mit Beachtung der Reihenfolge, aber ohne Mehrfachauswahl auszuwählen.

Wie immer gibt es viele Wege, um zum Ergebnis, hier $n!/(n-k)!$, zu kommen. Ein alternativer Weg nutzt die Erkenntnisse von Abschnitt 10.3.1 aus und betrachtet die Fragestellung als Lösung einer Umkehraufgabe.

Stellen wir uns dazu vor, wir entnehmen im ersten Schritt k aus den n Objekten ohne Mehrfachauswahl und unter Berücksichtigung der Reihenfolge. Die Anzahl der Möglichkeiten m wollen wir bestimmen. Dann entnehmen wir auf dieselbe Art und Weisen die restlichen Objekte - $(n-k)$ an der Zahl - und platzieren sie hinter die zuerst ausgewählten k Objekte. Mit diesem zweiten Schritt bilden wir Permutationen der $n-k$ Objekte. Dafür gibt es $(n-k)!$ viele Möglichkeiten.

Um die Schritte 1 und 2 hintereinander auszuführen, haben wir einerseits so viele Möglichkeiten wie das Produkt der Möglichkeiten in den Einzelschritten, also $m \cdot (n-k)!$. Andererseits bilden wir durch Hintereinanderausführen der Schritte 1 und 2 Permutationen aller n Objekte. Dafür gibt es $n!$ Möglichkeiten. Es muss also gelten

$$m \cdot (n-k)! = n!$$

Auflösen nach der gesuchten Anzahl m führt zum Ergebnis $n!/(n-k)!$. Das ist genau das Zählergebnis aus (10.8).

Beispiel 10.12: Wie viele mögliche Medaillenverteilungen gibt es bei einem Mannschaftsspielwettbewerb, an dem 32 Teams teilnehmen?

32 Teams können die Goldmedaille gewinnen. Wenn die Goldmedaille festliegt, gibt es noch 31 Möglichkeiten für Silber und 30 für Bronze. Insgesamt also $32 \cdot 31 \cdot 30$ Möglichkeiten.

Zum selben Ergebnis führt natürlich die Formel mit $n = 32$ und $k = 3$:

$$\frac{n!}{(n-k)!} = \frac{32!}{(32-3)!} = \frac{32!}{29!} = \frac{32 \cdot 31 \cdot 30 \cdot 29!}{29!} = 32 \cdot 31 \cdot 30$$

■

10.4.3 Reihenfolge nicht relevant, Mehrfachvorkommen nicht relevant

Nun haben wir es zum ersten Mal mit einer Situation zu tun, bei der die Reihenfolge der Objekte keine Rolle spielt. Nennen wir m die gesuchte Anzahl der Möglichkeiten, k aus n Objekten ohne Beachten der Reihenfolge und ohne mögliches Mehrfachvorkommen aus der Urne zu ziehen.

In jedem dieser m Fälle haben wir k Objekte „auf dem Tisch liegen“. Wenn wir diese nun doch noch in eine Reihenfolge bringen wollen (also zur Situation aus dem vorangehenden Abschnitt 10.4.2 zurückkehren), haben wir $k!$ viele Möglichkeiten dies zu tun. Am Ende haben wir also $m \cdot k!$ Möglichkeiten k Objekte ohne Mehrfachvorkommen, aber *mit* Beachten der Reihenfolge auszuwählen. Aus 10.4.2 wissen wir, dass diese Anzahl $n!/(n-k)!$ ist:

$$m \cdot k! = \frac{n!}{(n-k)!}$$

Daraus können wir die gesuchte Anzahl m der Möglichkeiten bestimmen, um k aus n Objekten ohne Mehrfachvorkommen und *ohne* Beachten der Reihenfolge auszuwählen. Es gibt also

$$\frac{n!}{k! \cdot (n-k)!} \quad (10.9)$$

solche Möglichkeiten.

Der etwas sperrige Ausdruck in (10.13) wird abkürzend als

$$\binom{n}{k} = \frac{n!}{k! \cdot (n-k)!} \quad (10.10)$$

geschrieben und als *Binomialkoeffizient* bezeichnet. Das Symbol $\binom{n}{k}$ wird üblicherweise „ n über k “ oder „ k aus n “ ausgesprochen.

Weitere Eigenschaften von Binomialkoeffizienten werden in Abschnitt 10.6.2 diskutiert.

Beispiel 10.13: Das klassische Beispiel für eine Auswahl ohne Beachten der Reihenfolge und ohne Mehrfachvorkommen ist die Lotterie „6 aus 49“. Beachten Sie die Bezeichnung, die wie die Aussprache des Binomialkoeffizient $\binom{49}{6}$ klingt.

Hier ist $n = 49$ und $k = 6$. Damit gibt es

$$\binom{49}{6} = \frac{49!}{6! \cdot (49 - 6)!} = \frac{49!}{6! \cdot 43!} = 13983816$$

unterschiedliche Lottoergebnisse. ■

10.4.4 Reihenfolge nicht relevant, Mehrfachvorkommen relevant

Hier könnte man sich die Situation so vorstellen, dass aus einer Urne mit n Objekten k viele entnommen werden - ohne Relevanz der Reihenfolge, aber mit der Möglichkeit der Mehrfachentnahme. Schauen wir uns dies zunächst an einem einfachen, überschaubaren Beispiel an. Das ist ja eine bewährte Strategie, um eine Vorstellung von einer Problemstellung zu bekommen.

Wir legen in die Urne die Ziffern 1, 2 und 3 (also $n = 3$) und entnehmen zweimal unter möglicher Mehrfachauswahl eine Ziffer (also $k = 2$). Folgende Ziffernkombinationen sind möglich: 11, 12, 13, 22, 23, 33.

Beachten Sie, dass in jeder Auswahlmöglichkeit die Ziffern *sortiert* aufgeschrieben wurden. 12 bedeutet aber bspw. nicht, dass die 1 vor der 2 aus der Urne entnommen wurde. Auch wenn die Reihenfolge der Entnahme hier keine Bedeutung hat, müssen wir die Ziffern in irgend einer Weise sortiert aufschreiben.

Wir haben also sechs mögliche Fälle. Aber wir wollen die Anzahl ja nicht durch Auflisten aller Fälle und anschließendes Abzählen bestimmen, sondern durch Berechnen. Das Urnenmodell ist für diese Situation (zunächst) nicht besonders geeignet. Statt den Fokus auf die Entnahme aus der Urne zu legen, schauen wir lieber wieder auf eine geeignete Datenstruktur, die wir zum Repräsentieren der Auswahlmöglichkeiten verwenden können. Eine geeignete Datenstruktur (bzw. Code) besteht aus drei Schubladen mit den Aufschriften 1, 2 und 3 und Kreisen, die Ziffern darstellen. Liegt ein Kreis in Schublade 1, stellt er eine 1 dar usw. Kennzeichnen wir die Schubladentrennwände mit |, können die obigen sechs Auswahlmöglichkeiten so kodiert werden:

$$\begin{array}{lll} 11 & \hat{=} & \circ \circ || \\ 22 & \hat{=} & | \circ \circ | \\ 12 & \hat{=} & \circ | \circ | \\ 23 & \hat{=} & | \circ | \circ \\ 13 & \hat{=} & \circ || \circ \\ 33 & \hat{=} & || \circ \circ \end{array}$$

In jedem Fall besteht der Code aus vier Symbolen, darunter zwei Trennstriche. Um die möglichen Codes zu erzeugen, können wir so vorgehen, dass wir „zwei aus vier“ Plätzen mit einem Trennstrich belegen. Dafür gibt es $\binom{4}{2} = 6$ Möglichkeiten - in Übereinstimmung mit der Anzahl der eben aufgelisteten Ziffernkombinationen.

Dieses Vorgehen lässt sich recht leicht verallgemeinern. Haben wir k Objekte, benötigen wir eben so viele Kreise. Sollen k Objekte (ohne Relevanz der Reihenfolge, aber mit möglicher Mehrfachauswahl) ausgewählt werden, brauchen wir n Schubladen, d. h. $n - 1$ Trennstriche als Schubladentrennwände. Unsere Codes bestehen insgesamt aus $n + k - 1$ Symbolen, von denen k mit Objekten belegt werden müssen. Dies ergibt

$$\binom{n + k - 1}{k} \tag{10.11}$$

Möglichkeiten Objekte gemäß vorliegender Spezifikation auszuwählen.

Zur Herleitung von (10.11) haben wir das Modell der Auswahl *aus* einer Urne zu Gunsten der Vorstellung der Verteilung *auf* Schubladen aufgeben. Letzteres ist eine mögliche Alternative auch in allen anderen Fällen, die wir oben mittels Urnenmodell modelliert hatten. Details dazu können Sie in Abschnitt 10.6.1 studieren.

Beispiel 10.14: Bei einer Tombola werden drei baugleiche Smartphones verlost. Insgesamt haben zehn Personen Lose gekauft (was nicht ausschließt, dass eine Person mehr als ein Los gekauft hat und daher auch mehr als einmal gewinnen kann). Wie viele mögliche Preisverteilungen gibt es?

Aus $n = 10$ Personen werden $k = 3$ Gewinner ausgewählt. Mehrfachgewinn ist möglich. Es werden quasi drei aus zehn Personen mit Mehrfachauswahl aus einer Urne gezogen. Die Reihenfolge der Gewinner ist nicht relevant, weil jeder den gleichen Preis bekommt. Es gibt also

$$\binom{n+k-1}{k} = \binom{10+3-1}{3} = \binom{12}{3} = 220$$

mögliche Gewinnerkonstellationen. ■

10.5 Mit dem Rechner zählen

Nicht immer ist es möglich mittels einer der in diesem Kapitel diskutierten Verfahren zu zählen. Dies ist dann der Fall, wenn sich die Situation nicht auf eine der Modellsituationen abbilden lässt. Dann bleibt einem nichts anderes übrig als zuerst alle Möglichkeiten „aufzuschreiben“ und diese anschließend zu zählen. Wenn die Anzahl groß ist, ist Rechnerunterstützung dabei natürlich mehr als angebracht.

Beispiel 10.15: Die Quersumme wievieler Primzahlen kleiner 1000 ist selbst wieder eine Primzahl?

Auch hier geht es um die Bestimmung der Mächtigkeit einer Menge. Es ist allerdings nicht offensichtlich, dass es sich bei der Menge um die Vereinigung zweier Mengen oder um das Kreuzprodukt zweier Mengen handelt. Somit helfen weder die additiven noch die multiplikativen Zählverfahren. Es bleibt also nichts anderes übrig als die Menge explizit hinzuschreiben und die Anzahl ihrer Elemente zu zählen. Der folgende *setlX*-Code tut dies:

```
=> quersumme:=procedure(n){
  if(n==0){
    return 0;
  } else {
    letzteziffer:=n%10;
    restziffern:=(n-letzteziffer)/10;
    return letzteziffer + quersumme(restziffern);
  }
};
=> gesucht:={n:n in {1..999}|isPrime(n) && isPrime(quersumme(n))};
~< Result: {2, 3, 5, 7, 11, 23, 29, 41, 43, 47, 61, 67, 83, 89, 101, 113,
131, 137, 139, 151, 157, 173, 179, 191, 193, 197, 199, 223, 227, 229, 241,
263, 269, 281, 283, 311, 313, 317, 331, 337, 353, 359, 373, 379, 397, 401,
409, 421, 443, 449, 461, 463, 467, 487, 557, 571, 577, 593, 599, 601, 607,
641, 643, 647, 661, 683, 719, 733, 739, 751, 757, 773, 797, 809, 821, 823,
827, 829, 863, 881, 883, 887, 911, 919, 937, 953, 971, 977, 991} >~
=> #gesucht;
~< Result: 89 >~
```

Es gibt also 89 Primzahlen kleiner 1000, deren Quersumme wieder eine Primzahl ist. ■

Für Sie bietet Zählen mit dem Rechner die gute Möglichkeit, berechnete Anzahlen durch konkretes Zählenlassen auf Korrektheit zu überprüfen.

Beispiel 10.16: Wir überprüfen die Korrektheit der im Beispiel 10.14 berechneten Anzahl der Tombolagewinnerverteilungen, indem wir per Programm alle möglichen Gewinnerverteilungen erzeugen und diese anschließend zählen. Eine Gewinnerverteilung besteht hier aus den Namen der drei Gewinner. Dabei kommt es auf die Reihenfolge der Gewinner nicht an (wie bei einer Menge), aber Mehrfachvorkommen eines Gewinners ist möglich (wie bei einem Tupel). In der Tabelle der

elementaren Kollektionen aus Abschnitt 1.4 sind wir also in der rechten oberen Ecke, ebenso wie die Zähltaufgabe in der rechten oberen Ecke der Tabelle auf S. 153 verortet ist.

Wir können solche Kollektionen in *setlX* durch Tupel repräsentieren, bei denen die für Tupel eigentlich vorliegende Relevanz der Reihenfolge der Komponenten durch Sortieren zerstört wird. Beispielsweise werden die unterscheidbaren Tupel $[2, 1, 3]$ und $[3, 1, 2]$ beide durch Sortieren zu $[1, 2, 3]$. Sie sind dadurch nicht mehr unterscheidbar. Die Reihenfolge hat durch Sortieren die Relevanz verloren.

Hier ist ein mögliches Programm zum Erzeugen aller möglichen Gewinnerverteilungen:

```
=> teilnehmer := {0..9};
=> gewinner := {sort([g1,g2,g3]): g1 in teilnehmer, g2 in teilnehmer, g3 in
    teilnehmer};
=> #gewinner;
~< Result: 220 >~
```

Die in Beispiel 10.14 berechnete Anzahl ist also korrekt. ■

Beispiel 10.17: Wir überprüfen die Korrektheit der Anzahl der möglichen Medaillenverteilungen aus dem Beispiel in Beispiel 10.12. Die Kollektionen sind hier Medaillenverteilungen. Ein geeignetes Modell ist das Tupel, das Gold-, Silber- und Bronzemedailengewinner in dieser Reihenfolgen nennt.

```
=> teams:={1..32};
=> gewinner:={ [g,s,b]: g in teams, s in teams-{g}, b in teams-{g,s}};
=> #gewinner==32*31*30;
~< Result: true >~
```

Die in Beispiel 10.12 berechnete Anzahl ist also korrekt. ■

Sie können nicht nur berechnete Anzahlen per Programm auf Korrektheit überprüfen, sondern auch umgekehrt die Korrektheit von Aufzählungen dadurch auf Korrektheit überprüfen, indem Sie die Anzahl berechnen. Unterscheidet sich die Anzahl von dem, was Sie mit dem Rechner aufgezählt haben, von der berechneten Anzahl, liegt ein Fehler vor.

10.6 Empfohlene Vertiefungen

10.6.1 Verteilen statt Auswählen

In Abschnitt 10.4 wurde die Modellvorstellung des Auswählens von Objekten aus einem Container („Urnenmodell“) verwendet, um kombinatorische Situationen zu klassifizieren und geeignete Zählverfahren abzuleiten. Das ist eine populäre Modellvorstellung, aber nicht die einzig mögliche.

Die Grenzen des Urnenmodells wurden insbesondere bei der Modellierung von Situationen der Kategorie „Reihenfolge nicht relevant, Mehrfachvorkommen relevant“ in Abschnitt 10.4.4 sichtbar. Dort sind wir von der Modellvorstellung der Auswahl aus einer Urne auf die Modellvorstellung „Verteilen in Schubladen“ gewechselt. In diesem Abschnitt soll diese Modellvorstellung als Alternative zum Urnenmodell diskutiert werden.

Beim Urnenmodell werden k aus n Objekten ausgewählt und anschließend - u. U. mehrfach - in einem Tupel, einer Menge usw. platziert. Beim „Schubladenmodell“ werden k Objekte auf Schubladen verteilt, wobei u. U. Mehrfachbelegung möglich ist. Der Reihenfolge beim Urnenmodell entspricht die Unterscheidbarkeit beim Schubladenmodell. Die folgende Tabelle kategorisiert die möglichen Fälle und nennt die Zählergebnisse:

Anzahl der Möglichkeiten, um k Objekte auf n Schubladen zu verteilen	Objekte sind unterscheidbar	Objekte sind nicht unterscheidbar
Mehrfachbelegung ist möglich oder relevant	n^k	$\binom{n+k-1}{k}$
Mehrfachbelegung ist nicht möglich oder relevant	$\frac{n!}{(n-k)!}$	$\binom{n}{k}$

Die Äquivalenz der einzelnen Fälle in Urnen- und Schubladenmodell werden im folgenden fallweise erläutert.

Unterscheidbare Objekte bei möglicher Mehrfachbelegung

Wenn k unterscheidbare Objekte auf n Schubladen verteilt werden, gibt es für jedes dieser Objekte n mögliche Schubladen. Insgesamt gibt es daher n^k Möglichkeiten die Objekte zu verteilen.

Abb. 10.1 visualisiert dies für $n = 5$ und $k = 3$, links für das Urnenmodell, rechts für das Schubladenmodell.

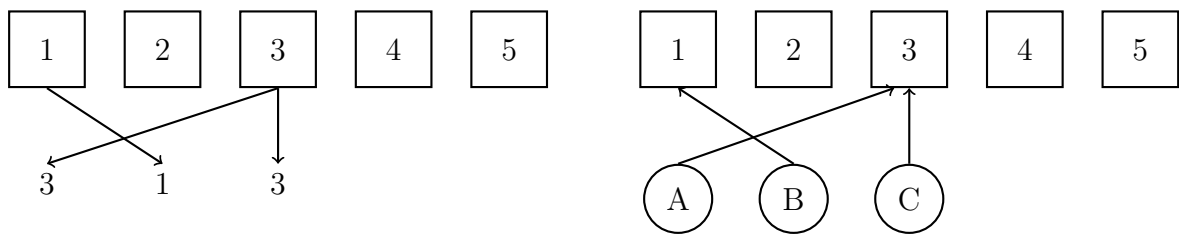


Abbildung 10.1: Gegenüberstellung von Urnenmodell (links) und Schubladenmodell (rechts) für das Verteilen von drei unterscheidbaren Objekten bei möglicher Mehrfachbelegung der Schubladen.

Unterscheidbare Objekte ohne Mehrfachbelegung

In diesem Fall gibt es für das erste Objekt n mögliche Schubladen, wegen ausgeschlossener Mehrfachbelegung einer Schublade für das zweite Objekt nur noch $n - 1$ viele Schubladen usw. Werden k Objekte verteilt, gibt es $n!/(n-k)!$ viele Möglichkeiten dies zu tun.

Abb. 10.2 visualisiert dies für $n = 5$ und $k = 2$, links für das Urnenmodell, rechts für das Schubladenmodell.

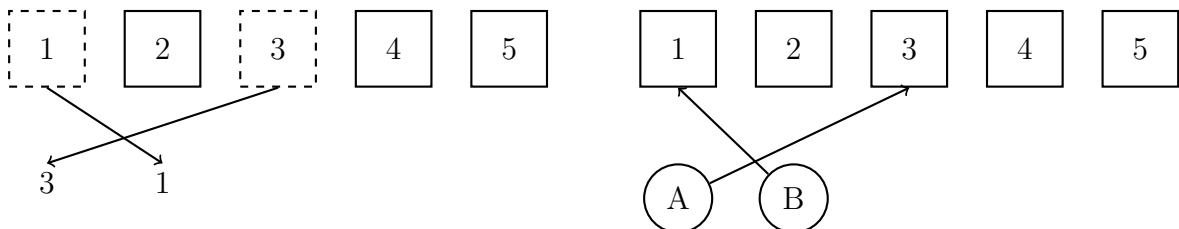


Abbildung 10.2: Gegenüberstellung von Urnenmodell (links) und Schubladenmodell (rechts) für das Verteilen von zwei unterscheidbaren Objekten ohne mögliche Mehrfachbelegung der Schubladen.

Ununterscheidbare Objekte ohne Mehrfachbelegung

Wenn im Vergleich zum vorausgehenden Fall die zu verteilenden k Objekte nicht mehr unterscheidbar sind, fallen $k!$ Schubladenverteilung zu einer einzigen zusammen. Also gibt es nur noch

$$\frac{n!}{k!(n-k)!} = \binom{n}{k}$$

mögliche Schubladenverteilungen.

Abb. 10.4 visualisiert dies für $n = 5$ und $k = 2$, links für das Urnenmodell, rechts für das Schubladenmodell.

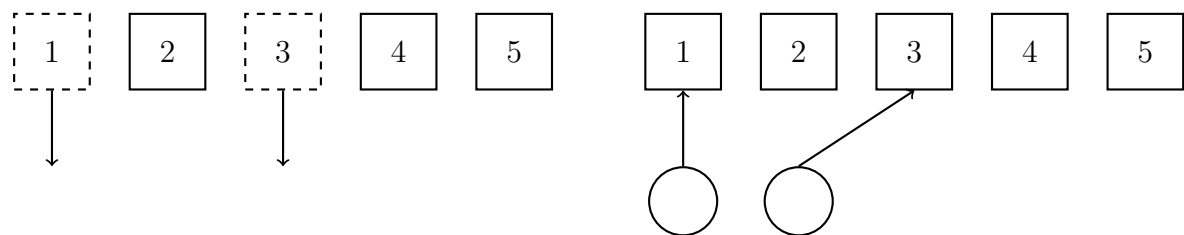


Abbildung 10.3: Gegenüberstellung von Urnenmodell (links) und Schubladenmodell (rechts) für das Verteilen von zwei nichtunterscheidbaren Objekten ohne mögliche Mehrfachbelegung der Schubladen.

Ununterscheidbare Objekte bei möglicher Mehrfachbelegung

Dieser Fall wurde schon in Abschnitt 10.4.4 aus der Sicht des Schubladenverteilungsmodells behandelt. Abb. 10.3 stellt Urnenmodell (links) und Schubladenmodell (rechts) für $n = 5$ und $k = 3$ nebeneinander.

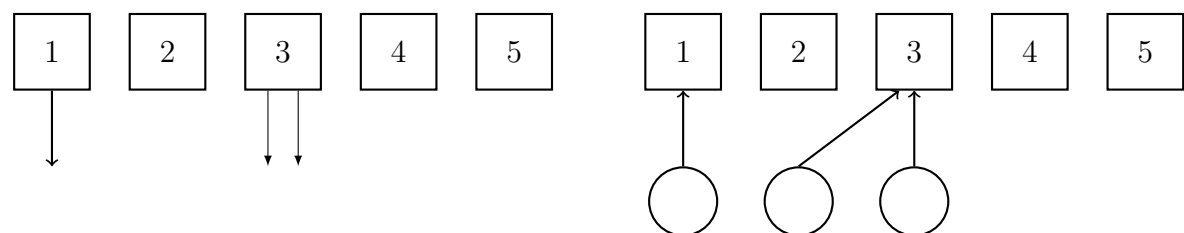


Abbildung 10.4: Gegenüberstellung von Urnenmodell (links) und Schubladenmodell (rechts) für das Verteilen von drei nichtunterscheidbaren Objekten bei möglicher Mehrfachbelegung der Schubladen.

10.6.2 Binomialkoeffizient

Binomialkoeffizienten sind nicht nur für das Zählen von Bedeutung, konkret für Situationen der Art „Mehrfachvorkommen und Reihenfolge nicht relevant“, sondern auch in anderen Kontexten. Namensgebend ist die binomische Reihe. Bei Binom denken Sie wahrscheinlich an die binomische Formel

$$(a + b)^2 = a^2 + 2ab + b^2. \quad (10.12)$$

In ihr sind Binomialkoeffizienten versteckt und zwar in der Form

$$(a+b)^2 = \binom{2}{0}a^2 + \binom{2}{1}a^1b^1 + \binom{2}{2}b^2 \quad (10.13)$$

bzw.

$$(a+b)^2 = \sum_{i=0}^2 \binom{2}{i} a^i b^{2-i}. \quad (10.14)$$

Testen Sie die Richtigkeit von (10.14), indem Sie auf (10.13) und dann auf (10.12) zurückrechnen!

Das Interessante an (10.14) ist, dass es nicht nur für Quadrate, sondern für allgemeine natürlich-zahlige Potenzen gilt:

$$(a+b)^n = \sum_{i=0}^n \binom{n}{i} a^i b^{n-i}. \quad (10.15)$$

Das ist die sogenannte *binomische Reihe*, die namensgebend für die Binomialkoeffizienten ist. Sie besagt für $n = 3$

$$\begin{aligned} (a+b)^3 &= \sum_{i=0}^3 \binom{3}{i} a^i b^{3-i} \\ &= \binom{3}{0} a^3 b^0 + \binom{3}{1} a^2 b^1 + \binom{3}{2} a^1 b^2 + \binom{3}{3} a^0 b^3 \\ &= a^3 + 3a^2b + 3ab^2 + b^3, \end{aligned}$$

für $n = 4$

$$\begin{aligned} (a+b)^4 &= \sum_{i=0}^4 \binom{4}{i} a^i b^{4-i} \\ &= \binom{4}{0} a^4 b^0 + \binom{4}{1} a^3 b^1 + \binom{4}{2} a^2 b^2 + \binom{4}{3} a b^3 + \binom{4}{4} a^0 b^4 \\ &= a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + b^4 \end{aligned}$$

usw.

Die Binomialkoeffizienten für benachbarte Werte von n hängen miteinander zusammen:

$$\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1} \quad (10.16)$$

Beispielsweise ist für $n = 3, k = 1$:

$$\begin{aligned} \binom{3}{1} + \binom{3}{2} &= \binom{4}{2} \\ \Leftrightarrow 3 + 3 &= 6 \end{aligned}$$

Geometrisch wird (10.16) in Form des *Pascalschen Dreiecks* visualisiert:

$$\begin{array}{ccccccccccc} & & & & & \binom{0}{0} & & & & & \\ & & & & & \binom{1}{0} & \binom{1}{1} & & & & \\ & & & \binom{2}{0} & \binom{2}{1} & \binom{2}{2} & & & & & \\ & & \binom{3}{0} & \binom{3}{1} & \binom{3}{2} & \binom{3}{3} & & & & & \\ & \binom{4}{0} & \binom{4}{1} & \binom{4}{2} & \binom{4}{3} & \binom{4}{4} & & & & & \\ \binom{5}{0} & \binom{5}{1} & \binom{5}{2} & \binom{5}{3} & \binom{5}{4} & \binom{5}{5} & & & & & \end{array}$$

bzw. mit Zahlenwerten:

				1					
				1		1			
			1		2		1		
		1		3		3		1	
	1		4		6		4		1
1		5		10		10		5	
	1								1

Dabei ist, wie (10.16) besagt, die Summe zweier benachbarten Koeffizienten (mit den Nummern k und $k+1$) in der n -ten Zeile gleich dem darunter stehenden Koeffizienten (mit der Nummer $k+1$) in der nächsten, d. h. $(n+1)$ -ten Zeile.

10.7 Lernziele

Studierende	Kennen	Können	Verstehen
fachlich	☐ benennen Definition und Bedeutung der Operation Fakultät ☐ kennen elementare Zählmodelle ☐ leiten Formeln für elementare Zählmodelle her	☐ berechnen gesuchte Anzahl bei gegebenem Zählmodell	☐ entscheiden, ob ein Sachverhalt mit einem elementaren Zählmodell gezählt werden kann ☐ identifizieren geeignete Zählmodelle oder deren Kombination, um gesuchte Anzahl zu bestimmen ☐ erkennen kombinatorische Fragestellungen in ihrem Alltag
methodisch		☐ überprüfen Zählergebnisse mittels Programm	

Hinzu kommen die themenübergreifenden Lernziele aus den Kategorien „sozial“ und „persönlich“ aus Abschnitt 1.7 sowie die Lernziele aus der Kategorie „methodisch“ aller vorangegangener Kapitel. Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben.

10.8 Übungsaufgaben

- Geben Sie geeignete kombinatorische Modelle für die folgenden Fragestellungen an.
 - Wie viele Passwörter der Länge 20 gibt es, die genau eine Ziffer beinhalten?
 - Wie viele Passwörter der Länge 20 gibt es, die je genau zur Hälfte aus Buchstaben und Ziffern bestehen?
 - Wie viele Passwörter der Länge N gibt es, die kein Zeichen mehrfach beinhalten?
 - Wie viele Passwörter der Länge N gibt es, die nur aus Ziffern bestehen?
- Berechnen Sie die in den Fragestellungen der vorherigen Aufgabe gesuchten Anzahlen.
- Schreiben Sie Programme, mit denen Sie die Ergebnisse aus der vorangegangenen Aufgabenstellung überprüfen können.
- In wievielen natürlichen Zahlen kleiner 1000 kommt die Ziffer 1 mindestens einmal vor?
- Bei einem Quadrat, das n Kästchen breit ist, sollen alle Kästchen auf der Diagonalen von links oben nach rechts unten und alle Kästchen unterhalb der Diagonalen schraffiert werden. Wieviele Kästchen müssen schraffiert werden?

6. Achten Sie eine Woche lang darauf, welchen kombinatorischen Fragestellungen Sie in Ihrem Alltag begegnen. Schreiben Sie diese auf und lösen Sie sie nach Möglichkeit.

Kapitel 11

Zahlentheorie

Die ganzen Zahlen hat der liebe Gott gemacht, alles andere ist Menschenwerk.
(Leopold Kronecker)

11.1 Aufwärmübungen

1. Was bleibt über, wenn 15 Murmeln „gerecht“ auf vier Kinder verteilt werden?
2. Was bleibt über, wenn vier Kinder ihre gemeinsamen Schulden im Umfang von 15 Murmeln „gerecht“ abbezahlen?
3. Welche möglichen Reste können bei der Division ganzer Zahlen durch 5 auftreten, welche bei Division durch 7? Was ist die Antwort auf die verallgemeinerte Frage, welche möglichen Reste bei der Division ganzer Zahlen durch m auftreten?
4. Bestimmen Sie $-1 \bmod 3$ und überprüfen Sie anschließend Ihr Resultat auf möglichst vielen Rechenmaschinen (Taschenrechner, Computer usw.) bzw. in möglichst vielen Programmiersprachen.
5. Wiederholen Sie die vorangegangene Aufgabenstellung für $-15 \bmod 4$. In welchem Zusammenhang steht $-15 \bmod 4$ mit Aufwärmübung 2?
6. Leitfragen:
 - (a) Wenn es auf dem Computer eine größte **integer**-Zahl MAXINT gibt, warum liefert dann MAXINT+1 eine **integer**-Zahl und warum ist MAXINT+1 kleiner als MAXINT?
 - (b) Wie löst man Gleichungen der Art $(7x + 11) \bmod 5 = 3$?
 - (c) Wann und wieso ist $1/7$ gleich 2?

11.2 Ganzzahlige Division

Bei der ganzzahligen Division geht es um zwei Fragen: Erstens, wie oft passt der Teiler in die zu teilende Zahl? Die Antwort ist der sogenannte *Quotient*. Zweitens, welcher *Rest* bleibt beim Teilen über?

Beispiel 11.1: Wie oft passt 9 in 42? Das Ergebnis, also der Quotient, ist 4.

Würden wir eine größere Zahl als 4 als Quotient angeben, z. B. 5, wäre das Ergebnis zu groß, denn $5 \cdot 9 = 45$. Dann hätten wir eine größere Zahl als 42 in Teile der Größe 9 zerlegt. Würden wir eine kleinere Zahl als 4 als Quotient angeben, z. B. 3, wäre das Ergebnis zu klein, denn $42 - 3 \cdot 9 = 15$. In den Rest würde der Teiler 9 dann noch einmal hineinpassen.

Da der Quotient 4 ist, bleibt als Rest $42 - 4 \cdot 9 = 6$. Machen wir die Probe: $4 \cdot 9 + 6 = 42$. Passt!

■

Nennen wir die zu teilende Zahl a , den Teiler m , den Quotienten q und den Rest r . Dann muss immer

$$a = q \cdot m + r \quad (11.1)$$

gelten, wenn Quotient und Teiler korrekt bestimmt wurden. In dieser Gleichung gibt es zwei Unbekannte, also gesuchte Größen: den Quotienten q und den Rest r . Allerdings werden durch (11.1) die Werte der beiden Unbekannten nicht eindeutig festgelegt.

Beispiel 11.2: Wie oft passt 9 in 42? Gesucht ist die Lösung der Gleichung

$$42 = q \cdot 9 + r.$$

Viele Paare $[q, r]$ erfüllen diese Gleichung, z. B. $[3, 15]$, $[4, 6]$, $[5, -3]$ oder auch $[-1, 51]$.

Vermutlich sind Sie der Meinung, dass 9 viermal in 42 passt und als Rest 6 übrig bleibt. ■

Was müssen wir tun, damit die Lösung von (11.1) eindeutig wird? Was müssen wir tun, damit im vorangegangenen Beispiel die von Ihnen präferierte Lösung die einzig mögliche ist und alle anderen Möglichkeiten ausgeschlossen sind?

Eine sinnvolle Anforderung ist sicher, dass in (11.1) der Rest kleiner als der Teiler sein soll: $r < m$. Denn wäre $r \geq m$, könnte der Rest nochmal durch Subtraktion von m verkleinert werden. Allerdings legt die Bedingung $r < m$ die Lösung von (11.1) immer noch nicht eindeutig fest, wie das folgende Beispiel zeigt:

Beispiel 11.3: Wie oft passt 4 in -15? Gesucht ist die Lösung der Gleichung

$$-15 = q \cdot 4 + r.$$

Viele Paare $[q, r]$ erfüllen diese Gleichung, z. B. $[-4, 1]$, $[-3, -3]$. Beide genannten Lösungen erfüllen die Bedingung $r < m$. Im ersten Fall gilt $1 < 4$, im zweiten Fall gilt $-3 < 4$. ■

Dieses Beispiel ist genau die Aufwärmübung 2. Wie sind Sie dort vorgegangen? Haben Sie sich überlegt, dass jedes der vier Kinder einen Beitrag von vier Murmeln leisten muss, damit die Schulden abbezahlt werden können? Dann liegen 16 Murmeln auf dem Tisch, der Gläubiger nimmt sich die geschuldeten 15 und ein Murmeln bleibt übrig. Also $q = -4$ und $r = 1$.

Dieses Beispiel macht Ihnen hoffentlich klar, dass es sinnvoll ist zu fordern, dass Reste nicht negativ sind, also $r \geq 0$. In der Tat wird Gleichung (11.1) durch die Einschränkung $0 \leq r < m$ eindeutig lösbar. Wir halten dieses wichtige Ergebnis fest:

In der Gleichung

$$a = q \cdot m + r \quad (11.2)$$

sind für gegebenes $a \in \mathbb{Z}$ und $m \in \mathbb{N}$ die *ganzzahligen* Unbekannten q, r eindeutig bestimmt, wenn man für den Rest r die Bedingung

$$0 \leq r \leq m - 1 \quad (11.3)$$

festlegt.

Die Modulo-Funktion mod macht nichts anderes als (11.2) unter der Einschränkung (11.3) zu lösen. Es gilt also

$$r = a \bmod m. \quad (11.4)$$

Diese Gleichung ist in Verbindung mit (11.2, 11.3) die Definitionsgleichung der Modulo-Funktion.

Die zweite wichtige Funktion ist die der ganzzahligen Division. Dafür wird oft das Symbol div verwendet. Diese Funktion bestimmt den Quotienten:

$$q = a \text{ div } m. \quad (11.5)$$

Der Teiler m wird übrigens auch *Modul* genannt.¹

In *setlX* wird `mod` mit `%` notiert und `div` mit `\`.²

Beispiel 11.4: Wir lösen die Aufgabenstellung aus Aufwärmübung 2

$$-15 = 4q + r$$

mit *setlX*:

```
=> r:=-15%4;
~< Result: 1 >~
=> q:=-15\4;
~< Result: -4 >~
```

Der Quotient ist also -4 und der Rest 1 - wie es wegen $0 \leq r \leq m - 1 = 3$ sein muss. ■

Wir geben der Menge der möglichen Reste modulo m noch ein Symbol:

$$\mathbb{Z}_m = \{0, 1, \dots, m-1\}$$

Eine Einschränkung der Art (11.3) ist absolut notwendig, damit (11.2) eindeutig lösbar ist. Das schließt aber nicht aus, dass die Einschränkung anders formuliert werden kann. Physiker, denen ja Symmetrien wichtig sind, wählen bspw. den Bereich der möglichen Reste symmetrisch um die 0 herum. Für gerades m ist dann $\mathbb{Z}_m = \{-m/2, \dots, m/2 - 1\}$.

11.3 Restklassen

Wir schauen uns die möglichen Reste modulo m am Zahlenstrahl an, siehe Abb. 11.1. Die Reste $(a \bmod m) \in \mathbb{Z}_m$ werden dabei periodisch durchlaufen. Die Zahlen auf dem Zahlenstrahl zerfallen dadurch in m viele Klassen, je eine für jeden möglichen Rest.

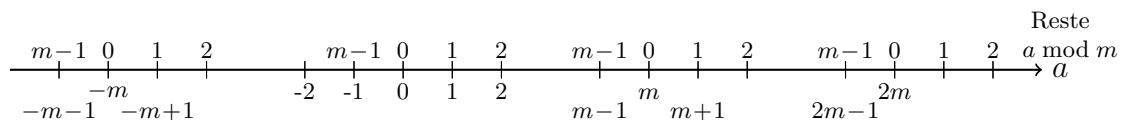


Abbildung 11.1: Die Operation `mod m` zerlegt die Elemente des Zahlenstrahls $\mathbb{Z}$ in Klassen gleichen Rests, die sich periodisch wiederholen.

Beispiel 11.5: Für $m = 3$ zerfällt der Zahlenstrahl in die folgenden drei Klassen:

$$\begin{aligned} \{\dots, -3, 0, 3, 6, \dots\} &= \{0 + 3 \cdot q \mid q \in \mathbb{Z}\} \\ \{\dots, -2, 1, 4, 7, \dots\} &= \{1 + 3 \cdot q \mid q \in \mathbb{Z}\} \\ \{\dots, -1, 2, 5, 8, \dots\} &= \{2 + 3 \cdot q \mid q \in \mathbb{Z}\} \end{aligned}$$

Die erste Klasse ist die Menge aller Zahlen, deren Rest modulo 3 gleich 0 ist. Das sind die Vielfachen von 3. Die zweite Klasse ist die Menge aller Zahlen, deren Rest modulo 3 gleich 1 ist. Entsprechend für die dritte Klasse. ■

Die Klassen, in die die Operation `mod m` die Menge $\mathbb{Z}$ aufteilt, sind die sogenannten *Restklassen*:

Die Menge

$$r + m\mathbb{Z} = \{r + m \cdot q \mid q \in \mathbb{Z}\}$$

wird als *Restklasse modulo m für den Rest r* bezeichnet.

¹Das Genus von Modul im Sinne von (11.1) ist maskulin, also *der* Modul.

²Die präzise Formulierung ist, dass der Ausdruck `a % m` in *setlX* $a \bmod m$ berechnet, nur für Fall dass $m > 0$.

Stören Sie sich nicht an dem ungewöhnlichen Symbol $r + m\mathbb{Z}$ für die Restklasse. Es ist nur ein Symbol. Sie müssen die Schreibweise nicht so interpretieren, dass $\mathbb{Z}$ mit m multipliziert wird.

Alle Elemente einer Restklasse sind bzgl. der Operation $\text{mod } m$ gleichwertig, d. h. äquivalent (mehr dazu in Abschnitt 11.4). Jedes Element einer Restklasse repräsentiert diese eindeutig für einen gegebenen Modul.

Beispiel 11.6: Für $m = 3$ ist die 1 ein eindeutiger Repräsentant für die Restklasse $1 + 3\mathbb{Z}$ (ebenso wie jedes andere Element dieser Restklasse). ■

Wenn wir den Zahlenstrahl in Abb. 11.1 modulo m durchlaufen, durchlaufen wir periodisch die Elemente von $\mathbb{Z}_m$. Effektiv durchlaufen wir damit $\mathbb{Z}$ im Kreis, wie dies Abb. 11.2 visualisiert. Der Zahlenstrahl wird quasi zu einem Kreis aufgewickelt.

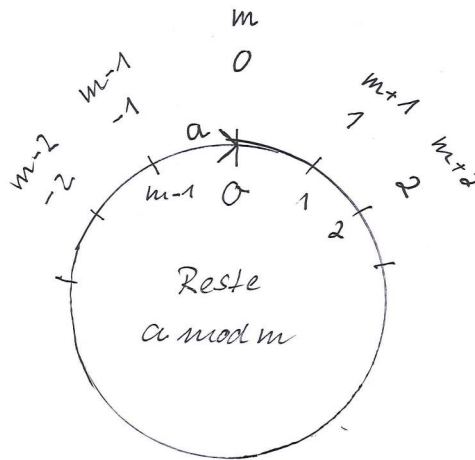


Abbildung 11.2: Durch die Operation modulo m wird der Zahlenstrahl effektiv zu einem Kreis. Die ganzen Zahlen werden mit einer Periode m durchlaufen.

Situationen, die mit modulo zu tun haben, sind immer periodisch. Umgekehrt ist modulo ein probates Mittel, um periodische Situationen zu beschreiben.

Beispiel 11.7: Die Uhrzeit ist periodisch. Bei den Minuten ist die Periode $m = 60$. Die Minuten haben Werte aus $\mathbb{Z}_{60}$. Wenn der Minutenzeiger auf 10 steht, bedeutet dies, dass seit Tagesbeginn 10 Minuten oder 70 Minuten oder 130 Minuten usw. vergangen sind. Die 10 repräsentiert alle Werte aus der Restklasse $10 + 60\mathbb{Z}$. Aus Sicht der Zahlentheorie steht der Minutenzeiger nicht auf der 10, sondern auf der Restklasse, deren Repräsentant die 10 ist.

Wenn der Zeiger auf 10 steht, sollte er nach 55 Minuten auf 65 stehen. Tatsächlich steht er auf 5, was modulo 60 „das Gleiche ist“, weil sowohl 5 als auch 65 Elemente der Restklasse $5 + 60\mathbb{Z}$ sind.

Wenn der Zeiger auf der höchstmöglichen Minutenzahl steht (59), zeigt er eine Minute später auf die kleinstmögliche Minutenzahl (0). ■

Auf dem Zahlenstrahl ist der nächste Nachbar immer um eins größer. Auf dem zyklischen Zahlenstrahl von $\text{mod } m$ ist das auch so, aber die nächste Zahl kann scheinbar kleiner sein. Bei der Uhr folgt auf die 59 die 0. Die bessere Beschreibung ist: Auf der Uhr folgen auf Zahlen aus der Restklasse $59 + 60\mathbb{Z}$ Zahlen aus der Restklasse $0 + 60\mathbb{Z}$. Das ist der Kern der *modularen Arithmetik*, d. h. dem Rechnen mit Zahlen, wenn „alles immer modulo m gesehen wird“. Wir beschäftigen uns mit dieser Art zu Rechnen eingehender in Abschnitt 11.5.

Beispiel 11.8: Auch Integer auf dem Rechner sind eigentlich Restklassen. Die Integer-Arithmetik ist modulo $2 \cdot (\text{MAXINT} + 1)$, wobei MAXINT für die „größte“ Integerzahl steht, d. h. ihr Repräsentant ist größer als der aller anderen Integer. Sie können dies daran sehen, dass Sie 1 auf MAXINT

addieren. Sie erhalten dann ein Resultat aus der benachbarten Restklasse, dessen Repräsentant aber kleiner als MAXINT ist.

Z. B. auf einer 32 bit-Maschine sind die Repräsentanten der Integer die ganzen Zahlen von $\text{MININT} = -2^{31}$ bis $\text{MAXINT} = 2^{31} - 1$. Die Berechnung $\text{MAXINT} + 1$ liefert das Ergebnis MININT. Probieren Sie es aus! ■

11.4 Kongruenz

Die Restklassen drücken aus, dass verschiedene Zahlen denselben Rest modulo m haben. Wir geben zwei Zahlen, die denselben Rest haben, eine besondere Bezeichnung:

Zwei ganze Zahlen a, b sind *kongruent modulo m* , wenn sie bei Division durch m denselben Rest haben. Man schreibt dafür

$$a \equiv b \pmod{m}. \quad (11.6)$$

Alle Zahlen einer Restklasse sind also kongruent zueinander.

Verwechseln Sie die Schreibweise $a \equiv b \pmod{m}$ nicht mit $a = b \bmod m$. Letztere sagt aus, dass a denselben Wert hat wie der Rest $b \bmod m$. Daher muss $a \in \mathbb{Z}_m$ gelten. Erstere sagt aus, dass sowohl a als auch b Elemente derselben Restklasse sind. Insbesondere muss nicht $a \in \mathbb{Z}_m$ gelten.

Beispiel 11.9: Für $m = 5$ gilt bspw.

$$\begin{aligned} 2 &\equiv 12 \pmod{5} \\ 7 &\equiv 12 \pmod{5} \\ -1 &\equiv 4 \pmod{5}. \end{aligned}$$

Auch

$$2 = 12 \bmod 5$$

ist eine wahre Aussage. Dagegen sind

$$\begin{aligned} 7 = 12 \bmod 5 &\Leftrightarrow 7 = 2 \\ -1 = 4 \bmod 5 &\Leftrightarrow -1 = 4 \end{aligned}$$

keine wahren Aussagen. ■

In $a \equiv b \pmod{m}$ steht $\bmod m$ in Klammern. Das ist ein visueller Hinweis, dass es um Kongruenz geht und nicht um die Berechnung eines Restes auf der rechten Seite. In $a = b \bmod m$ bezieht sich $\bmod m$ alleine auf b . In $a \equiv b \pmod{m}$ bezieht es auf a und b .

Eine gängige alternative Schreibweise für die Kongruenz (11.6) ist

$$a = b \pmod{m},$$

insbesondere wenn kein $\equiv$ zur Verfügung steht. Bei dieser Schreibweise ist das in Klammern gesetzte $\bmod m$ dann der kritische Hinweis, dass es sich um eine Kongruenzaussage und nicht um $a = b \bmod m$ handelt.

Gemäß Definition sind zwei ganze Zahlen a, b kongruent modulo m , wenn sie denselben Rest haben, also

$$a \equiv b \pmod{m} \Leftrightarrow a \bmod m = b \bmod m. \quad (11.7)$$

Dieser Ausdruck macht die unterschiedlichen Bedeutungen von $\bmod m$ „mit und ohne Klammern“ noch einmal deutlich und zeigt den Zusammenhang zwischen den beiden auf.

Beispiel 11.10: Mit Hilfe von (11.7) können wir die Kongruenz in *setlX* implementieren:

```
=> iscongruent:=[a,b,m] |-> a%m==b%m;
~< Result: [a, b, m] |-> a % m == b % m >~
=> iscongruent(-1,4,5);
```

```
~< Result: true >~
=> -1==4%5;
~< Result: false >~
```

Die letzten beiden Eingaben zeigen noch einmal den Unterschied von $\text{mod } m$ „mit und ohne Klammern“.

Die Kongruenz kann als Relation auf $\mathbb{Z}$ modelliert werden. Sie setzt all die ganzen Zahlen in Relation, die kongruent zueinander sind.

Beispiel 11.11: Um einen Eindruck von der Relation „ist kongruent zu“ zu bekommen, berechnen wir mit *setlX* die Elemente einer Teilmenge von $\mathbb{Z}$ für den Modul $m = 5$:

```
=> {[a,b]: a in {-6..6}, b in {-6..6} | iscongruent(a,b,5) };
~< Result: {[ -6, -6], [ -6, -1], [ -6, 4], [ -5, -5], [ -5, 0], [ -5, 5],
[ -4, -4], [ -4, 1], [ -4, 6], [ -3, -3], [ -3, 2], [ -2, -2], [ -2, 3], [ -1, -6],
[ -1, -1], [ -1, 4], [ 0, -5], [ 0, 0], [ 0, 5], [ 1, -4], [ 1, 1], [ 1, 6], [ 2, -3],
[ 2, 2], [ 3, -2], [ 3, 3], [ 4, -6], [ 4, -1], [ 4, 4], [ 5, -5],
[ 5, 0], [ 5, 5], [ 6, -4], [ 6, 1], [ 6, 6]} >~
```

Überlegen Sie sich, welche der Eigenschaften aus Abschnitt 7.3 die Relation „ist kongruent zu“ hat:

Reflexivität	 ☐ Ja ☐ Nein
Symmetrie	 ☐ Ja ☐ Nein
Transitivität	 ☐ Ja ☐ Nein
Antisymmetrie	 ☐ Ja ☐ Nein
Asymmetrie	 ☐ Ja ☐ Nein

Die Kongruenz ist also eine Äquivalenzrelation. Ihre Äquivalenzklassen sind die Restklassen.

Der Nutzen der Kongruenzrelation ist analog zum Nutzen der Gleichheitsrelation, die ja ebenfalls eine Äquivalenzrelation ist. Was die Gleichung für das Rechnen mit (reellen) Zahlen ist, ist die Kongruenz für das Rechnen mit Restklassen. So wie wir bspw. beim Lösen einer Gleichung nach einer Unbekannten die Gleichung durch eine äquivalente ersetzen, können wir beim Lösen einer Kongruenz nach einer Unbekannten eine Kongruenz durch eine äquivalente ersetzen:

$$\begin{aligned}
 2x + 3 &\equiv 14 \pmod{5} \\
 \Leftrightarrow 2x + 3 &\equiv 4 \pmod{5} \\
 \Leftrightarrow 2x &\equiv 1 \pmod{5} \\
 \Leftrightarrow x &\equiv 2^{-1} \cdot 1 \pmod{5} \\
 \Leftrightarrow x &\equiv 3 \pmod{5}
 \end{aligned}$$

Das Ersetzen einer Kongruenz durch eine andere erlaubt das Rechnen mit Restklassen. Wie dies für die arithmetischen Operationen Addition, Subtraktion, Multiplikation und Division funktioniert (und warum $2^{-1} \equiv 3 \pmod{5}$), ist Thema des nächsten Abschnitts.

11.5 Modulare Arithmetik

Um Kongruenzen umformen zu können, brauchen wir wie in der Arithmetik Äquivalenzumformungen, d. h. wir müssen wissen, welche Operationen auf Kongruenzen angewandt deren Wahrheitswert nicht ändern. Bei Gleichungen sind zwei wichtige Äquivalenzumformungen die Addition einer Zahl auf beiden Seiten und die Multiplikation mit einer von Null verschiedenen Zahl auf beiden Seiten.

Beginnen wir mit der Addition:

Wenn $a \equiv b \pmod{m}$ und $c \equiv d \pmod{m}$, dann folgt

$$a + c \equiv b + d \pmod{m} \quad (11.8)$$

$$a \cdot c \equiv b \cdot d \pmod{m}. \quad (11.9)$$

Formal notiert bedeutet dies:

$$a \equiv b \pmod{m} \wedge c \equiv d \pmod{m} \Rightarrow a + c \equiv b + d \pmod{m}.$$

Beachten Sie, dass in dieser Tautologie $\Rightarrow$ und nicht $\Leftrightarrow$ steht. Das ist auch bei der analogen Tautologie in der Arithmetik so:

$$a = b \wedge c = d \Rightarrow a + c = b + d.$$

Auch hier ist die Folgerichtung von links nach rechts, aber nicht von rechts nach links. Aus der wahren Aussage

$$7 + 5 = 0 + 12$$

kann nicht gefolgert werden, dass

$$7 = 0 \wedge 5 = 12,$$

was den Wahrheitswert 0 hat. Analog kann aus der wahren Aussage

$$7 + 5 \equiv 0 + 2 \pmod{5}$$

nicht gefolgert werden, dass

$$7 \equiv 0 \pmod{5} \wedge 5 \equiv 2 \pmod{5};$$

der Wahrheitswert ist 0. Aber aus $7 \equiv 2 \pmod{5}$ und $5 \equiv 0 \pmod{5}$ folgt $7 + 5 \equiv 2 + 0 \pmod{5}$ also $12 \equiv 2 \pmod{5}$.

Aufgrund (11.8) können Kongruenzen wie Gleichungen addiert werden

$$\begin{array}{rcl} a & = & b \\ c & = & d \\ \hline a + c & = & b + d \end{array} \quad \begin{array}{rcl} a & \equiv & b \pmod{m} \\ c & \equiv & d \pmod{m} \\ \hline a + c & \equiv & b + d \pmod{m} \end{array}$$

und aufgrund (11.9) auch multipliziert werden.

Beispiel 11.12: (11.8) erlaubt uns Kongruenzen der Art

$$x - 7 \equiv 12 \pmod{5} \quad (11.10)$$

rein algorithmisch nach der Unbekannten x aufzulösen. Bevor wir das gemeinsam tun, sollten Sie erst mal alleine versuchen Hand anzulegen. Was können Sie über die Lösung von (11.10) sagen?



Machen wir also gemeinsam weiter: Zusammen mit $7 \equiv 7 \pmod{5}$ folgt aus (11.10) aufgrund (11.8):

$$\begin{aligned} x - 7 + 7 &\equiv 12 + 7 \pmod{5} \\ \Leftrightarrow x &\equiv 19 \pmod{5} \end{aligned}$$

Effektiv haben wir dadurch 7 auf beiden Seiten addiert. Wegen $19 \equiv 4 \pmod{5}$ und der Transitivität der Kongruenzrelation bedeutet dies $x \equiv 4 \pmod{5}$. Jedes Element aus der Restklasse $4 + 5\mathbb{Z}$ erfüllt also (11.10). Wie sieht Ihre Lösung von oben im Vergleich zu dieser aus?

Machen wir abschließend noch die Probe und zwar exemplarisch für die Elemente 4 und 9 aus der Lösungsmenge $4 + 5\mathbb{Z}$:

$$\begin{aligned} \text{für } x = 4 : x - 7 &\equiv 12 \pmod{5} &\Leftrightarrow -3 &\equiv 12 \pmod{5} \\ &&\Leftrightarrow 1, &\text{weil beide Zahlen modulo 5 denselben Rest 2 haben} \\ \text{für } x = 9 : x - 7 &\equiv 12 \pmod{5} &\Leftrightarrow 2 &\equiv 12 \pmod{5} \\ &&\Leftrightarrow 1, &\text{weil beide Zahlen modulo 5 denselben Rest 2 haben} \end{aligned}$$

Zumindest in den überprüften Fällen ist unsere Lösung sicher korrekt. ■

Wie bei Gleichungen, die ihren Wahrheitswert nicht ändern, wenn auf beiden Seiten etwas *Gleiches* addiert wird, ändert sich der Wahrheitswert von Kongruenzen nicht, wenn auf beiden Seiten etwas *Kongruentes* addiert wird.

Bei Gleichungen wird gelegentlich etwas Negatives auf beiden Seiten addiert. Grund für uns zu klären, was das Negative im Kontext der modularen Arithmetik bedeutet:

Das *Negative* oder *additive Inverse* zu einer Zahl $z \in \mathbb{Z}_m$ ist die Zahl $\bar{z} \in \mathbb{Z}_m$, die

$$z + \bar{z} \equiv 0 \pmod{m}$$

erfüllt. Wie bei den reellen Zahlen schreibt man auch kurz $-z$ für das additive Inverse zu $z \in \mathbb{Z}_m$.

Beispiel 11.13: Bestimmen Sie jeweils das Negative von allen Elementen von $\mathbb{Z}_5$.



1	1	1	1	1	Probe: $z + -z \equiv 0 \pmod{5}$
5	5	5	5	0	Probe: $z + -z$
1	2	3	4	0	$-z$
4	3	2	1	0	z

Für die anderen Zahlen führen diese Überlegungen zu den folgenden additiven Inversen:

Anforderung, denn $4 + 1 \equiv 0 \pmod{5}$.
 $-z \in \mathbb{Z}_5$ ergibt mit z addiert ein Element aus der Restklasse $0 + 5\mathbb{Z}_5$? Die Zahl $-z = 1$ erfüllt diese
 Beginnen wir mit $z = 4$. Definitionsgemäß müssen wir folgende Frage beantworten: Welche Zahl

Wegen $5 \equiv 0 \pmod{5}$ geht die Probe in jedem Fall auf. Zu jeder Zahl aus $\mathbb{Z}_5$ gibt es also eine eindeutige additive Inverse. ■

Möglicherweise ist Ihnen bei der Bearbeitung dieses Beispiels aufgefallen, dass additive Inverse leicht berechnet werden können:

Zu jedem $z \in \mathbb{Z}_m$ gibt es genau ein additives Inverses $-z \in \mathbb{Z}_m$:

$$-z = \begin{cases} m - z & , \text{für } z \neq 0 \\ 0 & , \text{für } z = 0 \end{cases} \quad (11.11)$$

Auf dem Zahlenstrahl in Abb. 11.1 liegen z und $-z$ symmetrisch zur 0. Auf dem zyklischen Zahlenstrahl in Abb. 11.2 liegen z und $-z$ ebenfalls symmetrisch zur 0.

Beispiel 11.14: Auf der Uhr mit $m = 60$ ist die Inverse zur 1 die 59:

$$-1 \equiv 59 \pmod{60}$$

59 Minuten *nach* der vollen Stunde ist eine Minute *vor* der vollen Stunde. ■

Halten wir fest, wo wir gerade stehen: Wir können Kongruenzen umformen, indem wir auf beiden Seiten Kongruentes addieren. Wie ist es mit der Multiplikation bestellt? Aufgrund von (11.9) können Kongruenzen wie Gleichungen multipliziert werden. Typischerweise multiplizieren wir Gleichungen jedoch mit Kehrwerten; z. B. multiplizieren wir

$$4x = 3$$

mit dem dem Kehrwert $4^{-1} = 1/4$ von 4, um die Gleichung nach der Unbekannten aufzulösen. Wir sollten also anschauen, wie es bei Kongruenzen mit Kehrwerten aussieht:

Wenn es zu $z \in \mathbb{Z}_m$ eine Zahl $\hat{z} \in \mathbb{Z}_m$ gibt, die

$$z \cdot \hat{z} \equiv 1 \pmod{m} \quad (11.12)$$

erfüllt, dann nennt man $\hat{z}$ den *Kehrwert* oder das *multiplikative Inverse* zu $z \in \mathbb{Z}_m$. Wie bei den reellen Zahlen schreibt man auch kurz z^{-1} oder $\frac{1}{z}$ für das multiplikative Inverse zu $z \in \mathbb{Z}_m$.

Beispiel 11.15: Wir bestimmen die multiplikativen Inversen in $\mathbb{Z}_5$. Beginnen wir mit $z = 4$. Wir müssen also überlegen, welche Zahl $z^{-1} \in \mathbb{Z}_5$ die Kongruenz

$$4 \cdot z^{-1} \equiv 1 \pmod{5}$$

erfüllt. Da es nur fünf mögliche Kandidaten gibt, sind diese schnell durchprobiert:

z^{-1}	0	1	2	3	4
$4 \cdot z^{-1}$	0	4	8	12	16
$4 \cdot z^{-1} \equiv 1 \pmod{5}$	0	0	0	0	1

Damit gilt für die multiplikative Inverse von 4: $4^{-1} \equiv 4 \pmod{5}$.

Analoges Vorgehen führt für die verbleibenden Elemente von $\mathbb{Z}$ zu den folgenden multiplikativen Inversen:

z	0	1	2	3	4
z^{-1}	-	1	3	2	4

Für die 0 gibt es keine multiplikative Inverse, analog zur Multiplikation bei den ganzen Zahlen $\mathbb{Z}$. 4 und 1 sind zu sich selbst invers. In $\mathbb{Z}$ ist dies für 1 und -1 der Fall. 1 und -1 sind in $\mathbb{Z}$ additive Inverse zueinander, ebenso wie 4 und 1 additive Inverse zueinander in $\mathbb{Z}_5$ sind. ■

Beispiel 11.16: Wir bestimmen die multiplikativen Inversen in $\mathbb{Z}_4$:

z	0	1	2	3
z^{-1}	-	1	-	3

0 hat natürlich keine multiplikative Inverse. 2 kann auch keine haben, weil das Produkt mit der gesuchten Inversen immer gerade ist und damit modulo 4 nie 1 ergeben kann. ■

Beispiel 11.17: Wir bestimmen die multiplikativen Inversen in $\mathbb{Z}_{15}$:

z	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
z^{-1}	-	1	8	-	4	-	-	13	2	-	-	11	-	7	14

Hier gibt es sogar vier Zahlen, die zu sich selbst invers sind. ■

Offensichtlich ist es nicht so, dass für die Menge $\mathbb{Z}_m$ jedes Element eine multiplikative Inverse hat. Das folgende Theorem gibt Auskunft darüber, wie der Modul m beschaffen sein muss, damit dies der Fall ist:

Wenn $z \in \mathbb{Z}_m$ und $z \neq 0$, dann gibt es genau dann ein (eindeutiges) multiplikatives Inverse zu z , wenn z und m keine gemeinsamen Teiler haben.

Wenn m eine Primzahl ist, sind z und m garantiert immer teilerfremd und jedes Element von $\mathbb{Z}_m \setminus \{0\}$ hat dann eine multiplikative Inverse. Das ist der Grund, warum in den obigen Beispielen für $m = 5$ alle Elemente verschieden 0 eine multiplikative Inverse hatten, aber nicht für $m = 4$ und $m = 15$. In den letzten beiden Fällen haben nur die Zahlen ein Inverses, die teilerfremd zu m sind. Für $m = 4$ ist dies für $z = 1$ und $z = 3$ der Fall, aber nicht für $z = 2$, weil 2 und 4 einen gemeinsamen Teiler haben.

Das obige Theorem sagt uns viel über die Existenz von multiplikativen Inversen, aber nicht wie sie berechnet werden können. Das in den obigen Beispielen verwendete Verfahren, alle Möglichkeiten durchzuprobieren, ist einfach, aber langwierig und daher bestens für Rechner geeignet:

Beispiel 11.18: Wir berechnen $2^{-1} \in \mathbb{Z}_5$ mittels *setlX* und verwenden dazu die Definition der multiplikativen Inversen:

```
=> exists(zinv in {1..4} | 2*zinv % 5 ==1 );
~< Result: true >~
=> zinv;
~< Result: 3 >~
```

Da das Theorem garantiert, dass die Inverse eindeutig ist, muss die von *setlX* gefundene Lösung die einzige sein: $2^{-1} \equiv 3 \pmod{5}$. ■

Nun haben wir alles zusammen, um Kongruenzen arithmetisch manipulieren zu können:

Beispiel 11.19: Wir lösen die folgende Kongruenz nach der Unbekannten x :

$$7x + 11 \equiv 13 \pmod{5} \quad (11.13)$$

Im ersten Schritt entfernen wir links die Addition mit 11, im zweiten Schritt die Multiplikation mit 7:

$$\begin{array}{rcll} 7x + 11 & \equiv & 13 \pmod{5} & | -11 \\ \Leftrightarrow 7x & \equiv & 2 \pmod{5} & | \cdot 7^{-1} \\ \Leftrightarrow x & \equiv & 7^{-1} \cdot 2 \pmod{5} & \end{array}$$

Wegen $7 \equiv 2 \pmod{5}$ ist die Inverse von 7 gleich der Inversen von 2. Oben hatten wir bereits festgestellt, dass $2^{-1} \equiv 3 \pmod{5}$ und daher ist auch $7^{-1} \equiv 3 \pmod{5}$. Wir erhalten somit

$$\begin{array}{rcl} x & \equiv & 3 \cdot 2 \pmod{5} \\ \Leftrightarrow x & \equiv & 1 \pmod{5} \end{array}$$

Alle Elemente der Restklasse $1 + 5\mathbb{Z}$ erfüllen also (11.13). Machen Sie die Probe für einige Elemente dieser Lösungsmenge:



■

Beispiel 11.20: Wir lösen die folgende Kongruenz nach der Unbekannten x :

$$3x + 1 \equiv 5 \pmod{15} \quad (11.14)$$

Addition von -1 auf beiden Seiten führt auf

$$3x \equiv 4 \pmod{15}$$

Diese Kongruenz können wir allerdings nicht nach x auflösen. Dazu müssten wir mit der multiplikativen Inversen von 3 multiplizieren. Diese gibt es allerdings nicht in $\mathbb{Z}_{15}$.

Tatsächlich hat die vorliegende Kongruenz keine Lösung, weil sich kein Element der Restklasse von 4

$$4 + 15\mathbb{Z} = \{\dots, -11, 4, 19, 34, \dots\}$$

durch 3 teilen lässt. ■

Beispiel 11.21: Wir lösen die folgende Kongruenz nach der Unbekannten x :

$$3x + 1 \equiv 4 \pmod{15}$$

Addition von -1 auf beiden Seiten führt auf

$$3x \equiv 3 \pmod{15}$$

Die 3 hat zwar kein multiplikatives Inverses modulo 15, aber alle Element der Restklasse von 3

$$3 + 15\mathbb{Z} = \{\dots, -27, -12, 3, 18, 33, 48, \dots\} = \{3 + 15z \mid z \in \mathbb{Z}\}$$

lassen sich durch 3 teilen. Die Lösungsmenge ist daher

$$\{1 + 5z \mid z \in \mathbb{Z}\} = \{\dots, -9, -4, 1, 6, 11, 16, \dots\} = 1 + 5\mathbb{Z}.$$

Dies sind alle Zahlen, die modulo 15 kongruent zu den Elementen von $\{1, 6, 11\} \subseteq \mathbb{Z}_{15}$ sind. ■

11.6 Modulare arithmetische Operatoren

Um auf $\mathbb{Z}_m$ und damit mit Restklassen rechnen zu können, haben wir im Abschnitt 11.5 das Konzept der Kongruenz als Analogon zur Gleichheit verwendet. Alternativ könnten wir bei der Gleichheit bleiben, wenn wir statt der arithmetischen Operatoren analoge Operatoren verwenden, die garantieren, dass die Ergebnisse immer Elemente von $\mathbb{Z}_m$ sind. Die folgende *modulare Addition* und *modulare Multiplikation*

$$x \oplus_m y = (x + y) \bmod m \quad (11.15)$$

$$x \odot_m y = (x \cdot y) \bmod m \quad (11.16)$$

stellen dies für den Modul m sicher.

Eine modulare Subtraktion könnten wir über $x \ominus_m y = (x - y) \bmod m$ definieren. Wir sparen uns dies hier, weil wir die Subtraktion immer als Summe mit dem additiven Inversen des Subtrahenden schreiben können: $x \oplus_m -y$. Bei der modularen (ganzzahligen!) Division verfahren wir analog und rechnen dabei mit den modularen multiplikativen Inversen aus (11.12).

Beispiel 11.22: Wärmen wir uns mit einigen elementaren modularen Berechnungen auf:

$$\begin{aligned} 7 \oplus_5 13 &= (7 + 13) \bmod 5 = 20 \bmod 5 = 0 \\ -7 \oplus_5 13 &= (-7 + 13) \bmod 5 = 6 \bmod 5 = 1 \\ -7 \odot_5 13 &= (-7 \cdot 13) \bmod 5 = -91 \bmod 5 = 4 \\ 5 \odot_3 4 \oplus_3 -8 &= ((5 \cdot 4) \bmod 3 - 8) \bmod 3 = (2 - 8) \bmod 3 = -6 \bmod 3 = 0 \end{aligned}$$

■

Beispiel 11.23: Die Integer-Addition und -Multiplikation auf Rechnern können als $\oplus_m$ bzw. $\odot_m$ aufgefasst werden. Bei einer 32-Bit-Darstellung von Integern ist $m = 2^{32}$, bei einer 64-Bit-Darstellung entsprechend 2^{64} . ■

Beispiel 11.24: An Stelle der Kongruenz (11.13), d. h.

$$7x + 11 \equiv 13 \pmod{5}$$

können wir genauso gut die Gleichung

$$7 \odot_5 x \oplus_5 11 = 13$$

lösen:

$$\begin{aligned} 7 \odot_5 x \oplus_5 11 &= 13 && | \oplus_5 - 11 \\ \Leftrightarrow 7 \odot_5 x &= 13 \oplus_5 -11 \\ \Leftrightarrow 7 \odot_5 x &= 2 && | \odot_5 7^{-1} \\ \Leftrightarrow x &= 7^{-1} \odot_5 2 \\ \Leftrightarrow x &= 3 \odot_5 2 \\ \Leftrightarrow x &= 1 \end{aligned}$$

Da wir modular rechnen, ist $x = 1$ nicht das einzige Ergebnis, sondern jedes Element der Restklasse von $\mathbb{Z}_5$, das 1 repräsentiert, also $1 + 5\mathbb{Z}$. ■

Die möglichen Ergebnisse modularer Addition und Multiplikation lassen sich übersichtlich in sogenannten *Verknüpfungstafeln* darstellen, z. B. für den Modul 3:

$\oplus_3$	0	1	2
0	0	1	2
1	1	2	0
2	2	0	1

$\odot_3$	0	1	2
0	0	0	0
1	0	1	2
2	0	2	1

Die Verknüpfungstafeln beschränken sich hier auf Zahlen aus $\mathbb{Z}_m$, hier also $\mathbb{Z}_3$, weil die arithmetischen Operationen ja nur Ergebnisse aus diesen Mengen liefern können.

Im Allgemeinen stehen in Verknüpfungstafeln in den Zeilen die linken Operanden und in den Spalten die rechten:

$\star$	$\cdots$	i	$\cdots$	j	$\cdots$
$\vdots$	$\ddots$	$\vdots$	$\ddots$	$\vdots$	$\ddots$
i	$\cdots$		$\cdots$	$i \star j$	$\cdots$
$\vdots$	$\ddots$	$\vdots$	$\ddots$	$\vdots$	$\ddots$
j	$\cdots$	$j \star i$	$\cdots$		$\cdots$
$\vdots$	$\ddots$	$\vdots$	$\ddots$	$\vdots$	$\ddots$

Das Ergebnis für $1 \oplus_3 2$ ist also die fett hervorgehobene **0** in der Verknüpfungstafeln für $\oplus_3$ und nicht etwa die 0 in der letzten Zeile (auch wenn der Zahlenwert hier der gleiche ist).

Beispiel 11.25: Als weiteres Beispiel stellen wir die Verknüpfungstafeln für $\oplus_5$ und $\odot_5$ auf. Gelegenheit für Sie, selbst Hand anzulegen:

$\oplus_5$	0	1	2	3	4
0					
1					
2					
3					
4					

$\odot_5$	0	1	2	3	4
0					
1					
2					
3					
4					

■

11.7 Empfohlene Vertiefungen

Da die Integer-Arithmetik auf Rechnern eine modulare Arithmetik ist, basieren zahlreiche Verfahren der Informatik auf der Zahlentheorie. Schauen Sie sich bspw. das Hash-Verfahren an (z. B. im Lehrbuch von Teschl und Teschl im Abschnitt 3.1.1). Auch außerhalb der Informatik basieren viele Technologien auf der Zahlentheorie, z. B. Prüfnr.-systeme wie ISBN oder IBAN (siehe z. B. im Lehrbuch von Teschl und Teschl im Abschnitt 3.1) oder Kryptographie. Sie haben also viele Möglichkeiten die Inhalte dieses Kapitels zu vertiefen.

11.8 Lernziele

Studierende	Kennen	Können	Verstehen
fachlich	☐ kennen Eigenschaften der modularen Addition, Multiplikation, Subtraktion und Division ☐ kennen Rechengesetze der modularen Arithmetik ☐ beschreiben Gemeinsamkeiten und Unterschiede zwischen modulo-Operation und Kongruenzrelation ☐ beschreiben die Bedeutung von Restklassen	☐ führen modulare arithmetische Berechnungen geringer Komplexität per Hand durch ☐ lösen Kongruenzen nach Unbekannten	☐ ☐
methodisch	☐ benennen den Zusammenhang zwischen modularer Arithmetik und Integer-Arithmetik ☐ erläutern Bedeutung der Zahlentheorie für die Informatik ☐ beschreiben Gemeinsamkeiten und Unterschiede zwischen modulo-Operation und verwandter Operationen in Programmiersprachen	☐ schreiben Programm, das entscheidet, ein Objekt bestimmte zahlentheoretische Eigenschaften hat	☐

Hinzu kommen die themenübergreifenden Lernziele aus den Kategorien „sozial“ und „persönlich“ aus Abschnitt 1.7 sowie die Lernziele aus der Kategorie „methodisch“ aller vorangegangener Kapitel. Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben.

11.9 Übungsaufgaben

1. Berechnen Sie in **Java** die nächsten Integer-Werte zu `MAX_VALUE` und `MIN_VALUE`, d. h. berechnen Sie `MAX_VALUE+1` und `MIN_VALUE-1`.
2. Schlagen Sie die Definition von modulo, wie es auf Ihrem Taschenrechner oder in einer Programmiersprache Ihrer Wahl verwendet wird, nach. Zeigen Sie entweder, dass diese Definition konform mit der von `mod` ist, oder arbeiten Sie die Unterschiede zwischen den beiden Definitionen heraus.
3. Bestimmen Sie die Äquivalenzklassen der Relation „Kongruenz modulo 5“ auf $\{-20..20\} \times \{-20..20\}$ per *setLX*.
4. Schreiben Sie in *setLX* eine Funktion, die für gegebenen Modul m das multiplikative Inverse z^{-1} zu $z \in \mathbb{Z}_m$ berechnet, falls dieses existiert.
5. Bestimmen Sie die Äquivalenzklassen der Relation „ist kongruent modulo 3 zu“. Was können Sie aufgrund des Ergebnisses allgemein über die Äquivalenzklassen der Relation „ist kongruent modulo m zu“ aussagen?

6. Entscheiden Sie für jeden grauen Kasten in diesem Kapitel, ob es sich um eine Definition, ein Theorem oder etwas anderes handelt, und begründen Sie Ihre Antwort.
7. Implementieren Sie die modularen Operationen $\oplus_m$ und $\odot_m$ aus (11.15, 11.16) in *setlX*.

Kapitel 12

Algebraische Strukturen

Wir kommen nun zu dem entscheidenden Schritt mathematischer Abstraktion: wir vergessen, wofür die Symbole stehen. ... [Der Mathematiker] braucht die Hände nicht in den Schoß zu legen; es gibt viele Operationen, die er mit diesen Symbolen ausführen kann, ohne jemals die Dinge betrachten zu müssen, für die sie stehen.
(Hermann Weyl)

12.1 Aufwärmübungen

1. Stellen Sie die Verknüpfungstabellen für die nachfolgend genannten Operationen und Mengen auf:
 - (a) $\oplus_5$ auf $\mathbb{Z}_5$
 - (b) $\odot_5$ auf $\mathbb{Z}_5$
 - (c) $\wedge$ auf $\{0, 1\}$
 - (d) $\rightarrow$ auf $\{0, 1\}$
 - (e) $+$ auf $\{-1, 1\}$
 - (f) $\cdot$ auf $\{-1, 1\}$
 - (g) $+$ auf $\{-1, 0, 1\}$
 - (h) Division auf $\{-1, 1\}$
 - (i) Division auf $\{\frac{1}{2}, 1, 2\}$
 - (j) Die Nacheinanderausführung von Drehungen auf der Menge der beiden Drehungen um 0° und um 180°
 - (k) Multiplikation von Funktionen auf der Menge $\{(\mathbb{R} \rightarrow \mathbb{R}, x \mapsto 1), (\mathbb{R} \rightarrow \mathbb{R}, x \mapsto x)\}$
2. Die Verknüpfungstabellen aus der vorherigen Aufgaben können auf verschiedene Art und Weise klassifiziert werden.
 - (a) Welche Eigenschaften haben die Verknüpfungstabellen aus (1a), (1b), (1c), (1d), (1f), (1h), (1j) gemeinsam, die die Verknüpfungstabellen aus (1e), (1g), (1i), (1k) alle nicht haben?
 - (b) Welche Eigenschaften haben die Verknüpfungstabellen aus (1a), (1b), (1c), (1d), (1f), (1g), (1h), (1i), (1j), (1k) gemeinsam, die die Verknüpfungstabelle aus (1e) nicht hat?
 - (c) Welche Eigenschaften haben die Verknüpfungstabellen aus (1a), (1b), (1c), (1e), (1f), (1g), (1j), (1k) gemeinsam, die die Verknüpfungstabellen aus (1d), (1h), (1i) alle nicht haben?
 - (d) Welche Operationen sind kommutativ, welche sind assoziativ?
3. Leitfragen:

- (a) Wie kann man unterschiedliche Operationen wie Konjunktion, Disjunktion, Addition, Multiplikation, Konkatenation, Permutation *etc.* in Klassen mit gemeinsamer Struktur zusammenfassen?
- (b) In welcher Beziehung stehen Gruppen und Körper zueinander?
- (c) Warum kann man Addition und Multiplikation ganzer Zahlen durch logische Berechnungen ausführen?

12.2 Übergreifende Strukturen

Kapitel 11 hat viele Gemeinsamkeiten, aber auch einige Unterschiede zwischen $\mathbb{Z}_m$ und $\mathbb{Z}$ (oder auch $\mathbb{R}$) aufgezeigt, von denen wir einige noch gar nicht explizit benannt haben:

- Die Summe und auch das Produkt zweier Zahlen aus $\mathbb{Z}$ ist wieder eine Zahl aus $\mathbb{Z}$. Rechnen wir mit $\oplus_m$ und $\odot_m$ sind auch Summe und Produkt zweier Zahlen aus $\mathbb{Z}_m$ wieder eine Zahl aus $\mathbb{Z}_m$. Diese Eigenschaft, die *Abgeschlossenheit* genannt wird, ist nicht selbstverständlich, denn z. B. liefert die Division zweier Zahlen aus $\mathbb{Z}$ nicht unbedingt eine Zahl aus $\mathbb{Z}$.
- In $\mathbb{Z}_m$ gibt es zu jeder Zahl eine additive Inverse. Das Entsprechende ist bei $\mathbb{Z}$ der Fall.
- In $\mathbb{Z}$ gibt es nicht zu jeder Zahl eine multiplikative Inverse, ebenso i. A. nicht in $\mathbb{Z}_m$. Allerdings gibt es in $\mathbb{Z}_m \setminus \{0\}$ immer eine multiplikative Inverse, wenn m eine Primzahl ist. Auch z. B. in $\mathbb{R} \setminus \{0\}$ gibt es immer eine multiplikative Inverse.
- Die Addition und Multiplikation von Zahlen aus $\mathbb{Z}$ ist kommutativ. Auch die Addition $\oplus_m$ und Multiplikation $\odot_m$ von Zahlen aus $\mathbb{Z}_m$ ist kommutativ.
- Die Addition und Multiplikation von Zahlen aus $\mathbb{Z}$ ist assoziativ. Auch die Addition $\oplus_m$ und Multiplikation $\odot_m$ von Zahlen aus $\mathbb{Z}_m$ ist assoziativ.

Es liegen hier offenbar übergreifende Strukturen vor, d. h. Strukturen von denen einerseits $\mathbb{Z}_m$ mit den Operationen $\oplus_m$ und $\odot_m$ und andererseits $\mathbb{Z}$ (und auch $\mathbb{R}$) mit den Operationen $+$ und $\cdot$ Spezialfälle sind. In einigen Fällen gibt es aber auch strukturelle Unterschiede, so z. B. im Fall der multiplikativen Inversen zwischen den Mengen $\mathbb{Z} \setminus \{0\}$ und $\mathbb{R} \setminus \{0\}$.

Bei der Bearbeitung der Aufwärmübung 2 sollten Sie ebenfalls auf solche strukturellen Unterschiede und Gemeinsamkeiten zwischen den zu untersuchenden Mengen und Operationen gestoßen sein.

In diesem Kapitel geht es um zwei Arten von übergreifenden Strukturen, die häufig vorkommen: Gruppen und Körper.

12.3 Gruppen

Eine *Gruppe* ist eine Menge G mit einem binären Operator $\star$ mit folgenden Eigenschaften:

1. Abgeschlossenheit: Für alle $a \in G$, $b \in G$ ist auch immer $a \star b \in G$.
2. Assoziativität: Für alle $a \in G$, $b \in G$, $c \in G$ gilt: $a \star (b \star c) = (a \star b) \star c$
3. Neutrales Element: Es gibt ein neutrales Element $n \in G$, so dass für alle $a \in G$ gilt:
 $n \star a = a \star n = a$
4. Inverse Elemente: Für alle $a \in G$ gibt es ein inverses Element $a^{-1} \in G$ mit der Eigenschaft:
 $a \star a^{-1} = a^{-1} \star a = n$

Gilt zusätzlich

5. Kommutativität: Für alle $a \in G$, $b \in G$ gilt: $a \star b = b \star a$,

dann wird eine Gruppe *kommutative* oder *abelsche* Gruppe genannt.

Vielleicht haben Sie einige dieser Eigenschaften in der Aufwärmübung 2 erkannt.

Eine Gruppe ist also eine Kollektion aus zwei Teilen: eine Menge G und eine Operation $\star$. Da G und $\star$ unterscheidbar sind, wäre es ungeeignet diese Kollektion als Menge zu modellieren. Das Tupel $[G, \star]$ ist dagegen ein geeignetes Modell.

Beispiel 12.1: Die Menge $\mathbb{Z}_5$ mit der Addition $\oplus_5$ ist eine kommutative Gruppe. Die Verknüpfungstafel hatten Sie schon in Aufwärmübung 1a aufgestellt:

$\oplus_5$	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

1. Abgeschlossenheit: Aus der Verknüpfungstafel geht hervor, dass alle Verknüpfungen zweier Elemente aus $\mathbb{Z}_5$ wieder ein Element aus $\mathbb{Z}_5$ ist. $[\mathbb{Z}_5, \oplus_5]$ ist daher abgeschlossen.
2. Assoziativität: Um die Assoziativität zu überprüfen, müssen wir drei Variablen mit Elementen aus $\mathbb{Z}_5$ belegen. Dafür gibt es 5^3 Möglichkeiten - etwas zu aufwändig, um zu überprüfen, ob die geforderte Gleichheit für alle Möglichkeiten erfüllt ist.

Dies ist genau ein Bereich, wo Rechner stark sind - viele einfache Berechnungen durchführen:

```
=> plusmod5:=[x,y] |->(x+y)%5;
~< Result: [x, y] |-> x + y % 5 >~
=> Z5:={0..4};
~< Result: {0, 1, 2, 3, 4} >~
=> forall(a in Z5, b in Z5, c in Z5 |
      plusmod5(a,plusmod5(b,c)) == plusmod5(plusmod5(a,b),c) );
~< Result: true >~
```

$[\mathbb{Z}_5, \oplus_5]$ ist also assoziativ.

3. Neutrales Element: Es ist herauszufinden, ob es ein Element aus $\mathbb{Z}_5$ gibt, dass in der Summe modulo 5 mit jedem Element wieder genau dieses Element ergibt. Dies ist für die 0 der Fall, denn

$$\forall(a \in \mathbb{Z}_5 \mid 0 \oplus_5 a = a \oplus_5 0 = a) \Leftrightarrow 1.$$

Das neutrale Element ist auch direkt aus der Verknüpfungstafel ablesbar. Es ist die „Überschrift“ der Zeile, die die Überschriftenzeile reproduziert, und die Überschrift der Spalte, die die „Überschriftenspalte“ reproduziert:

$\oplus_5$	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

4. Inverse Elemente: Wir wissen schon aus Abschnitt 11.5, dass es für jedes Element aus $\mathbb{Z}_5$ immer eine additive Inverse, d. h. das Negative gibt. (11.11) gab an, wie diese berechnet werden.

Die additiven Inversen sind aber auch direkt aus der Verknüpfungstabelle ablesbar. Um bspw. zu schauen, was die Inverse von 4 ist, gehen wir zuerst in die Zeile mit der 4 und schauen, mit welcher Zahl die 4 verknüpft das neutrale Element 0 ergibt. Das ist die 1. Dann müssen wir noch schauen, ob auch die 1 (Zeile) mit der 4 (Spalte) verknüpft 0 ergibt. Das ist der Fall.

5. Kommutativität: Wie im Fall der Assoziativität haben wir hier ziemlich viel zu überprüfen, wenn auch mit 5^2 möglichen Fällen nicht ganz so viel. Nutzen wir also die Hilfe des Rechners:

```
=> forall(a in Z5, b in Z5 | plusmod5(a,b) == plusmod5(b,a) );
~< Result: true >~
```

Die Symmetrie ist auch direkt aus der Verknüpfungstabelle ablesbar, nämlich als Spiegelsymmetrie zur Diagonalen von links oben nach rechts unten:

$\oplus_5$	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

Damit sind alle fünf definierenden Eigenschaften einer kommutativen Gruppe erfüllt. $[\mathbb{Z}_5, \oplus_5]$ ist eine kommutative Gruppe. ■

Das Ergebnis des vorangegangenen Beispiels lässt sich leicht verallgemeinern: Unabhängig von m ist $[\mathbb{Z}_m, \oplus_m]$ immer eine kommutative Gruppe. Das gilt auch, wenn wir nicht mehr modular addieren und zu der vollen Menge der ganzen Zahlen zurückkehren. Auch $[\mathbb{Z}, +]$ ist eine Gruppe (mit dem neutralen Element 0), ebenso $[\mathbb{R}, +]$.

Bei der Multiplikation ist die Sache jedoch nicht ganz so einfach:

Beispiel 12.2: Die Menge $\mathbb{Z}_5$ ist mit der Multiplikation $\odot_5$ keine Gruppe. Wir überprüfen die fünf definierenden Eigenschaften wieder an Hand der Verknüpfungstabelle, die Sie ebenfalls schon in Aufwärmübung 1a aufgestellt haben:

$\odot_5$	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

1. $[\mathbb{Z}_5, \odot_5]$ ist abgeschlossen, weil die erzeugten Produkte alle Element von $\mathbb{Z}_5$ sind.
2. $[\mathbb{Z}_5, \odot_5]$ ist assoziativ. Wir überprüfen dies wieder mit *setlX*:

```
=> timesmod5:=[x,y] |-(x*y)%5;
~< Result: [x, y] |-> x * y % 5 >~
=> Z5:={0..4};
~< Result: {0, 1, 2, 3, 4} >~
=> forall(a in Z5, b in Z5, c in Z5 |
    timesmod5(a,timesmod5(b,c)) == timesmod5(timesmod5(a,b),c) );
~< Result: true >~
```

3. Es gibt auch ein neutrales Element: die 1.
4. Aber die 0 hat kein Inverses, denn es gibt kein Element von $\mathbb{Z}_5$, das mit 0 multipliziert das neutrale Element 1 ergibt.

Weil das Kriterium der Existenz der Inversen verletzt ist, kann $[\mathbb{Z}_5, \odot_5]$ keine Gruppe sein.

Anders sieht es aus, wenn wir die 0 ausschließen und $[\mathbb{Z}_5 \setminus \{0\}, \odot_5]$ betrachten. In der Abgeschlossenheit, Assoziativität und der Existenz des neutralen Elements ändert sich nichts im Vergleich zu $[\mathbb{Z}_5, \odot_5]$. Prüfen Sie das nach!

Wir machen weiter mit der Existenz der inversen Elemente:

4. Inverse Elemente: Einerseits können Sie an der Verknüpfungstafel die inversen Elemente direkt ablesen:

a	1	2	3	4
a^{-1}	1	3	2	4

Andererseits wissen wir aus Abschnitt 11.5, dass für alle Elemente aus $\mathbb{Z}_5 \setminus \{0\}$ multiplikative Inverse existieren, weil 5 eine Primzahl ist.

5. Kommutativität: Um Kommutativität zu überprüfen, bemühen wir wieder *setlX*.

```
=> forall(a in Z5, b in Z5 | timesmod5(a,b) == timesmod5(b,a) );
~< Result: true >~
```

Damit ist $[\mathbb{Z}_5 \setminus \{0\}, \odot_5]$ eine kommutative Gruppe. ■

Auch hier können wir die gewonnene Erkenntnis auf andere Modulwerte m verallgemeinern: Dazu müssen wir auf die Erkenntnisse aus Kapitel 11 zurückgreifen, dass die Existenz multiplikativer Inversen entscheidend von m abhängt. $[\mathbb{Z}_m \setminus \{0\}, \odot_m]$ ist eine kommutative Gruppe, aber nur wenn m eine Primzahl ist.

Beispiel 12.3: Die natürlichen Zahlen $\mathbb{N}$ bilden zusammen mit ihrer Addition keine Gruppe. $[\mathbb{N}, +]$ ist zwar abgeschlossen und assoziativ, aber es gibt kein neutrales Element. Das neutrale Element für die Addition wäre die Null, denn

$$\forall(m \in \mathbb{N} | m + 0 = 0 + m = m) \Leftrightarrow \mathbb{1},$$

aber die Null ist nicht Element von $\mathbb{N}$.

Wir können diesen „Makel heilen“, wenn wir stattdessen $[\mathbb{N}_0, +]$ betrachten. Diese Struktur hat mit der Null ein neutrales Element. Eine Gruppe ist sie dennoch nicht, denn es gibt (außer für die Null) keine inversen Elemente (das wären die negativen Zahlen $\{-m : m \in \mathbb{N}\}$). ■

Verknüpfungstafeln von Gruppen werden übrigens auch als *Gruppentafeln* bezeichnet.

Beispiel 12.4: Die Menge der Wahrheitswerte zusammen mit dem exklusiven Oder $\nleftrightarrow$ bilden ebenfalls eine kommutative Gruppe. Hier ist die Gruppentafel:

$\nleftrightarrow$	0	1
0	0	1
1	1	0

$\oplus_2$	0	1
0	0	1
1	1	0

Aus der Gruppentafel ist ersichtlich, dass Abgeschlossenheit vorliegt. Dass Assoziativität und Kommutativität vorliegt, wissen wir aus Kapitel 4. Das neutrale Element ist 0. Die Inversen sind: $0^{-1} = 0$ und $1^{-1} = 1$. ■

Im diesem Beispiel wurde neben der Gruppentafel für $[\{0, 1\}, \nleftrightarrow]$ die Gruppentafel für $[\mathbb{Z}_2, \oplus_2]$ gezeigt. Die strukturelle Ähnlichkeit ist ziemlich deutlich sichtbar! Es ist nicht nur so, dass beide Strukturen Gruppen sind, sondern sogar dass beide Strukturen eins zu eins aufeinander abgebildet werden können: $0 \doteq 0$, $1 \doteq 1$, $\nleftrightarrow \doteq \oplus_2$. Wir haben es hier also mit einer Metastruktur zu tun, die mehrere, „gleich gebaute“ (der Fachbegriff ist *isomorph*) Gruppen umfasst.

Die Isomorphie von $[\{0, 1\}, \nleftrightarrow]$ und $[\mathbb{Z}_2, \oplus_2]$ hat ganz praktische Bedeutungen: Anstatt mit 0, 1 und $\nleftrightarrow$ zu rechnen, können wir modulo 2 mit 0 und 1 rechnen - was dem Gehirn oft leichter fällt. Und umgekehrt können wir, anstatt mit den Binärziffern aus $\mathbb{Z}_2$ zu rechnen, Wahrheitswerte verknüpfen. Das ist die Grundlage der digitalen Rechentechnik, die binäre Arithmetik durch logische

Verknüpfungen realisiert. Es hat also seinen guten Grund, dass z. B. für wahr ein eins-ähnliches Symbol verwendet wird.

Beispiel 12.5: Auch die folgenden drei Strukturen sind isomorph:

$\wedge$	0	1	$\odot_2$	0	1	$\star$	a	b
0	0	0	0	0	0	a	a	a
1	0	1	1	0	1	b	a	b

In der Mitte befindlich ist $[\mathbb{Z}_2 \setminus \{0\}, \odot_2]$ eine Gruppe mit dem neutralen Element 1. (Eine ziemlich kleine Gruppe mit nur einem Element: $\mathbb{Z}_2 \setminus \{0\} = \{1\}$). $[\{0, 1\} \setminus \{0\}, \wedge]$ ist eine Gruppe mit dem neutralen Element 1. Sie erlaubt die Multiplikation binärer Zahlen durch die Konjunktion zu realisieren.

Was auch immer die Bedeutung von a , b und $\star$ in der rechten Verknüpfungstafel ist, die Struktur ist isomorph zu den beiden anderen und kann daher durch eine der beiden anderen realisiert werden, z. B. durch die linke, indem a als 0, b als 1 und $\star$ als $\wedge$ interpretiert wird. ■

Beispiel 12.6: Auch die komponentenweise Addition von Tupeln bildet eine Gruppe. Wir untersuchen dies hier exemplarisch für $[\mathbb{Z} \times \mathbb{Z}, +]$ mit der Addition:

$$[k_1, k_2] + [l_1, l_2] = [k_1 + l_1, k_2 + l_2] \quad (12.1)$$

1. Abgeschlossenheit: Da die Addition auf $\mathbb{Z}$ abgeschlossen ist, ist die rechte Seite von (12.1) auch wieder ein Tupel aus $\mathbb{Z} \times \mathbb{Z}$.
2. Assoziativität erfordert die Gleichheit von

$$[k_1, k_2] + ([l_1, l_2] + [m_1, m_2]) = [k_1, k_2] + [l_1 + m_1, l_2 + m_2] = [k_1 + l_1 + m_1, k_2 + l_2 + m_2]$$

und

$$([k_1, k_2] + [l_1, l_2]) + [m_1, m_2] = [k_1 + l_1, k_2 + l_2] + [m_1, m_2] = [k_1 + l_1 + m_1, k_2 + l_2 + m_2],$$

was der Fall ist, weil die beiden entstehenden Zweitupel identisch sind.

3. neutrales Element: Gesucht ist das Tupel $[n_1, n_2]$, das

$$[n_1, n_2] + [m_1, m_2] = [m_1, m_2]$$

und

$$[m_1, m_2] + [n_1, n_2] = [m_1, m_2]$$

erfüllt. Das ist das *Nulltupel* $[0, 0]$.

4. inverse Elemente: Das inverse Element zu $[m_1, m_2]$ ist $[-m_1, -m_2]$, denn

$$[m_1, m_2] + [-m_1, -m_2] = [-m_1, -m_2] + [m_1, m_2] = [0, 0].$$

5. Kommutativität: Wegen

$$[k_1, k_2] + [l_1, l_2] = [k_1 + l_1, k_2 + l_2]$$

und

$$[l_1, l_2] + [k_1, k_2] = [k_1 + l_1, k_2 + l_2]$$

und der Gleichheit der rechten Seiten, liegt Kommutativität vor.

Die untersuchte Struktur $[\mathbb{Z} \times \mathbb{Z}, +]$ ist also eine kommutative Gruppe. ■

Gegenstand von Abschnitt 10.3.3 waren Permutationen. Damit modellieren wir Situationen, wo Vielfachheit nicht vorkommt und die Reihenfolge eine Rolle spielt. Daher ist die geeignete Repräsentationsform für die Permutation das Tupel. Für Tupel der Länge 3 gibt es 3! mögliche Permutationen. Diese sind:

```
=> permutations([1,2,3]);
~< Result: {[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2],
[3, 2, 1]} >~
```

Diese Permutationen können wir als Operatoren verwenden, um Objekte zu vertauschen. Die Permutation $[3, 2, 1]$ bedeutet dann z. B. „Stelle das dritte Objekt von dem, was permutiert werden soll, an die erste Stelle, belasse das zweite Objekt an zweiter Stelle und stelle das erste Objekt an die dritte Stelle“. Lassen Sie uns das exemplarisch auf drei unterscheidbare Objekte anwenden, nennen wir Sie A, B und C:

- $[3, 2, 1]$ bringt A, B, C in die Reihenfolge C, B, A
- $[2, 1, 3]$ bringt A, B, C in die Reihenfolge B, A, C
- $[2, 1, 3]$ bringt C, B, A in die Reihenfolge B, C, A

Wir können Permutationen auch hintereinander ausführen, z. B.

$$[A, B, C] \xrightarrow{[2,1,3]} [B, A, C] \xrightarrow{[3,2,1]} [C, A, B] \quad (12.2)$$

$$[A, B, C] \xrightarrow{[3,2,1]} [C, B, A] \xrightarrow{[2,1,3]} [B, C, A]. \quad (12.3)$$

Verwenden wir als Operatorsymbol für das Hintereinanderausführen wieder $\circ$, können wir z. B. für (12.3) auch kürzer schreiben

$$[2, 1, 3] \circ [3, 2, 1] = [2, 3, 1],$$

denn

$$[A, B, C] \xrightarrow{[2,3,1]} [B, C, A].$$

Beispiel 12.7: Die Menge aller Permutationen der Länge 3 zusammen mit der Komposition bildet eine Gruppe, die sogenannte *Permutationsgruppe*.

Wir beginnen mit dem Aufstellen der Gruppentafel - Gelegenheit für Sie einige Kompositionen von Permutationen selbst zu bestimmen:

$\circ$	$[1, 2, 3]$	$[2, 3, 1]$	$[3, 1, 2]$	$[3, 2, 1]$	$[2, 1, 3]$	$[1, 3, 2]$
$[1, 2, 3]$	$[1, 2, 3]$			$[3, 2, 1]$	$[2, 1, 3]$	$[1, 3, 2]$
$[2, 3, 1]$	$[2, 3, 1]$	$[3, 1, 2]$	$[1, 2, 3]$			
 $[3, 1, 2]$			$[2, 3, 1]$	$[1, 3, 2]$	$[3, 2, 1]$	$[2, 1, 3]$
$[3, 2, 1]$		$[1, 3, 2]$		$[1, 2, 3]$	$[3, 1, 2]$	$[2, 3, 1]$
$[2, 1, 3]$			$[1, 3, 2]$		$[1, 2, 3]$	$[3, 1, 2]$
$[1, 3, 2]$	$[1, 3, 2]$	$[2, 1, 3]$	$[3, 2, 1]$	$[3, 1, 2]$	$[2, 3, 1]$	$[1, 2, 3]$

1. Abgeschlossenheit: Sie liegt vor, weil die Komposition zweier Permutationen aus der Menge `permutations([1,2,3])` wieder eine solche Permutation ist, wie die vollständig ausgefüllte Verknüpfungstafel zeigt.
2. Assoziativität: Wir delegieren die Überprüfung wieder an `setlX`, um nicht alle 6^3 zu überprüfenden Gleichungen aufschreiben zu müssen:

```
=> permkompo:=procedure(perm2,perm1){
    return [perm1[perm2[1]],perm1[perm2[2]],perm1[perm2[3]]];
};
=> perms:=permutations([1,2,3]);
=> forall(a in perms, b in perms, c in perms |
    permkompo(a,permkompo(b,c))==permkompo(permkompo(a,b),c) );
~< Result: true >~
```

3. neutrales Element: Die Permutation $[1, 2, 3]$ verändert die Objektreihenfolge nicht. Sie ist das neutrale Element. Formal:

```
=> forall(a in perms | permkompo([1,2,3],a)==a && permkompo(a,[1,2,3])==a );
~< Result: true >~
```

4. inverse Elemente: Die inversen Elemente können wir, wie in Beispiel 12.1 beschrieben, aus der Gruppentafel ablesen:

a	$[1, 2, 3]$	$[2, 3, 1]$	$[3, 1, 2]$	$[3, 2, 1]$	$[2, 1, 3]$	$[1, 3, 2]$
a^{-1}	$[1, 2, 3]$	$[3, 1, 2]$	$[2, 3, 1]$	$[3, 2, 1]$	$[2, 1, 3]$	$[1, 3, 2]$

Beispielsweise vertauscht $[1, 3, 2]$ die zweite und dritte Komponente und lässt die erste unverändert. Die Inverse macht das rückgängig, d. h. sie muss die zweite und dritte Komponente zurücktauschen und die erste ebenfalls unverändert lassen. Das ist genau die Permutation $[1, 3, 2]$. Es gilt also $[1, 3, 2]^{-1} = [1, 3, 2]$. Diese Permutation ist ihre eigene Inverse.

Die sogenannte Permutationsgruppe ist also tatsächlich eine Gruppe. Sie ist keine kommutative Gruppe, denn die Kommutativitätsbedingung ist verletzt, wie Sie schon in (12.2, 12.3) sehen konnten. ■

Auch die Komposition von Funktionen aus Kapitel 9 hat auf einigen Mengen Gruppeneigenschaften:

Beispiel 12.8: Die Menge $P = \{(\mathbb{R}^+ \rightarrow \mathbb{R}, x \mapsto p_n(x) = x^n) \mid n \in \mathbb{Q} \setminus \{0\}\}$ der Potenzfunktionen zusammen mit der Operation Komposition $\circ$ bildet eine Gruppe $[P, \circ]$. Diese Gruppe hat unendlich viele Elemente, weshalb wir keine vollständige Gruppentafel aufstellen können. Dennoch lohnt es sich, diese ansatzweise aufzuschreiben, um ein Gefühl von der Gruppe zu bekommen:

$\circ$	x	x^2	x^3	$\dots$
x	x	x^2	x^3	$\dots$
x^2	x^2	x^4	x^6	$\dots$
x^3	x^3	x^6	x^9	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Untersuchen wir also formal, ob die Gruppeneigenschaften erfüllt sind:

1. Für zwei beliebige Elemente mit den Potenzen m und n aus P gilt

$$(p_m \circ p_n)(x) = (x^n)^m = x^{m \cdot n} = p_{m \cdot n}(x).$$

Die Komposition zweier Funktionen aus P ergibt also immer eine Funktion aus P . Die Abgeschlossenheit ist somit erfüllt.

2. Wegen

$$\begin{aligned} (p_l \circ (p_m \circ p_n))(x) &= (x^{n \cdot m})^l = x^{l \cdot m \cdot n} = p_{l \cdot m \cdot n}(x) \\ ((p_l \circ p_m) \circ p_n)(x) &= (x^n)^{l \cdot m} = x^{l \cdot m \cdot n} = p_{l \cdot m \cdot n}(x) \end{aligned}$$

gilt auch Assoziativität.

3. Das neutrale Element der Gruppe ist die Identitätsfunktion auf $\mathbb{R}^+$

$$\mathbb{R}^+ \rightarrow \mathbb{R}, x \mapsto \text{id}(x) = x,$$

die hier unter dem Namen p_1 auftritt. Die Identitätsfunktion erfüllt die definierende Eigenschaft des neutralen Elements:

$$\text{id} \circ p_m = p_m \circ \text{id} = p_m,$$

denn

$$\begin{aligned}(\text{id} \circ p_m)(x) &= \text{id}(x^m) = x^m = p_m(x) \\ (p_m \circ \text{id})(x) &= (\text{id}(x))^m = x^m = p_m(x).\end{aligned}$$

4. Das inverse Element zu p_m ist $p_{1/m}$, denn wegen

$$\begin{aligned}(p_{1/m} \circ p_m)(x) &= (x^m)^{\frac{1}{m}} = x = \text{id}(x) \\ (p_m \circ p_{1/m})(x) &= \left(x^{\frac{1}{m}}\right)^m = x = \text{id}(x)\end{aligned}$$

gilt

$$p_m \circ p_{1/m} = p_{1/m} \circ p_m = \text{id}. \quad (12.4)$$

5. Wegen

$$\begin{aligned}(p_m \circ p_n)(x) &= (x^n)^m = x^{m \cdot n} = p_{m \cdot n}(x) \\ (p_n \circ p_m)(x) &= (x^m)^n = x^{m \cdot n} = p_{m \cdot n}(x)\end{aligned}$$

gilt sogar Kommutativität.

$[P, \circ]$ ist also eine kommutative Gruppe. ■

Dieses Beispiel hat auch die besondere Bedeutung der Identitätsfunktion id beleuchtet: Sie ist das neutrale Element zur Komposition. Damit ist sie gemäß der Definition des Konzepts Gruppe zentral für die Bestimmung der Inversen. Diese Inversen sind nichts anderes als die Umkehrfunktionen, siehe (9.13) und auch (12.4) im obigen Beispiel. Dort sind Wurzelfunktionen die Umkehrfunktionen zu den Potenzfunktionen mit ganzzahligem Exponenten (und natürlich umgekehrt).

12.4 Körper

Die „nächstgrößere“ bedeutende Struktur umfasst eine Menge mit zwei Operatoren mit besonderen Eigenschaften:

Ein *Körper* ist eine Menge K mit mindestens zwei Elementen, dem *Null-Element* $\tilde{0}$ und dem *Eins-Element* $\tilde{1}$, und mit zwei binären Operatoren $\oplus$ und $\otimes$ mit folgenden Eigenschaften:

1. $[K, \oplus]$ ist eine kommutative Gruppe mit neutralem Element $\tilde{0}$.
2. $[K \setminus \{\tilde{0}\}, \otimes]$ ist eine kommutative Gruppe mit neutralem Element $\tilde{1}$.
3. Distributivität: Für alle $a \in K$, $b \in K$, $c \in K$ gilt: $a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$

Während wir bei der Gruppe das neutrale Element noch mit n bezeichnet hatten, brauchen wir hier zwei Symbole: das *Null-Element* $\tilde{0}$ und das *Eins-Element* $\tilde{1}$. Die Bezeichnungen sind in Anlehnung an die Null als neutrales Element der Gruppe $[\mathbb{R}, +]$, also der Addition, und die Eins als neutrales Element der Gruppe $[\mathbb{R} \setminus \{0\}, \cdot]$, also der Multiplikation.

Ein geeignetes Modell für die Kollektion Körper ist das Tripel $[K, \oplus, \otimes]$.

Beispiel 12.9: Die reellen Zahlen mit ihrer Addition und Multiplikation, d. h. $[\mathbb{R}, +, \cdot]$ bilden einen Körper, denn:

1. $[\mathbb{R}, +]$ ist eine kommutative Gruppe mit dem neutralen Element 0. Dies hatten wir in Abschnitt 12.3 festgestellt.
2. $[\mathbb{R} \setminus \{0\}, \cdot]$ ist eine kommutative Gruppe mit dem neutralen Element 1. Dies hatten wir in Abschnitt 12.3 festgestellt.

3. Es gilt das Distributivgesetz

$$\forall (a \in \mathbb{R}, b \in \mathbb{R}, c \in \mathbb{R} \mid a \cdot (b + c) = a \cdot b + a \cdot c) \Leftrightarrow \mathbb{1},$$

d. h. die Multiplikation ist distributiv über die Addition.

Alle Körpereigenschaften sind also erfüllt. ■

Der Körper, der Ihnen vermutlich am vertrautesten ist, ist $[\mathbb{R}, +, \cdot]$. Wenn eine Struktur ein Körper ist, ist alles genauso wie bei den reellen Zahlen, d. h. man kann nach Herzens Lust addieren, subtrahieren, multiplizieren und dividieren - was immer die analogen Operationen in dieser Struktur bedeuten.

Beispiel 12.10: Die Menge $P_0 = \{(\mathbb{R}^+ \rightarrow \mathbb{R}, x \mapsto p_n(x) = x^n) \mid n \in \mathbb{Q}\}$ der Potenzfunktionen zusammen mit den Operationen Multiplikation $\cdot$ und Komposition $\circ$ bildet einen Körper $[P_0, \cdot, \circ]$.

1. $[P_0, \cdot]$ ist eine kommutative Gruppe. Dies bietet eine gute Gelegenheit für Sie, das Überprüfen von Gruppeneigenschaften zu üben:



Das neutrale Element (das ist hier das Null-Element $\tilde{0}$) ist

$$\mathbb{R}^+ \rightarrow \mathbb{R}, x \mapsto p_0(x) = 1.$$

2. $[P_0 \setminus \{\tilde{0}\}, \circ]$ ist ebenfalls eine kommutative Gruppe. Das hatten wir bereits in Beispiel 12.8 festgestellt, als wir $[P, \circ]$ untersucht hatten. Für das dortige P gilt $P = P_0 \setminus \{\tilde{0}\}$, denn P ist die Menge der Potenzfunktionen und hat nicht die Konstante Funktion p_0 als Element.
3. Es gilt das Distributivgesetz

$$p_l \circ (p_m \cdot p_n) = (p_l \circ p_m) \cdot (p_l \circ p_n),$$

denn

$$\begin{aligned} (p_l \circ (p_m \cdot p_n))(x) &= p_l((p_m(x) \cdot p_n(x))) = (x^m \cdot x^n)^l = x^{l \cdot (m+n)} = x^{l \cdot m + l \cdot n} \\ (p_l \circ p_m \cdot p_l \circ p_n)(x) &= p_l(p_m(x)) \cdot p_l(p_n(x)) = (x^m)^l \cdot (x^n)^l = x^{l \cdot m + l \cdot n}. \end{aligned}$$

Damit ist nachgewiesen, dass $[P_0, \cdot, \circ]$ alle Eigenschaften eines Körpers hat. ■

Die Gebilde $[\mathbb{R}, +, \cdot]$ und $[P_0, \cdot, \circ]$ gehören also zur selben Strukturklasse. Beides sind Körper. Was für die reellen Zahlen die Addition ist, ist für die Potenzfunktionen also die Multiplikation. Die Multiplikation von Potenzfunktionen hat ja auch mit der Addition der Exponenten zu tun. Und was für die reellen Zahlen die Multiplikation ist, ist für die Potenzfunktionen die Komposition. Die Komposition zweier Potenzfunktionen hat ja auch mit der Multiplikation der Exponenten zu tun.

Der Körper $[\mathbb{Q}, +, \cdot]$ steht sogar in einer noch engeren Beziehung zum Körper $[P_0, \cdot, \circ]$, denn es gibt eine eins-zu-eins-Abbildung zwischen ihren Elementen:

$$\mathbb{Q} \rightarrow P_0, q \mapsto (x \mapsto x^q)$$

12.5 Lernziele

Studierende	Kennen	Können	Verstehen
fachlich	☐ nennen Eigenschaften von Gruppen und Körpern	☐ stellen Gruppentafeln/Verknüpfungstabellen auf ☐ bestimmen neutrale Elemente und Inverse	☐ entscheiden, ob etwas eine Gruppe ist ☐ entscheiden, ob etwas ein Körper ist ☐ entscheiden, ob eine binäre Operation assoziativ bzw. kommutativ ist ☐ entscheiden, ob eine binäre Operation distributiv über eine zweite binäre Operation ist
methodisch	☐	☐ schreiben Programm, das entscheidet, ein Objekt bestimmte algebraische Eigenschaften hat	☐

Hinzu kommen die themenübergreifenden Lernziele aus den Kategorien „sozial“ und „persönlich“ aus Abschnitt 1.7 sowie die Lernziele aus der Kategorie „methodisch“ aller vorangegangener Kapitel. Haken Sie die Lernziele ab, von denen Sie zuversichtlich sind, dass Sie sie bereits erreicht haben.

12.6 Übungsaufgaben

1. Untersuchen Sie jeweils, ob die folgenden Strukturen eine Gruppe bilden:

- (a) $[\mathbb{Z}, +]$
- (b) $[\mathbb{Z} \setminus \{0\}, \cdot]$
- (c) $[\mathbb{Q}, +]$
- (d) $[\mathbb{Q} \setminus \{0\}, \cdot]$
- (e) $[\mathbb{R}, +]$
- (f) $[\mathbb{R} \setminus \{0\}, \cdot]$
- (g) $[\mathbb{N}, \cdot]$
- (h) $[E, +]$ mit der Menge der geraden Zahlen $E = \{2m : m \in \mathbb{Z}\}$
- (i) $[E, \cdot]$ mit der Menge der geraden Zahlen $E = \{2m : m \in \mathbb{Z}\}$
- (j) $[\{0, 1\}, \rightarrow]$
- (k) $[\{0, 1\}, \leftrightarrow]$

2. Die Konkatenation $\bowtie$ ist eine binäre Operation, die zwei Zeichenketten (*Strings*) zu einer neuen Zeichenkette verbindet. So ist z. B.

$$\text{Kon} \bowtie \text{katenation} = \text{Konkatenation}$$

Untersuchen Sie, welche Eigenschaften einer Gruppe die Konkatenation auf der Menge aller Zeichenketten erfüllt.

3. Bezeichnen wir mit $\mathcal{D}_i^{\pm 90}$ die Drehung um $\pm 90^\circ$ um die i -Achse, also z. B. mit $\mathcal{D}_z^{-90}$ die Drehung um -90° um die z -Achse. Entsprechend soll $\mathcal{D}_i^{180}$ die Drehung um 180° um die i -Achse bezeichnen und $\mathcal{D}_i^0$ die Drehung um 0° . Untersuchen Sie, ob die Menge dieser zwölf Drehungen mit der Operation „Hintereinanderausführung von Drehungen“ eine Gruppe ist.
4. Untersuchen Sie jeweils, ob die folgenden Strukturen einen Körper bilden:
- $[\mathbb{N}, +, \cdot]$
 - $[\mathbb{Z}, +, \cdot]$
 - $[\mathbb{Q}, +, \cdot]$
 - $[\mathbb{Z}_m, \oplus_m, \odot_m]$
 - $[\{\emptyset, \mathbb{1}\}, \wedge, \nrightarrow]$
 - $[\mathbb{P}, +, \circ]$ mit der Menge der Polynome

$$\mathbb{P} = \left\{ \left(\mathbb{R} \rightarrow \mathbb{R}, x \mapsto \sum_{i=0}^N a_i x^i \right) \mid N \in \mathbb{N}_0, a_i \in \mathbb{R} \right\}$$

5. Stellen Sie die Gruppentafel für die Gruppe $[\{a, b, c, d\}, \otimes]$ auf, die die folgenden Eigenschaften hat:
- Das neutrale Element ist a .
 - Jedes Element ist seine eigene Inverse.
6. Schreiben Sie in *setlX* ein Programm, das überprüft, ob eine gegebenen Struktur eine Gruppe ist.
- Was ergibt Ihr Programm für $[\{0..4\}, \text{plusmod5}]$, wobei $\text{plusmod5} := [x, y] \mapsto (x+y) \% 5$?
7. Schreiben Sie in *setlX* ein Programm, das überprüft, ob eine gegebene Struktur ein Körper ist.